

ASP.NET Core MVC ile C# Web Geliştirme — Teknik Referans Belgesi

Kapsamlı, kitap düzeyinde referans. .NET 8 + ASP.NET Core MVC + C# ekosistemi için tek başvuru kaynağı olarak yazılmıştır.

ORTAM

Bileşen	Sürüm / Değer
.NET SDK	8.0.x
C#	12
ASP.NET Core	8.0 (MVC)
Entity Framework Core	8.0.x
SQL Server	LocalDB veya SQL Server 2019+
Newtonsoft.Json	13.0.x
jQuery	3.x (proje şablonu ile gelen)
Bootstrap	5.x (proje şablonu ile gelen)
Bootstrap Icons	1.11+ (opsiyonel)
Bootswatch	5.x (opsiyonel tema)
Toastr	2.1.4 (CDN)
IDE	Visual Studio 2022 (17.8+) veya Rider 2024+
NuGet Paketleri (özet)	Microsoft.EntityFrameworkCore.SqlServer, Microsoft.EntityFrameworkCore.Tools, Microsoft.EntityFrameworkCore.Design, Microsoft.AspNetCore.Mvc.Razor.RuntimeCompilation
Package Manager Console (PMC)	Visual Studio → Tools → NuGet Package Manager → PMC
Konvansiyon	Türkçe entity/property adları (proje bağlamıyla uyumlu)

İçindekiler

- **Bölüm I — Programlama Temelleri**
 - 1.1 JavaScript Temelleri
 - 1.2 jQuery ile DOM ve Olay Yönetimi
 - 1.3 Nesne Yönelimli Programlamaya Genel Bakış
 - 1.4 Encapsulation (Kapsülleme)
 - 1.5 Inheritance (Kalıtım)
 - 1.6 Polymorphism (Çokbiçimlilik)
 - 1.7 Abstraction (Soyutlama)
 - 1.8 SOLID Prensipleri
- **Bölüm II — C# Dili**
 - 2.1 Sınıf, Nesne, Constructor, Property
 - 2.2 Interface ve Abstract Class
 - 2.3 Erişim Belirleyiciler
 - 2.4 Değişkenler, Veri Tipleri, Bellek Modeli
 - 2.5 Tip Dönüşümü, Boxing / Unboxing
 - 2.6 Kontrol Yapıları ve Pattern Matching
 - 2.7 Döngüler
 - 2.8 Try-Catch (Hata Yönetimi)
 - 2.9 Diziler ve Çok Boyutlu Diziler
 - 2.10 Koleksiyonlar (IEnumerable, ICollection, List, Dictionary)
 - 2.11 String İşlemleri
 - 2.12 Metotlar ve Parametre Türleri
 - 2.13 Tuple
 - 2.14 Debug ve Watch Penceresi
 - 2.15 Top-Level Statements, Random, using Direktifleri

- **Bölüm III — Veritabanı ve Entity Framework Core**
 - 3.1 İlişkisel Veritabanı Temelleri
 - 3.2 İlişkiler (1-1, 1-N, N-M) ve Foreign Key Davranışları
 - 3.3 ORM Kavramı ve EF Core'a Genel Bakış
 - 3.4 EF vs EF Core Karşılaştırması
 - 3.5 EF Core Kurulumu ve NuGet Paketleri
 - 3.6 DbContext ve DbSet
 - 3.7 Data Annotations
 - 3.8 OnModelCreating ve Fluent API
 - 3.9 Code First ve Migration
 - 3.10 Database First ve Scaffold-DbContext
 - 3.11 LINQ Sorguları
 - 3.12 Eager Loading, Lazy Loading, Projection, Materialization
 - 3.13 CRUD Operasyonları ve SaveChanges
 - 3.14 Unique Constraint ve HasIndex
- **Bölüm IV — ASP.NET Core MVC**
 - 4.1 MVC Tasarım Deseni
 - 4.2 Proje Yapısı ve Anahtar Dosyalar
 - 4.3 Program.cs (Minimal Hosting)
 - 4.4 appsettings.json ve ConnectionStrings
 - 4.5 SDK vs Runtime
 - 4.6 Routing
 - 4.7 Controller ve ActionResult Tipleri
 - 4.8 Razor View Engine
 - 4.9 Layout, _ViewStart, _ViewImports, Sections, Partial View
 - 4.10 Controller'dan View'a Veri Taşıma (ViewBag/ViewData/TempData/Model)
 - 4.11 Tag Helpers
 - 4.12 HTML Form Elemanları
 - 4.13 Model Binding
 - 4.14 ViewModel ve DTO
 - 4.15 Model Validation
 - 4.16 Dependency Injection
 - 4.17 Middleware ve Pipeline
 - 4.18 Anti-forgery Koruması
 - 4.19 AJAX (jQuery \$.ajax, JsonResult, Cascading Dropdown)
 - 4.20 Base Sınıflar (BaseEntity, BaseController, BaseService)
 - 4.21 Soft Delete
 - 4.22 Toastr Bildirimleri
- **Bölüm V — Proje Dokümantasyonları**
 - 5.1 BulkyBookWeb — Kategori CRUD Modülü
 - 5.2 HastaneMVC — Kapsamlı Randevu Yönetimi
 - 5.3 HastaneWeb (AspNet7Hastane) — Database First + AJAX
 - 5.4 ÖrnekUygulama — Basit Product CRUD
- **Ekler**
 - Ek A. Tag Helper Hızlı Referansı
 - Ek B. Migration Komutları (PMC + .NET CLI)
 - Ek C. Bootswatch ve Bootstrap Icons Kurulumu
 - Ek D. Razor Runtime Compilation
 - Ek E. Git ve GitHub Entegrasyonu
 - Ek F. Terim Sözlüğü

Bölüm I — Programlama Temelleri

1.1 JavaScript Temelleri

1.1.1 Genel Bakış

JavaScript, tarayıcının içine gömülü olarak çalışan, dinamik sayfa davranışını mümkün kılan bir betik dilidir. HTML sayfanın iskeletini, CSS görünümünü tanımlarken JavaScript sayfayı etkileşimi (buton tıklaması, form doğrulama, sayfa yenilemeden veri çekme) yönetir.

ASP.NET Core MVC projelerinde JavaScript'in iki temel rolü vardır: (a) Razor tarafından üretilen HTML üzerinde son kullanıcı etkileşimini yürütmek, (b) Controller action'larıyla AJAX üzerinden asenkron veri alışverişi kurmak. Bu iki rol, sonraki bölümlerde detaylandırılan Tag Helper, Model Binding ve JsonResult kavramlarıyla iç içedir.

Modern JavaScript (ES6+), değişken bildirimini `let` ve `const` ile yapar; eski `var` anahtar kelimesi kapsam sorunları nedeniyle önerilmez.

1.1.2 Değişken Bildirimi

```
// ornekler/01-degisken-tanimlama.js
let degisken = 10;           // değeri değiştirilebilir
const sabit = 3.14;         // yeniden atanamaz
var eskiYontem = "eski";    // kapsamı fonksiyonel, kaçınılmalı
```

- `let`: blok kapsamlı (`{ }` içinde tanımlanır, dışında görünmez), yeniden atanabilir.
- `const`: blok kapsamlı, yeniden **atanamaz** — ancak içeriği (nesne alanları, dizi elemanları) değiştirilebilir.
- `var`: fonksiyon kapsamlı, hoisting nedeniyle bildirimden önce kullanılabilir (undefined döner). Yeni kodda tercih edilmez.

1.1.3 Temel Veri Tipleri

Tip	Örnek	typeof Sonucu
string	"Ayşe"	"string"
number	22, 3.14	"number"
boolean	true, false	"boolean"
null	null	"object" (tarihsel hata)
undefined	atanmamış değişken	"undefined"
object	{ ad: "Ali" }	"object"
array	[1, 2, 3]	"object"
function	function(){}	"function"

Örnek 1.1.1 — typeof ile tip kontrolü

```
// ornekler/02-typeof.js
let isim = "Ayşe";
let yas = 22;
let aktif = true;
let skor = null;
let adres;

console.log(typeof isim); // string
console.log(typeof yas); // number
console.log(typeof aktif); // boolean
console.log(typeof skor); // object ← tarihsel JS hatası
console.log(typeof adres); // undefined
```

Çıktı:

```
string
number
boolean
object
undefined
```

`typeof null` sonucunun "object" dönmesi, JavaScript'in ilk sürümlerinden kalma geri uyumluluk sorunudur; standartlaşmıştır ve değiştirilmez.

1.1.4 const ve Nesne Mutasyonu

`const` bir *referansı* sabitler, işaret ettiği nesnenin içeriğini değil.

Örnek 1.1.2

```
// ornekler/03-const-nesne.js
const kullanıcı = { ad: "Ali", yas: 30 };
kullanıcı.yas = 31;           // izinli – nesne alanı değişti
kullanıcı.email = "ali@x.com"; // izinli – yeni alan eklendi
// kullanıcı = { ad: "Veli" }; // TypeError: Assignment to constant variable
```

Çıktı (yorum satırı açılırsa):

Uncaught TypeError: Assignment to constant variable.

1.1.5 Karar Yapıları

Örnek 1.1.3 — Not harfine dönüştürme

```
// ornekler/04-not-harf.js
let not = 78;
let harf;

if (not >= 90) harf = "AA";
else if (not >= 80) harf = "BB";
else if (not >= 70) harf = "CC";
else if (not >= 60) harf = "DD";
else harf = "FF";

console.log(`Notunuz: ${not}, Harf: ${harf}`);
```

Çıktı:

Notunuz: 78, Harf: CC

Aynı örneğin C# karşılığı §2.6.1’de bulunmaktadır; iki dilin sözdizimsel yakınlığı bu karşılaştırmayla belirginleşir.

1.1.6 Döngüler ve Dizi Metotları

Örnek 1.1.4 — forEach ile dizi tarama

```
// ornekler/05-foreach.js
const ogrenciler = ["Ali", "Veli", "Ayşe"];
ogrenciler.forEach((ad, indeks) => {
  console.log(`[${indeks}] ${ad.toUpperCase()}`);
});
```

Çıktı:

```
[0] ALI
[1] VELI
[2] AYŞE
```

Örnek 1.1.5 — filter + map zincirlemesi

```
// ornekler/06-filter-map.js
const notlar = [85, 42, 91, 60, 73];
const gecenler = notlar
  .filter(n => n >= 60)
  .map(n => `${n} ✓`);
console.log(gecenler);
```

Çıktı:

```
[ '85 ✓', '91 ✓', '60 ✓', '73 ✓' ]
```

filter bir yordam (predicate) alır, koşulu sağlayan elemanları toplar. map her elemanı dönüştürür. İkisi zincirlenerek yeni bir dizi üretir; orijinal dizi değişmez (immutable pattern).

1.1.7 Fonksiyonlar

```
// Klasik ifade
function topla(a, b) {
  return a + b;
}

// Arrow function
const carp = (a, b) => a * b;

// Varsayılan parametre
function selamla(ad = "misafir") {
  return `Merhaba ${ad}`;
}

console.log(topla(3, 5)); // 8
console.log(carp(4, 6)); // 24
console.log(selamla()); // Merhaba misafir
console.log(selamla("Deniz")); // Merhaba Deniz
```

Çıktı:

```
8
24
Merhaba misafir
Merhaba Deniz
```

1.1.8 Yaygın Hatalar

Hatalı Kod	Hata Mesajı / Sonuç	Neden	Düzeltilmiş
const x = 5; x = 6;	TypeError: Assignment to constant variable.	const yeniden atanamaz	let x = 5; x = 6;
let a; console.log(b); console.log([1,2] + [3,4]);	ReferenceError: b is not defined "1,23,4"	b hiç bildirilmemiş Diziler string'e dönüştürülüp birleştiriliyor	Önce let b; bildirin [...a, ...b] veya a.concat(b)
typeof null === "object"	Beklenmedik true	Tarihsel hata	x === null ile açık kontrol

1.1.9 Mini Sözlük

- **Değişken (variable):** Bellekte bir değer için verilen ad.
- **Kapsam (scope):** Bir değişkenin görünür/erişilebilir olduğu bölge.
- **Hoisting:** var bildirimlerinin kapsamın başına taşındığı ilüzyonu; let/const'ta yoktur.
- **Predicate:** Boolean döndüren fonksiyon; filter, some, every gibi metotların aldığı bağımsız değişken.
- **Immutable pattern:** Orijinal veriyi değiştirmeden yeni veri üretme yaklaşımı.

1.1.10 Bölüm Özeti

- let/const blok kapsamlıdır; var yerine bunlar kullanılır.
- JavaScript zayıf tipli bir dildir; typeof operatörüyle tip sorgulanır.
- null için typeof "object" döner — dilin tarihsel bir kusuru.
- Dizi metotları (forEach, filter, map, reduce) zincirlenebilir ve orijinal diziyi değiştirmez.
- const, referansı sabitler; nesnenin içeriğini değil.

İlgili konular: §1.2 (jQuery), §4.19 (AJAX), §2.6 (C# karar yapıları).

1.2 jQuery ile DOM ve Olay Yönetimi

1.2.1 Genel Bakış

jQuery, saf JavaScript ile yazılması uzun süren DOM manipülasyonu, olay bağlama ve AJAX çağrılarını tek satırlarla ifade eden bir kütüphanedir. Modern tarayıcıların hepsinde bulunan Fetch API ve Web Component'ler jQuery'nin bir kısım rolünü devralsa da, ASP.NET Core MVC şablonu jQuery'yi varsayılan olarak `wwwroot/lib/jquery/` altına dahil eder ve `_ValidationScriptsPartial` gibi partial view'lar jQuery'ye bağımlıdır. Bu nedenle jQuery, kurumsal ASP.NET Core MVC projelerinde hâlâ birinci sınıf araçtır.

jQuery ile tüm işlemler `$` (dolar) sembolü üzerinden yapılır. `$` teknik olarak bir değişkendir; jQuery kütüphanesi yüklendiğinde global kapsama jQuery ve `$` adlarıyla iki referans olarak yerleşir.

1.2.2 Kütüphanenin Sayfaya Dahil Edilmesi

Layout dosyasında (`Views/Shared/_Layout.cshtml`) `<head>` veya `body` sonunda:

```
<!-- Views/Shared/_Layout.cshtml (parça) -->
<script src="~/lib/jquery/dist/jquery.min.js"></script>
```

Proje şablonu bu satırı otomatik olarak ekler; sayfaya özel jQuery kodu `@section Scripts { ... }` içinde yazılır (bkz. §4.9).

1.2.3 Seçiciler

Seçici	Anlamı	Örnek
<code>\$("#id")</code>	id ile eleman	<code>\$("#kaydetBtn")</code>
<code>\$(".sinif")</code>	class ile eleman(lar)	<code>(".hata-alani")</code>
<code>\$("etiket")</code>	HTML etiketi	<code>("input")</code>
<code>\$("#a, #b")</code>	çoklu id	<code>("#ad, #soyad")</code>
<code>\$(this)</code>	bir olay yakalayıcısı içinde tetikleyen eleman	<code>\$(this).val()</code>

1.2.4 Olay Metotları

Örnek 1.2.1 — Tıklama sayacı

```
<!-- Views/Home/Privacy.cshtml (parça) -->
<button id="btn">Tıkla</button>
<span id="sayac">0</span>
```

```
@section Scripts {
<script>
    let sayi = 0;
    $("#btn").click(function () {
        sayi++;
        $("#sayac").text(sayi);
    });
</script>
}
```

Beklenen davranış: Butona her tıklandığında `<span>` içindeki değer bir artar.

Örnek 1.2.2 — Select değişimi (`.change`)

```
<select id="sehir">
    <option value="">-- Şehir --</option>
    <option>İstanbul</option>
    <option>Ankara</option>
    <option>İzmir</option>
</select>
<p id="sonuc"></p>
```

```
<script>
  $("#sehir").change(function () {
    $("#sonuc").text("Seilen: " + $(this).val());
  });
</script>
```

Beklenen davranış: <select>'ten bir öge seçildiğinde <p> içeriğı güncellenir.

Örnek 1.2.3 — Karakter sayacı (.keyup)

```
<input id="metin" type="text" />
<p>Karakter: <span id="adet">0</span></p>

<script>
  $("#metin").keyup(function () {
    $("#adet").text($(this).val().length);
  });
</script>
```

Beklenen davranış: Her tuşa basıldığında mevcut karakter sayısını gösterir.

1.2.5 İçerik ve Stil Metotları

Metot	Ne yapar	Örnek
.text(x)	Elemanın metnini ayarlar; HTML olarak <i>işlemez</i>	\$("#a").text("x") düz metin
.html(x)	Elemanın iç HTML'ini ayarlar; HTML işlenir	\$("#a").html("x") kalın yazı
.val()	Form kontrolünün değeri	\$("#ad").val(), \$("#ad").val("Ali")
.attr(name, val)	HTML attribute okur / yazar	\$("#img").attr("src", "x.png")
.css(prop, val)	CSS özelliğini ayarlar	\$("#a").css("color", "red")
.addClass(name)	Sınıf ekler	\$("#a").addClass("aktif")
.removeClass(name)	Sınıf çıkarır	\$("#a").removeClass("aktif")
.toggleClass(name)	Varsa çıkarır yoksa ekler	\$("#a").toggleClass("acik")

Örnek 1.2.4 — .text() ile .html() farkı

```
<div id="a"></div>
<div id="b"></div>

<script>
  $("#a").text("<b>Kalın</b>"); // ekranda: <b>Kalın</b> (metin olarak)
  $("#b").html("<b>Kalın</b>"); // ekranda: Kalın (kalın)
</script>
```

Bu ayrım güvenlik açısından da önemlidir: kullanıcıdan gelen içeriğı .html() ile enjekte etmek XSS zafiyeti doğurur. Bilinmeyen içerik için .text() kullanılır.

1.2.6 AJAX Yardımcıları (Kısa)

jQuery'nin AJAX metotlarının tam anlatımı §4.19'dadır. Kısaca:

- .load(url) — HTML parçası çekip elemana yerleştirir.
- \$.get(url, callback) — GET isteğı yollar.
- \$.post(url, data, callback) — POST isteğı yollar.
- \$.ajax({ ... }) — Tüm parametreleri açıkça verilen ana metot.

1.2.7 Yaygın Hatalar

Hatalı Kod	Sonuç	Neden	Düzeltilmiş
<code>\$(...).click(...)</code> sayfa yükleme öncesi <code>.text(userInput)</code> yerine <code>.html(userInput)</code>	Bağlanma sessizce başarısız XSS zafiyeti	DOM henüz oluşmadı HTML işleniyor	<code>\$(function(){ ... })</code> içinde çalıştır Kullanıcı içeriği için <code>.text()</code>
<code>\$("#a").val = "x"</code> Script_Layout içinde @section dışına yazılmış	Değer atanmaz Bazı sayfalarda çalışmıyor	val bir metottur Global scope'a kaçtı	<code>\$("#a").val("x")</code> Sayfa scripti @section Scripts içinde

1.2.8 Bölüm Özeti

- Tüm jQuery işlemleri \$ üzerinden yapılır; \$ bir global değişkendir.
- Seçiciler CSS ile aynı sözdizimidir: #id, .class, etiket.
- Olay metotları .click, .change, .keyup, .submit gibi kısa adlar taşır.
- .text() metni, .html() HTML'i yerleştirir; XSS için .html() dikkatli kullanılmalı.
- Sayfaya özel script @section Scripts içine yazılır (§4.9).

İlgili konular: §4.19 (AJAX), §4.9 (Sections / Layout), §4.11 (Tag Helpers).

1.3 Nesne Yönelimli Programlamaya Genel Bakış

1.3.1 Genel Bakış

Nesne yönelimli programlama (OOP), yazılımı birbirleriyle mesaj alışverişinde bulunan **nesneler** üzerinden kurgulayan paradigmadır. Her nesne kendi verisini (state) ve bu veri üzerinde işlemleri (behavior) taşır. C#, tümüyle OOP tabanlı bir dildir; Program sınıfı bile — top-level statements kullanılsa dahi — arka planda bir sınıf olarak derlenir.

OOP'nin dört temel prensibi vardır: **Encapsulation** (kapsülleme), **Inheritance** (kalıtım), **Poly-morphism** (çokbiçimlilik), **Abstraction** (soyutlama). Bu prensiplerin her biri, ayrı bir tasarım problemine yanıt olarak evrilmiştir:

- Kapsülleme, veriyi kontrolsüz değişikliklerden korur.
- Kalıtım, kod tekrarını ortak bir taban sınıfa taşıyarak azaltır.
- Çokbiçimlilik, aynı arayüzü paylaşan farklı davranışları tek bir noktadan çağırmayı sağlar.
- Soyutlama, gereksiz detayı gizleyerek karmaşıklığı yönetilebilir kılar.

Aşağıdaki başlıklar dördünü ayrı ayrı işler. Örneklerin tamamı C#'tadır; C# sınıf temellerinin sözdizimsel referansı §2.1'de yer alır.

1.3.2 OOP'nin Sağladığı Kazanımlar

- **Kod tekrarının azalması:** Ortak alanlar taban sınıfa taşınır.
- **Modülerlik:** Her sınıf tek bir sorumluluk taşır (§1.8.1 SRP ile bağlantılı).
- **Bakım kolaylığı:** Bir sınıftaki değişiklik, o sınıfın kapsülünde kalır.
- **Test edilebilirlik:** Interface'ler üzerinden sahte (mock) uygulamalar yazılır.
- **Genişletilebilirlik:** Var olan davranışa dokunmadan yeni tipler eklenebilir (§1.8.2 OCP).

1.3.3 Örnek: OOP Olmayan vs OOP Yaklaşımı

Aynı problem — bir ürünün fiyatını KDV'li olarak hesaplama — iki yaklaşımla:

```
// Prosedürel yaklaşım
double KdvliFiyat(double fiyat, double oran) => fiyat * (1 + oran);

double urun1Fiyat = 100;
double urun2Fiyat = 250;
```

```
Console.WriteLine(KdvliFiyat(urun1Fiyat, 0.20));
Console.WriteLine(KdvliFiyat(urun2Fiyat, 0.20));
```

```
// Nesne yönelimli yaklaşım – Models/Urun.cs
public class Urun
{
    public string Ad { get; set; }
    public double Fiyat { get; set; }
    public double KdvOrani { get; set; } = 0.20;

    public double KdvliFiyat() => Fiyat * (1 + KdvOrani);
}

// Kullanım
var urun1 = new Urun { Ad = "Kalem", Fiyat = 100 };
var urun2 = new Urun { Ad = "Defter", Fiyat = 250 };
Console.WriteLine($"{urun1.Ad}: {urun1.KdvliFiyat()}");
Console.WriteLine($"{urun2.Ad}: {urun2.KdvliFiyat()}");
```

Çıktı:

Kalem: 120
Defter: 300

İkinci yaklaşımda hem veri (ad, fiyat, oran) hem de davranış (KdvliFiyat) tek bir birimde toplanmıştır. Bu birim, projede tekrar tekrar Urun tipiyle taşınabilir; entity'ye dönüştüğünde EF Core ile veritabanına aynen eşlenir (§3.6).

1.3.4 Bölüm Özeti

- OOP, veri + davranışı bir arada tutan nesneler üzerinde kuruludur.
- Dört prensip birlikte çalışır: Encapsulation, Inheritance, Polymorphism, Abstraction.
- OOP tasarım kararları, SOLID (§1.8) ile pekiştirilir.

İlgili konular: §1.4–§1.7 (dört prensip), §1.8 (SOLID), §2.1 (C# sınıfları).

1.4 Encapsulation (Kapsülleme)

1.4.1 Genel Bakış

Kapsülleme, bir nesnenin iç durumunu (private alanlar) dışarıdan doğrudan erişime kapatıp, kontrollü giriş noktaları (property, metot) üzerinden ulaşılabilir kılmaktır. Amaç, geçersiz durumların oluşmasını sınıfın sınırında yakalamaktır. Örneğin bir banka hesabı bakiyesinin negatife düşmesi, sınıf içinde tek bir setter'da engellenebilir; her çağırın kodda ayrıca kontrol etmek gerekmez.

C#'ta kapsülleme iki araçla sağlanır: **erişim belirleyicileri** (§2.3) ve **property**'ler (§2.1). Bunlar birlikte, "iç veri gizli, kontrollü giriş açık" prensibini uygular.

1.4.2 Sözdizimi

```
public class SinifAdi
{
    private tip _alanAdi; // gizli iç durum

    public tip AlanAdi // dışa açık property
    {
        get => _alanAdi;
        set { /* validation */ _alanAdi = value; }
    }

    public void EylemAdi() // davranış
}
```

```

    {
        // _alanAdi'na burada dokunulur
    }
}

```

1.4.3 Örnek 1.4.1 — Banka Hesabı

Bakiyenin negatife düşmesini kapsüller.

```

// Models/BankaHesabi.cs
public class BankaHesabi
{
    private decimal _bakiye;

    public decimal Bakiye
    {
        get => _bakiye;
        set
        {
            if (value >= 0)
                _bakiye = value;
        }
    }

    public void ParaYatir(decimal miktar)
    {
        if (miktar > 0)
            _bakiye += miktar;
    }

    public void ParaCek(decimal miktar)
    {
        if (miktar > 0 && miktar <= _bakiye)
            _bakiye -= miktar;
    }
}

// Kullanım
var h = new BankaHesabi();
h.ParaYatir(1000);
h.ParaCek(300);
Console.WriteLine(h.Bakiye);
h.Bakiye = -50; // set içinde reddedilir, değişmez
Console.WriteLine(h.Bakiye);

```

Çıktı:

```

700
700

```

_bakiye private olduğu için sınıfın dışından hesap._bakiye = -1000 yazmak mümkün değildir; tek yol Bakiye property'sidir ve orada koşul vardır.

1.4.4 Örnek 1.4.2 — Salt-Okunur Property

get var, set yok:

```

// Models/Urun.cs
public class Urun
{
    public string Ad { get; } // salt-okunur; sadece constructor'da atanır
    public decimal Fiyat { get; private set; }
}

```

```

public Urun(string ad, decimal fiyat)
{
    Ad = ad;
    Fiyat = fiyat;
}

public void Zamla(decimal oran)
{
    Fiyat *= (1 + oran);
}
}

```

Ad dışarıdan hiç değiştirilemez; Fiyat sınıf içinde Zamla metodu tarafından değiştirilir ama dışarıdan doğrudan yazılamaz. Bu, “durum değişikliği anlamlı bir işlem üzerinden geçer” tasarımıdır.

1.4.5 Erişim Belirleyicileri (Özet)

Detaylı tablo §2.3’tedir. Kapsülleme bağlamında en yaygın kullanılanlar:

- **private**: yalnızca aynı sınıfın içinden erişilir. İç durum için varsayılan.
- **public**: her yerden erişilir. Sınıfın dış API’si.
- **protected**: aynı sınıftan **veya** ondan türeyen alt sınıflardan erişilir.

Kapsülleme kuralı: **alan private, property public**. Doğrudan `public int x` yerine `private int _x; public int X { get; set; }`.

1.4.6 Yaygın Hatalar

Hatalı Kod	Sonuç	Neden	Düzeltilmiş
<code>public decimal _bakiye;</code>	Her yerden değiştirilir	Kapsülleme yok	<code>private decimal _bakiye;</code> + <code>property</code>
Property setter’ında koşul yok	Geçersiz değer atanır	Validation eksik	set içinde koşul veya <code>exception</code>
Field’a doğrudan setter dışından yazma	Property atlanır	Public alan	Sadece <code>property</code> üzerinden

1.4.7 Anti-Pattern ve İdiyomatik Kullanım

Anti-Pattern	İdiyomatik	Neden Daha İyi
Bütün alanlar <code>public</code>	<code>private</code> alan + <code>public</code> <code>property</code>	Değişiklikleri tek yerde kontrol
Property’de koşul yerine çağırın her yerde tekrar kontrol	Koşul setter’da tek noktada	DRY, hatasız
Aynı sınıfta 15+ setter	Değişikliği anlamlı bir metotla ifade	<code>ParaCek(miktar)</code> gibi niyet açık

1.4.8 Mini Sözlük

- **Field (alan)**: Sınıf içinde tanımlı değişken; genellikle `private`.
- **Property**: Dış dünyaya alan gibi görünen ama gerçekte metot çağırısı olan üye (`get/set`).
- **Setter validation**: set bloğunun içinde geçersiz değerleri reddetme mantığı.
- **Backing field**: Bir `property`’nin arkasındaki `private` alan (`_bakiye` gibi).
- **Immutable**: Oluşturulduktan sonra durumu değiştirilemeyen nesne (yalnızca `get`, `set` yok).

1.4.9 Bölüm Özeti

- Kapsülleme: iç veriyi gizle, kontrollü erişim ver.
- Kural: alan `private`, `property` `public`.

- Setter validation, hatalı durumları sınıfın sınırında yakalar.
- Salt-okunur property'lerle immutable sınıflar kurulur.
- Erişim belirleyicileri kapsüllemenin araçlarıdır (§2.3).

İlgili konular: §1.5 (Inheritance), §2.3 (Erişim belirleyiciler), §2.1 (Property sözdizimi).

1.5 Inheritance (Kalıtım)

1.5.1 Genel Bakış

Kalıtım, bir sınıfın (türeyen / derived / child) başka bir sınıfın (taban / base / parent) üyelerini otomatik olarak edinmesidir. Amaç, birden çok tipe ortak olan alanları ve davranışları tek bir yerde tanımlamak, alt tiplerde yalnızca farkı yazmaktır. C# tek kalıtımı destekler (bir sınıf en fazla bir sınıftan türeyebilir) ancak birden fazla interface uygulayabilir. Bütün sınıflar dolaylı olarak System.Object'ten türer; dolayısıyla her sınıfta ToString(), Equals(), GetHashCode() gibi üyeler hazır bulunur.

Bu prensip ASP.NET Core MVC projelerinde iki yerde belirgin biçimde kullanılır: **entity'ler için taban sınıf** (bkz. §4.20 BaseEntity) ve **controller'lar için taban sınıf** (BaseController). Her ikisi de ortak alanları (ID, IsDeleted, ortak DbContext, ortak dropdown metotları) her entity/controller'da tekrar yazmak yerine tek noktada toplar.

1.5.2 Sözdizimi

```
public class Taban
{
    public string OrtakAlan { get; set; }
    public void OrtakMetot() { }
}

public class Turemis : Taban           // ':' ile kalıtım
{
    public string OzelAlan { get; set; }
}
```

- class Turemis : Taban sözdizimi, Turemis sınıfının Taban'daki tüm public/protected üyeleri kalıtıgını ifade eder.
- private üyeler kalıtılır ancak alt sınıftan erişilemez.
- sealed anahtar kelimesiyle bir sınıf mühürlenerek türetilmesi engellenebilir.

1.5.3 Örnek 1.5.1 — Hayvan → Köpek

```
// Models/Hayvan.cs
public class Hayvan
{
    public string Ad { get; set; }
    public void Nefes()
    {
        Console.WriteLine($"{Ad} nefes alıyor.");
    }
}

public class Kopek : Hayvan
{
    public void Havla()
    {
        Console.WriteLine($"{Ad}: Hav hav!");
    }
}
```

```
// Kullanım
var kopek = new Kopek { Ad = "Karabaş" };
kopek.Nefes(); // Hayvan'dan kalıtıldı
kopek.Havla(); // Kopek'e özgü
```

Çıktı:

```
Karabaş nefes alıyor.
Karabaş: Hav hav!
```

Kopek sınıfı yalnızca Havla metodunu tanımladı; Ad property'si ve Nefes metodu Hayvan'dan geldi. Kod tekrarı yok.

1.5.4 Örnek 1.5.2 — Çalışan → Yazılımcı → Müdür

Zincirleme kalıtım. Her düzey bir öncekinin üstüne yeni alan/metot ekler.

```
// Models/Calisan.cs
public class Calisan
{
    public string Ad { get; set; }
    public decimal Maas { get; set; }

    public virtual void BilgiYazdir()
    {
        Console.WriteLine($"{Ad} - {Maas:C}");
    }
}

public class Yazilimci : Calisan
{
    public string ProgramlamaDili { get; set; }

    public override void BilgiYazdir()
    {
        base.BilgiYazdir();
        Console.WriteLine($" Dil: {ProgramlamaDili}");
    }
}

public class Mudur : Yazilimci
{
    public int EkipBoyut { get; set; }

    public override void BilgiYazdir()
    {
        base.BilgiYazdir();
        Console.WriteLine($" Ekip: {EkipBoyut} kişi");
    }
}

// Kullanım
var m = new Mudur { Ad = "Ayşe", Maas = 60000, ProgramlamaDili = "C#", EkipBoyut = 8 };
m.BilgiYazdir();
```

Çıktı:

```
Ayşe - ₺60.000,00
Dil: C#
Ekip: 8 kişi
```

base.BilgiYazdir() ile bir üst sınıfın metodu çağrılıyor; sonra alt sınıfın ek çıktısı ekleniyor. Bu, tipik kalıtım kompozisyonudur.

1.5.5 System.Object — Herşeyin Atası

C#’ta hiçbir taban belirtilmemişse sınıf otomatik olarak System.Object’ten türer. object şu üyelere sahiptir:

Üye	Görevi
ToString()	Nesnenin string temsili. Override edilebilir.
Equals(obj)	İki nesnenin eşitliğini karşılaştırır.
GetHashCode()	Hash tablolarında kullanılan sayısal özet.
GetType()	Runtime tip bilgisi (Type nesnesi) döner.

```
var u = new Urun { Ad = "Kalem", Fiyat = 15 };
Console.WriteLine(u.ToString()); // varsayılan: "Namespace.Urun"
Console.WriteLine(u.GetType().Name); // "Urun"
```

ToString’i override etmek, debug ve log için sınıfın anlamlı bir temsili üretir:

```
public class Urun
{
    public string Ad { get; set; }
    public decimal Fiyat { get; set; }

    public override string ToString() => $"{Ad} ({Fiyat:C})";
}
```

1.5.6 Tek Kalıtım + Çoklu Interface

C# yalnızca **tek sınıf** kalıtımına izin verir; ancak sınırsız sayıda interface uygulanabilir.

```
public class Kitap : Materyal, IKaydedilebilir, IYazdirilabilir
{
    // Materyal'den kalıtılan üyeler + iki interface'in imzaları
}
```

- Materyal: taban sınıf, kalıtım (: sonrası ilk isim).
- IKaydedilebilir, IYazdirilabilir: interface, virgülle eklenir.

Neden çoklu sınıf kalıtımı yok? “Elmas problemi” (aynı imzayı iki üst sınıftan farklı biçimde kalıtma) belirsizliği vardır. Interface’ler yalnız imza taşıdığı için bu belirsizlik oluşmaz.

1.5.7 Yaygın Hatalar

Hatalı Kod	Hata Mesajı / Sonuç	Neden	Düzeltilmiş
public class B : A1, A2 { } (A1, A2 sınıf)	CS1721: class 'B' cannot have multiple base classes	Tek kalıtım	Bir sınıf, birden çok interface
Taban sınıfın private alanına türetilenden erişim	CS0122: '_x' is inaccessible due to its protection level	private alt sınıfa açık değil	protected yap
sealed sınıftan türeme	CS0509: cannot derive from sealed type	Mühürlenmiş	sealed’i kaldır ya da başka taban seç
Base sınıfta parametresiz constructor yok, alt sınıf çağırıyor	CS7036: no argument given for required parameter	Constructor chaining eksik	: base(...) ile parametreleri geçir

1.5.8 Anti-Pattern ve İdiyomatik Kullanım

Anti-Pattern	İdiyomatik	Neden Daha İyi
Derin (5+ seviyeli) kalıtım hiyerarşisi	Kısa hiyerarşi + kompozisyon	Bakımı zorlaşır, LSP kırılır
"Kod paylaşımı" için tabana atma	Ortak davranış interface'e, ortak durum taban sınıfa	Anlam belirsiz olur
Türetilenin taban davranışını tamamen bozması	virtual/override ile kontrollü genişletme	Beklenti kırılmaz

1.5.9 Mini Sözlük

- **Base class / taban sınıf:** Kalıtımda üst sınıf.
- **Derived class / türetilmiş sınıf:** Alt sınıf.
- **virtual:** Taban sınıfta override edilebilir işaretlemesi.
- **override:** Alt sınıfta taban metodun yerini alma.
- **base anahtar kelimesi:** Alt sınıftan taban sınıfın üyesine erişim (`base.Metod()`).
- **sealed:** Türetilmesi engellenmiş sınıf.
- **Constructor chaining:** `: base(...)` ile taban constructor'a parametre geçirme.

1.5.10 Bölüm Özeti

- Kalıtım: ortak alan/davranışı tek yerde topla; alt tipte yalnız farkı yaz.
- Sözdizim: `class B : A { }`.
- Tek sınıf, sınırsız interface.
- `System.Object` her sınıfın örtük atası.
- `virtual/override/base` üçlüsü, taban davranışını genişletmenin standart yoludur.
- Aşırı derin hiyerarşiden kaçınılır; kompozisyon çoğu zaman daha esnektir.

İlgili konular: §1.6 (Polymorphism), §1.7 (Abstraction), §4.20 (BaseEntity), §2.2 (Interface).

1.6 Polymorphism (Çokbiçimlilik)

1.6.1 Genel Bakış

Çokbiçimlilik, bir üst tip referansı üzerinden çağrılan bir metodun, referansın gerçekte işaret ettiği alt tipin metoduna yönlendirilmesidir. Böylece aynı çağrı satırı, çalışma zamanında farklı davranışlar üretir. Bu, OOP'nin en güçlü yönlerinden biridir: yeni bir alt tip eklendiğinde, çağırıcı kod değişmeden yeni tip devreye girer.

C#'ta çokbiçimlilik iki mekanizma üzerinden gerçekleşir:

1. **Method Overriding** (`virtual + override`): Taban sınıftaki `virtual` bir metod, alt sınıfta `override` ile yeniden tanımlanır. Çalışma zamanında (runtime) doğru sürüm seçilir.
2. **Interface tabanlı çokbiçimlilik:** Bir interface referansı, o interface'i uygulayan farklı sınıfların örneklerini tutar; `.Metot()` çağrısı hangi sınıfın uyguladığı yönlendirilir.

1.6.2 Sözdizimi

```
public class Taban
{
    public virtual void Yap() { Console.WriteLine("Taban"); }
}

public class Alt : Taban
{
    public override void Yap() { Console.WriteLine("Alt"); }
}

// Referans tipi Taban, gerçek nesne Alt
Taban t = new Alt();
t.Yap(); // "Alt" - override çağrılır
```

- virtual: Taban sınıfta “bu metot alt sınıflarca özelleştirilebilir” işareti.
- override: Alt sınıfta “taban metodun yerini alıyorum” beyanı.
- new (method hiding): Farklı bir mekanizma; polimorfik davranış üretmez, sadece derleme zamanı tipiyle seçim yapar. Genellikle kaçınılır.

1.6.3 Örnek 1.6.1 — Şekil Polimorfizmi

Farklı geometrik şekillerin alanını hesaplayan tek bir döngü.

```
// Models/Sekil.cs
public abstract class Sekil
{
    public abstract double Alan();
}

public class Daire : Sekil
{
    public double Yaricap { get; set; }
    public override double Alan() => Math.PI * Yaricap * Yaricap;
}

public class Kare : Sekil
{
    public double Kenar { get; set; }
    public override double Alan() => Kenar * Kenar;
}

public class Ucgen : Sekil
{
    public double Taban { get; set; }
    public double Yukseklik { get; set; }
    public override double Alan() => 0.5 * Taban * Yukseklik;
}

// Kullanım
Sekil[] sekiller =
{
    new Daire { Yaricap = 5 },
    new Kare { Kenar = 4 },
    new Ucgen { Taban = 6, Yukseklik = 3 }
};

foreach (var s in sekiller)
    Console.WriteLine($"{s.GetType().Name}: {s.Alan():F2}");
```

Çıktı:

```
Daire: 78,54
Kare: 16,00
Ucgen: 9,00
```

Döngü Sekil tipini bilir; her elemanın gerçek tipi runtime’da belirlenir ve doğru Alan() metodu çağrılır. Yeni bir şekil (örn. Dikdortgen) eklemek için döngüyü değiştirmeye gerek yoktur.

1.6.4 Örnek 1.6.2 — Interface ile Çokbiçimlilik

```
// Models/I0deme.cs
public interface I0deme
{
    void Ode(decimal miktar);
}
```

```

public class KrediKarti : IOdeme
{
    public void Ode(decimal miktar) => Console.WriteLine($"Kart ile {miktar} ödendi");
}

public class Kripto : IOdeme
{
    public void Ode(decimal miktar) => Console.WriteLine($"Kripto ile {miktar} transfer edildi");
}

// Kullanım
IOdeme[] yontemler = { new KrediKarti(), new Kripto() };
foreach (var y in yontemler)
    y.Ode(500);

```

Çıktı:

```

Kart ile 500 ödendi
Kripto ile 500 transfer edildi

```

Kalıtım hiyerarşisi olmadan da çokbiçimlilik elde edildi; ortak “arayüz” (interface) yeterlidir.

1.6.5 is ve as Operatörleri

Runtime tipi kontrol etmek ve güvenli dönüşüm yapmak için:

```

Sekil s = new Daire { Yaricap = 3 };

```

```

if (s is Daire d1)
    Console.WriteLine($"Daire yarıçapı: {d1.Yaricap}");

// Alternatif: as, uyumsuzsa null döner
var d2 = s as Daire;
if (d2 != null)
    Console.WriteLine("Daire olarak dönüştürüldü.");

```

Çıktı:

```

Daire yarıçapı: 3
Daire olarak dönüştürüldü.

```

is operatörü hem tip kontrolü hem de pattern matching yaparak yeni değişkene bağlar (§2.6). as operatörü ise sadece güvenli cast’tır — başarısız olursa null döner.

1.6.6 Yaygın Hatalar

Hatalı Kod	Hata Mesajı / Sonuç	Neden	Düzeltilmiş
Taban sınıfta metot virtual değil, alt sınıfta override	CS0506: cannot override inherited member because it is not marked virtual	Taban virtual işareti eksik	Taban metoda virtual ekle
override yerine new kullanmak	Polimorfik davranış yok	Method hiding	override kullan
Taban t = new Alt(); (t as Alt).OzelMetot(); — Alt değilse crash	NullReferenceException as null döndü, kontrol yok		if (t is Alt a) a.OzelMetot();
Explicit cast (Alt)t başarısız	InvalidCastException	Runtime uyumsuzluk	Önce is ile kontrol et

1.6.7 Anti-Pattern ve İdiyomatik Kullanım

Anti-Pattern	İdiyomatik	Neden Daha İyi
Uzun if (x is Daire) ... else if (x is Kare) ... Her yerde GetType() == typeof(...) karşılaştırması	Virtual metot ile davranış üst tipe delegasyon is operatörü	Yeni tip eklerken kod değişmez Kalıtımı da göz önüne alır
Alt sınıfta new ile aynı imzalı metot yazmak	virtual/override	Polimorfik davranış

1.6.8 Mini Sözlük

- **Method overriding**: Taban virtual metodun alt sınıfta override ile yeniden yazılması.
- **Method hiding**: new anahtarıyla taban metodu gizleme; polimorfizm sağlamaz.
- **Dynamic dispatch**: Çalışma zamanında doğru metodun seçilme mekanizması.
- **Upcast**: Alt tip referansını üst tipe atamak (Sekil s = new Daire();), her zaman güvenli.
- **Downcast**: Üst tip referansını alt tipe indirmek; is/as ile güvenli hale getirilir.

1.6.9 Bölüm Özeti

- Çokbiçimlilik, aynı çağrının farklı türlerde farklı davranmasıdır.
- virtual + override ikilisi klasik yoldur.
- Interface üzerinden de polimorfizm kurulabilir; kalıtım hiyerarşisi gerekmez.
- is / as operatörleri runtime tip kontrolünün güvenli yollarıdır.
- Yeni tip ekleyince çağırıcı kodu değiştirmemek, iyi polimorfizm işaretidir.

İlgili konular: §1.7 (Abstraction), §1.8.3 (OCP), §2.2 (Interface).

1.7 Abstraction (Soyutlama)

1.7.1 Genel Bakış

Soyutlama, bir tipin **ne yaptığını** ifade edip **nasıl yaptığını** gizlemektir. Bu, kullanıcının (bir sınıfı çağırıcı kodun) sadece anlamlı imzalara — metot adları ve parametreleri — bağımlı kalmasını, uygulama detayına bağımlı kalmamasını sağlar. Kullanıcı Console.WriteLine("Merhaba") çağırırken, arka planda hangi buffer'a yazıldığını, hangi encoding'ın kullanıldığını, hangi işletim sistemi çağrılarının yapıldığını bilmez ve bilmesine gerek yoktur. Bu bir soyutlama örneğidir.

C#'ta soyutlama iki yapıyla ifade edilir: **abstract class** (§1.7.3) ve **interface** (§2.2). İkisinin ayrımı şu şekildedir:

- **abstract class**: Bir "kısmen tamamlanmış" taban sınıftır. Bazı üyeleri uygulanmış, bazıları abstract (imzası var, gövdesi yok) olabilir. Ortak durum (property, field) taşıyabilir.
- **interface**: Sadece imzalar (C# 8'e kadar). Ortak durum taşımaz (static üyeler dışında). Bir sınıf birden fazla interface uygulayabilir.

1.7.2 Sözdizimi

```
public abstract class TabanAdi
{
    public abstract void SoyutMetot();           // gövdesi yok, alt sınıf zorunlu tanımlar
    public virtual void OzellestirilebilirMetot() { /* varsayılan */ }
    public void OrtakMetot() { /* alt sınıflarca değiştirilemez */ }
}

public class AltAdi : TabanAdi
{
    public override void SoyutMetot() { /* uygulama */ }
}
```

- abstract işaretli sınıf new ile doğrudan örneklenemez.
- abstract işaretli metot gövdesizdir; alt sınıf override ile uygulamak zorundadır.

1.7.3 Örnek 1.7.1 — Abstract Sınıf: Şekil

```
// Models/Sekil.cs
public abstract class Sekil
{
    public string Renk { get; set; } = "Siyah"; // ortak durum
    public abstract double Alan(); // her alt sınıf uygular
    public virtual void Ciz() => Console.WriteLine($"{Renk} bir şekil çizildi.");
}

public class Daire : Sekil
{
    public double Yaricap { get; set; }
    public override double Alan() => Math.PI * Yaricap * Yaricap;
    public override void Ciz() => Console.WriteLine($"{Renk} daire ({Yaricap} yarıçap)");
}

// var s = new Sekil(); // HATA: abstract sınıf örneklenemez
var d = new Daire { Yaricap = 3, Renk = "Kırmızı" };
d.Ciz();
Console.WriteLine($"Alan: {d.Alan():F2}");
```

Çıktı:

Kırmızı daire (3 yarıçap)
Alan: 28,27

Sekil, kendi başına anlam taşımaz; sadece “her şeklin bir rengi ve bir alan hesaplama biçimi vardır” tarafını belirtir. Alan() soyut, çünkü daire ile karenin alan formülü farklıdır. Ciz() varsayılan davranışa sahip, gerekirse override edilebilir.

1.7.4 Örnek 1.7.2 — Interface: IKaydedilebilir + IYazdirilabilir

Farklı yeteneklerin ayrı ayrı ifade edilmesi:

```
// Models/IKaydedilebilir.cs
public interface IKaydedilebilir
{
    void Kaydet();
}

public interface IYazdirilabilir
{
    void Yazdir();
}

public class Kitap : IKaydedilebilir, IYazdirilabilir
{
    public string Baslik { get; set; }

    public void Kaydet() => Console.WriteLine($"'{Baslik}' veritabanına kaydedildi.");
    public void Yazdir() => Console.WriteLine($"'{Baslik}' yazıcıya gönderildi.");
}

// Kullanım
IKaydedilebilir k = new Kitap { Baslik = "OOP Rehberi" };
k.Kaydet();

IYazdirilabilir y = new Kitap { Baslik = "MVC Notları" };
y.Yazdir();
```

Çıktı:

'OOP Rehberi' veritabanına kaydedildi.
'MVC Notları' yazıcıya gönderildi.

Kitap her iki “yetenek”i de taşır. Bir başka sınıf (Resim) sadece `IYazdirilabilir` olabilir; kayıt yeteneği zorlanmaz. Bu, tekli büyük sınıflar yerine küçük, odaklı sözleşmeler tanımlamayı sağlar (§1.8.5 ISP).

1.7.5 Somut Örnek: `Console.WriteLine`

Günlük hayatta hep kullanılan `Console.WriteLine("Merhaba")` çağrısı, mükemmel bir soyutlama örneğidir:

- Çağıran kod bilmiyor: karakterlerin nasıl encode edildiği (UTF-8?), hangi işletim sistemi çağrısının yapıldığı (Windows `WriteConsole` mu, POSIX `write` mi), `stdout`’un yönlendirilip yönlendirilmediği, buffer boyutunun ne olduğu.
- Çağıran kod biliyor: metot adı `WriteLine`, parametre `string`, davranış “verilen metni satır sonu ile yazdırır”.

İşte soyutlama tam olarak bu ayrımdır: kullanıcı için yeterince güçlü, anlamlı bir sözleşme; detaylar sağlayıcının sorunu.

1.7.6 Abstract Class ile Interface Ne Zaman?

Durum	Tercih	Gerekçe
Ortak alan/durum paylaşılacak	Abstract class	Interface (C# 7 ve öncesinde) durum tutamaz
Bir tipe birden fazla yetenek eklenecek	Interface	Tek sınıf, çoklu interface
“Kısmen tamamlanmış” ortak davranış var	Abstract class	Varsayılan uygulamalar taşınabilir
Sadece imza sözleşmesi gerekli	Interface	En hafif seçenek
DI ile inject edilecek (mock için)	Interface	Test yazımı kolay (§1.8.6 DIP)

Çoğu modern C# projesi, katman sınırlarında (servis, repository) interface kullanır; ortak durum barındıran taban sınıflar için abstract class kullanır. §4.20’deki `BaseEntity` bu ayrımın somut örneğidir.

1.7.7 Yaygın Hatalar

Hatalı Kod	Hata Mesajı / Sonuç	Neden	Düzeltilmiş
<code>var s = new Sekil();</code> (<code>Sekil</code> abstract)	CS0144: cannot create an instance of the abstract type	Abstract örneklenemez	Türetilmiş sınıfı örnekle
Abstract metot gövdeyle yazılmış	CS0500: '...' cannot declare a body because it is marked abstract	Abstract gövdesiz olur	Ya gövde sil ya abstract kaldır
Alt sınıf abstract metodu override etmiyor	CS0534: does not implement inherited abstract member	Zorunlu üye tamamlanmadı	override ile uygula
Interface’e default { get; set; } yazmak (C# 7 ve öncesi)	Uyarı / eski derleyicide hata	Interface durum tutamaz	Property’yi implement eden sınıfa taşı

1.7.8 Anti-Pattern ve İdiyomatik Kullanım

Anti-Pattern	İdiyomatik	Neden Daha İyi
Her sınıfa "generic" olacak diye object parametresi	Interface veya abstract class ile net sözleşme	Tip güvenliği
Tek büyük "Manager" sınıfı	Küçük, tek amaçlı sınıflar + interface	Test ve bakım kolay
Abstract class yerine tam soyut interface, ama ortak durum kopyalanıyor	Abstract class'a taşı	DRY

1.7.9 Mini Sözlük

- **Abstract class:** Kısmen soyut sınıf; doğrudan örneklenemez, bazı üyeleri abstract olabilir.
- **Abstract member:** İmzası var, gövdesi yok; alt sınıf uygulamak zorundadır.
- **Interface:** Sadece imzalardan oluşan sözleşme.
- **Concrete class / somut sınıf:** Tüm üyeleri uygulanmış, doğrudan örneklenabilir sınıf.
- **Contract / sözleşme:** Bir tip ve onu kullanan kod arasındaki imzalar üzerinden kurulan anlaşma.

1.7.10 Bölüm Özeti

- Soyutlama: **ne** görünsün, **nasıl** gizlensin.
- Abstract class = kısmen tamamlanmış taban; interface = sadece imza.
- Console.WriteLine günlük bir soyutlama örneğidir.
- Ortak durum taşınacaksa abstract class; sadece yetenek eklenecekse interface.
- Modern projelerde katman sınırlarında interface (test edilebilirlik + DI için) idiomatiktir.

İlgili konular: §1.5 (Inheritance), §2.2 (Interface ve abstract class detayı), §1.8.6 (DIP).

1.8 SOLID Prensipleri

1.8.1 Genel Bakış

SOLID, Robert C. Martin tarafından popülerleştirilen beş nesne yönelimli tasarım prensibinin kısaltmasıdır:

Harf	Prensip	Türkçe
S	Single Responsibility Principle	Tek sorumluluk prensibi
O	Open/Closed Principle	Açık/kapalı prensibi
L	Liskov Substitution Principle	Liskov yer değiştirme prensibi
I	Interface Segregation Principle	Arayüz ayrımı prensibi
D	Dependency Inversion Principle	Bağımlılıkların tersine çevrilmesi

Bu prensipler kural değil, rehberdir. Amaç, zaman içinde büyüyen bir kod tabanında **değişikliğin maliyetini** düşük tutmak, hata izini kolaylaştırmak ve yeni davranış eklerken var olanı bozmamaktır. Her prensip, sınıf ölçeğinde belirli bir kokuya (code smell) karşı bir yanıt sunar; hepsi birlikte "sağlıklı OOP tasarımı" olarak paketlenir.

Aşağıdaki alt bölümlerde her prensip için: sorun (anti-pattern) → çözüm (idiomatic) → gerekçe verilmiştir.

1.8.2 Single Responsibility Principle (SRP)

Prensip: Bir sınıfın değişmek için yalnızca **tek bir sebebi** olmalıdır. Robert C. Martin'in ifadesiyle "bir sınıfın değişmek için bir ve sadece bir nedeni olmalıdır".

Sınıfın “sorumluluk” sınırı, kime hizmet ettiğidir; kullanıcı yönetimi, raporlama, e-posta gönderimi farklı aktörlere hizmet eden farklı sorumluluklardır — dolayısıyla farklı sınıflara ait olmalıdır.

Anti-pattern: Bir sınıfın üç işi birden yapması.

```
// Models/Ogrenci.cs – anti-pattern
public class Ogrenci
{
    public void KayitOl() { /* DB'ye kaydet */ }
    public void NotHesapla() { /* iş kuralı uygula */ }
    public void EmailGonder() { /* SMTP çağır */ }
}
```

Sınıf üç ayrı sebepten değişebilir: DB şeması değişince, not politikası değişince, SMTP sağlayıcısı değişince. Bu, bakımı kırılgan kılar.

Idiomatic: Her sorumluluğu ayrı sınıfa taşı.

```
public class OgrenciKayitServisi { public void KayitOl(Ogrenci o) { } }
public class NotHesaplamaServisi { public double Hesapla(Ogrenci o) => 0; }
public class EmailServisi { public void Gonder(string kime, string konu) { } }
```

Artık her sınıf tek bir aktöre hizmet eder; bir sorumluluk değişince yalnızca ilgili sınıf değişir. §4.20'deki DoktorService, RandevuService bu prensibin doğrudan uygulamasıdır.

1.8.3 Open/Closed Principle (OCP)

Prensip: Sınıflar **genişletmeye açık, değişikliğe kapalı** olmalıdır. Yeni davranış eklerken var olan sınıfa dokunmadan yapılabilmelidir. Bu genellikle soyutlama (interface / abstract class) ile başarılır.

Anti-pattern: Yeni indirim türü eklerken var olan Fatura sınıfını her seferinde değiştirmek.

```
// Models/Fatura.cs – anti-pattern
public class Fatura
{
    public decimal Hesapla(decimal tutar, string tur)
    {
        if (tur == "Ogrenci") return tutar * 0.80m;
        if (tur == "Emekli") return tutar * 0.85m;
        if (tur == "Personel") return tutar * 0.90m;
        return tutar;
    }
}
```

Her yeni indirim türü, bu sınıfın gövdesini genişletmeyi ve testleri yeniden çalıştırmayı gerektirir.

Idiomatic: Strategy pattern — indirim bir soyutlama arkasında.

```
// Strategies/IIndirimStratejisi.cs
public interface IIndirimStratejisi
{
    decimal Uygula(decimal tutar);
}

public class OgrenciIndirimi : IIndirimStratejisi { public decimal Uygula(decimal t) => t * 0.80m; }
public class EmekliIndirimi : IIndirimStratejisi { public decimal Uygula(decimal t) => t * 0.85m; }
public class PersonelIndirimi : IIndirimStratejisi { public decimal Uygula(decimal t) => t * 0.90m; }

// Fatura artık indirim tipini bilmiyor, sadece "bir strateji uygula" diyor
public class Fatura
{
    private readonly IIndirimStratejisi _strateji;
    public Fatura(IIndirimStratejisi strateji) => _strateji = strateji;
    public decimal Hesapla(decimal tutar) => _strateji.Uygula(tutar);
}
```

```
// Kullanım
var f1 = new Fatura(new OgrenciIndirimi());
Console.WriteLine(f1.Hesapla(100)); // 80
```

Çıktı:

80

Yeni bir “Askeri İndirimi” eklemek için Faturaya dokunulmaz; yalnızca yeni bir sınıf yazılır. Bu, yatay büyümedir; dikey (mevcut kodun içine ekleme) değildir.

1.8.4 Liskov Substitution Principle (LSP)

Prensip: Alt sınıf nesnesi, üst sınıf nesnesinin yerine kullanıldığında programın davranışı bozulmamalıdır. Barbara Liskov’un 1987’de formüle ettiği bu prensip, kalıtımın “sözünde durmasını” ister.

Anti-pattern: Kuş sınıfının tüm alt tiplerinin uçacağını varsaymak, ama penguen uçamaz.

```
public class Kus
{
    public virtual void Uc() => Console.WriteLine("Uçuyor");
}

public class Penguen : Kus
{
    public override void Uc() => throw new NotImplementedException("Penguen uçamaz!");
}

// Bu döngü, Penguen ile bozulur
Kus[] kuslar = { new Kus(), new Penguen() };
foreach (var k in kuslar) k.Uc(); // penguen exception fırlatır
```

Penguen “bir Kuş’tur” mirasını üstlense de, Uc() sözleşmesini yerine getiremez. LSP kırılmıştır.

Idiomatic: Yeteneği ayrı bir interface’e taşı.

```
public interface IUcabilir { void Uc(); }

public class Kus { public string Ad { get; set; } }
public class Serçe : Kus, IUcabilir { public void Uc() => Console.WriteLine("Serçe uçuyor"); }
public class Penguen : Kus { /* uçmaz, IUcabilir'e sahip değil */ }

IUcabilir[] ucanlar = { new Serçe() }; // penguen zaten burada yer alamaz
foreach (var u in ucanlar) u.Uc();
```

Çıktı:

Serçe uçuyor

Artık IUcabilir[] dizisi güvenle iter edilebilir; her elemanın Uc() sözünü tutacağı derleme zamanı ile garanti altındadır.

1.8.5 Interface Segregation Principle (ISP)

Prensip: Bir sınıf, kullanmayacağı metotları uygulamak zorunda bırakılmamalıdır. Büyük “her şeyi kapsayan” interface’ler yerine, küçük ve odaklı interface’ler tercih edilir.

Anti-pattern: Tek büyük interface.

```
public interface ICalisan
{
    void Calis();
    void Yemek();
    void UyuMoralden();
}
```

```
public class Robot : ICalisan
{
    public void Calis() { }
    public void Yemek() => throw new NotSupportedException(); // robot yemek yemez
    public void UyuMoralDen() => throw new NotSupportedException(); // robot uyumaz
}
```

Robot, iki metodu boşuna uygulamak zorunda kalıyor.

Idiomatic: Küçük interface'ler.

```
public interface ICalisabilen { void Calis(); }
public interface IYiyabilen { void Yemek(); }
public interface IUyuyabilen { void Uyu(); }

public class Robot : ICalisabilen { public void Calis() { } }
public class Insan : ICalisabilen, IYiyabilen, IUyuyabilen
{
    public void Calis() { } public void Yemek() { } public void Uyu() { }
}
```

Her sınıf, sadece gerçekten uyguladığı yetenekleri deklare eder. Robotu IYiyabilen gibi bir yerde kullanmaya kalkmak derleme zamanı hatasıyla önlenir.

1.8.6 Dependency Inversion Principle (DIP)

Prensip: Üst seviye modüller alt seviye modüllere bağımlı olmamalıdır; ikisi de soyutlamalara (interface) bağımlı olmalıdır. Ayrıca soyutlamalar detaylara değil, detaylar soyutlamalara bağımlı olmalıdır.

Pratikte bu, “somut sınıfları new ile üretmek” yerine “interface’e ihtiyaç duyduğunu belirtmek ve dışardan gelmesini beklemek” anlamına gelir. ASP.NET Core’un yerleşik Dependency Injection container’ı (§4.16) bu prensibin motorudur.

Anti-pattern: Doğrudan somut bağımlılık.

```
public class SiparisServisi
{
    private readonly SqlVeritabani _db = new SqlVeritabani(); // sabitlendi

    public void KaydetSiparis(Siparis s) => _db.Ekle(s);
}
```

SiparisServisi artık SqlVeritabani’na kilitli; test için sahte bir DB koyulamaz, MongoDB’ye geçilemez.

Idiomatic: Soyutlamaya bağımlı ol, uygulama dışarıdan gelsin.

```
public interface IVeritabani
{
    void Ekle(object nesne);
}

public class SqlVeritabani : IVeritabani { public void Ekle(object n) { /* SQL */ } }
public class MongoVeritabani : IVeritabani { public void Ekle(object n) { /* Mongo */ } }

public class SiparisServisi
{
    private readonly IVeritabani _db;
    public SiparisServisi(IVeritabani db) => _db = db;

    public void KaydetSiparis(Siparis s) => _db.Ekle(s);
}

// DI container'da (Program.cs)
```

```
builder.Services.AddScoped<IVeritabani, SqlVeritabani>();
builder.Services.AddScoped<SiparisServisi>();
```

Artık SiparisServisi “hangi DB” olduğunu bilmiyor; yalnız “bir DB var” biliyor. Test ortamında IVeritabani’yi sahte (mock) bir sınıfla değiştirmek serbesttir.

1.8.7 Prensiplerin Etkileşimi

Bu beş prensip birbirinden kopuk değildir; birlikte çalışırlar:

- **SRP** sınıf sayısını artırır.
- **OCP + LSP + ISP** yeni sınıfları güvenle bağlayabilmenin altyapısını kurar.
- **DIP** bu sınıfları birbirine soyutlamalarla, gevşek biçimde bağlar.

Sonuç: her sorumluluk kendi yerinde, her genişleme yeni sınıfla, her bağımlılık container üzerinden. ASP.NET Core’un mimari bakış açısı zaten bu prensipler etrafında şekillenmiştir; bir Web MVC projesinde Controller’ın Service’e, Service’in DbContext’e bağımlı olması (hepsi Constructor Injection ile), pratikte SOLID’in canlı halidir.

1.8.8 Yaygın Hatalar

Hatalı Kod	Sonuç	Neden	Düzeltilmiş
Constructor’da new ile bağımlılık üretmek	Test edilemez, sıkı bağıllık	DIP kırık	Interface parametresi + DI container
Alt sınıfın taban metodun sözleşmesini bozması (exception fırlatma)	Beklenmeyen crash’ler	LSP kırık	Yeteneği ayrı interface’e taşı
Interface’e 15+ metod koyma	Uygulayan sınıflar NotSupported fırlatır	ISP kırık	Küçük interface’lere böl
Bir sınıfta hem persistence hem business hem UI	Değişiklik dalga yayar	SRP kırık	Katmanlara ayır

1.8.9 Mini Sözlük

- **Coupling / bağıllık**: Bir modülün başka bir modüle olan bağımlılığı. Az bağıllık iyidir.
- **Cohesion / uyum**: Bir modül içindeki üyelerin ne kadar aynı amaca hizmet ettiği. Yüksek uyum iyidir.
- **Strategy pattern**: Farklı algoritmaları interface arkasında değiştirilebilir kılan tasarım deseni.
- **Mock**: Test için gerçek bir bağımlılığın yerine kullanılan sahte uygulama.
- **DI container**: Bağımlılıkları kayıt eden ve talep edildikçe otomatik enjekte eden çatı bileşeni.

1.8.10 Bölüm Özeti

- **SRP**: Bir sınıf, bir değişim sebebi.
- **OCP**: Genişlet, değiştirme.
- **LSP**: Alt sınıf, üstün sözünü tutar.
- **ISP**: Küçük, odaklı interface’ler.
- **DIP**: Somut değil, soyut bağımlılık.
- Beş prensip birlikte, kalıcı bakım kolaylığı doğurur; ASP.NET Core’un DI’i bunun altyapısıdır.

İlgili konular: §4.16 (DI), §4.20 (BaseService), §5.2 (HastaneMVC servisleri).

Bölüm II — C# Dili

C# 12, .NET 8 üzerinde çalışan, tam nesne yönelimli, statik tipli bir dildir. Bu bölüm dilin ASP.NET Core MVC projelerinde en sık kullanılan yapılarını ele alır: sınıflar, tip sistemi, kontrol akışı, kolek-

siyonlar, string işlemleri, metotlar ve modern C# özellikleri (pattern matching, tuple, top-level statements). Amaç, sonraki bölümlerde (EF Core, MVC) karşılaşılan tüm C# sözdiziminin bir başvuru noktası olmaktır.

2.1 Sınıf, Nesne, Constructor, Property

2.1.1 Genel Bakış

C#'ta bir **sınıf** (class), nesnelerin nasıl üretileceğini belirleyen şablondur. Sınıf bir plan gibidir: hangi alanlara (durum) ve hangi metotlara (davranış) sahip olacağı burada tanımlanır. Bu plandan new operatörü ile üretilen somut örneğe **nesne** (instance / object) denir. Bir sınıftan sınırsız sayıda nesne üretilebilir; her nesne kendi durumunu tutar.

Sınıflar iki türlü üye taşır:

- **Field (alan):** Sınıf içinde tanımlı değişken; genellikle private.
- **Property:** Dış dünyaya alan gibi görünen ama gerçekte metot çağrısı olan üye. get ve set accessor'ları vardır.

Constructor, nesne oluşturulurken otomatik çalışan ve başlangıç değerlerini ayarlayan özel bir metottur; adı sınıf adıyla aynıdır ve dönüş tipi belirtilmez.

2.1.2 Sözdizimi

```
public class SinifAdi
{
    // Field
    private tip _alanAdi;

    // Property (otomatik)
    public tip PropertyAdi { get; set; }

    // Constructor (parametresiz)
    public SinifAdi() { }

    // Constructor (parametrelili)
    public SinifAdi(tip parametre)
    {
        _alanAdi = parametre;
    }

    // Metot
    public donusTipi MetotAdi(tip parametre)
    {
        return sonuc;
    }
}

// Nesne üretimi
var nesne = new SinifAdi(deger);
```

Nesne üretimi (Katalog B'deki "new Operatörü İle Nesne Üretimi" slaytı) şu adımlarla gerçekleşir: (a) Heap'te sınıfın alan boyutu kadar yer ayrılır, (b) constructor çağrılır, (c) constructor tamamlanınca nesnenin adresi geriye döner ve new çağrısı bu adresi bir referans olarak (stack'teki değişkene) yazar.

2.1.3 Property Türleri

Otomatik property (auto-property):

```
public string Ad { get; set; }           // en yaygın
public string Ad { get; }               // salt-okunur; sadece constructor'da atanır
public string Ad { get; private set; } // dışarıya salt-okunur, içeriye yazılabilir
```

Hesaplanmış property (expression-bodied):

```
public class Dikdortgen
{
    public double Genislik { get; set; }
    public double Yukseklik { get; set; }
    public double Alan => Genislik * Yukseklik; // sadece get, formül ile
}
```

Backing field'lı property:

```
private decimal _bakiye;
public decimal Bakiye
{
    get => _bakiye;
    set
    {
        if (value >= 0) _bakiye = value;
        else throw new ArgumentException("Bakiye negatif olamaz.");
    }
}
```

2.1.4 Örnek 2.1.1 — Araba Sınıfı (Auto-property + Metot)

```
// Models/Araba.cs
public class Araba
{
    public string Marka { get; set; }
    public string Model { get; set; }
    public int Yil { get; set; }
    public int Hiz { get; private set; } // sadece sınıf içinden değiştirilir

    public void Hizlan(int miktar)
    {
        Hiz += miktar;
        Console.WriteLine($"{Marka} {Model} – hız: {Hiz} km/h");
    }
}

// Kullanım
var a = new Araba { Marka = "Ford", Model = "Focus", Yil = 2022 };
a.Hizlan(30);
a.Hizlan(20);
```

Çıktı:

```
Ford Focus – hız: 30 km/h
Ford Focus – hız: 50 km/h
```

Marka, Model, Yil object initializer sözdizimi ({ ... }) ile atandı; bu, parametresiz constructor ve public setter varsa çalışan kısayoldur. Hiz'in setter'ı private olduğu için dışarıdan a.Hiz = 100 yazmak derleme hatası verir.

2.1.5 Örnek 2.1.2 — Constructor ile Zorunlu Alanlar

Constructor'sız bir sınıfta tüm alanlar isteğe bağlıdır; constructor eklenerek bazı alanlar **zorunlu** kılınır.

```
// Models/Hesap.cs
public class Hesap
{
    public string HesapNo { get; }
    public string Sahip { get; }
```

```

public decimal Bakiye { get; private set; }

public Hesap(string hesapNo, string sahip, decimal baslangicBakiye = 0)
{
    HesapNo = hesapNo;
    Sahip = sahip;
    Bakiye = baslangicBakiye;
}

public void ParaYatir(decimal miktar)
{
    if (miktar > 0) Bakiye += miktar;
}
}

var h = new Hesap("TR001", "Ali Yılmaz", 1500);
h.ParaYatir(500);
Console.WriteLine($"{h.HesapNo} - {h.Sahip} - {h.Bakiye:C}");

```

Çıktı:

```
TR001 - Ali Yılmaz - ₺2.000,00
```

Constructor parametreleri zorunludur; new Hesap() demek derleme hatası üretir çünkü parametresiz constructor tanımlı değildir. baslangicBakiye'ye varsayılan değer verilerek son parametre isteğe bağlı kılınmıştır.

2.1.6 Örnek 2.1.3 — Constructor Overload

Aynı sınıfta farklı imzalarla birden çok constructor tanımlanabilir. this(...) ile bir constructor başka bir constructor'a delegasyon yapabilir (constructor chaining).

```

public class Urun
{
    public string Ad { get; }
    public decimal Fiyat { get; }
    public int Stok { get; }

    public Urun(string ad) : this(ad, 0, 0) { }
    public Urun(string ad, decimal fiyat) : this(ad, fiyat, 0) { }
    public Urun(string ad, decimal fiyat, int stok)
    {
        Ad = ad;
        Fiyat = fiyat;
        Stok = stok;
    }
}

var u1 = new Urun("Kalem");
var u2 = new Urun("Defter", 25);
var u3 = new Urun("Silgi", 5, 100);

```

Üç overload da nihayetinde üçüncü (tam parametrelili) constructor'u çağırır; başlatma mantığı tek noktada toplanır.

2.1.7 Nesnenin Bellek Yerleşimi

Katalog B'deki "RAM'in Yapısı" diyagramına göre:

- **Stack:** Değer tipli değişkenler (int, bool, struct) ve referans değişkenlerinin kendisi burada tutulur.
- **Heap:** new ile üretilen nesneler burada yer alır.

```
var a = new Araba();
```

Bellek görünümü:

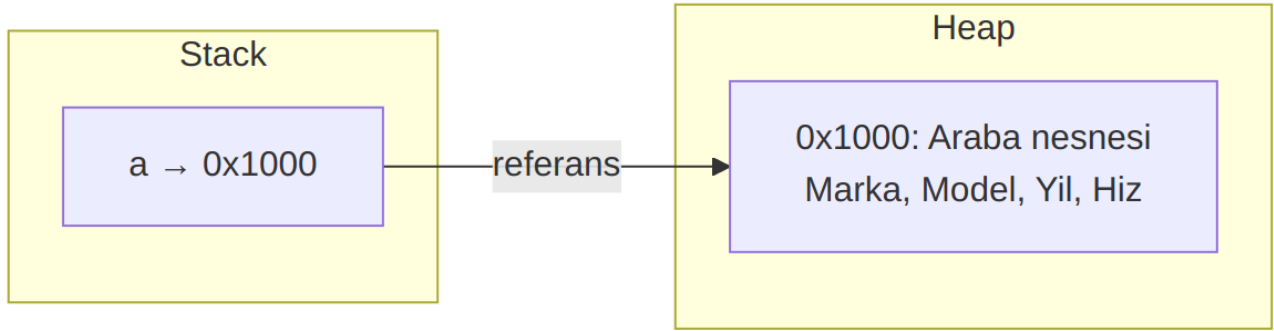


Figure 1: Diyagram 1

Bu ayrımın pratik sonuçları: iki referans aynı nesneye işaret ediyorsa (`var b = a;`), birinden değişiklik diğ erinde görülür (bkz. §2.5.8 Shallow Copy).

2.1.8 Yaygın Hatalar

Hatalı Kod	Hata Mesajı	Neden	Düzeltilmiş
<code>public Hesap() { } yok, new Hesap() çağ ırısı</code>	CS7036: There is no argument given that corresponds to the required parameter	Parametrelili constructor tanımlanınca parametresiz otomatik gelmez	Parametresiz constructor ekle veya parametrelili çağ ır
Property'ye sınıf dış ından atama, setter private	CS0272: The property or indexer cannot be used in this context because the set accessor is inaccessible	Setter private	Setter'ı public yap veya değ iş ikliğı metot üzerinden
Get-only property'ye setter dış ında atama	CS0200: Property or indexer '...' cannot be assigned to -- it is read only	Get-only sadece constructor'da atanır	Ya constructor'a taşı ya private set yap
<code>class A { A(); } — sadece imza</code>	CS0501: '...' must declare a body	Constructor gövdesiz olamaz	{ } ekle

2.1.9 Anti-Pattern ve İdiyomatik Kullanım

Anti-Pattern	İdiyomatik	Neden Daha İyi
Tüm alanlar public field	private field + public property	Kapsülleme, ileride validation eklenebilir
Constructor içinde <code>throw new Exception("...")</code> genel	<code>throw new ArgumentException(nameof(param))</code>	Belirli exception tipi, IntelliSense
Aynı iş i yapan 5 parametrelili constructor tekrarı	: <code>this(...)</code> ile chaining	DRY
Object initializer için <code>new SinifAdi() { ... }</code> yazımı ama constructor zorunlu parametre bekliyor	Ya constructor'ı gevş et, ya initializer yerine parametre kullan	Karışıklığı önler

2.1.10 Mini Sözlük

- **Sınıf (class):** Nesnelerin nasıl üretileceğini belirleyen şablon.
- **Nesne (object / instance):** Bir sınıftan new ile üretilen somut örnek.
- **Field:** Sınıf içinde tanımlı değişken; genellikle private.
- **Property:** Dış dünyaya alan gibi görünen ama arka planda get/set metotları olan üye.
- **Constructor:** Nesne üretilirken otomatik çalışan, adı sınıf adıyla aynı olan özel metot.
- **Object initializer:** new Sinif { Prop = deger } sözdizimi; setter'lar üzerinden atama yapar.
- **Constructor chaining:** : this(...) ile bir constructor'un başka bir constructor'ı çağırması.

2.1.11 Bölüm Özeti

- Sınıf şablon, nesne örnek. new ile üretilir.
- Property = akıllı alan; get/set ile kontrollü erişim.
- Auto-property: { get; set; }; hesaplanmış: => ifade; salt-okunur: { get; }.
- Constructor, nesnenin ilk durumunu ayarlar; parametrelili varsa varsayılan parametresiz gelmez.
- Object initializer, setter tabanlı hızlı atama sağlar.
- Nesne heap'te, referansı stack'te durur.

İlgili konular: §1.4 (Encapsulation), §2.3 (Erişim belirleyiciler), §2.5.8 (Shallow copy), §3.6 (DbSet: entity'ler POCO sınıflardır).

2.2 Interface ve Abstract Class

2.2.1 Genel Bakış

Bir **interface**, sadece imzaları (metot adı, parametreler, dönüş tipi) tanımlayan bir sözleşmedir. Uygulama detayı taşımaz. Bir sınıf birden fazla interface uygulayabilir; bu durumda sözleşmedeki tüm üyeleri sağlamak zorundadır. Interface'ler, "bir tipin ne yapabileceğini" (yeteneklerini) tanımlar; kalıtım hiyerarşisinden bağımsızdırlar.

Bir **abstract class**, kalıtım hiyerarşisinin tepesinde duran ve doğrudan örneklenemeyen sınıftır. Bazı üyeleri tam uygulanmış (concrete), bazıları imzasız (abstract) olabilir. Alt sınıflar, abstract üyeleri override ile uygulamak zorundadır.

2.2.2 Interface Sözdizimi

```
// Konvansiyon: interface adı 'I' harfiyle başlar
public interface IArayuz
{
    // Metot imzaları (gövde yok)
    void Metot1();
    int Metot2(string parametre);

    // Property imzaları (get/set)
    string Property1 { get; set; }
    int Property2 { get; } // sadece get
}
```

- Interface üyeleri örtük olarak public ve abstract'tır.
- Alan (field) tutamaz (C# 7'ye kadar); C# 8+ ile default implementation'lar eklenmiş olsa da geleneksel kullanımda sadece imzalar bulunur.

2.2.3 Örnek 2.2.1 — IHesap Sözleşmesi

Bir banka hesabının uyacağı yetenekler:

```
// Models/IHesap.cs
public interface IHesap
{
    string HesapNo { get; }
}
```

```

    decimal Bakiye { get; }

    void ParaYatir(decimal miktar);
    bool ParaCek(decimal miktar);
}

// Models/VadesizHesap.cs
public class VadesizHesap : IHesap
{
    public string HesapNo { get; }
    public decimal Bakiye { get; private set; }

    public VadesizHesap(string hesapNo, decimal baslangic = 0)
    {
        HesapNo = hesapNo;
        Bakiye = baslangic;
    }

    public void ParaYatir(decimal miktar)
    {
        if (miktar > 0) Bakiye += miktar;
    }

    public bool ParaCek(decimal miktar)
    {
        if (miktar <= 0 || miktar > Bakiye) return false;
        Bakiye -= miktar;
        return true;
    }
}

// Kullanım – referans interface tipi
IHesap h = new VadesizHesap("TR001", 1000);
h.ParaYatir(500);
bool basarili = h.ParaCek(200);
Console.WriteLine($"{h.HesapNo}: {h.Bakiye:C} – çekim: {basarili}");

```

Çıktı:

TR001: ₺1.300,00 – çekim: True

Referansın tipi IHesap; bu, sınıfın yalnızca sözleşmede tanımlı üyelerine erişim verir. Yarın bir Vadelihesap sınıfı yazılıp aynı IHesap sözleşmesini uygularsa, bu kod hiç değişmez.

2.2.4 Interface vs Abstract Class

Özellik	Interface	Abstract Class
Örneklenebilir mi?	Hayır	Hayır
Alan (field) tutar mı?	Hayır	Evet
Constructor'ı olabilir mi?	Hayır	Evet
Metot gövdesi taşıyabilir mi?	C# 7 ve önce: Hayır. C# 8+ default: Evet ama sınırlı	Evet (concrete metotlar)
Bir sınıf kaç tane uygulayabilir?	Sınırsız	Sadece 1 (tek kalıtım)
Erişim belirleyicisi	Üyeler örtük public	Her seviyede olabilir
Kullanım amacı	Yetenek sözleşmesi	Ortak durum + kısmen uygulanmış davranış

Karar rehberi:

- Ortak durum (alan, property varsayılan değeri) paylaşılacaksa → abstract class.

- Sadece “bunu yapabilirim” sözleşmesi lazımsa → interface.
- DI ile inject edilecekse (test için mock kolaylığı) → interface.
- Bir tipe birden fazla yetenek eklenecekse → interface.

2.2.5 Örnek 2.2.2 — Abstract Class’a Örnek: Materyal Hiyerarşisi

```
// Models/Materyal.cs
public abstract class Materyal
{
    public string Ad { get; set; }
    public int SayfaSayisi { get; set; }

    // Concrete metot – tüm alt sınıflar aynısını kullanır
    public void OzetYazdir() => Console.WriteLine($"{Ad} ({SayfaSayisi} sf)");

    // Abstract metot – her alt sınıf kendine göre uygular
    public abstract decimal FiyatHesapla();
}

public class Kitap : Materyal
{
    public decimal Fiyat { get; set; }
    public override decimal FiyatHesapla() => Fiyat;
}

public class Dergi : Materyal
{
    public decimal AboneUcreti { get; set; }
    public int AylikSayi { get; set; }
    public override decimal FiyatHesapla() => AboneUcreti / AylikSayi;
}
```

Materyal sınıfı doğrudan örneklenemez; sadece alt sınıflar üzerinden kullanılır. OzetYazdir her alt sınıfta aynı davranışa sahiptir, FiyatHesapla ise her birinde farklı.

2.2.6 Yaygın Hatalar

Hatalı Kod	Hata Mesajı	Neden	Düzeltilmiş
interface IX { public void M(); }	CS0106: modifier 'public' is not valid (bazı sürümlerde uyarı)	Interface üyeleri örtük public	public sözcüğünü kaldır
interface IX { int x; }	CS0525: interfaces cannot contain fields	Interface field tutmaz	Property'ye dönüştür: int X { get; set; }
Bir sınıf interface'i uyguladığını beyan etti ama üye eksik	CS0535: '...' does not implement interface member '...'	Sözleşme eksik	Eksik üyeyi ekle
abstract class A { public abstract void M() { } }	CS0500: '...' cannot declare a body because it is marked abstract	Abstract gövdesiz olur	Gövdeyi sil ya abstract'ı kaldır

2.2.7 Bölüm Özeti

- Interface = sadece imza sözleşmesi.
- Abstract class = kısmen tamamlanmış taban sınıf.
- Interface'ler yetenek eklerken çokludur; abstract class tek kalıttır.
- DI ile enjekte edilen katmanlar (servis, repository) tipik olarak interface tabanlıdır.

İlgili konular: §1.7 (Abstraction), §1.8.6 (DIP), §4.16 (DI ile interface enjeksiyonu).

2.3 Erişim Belirleyiciler

2.3.1 Genel Bakış

Erişim belirleyiciler (access modifiers), bir tipin veya üyenin nereden görünür olacağını belirler. Kapsüllemenin (§1.4) doğrudan uygulama aracıdır. C#'ta beş erişim belirleyicisi vardır; public en açık, private en kapalıdır.

Bir “assembly”, derleme çıktısı olan .dll veya .exe dosyasıdır. Pratikte bir proje bir assembly'ye derlenir; dolayısıyla “aynı assembly” çoğunlukla “aynı proje” anlamına gelir.

2.3.2 Beş Erişim Belirleyicisi

Belirleyici	Erişim Kapsamı	Tipik Kullanım
public	Her yerden (aynı proje + başka projeler)	Sınıfın dış API'si
private	Sadece tanımlandığı sınıfın içinden	İç durum (_bakiye gibi)
internal	Sadece aynı assembly (proje) içinden	Aynı proje içi yardımcı sınıflar
protected	Tanımlandığı sınıftan veya ondan türeyen alt sınıflardan	Alt sınıflara açık, dışa kapalı
protected internal	protected veya internal — aynı assembly'deki herhangi bir yer, ya da başka assembly'deki alt sınıflar	Kütüphane geliştirenlerin alt sınıflara + iç kullanıma açtığı üyeler

private protected (C# 7.2+) da vardır: protected **ve** aynı assembly. Nadir kullanılır, master defterin standart repertuarı içinde değildir.

Not: Bir üye için erişim belirleyicisi yazılmazsa varsayılan private'dır. Bir top-level sınıf için varsayılan internal'dır.

2.3.3 Örnek 2.3.1 — Beş Belirleyicinin Bir Sınıfta Uygulanması

```
// Models/Ornek.cs
public class Ornek
{
    public string PublicAlan = "her yerden";
    private string PrivateAlan = "sadece bu sınıftan";
    internal string InternalAlan = "aynı projeden";
    protected string ProtectedAlan = "bu sınıftan veya alt sınıftan";
    protected internal string ProtectedInternalAlan = "aynı proje veya alt sınıflardan";
}

public class AltOrnek : Ornek
{
    public void Test()
    {
        Console.WriteLine(PublicAlan); // OK
        // Console.WriteLine(PrivateAlan); // HATA: private
        Console.WriteLine(InternalAlan); // OK (aynı proje)
        Console.WriteLine(ProtectedAlan); // OK (alt sınıf)
        Console.WriteLine(ProtectedInternalAlan); // OK
    }
}
```

2.3.4 protected ile protected internal Farkı

Bu ikisi karıştırılır. `protected internal` her iki koşulun birleşimidir — yani “aynı assembly VEYA alt sınıf”:

- **protected**: Sadece kalıtım yoluyla erişim. Başka bir assembly'deki alt sınıf da erişebilir, ama aynı assembly'deki alakasız bir sınıf erişemez.
- **protected internal**: `protected` erişiminin üzerine, aynı assembly'deki tüm sınıflara da açar. Yani “`protected`'ın gevşetilmiş hali”.

Katalog A ve B'deki bazı özet cümleleri “aynı proje içerisinde olma şartı yoktur” gibi ifadelerle yanıltıcı olabilir. Standart C# tanımı yukarıdaki tablodadır ve pratikte kütüphane API'lerinde “kullanıcı türetirse görsün ya da aynı projede yardımcı sınıflarımdan görülsün” niyetini karşılar.

2.3.5 Tip Seviyesinde Erişim

Erişim belirleyicileri yalnız üyelere değil, tiplere (sınıf, interface, enum) de uygulanır.

```
public class Servis { }           // her yerden görünür
internal class Yardimci { }      // sadece bu projede
```

Bir tip `public` iken bir üyesi `internal` olabilir; bu sınıfın dışına çıksa da o üye görülemez.

2.3.6 Yaygın Hatalar

Hatalı Kod	Sonuç	Neden	Düzeltilmiş
Alt sınıftan tabandaki <code>private</code> alana erişim	CS0122: inaccessible due to its protection level	<code>private</code> alt sınıfa açık değil	<code>protected</code> yap
Başka projeden <code>internal</code> sınıfa erişim	CS0122	<code>internal</code> proje sınırında	<code>public</code> yap ya da <code>InternalsVisibleTo</code> kullan
<code>private class</code> OrtaSeviye (top-level)	CS1527: elements defined in a namespace cannot be explicitly declared as private	Top-level tip <code>private</code> olamaz	<code>internal</code> veya <code>public</code>

2.3.7 Bölüm Özeti

- Beş belirleyici: `public`, `private`, `internal`, `protected`, `protected internal`.
- `private` en kısıtlı, `public` en açık.
- Assembly $\approx$ proje (`.dll` / `.exe`).
- `protected internal` = “`protected` VEYA `internal`” — kütüphane API tasarımında kullanışlı.
- Varsayılan üye erişimi `private`; varsayılan tip erişimi `internal`.

İlgili konular: §1.4 (Encapsulation), §2.2 (Interface — üyeler örtük `public`).

2.4 Değişkenler, Veri Tipleri, Bellek Modeli

2.4.1 Genel Bakış

C# **statik tipli** bir dildir: her değişkenin bir tipi vardır ve bu tip derleme zamanında bilinir. Değişkenler ya **değer tipi** (value type) ya da **referans tipi** (reference type) olarak sınıflandırılır. Bu ayrım sadece bellek yerleşimini değil, atama semantiğini de belirler: değer tipleri kopyalanarak taşınır, referans tipleri paylaşılarak.

2.4.2 Değer Tipleri vs Referans Tipleri

Kategori	Örnek Tipler	Bellek	Atama Davranışı
Değer tipi	int, bool, char, double, decimal, struct, enum	Stack (yerel değişkene) veya nesnenin içi (alan ise)	Kopyalanır — iki değişken bağımsız
Referans tipi	class, interface, delegate, string, object, dizi	Nesne heap'te, referans stack'te	Referans kopyalanır — iki değişken aynı nesneyi paylaşır

string teknik olarak referans tipidir ama immutable (değiştirilemez) olduğu için pratikte değer tipi gibi davranır.

2.4.3 Stack vs Heap

Katalog B'deki "RAM'in Yapısı" diyagramına göre:

- **Stack:** Metot çağrıları sırasında yerel değişkenler için ayrılan LIFO (last-in-first-out) alan. Değer tipli yerel değişkenler ve referans değişkenlerinin kendisi burada tutulur. Metot bitince alan otomatik serbest kalır.
- **Heap:** new ile üretilen tüm nesnelerin yaşadığı alan. Garbage Collector (GC) tarafından, artık referans edilmeyenler temizlenir.

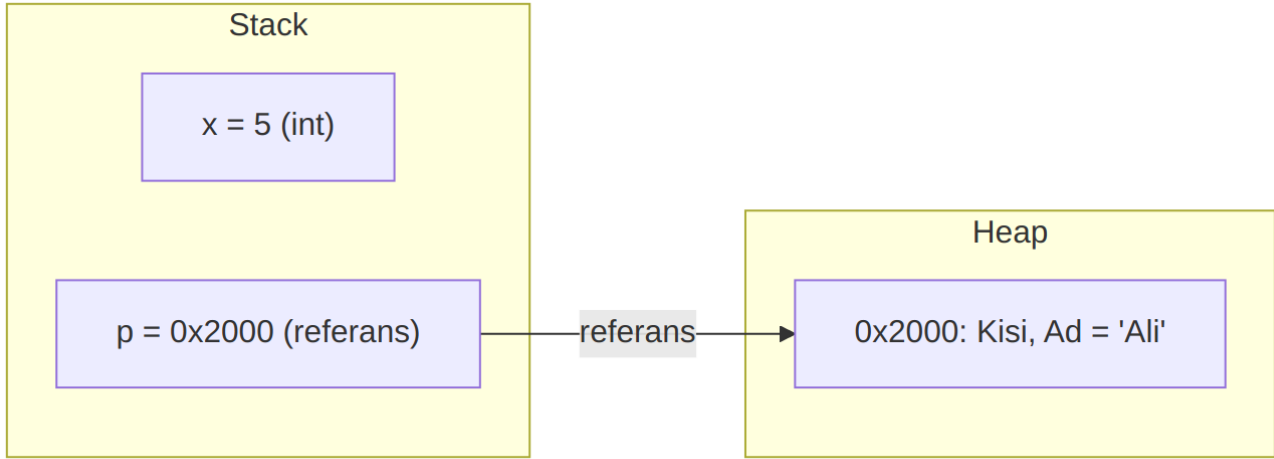


Figure 2: Diyagram 2

Değer tipli x, doğrudan değeri stack'te tutar. Referans p, heap'teki nesnenin adresini stack'te tutar.

2.4.4 Temel Veri Tipleri (Katalog B veri tipi tablosu, düzeltilmiş)

Tür	Açıklama ve Bellek Alanı	Aralık
bool	Doğru veya Yanlış (1 bit)	true / false
char	Karakter (16 bit, Unicode)	\u0000 - \uFFFF
sbyte	İşaretili Tam Sayı (8 bit)	-128 ... 127
byte	İşaretsiz Tam Sayı (8 bit)	0 ... 255
short	İşaretili Tam Sayı (16 bit)	-32.768 ... 32.767
ushort	İşaretsiz Tam Sayı (16 bit)	0 ... 65.535
int	İşaretili Tam Sayı (32 bit)	-2.147.483.648 ... 2.147.483.647
uint	İşaretsiz Tam Sayı (32 bit)	0 ... 4.294.967.295
long	İşaretili Tam Sayı (64 bit)	$\approx \pm 9,22 \times 10^{18}$
ulong	İşaretsiz Tam Sayı (64 bit)	0 ... 18.446.744.073.709.551.615

Tür	Açıklama ve Bellek Alanı	Aralık
float	Tek Kayan Nokta (32 bit)	$\approx \pm 1,5 \cdot 10^{-45} \dots \pm 3,4 \cdot 10^{38}$ (7 basamak hassasiyet)
double	Çift Kayan Nokta (64 bit)	$\approx \pm 5 \cdot 10^{-324} \dots \pm 1,7 \cdot 10^{308}$ (15-16 basamak hassasiyet)
decimal	Ondalıklı Sayı (128 bit)	$\approx \pm 1,0 \cdot 10^{-28} \dots \pm 7,9 \cdot 10^{28}$ (28-29 basamak hassasiyet)

Pratik seçim rehberi:

- Tam sayı: varsayılan int. Büyük değerlerde long.
- Ondalık: bilimsel hesaplamalar için double. **Para işlemleri için mutlaka decimal** (yuvarlama hatasız).
- Karakter: char; metin: string.
- Küçük tam sayı optimizasyonu (byte/short) genellikle gereksiz; derleyici ve JIT işi halleder.

2.4.5 Değişken Bildirimi

```
// Açık tip
int yas = 25;
string ad = "Ayşe";
bool aktif = true;
double pi = 3.14;
decimal fiyat = 199.99m; // 'm' suffix: decimal literal
float oran = 0.5f; // 'f' suffix: float literal
long nufus = 84000000L; // 'L' suffix: long literal
char harf = 'A'; // tek tırnak

// Değişken önce bildirilip sonra atanabilir
int sayi;
sayi = 42;

// var: derleyici tipi sağdaki ifadeden çıkarır
var metin = "Merhaba"; // string
var x = 10; // int
```

2.4.6 var Anahtar Kelimesi

var, tipin sağdaki ifadeden çıkarılmasını sağlar. **Örtük tiplendirme (implicit typing)** olarak da bilinir. Kurallar:

- Yalnızca yerel değişkenlerde kullanılır (metot içinde). Sınıf alanlarında kullanılamaz.
- Başlangıç değeri olmadan tanımlanamaz (var x; derleme hatası — tip çıkarılamaz).
- Bir kez tanımlandıktan sonra tipi değişmez; runtime'da int olan x, sonradan string olamaz.

```
var isim = "Ayşe"; // string
var sayi = 100; // int
var pi = 3.14; // double
var list = new List<int>(); // List<int>
// var x; // HATA — tip çıkarılamaz
```

Katalog B'deki "var Keyword" slaytında verilen bool medeniHal = true; örneği açık tiplendirme; var medeniHal = true; yazılsa da yine bool olurdu.

2.4.7 Sabitler ve Değişmezler

```
const double Pi = 3.14159; // derleme zamanı sabit
readonly int MaxLogin; // constructor'da atanır, sonra değişmez

public class Config
```

```
{
    public const int OturumSaniye = 3600;
    public readonly string Kaynak;

    public Config(string kaynak) => Kaynak = kaynak;
}
```

- const: Derleme zamanında bilinir; sabit ifadelerle atanır.
- readonly: Runtime’da constructor içinde atanır; sonra değiştirilemez.

2.4.8 String — Referans Tipi, Immutable

string bir referans tipidir ancak “değer semantiği” gibi davranır: tüm string işlemleri (birleştirme, Replace, Trim) yeni bir string üretir; orijinal değişmez.

```
string s1 = "Merhaba";
string s2 = s1;
s1 = s1 + " Dünya";
// s1 → "Merhaba Dünya"
// s2 → "Merhaba" (değişmedi)
```

Bu davranış, s1 = s1 + ... işlemi sırasında yeni bir string nesnesi üretildiği, s2’nin hâlâ eskisine işaret ettiği gerçeğinden gelir.

2.4.9 null, string.Empty, “Boş” Farkları

Karışıklığı olan üç durum:

Değer	Bellek Durumu	Semantik
null	Referans hiçbir nesneye işaret etmiyor	“Değer yok”
string.Empty (aynısı "")	Heap’te uzunluğu 0 olan bir string nesnesi var, referans oraya işaret ediyor	“Değer var ama içi boş”
" " (tek boşluk)	Heap’te uzunluğu 1 olan bir string	“Bir karakter boşluk var”

```
string a = null;
string b = string.Empty;
string c = "";
string d = " ";

Console.WriteLine(a == null); // True
Console.WriteLine(string.IsNullOrEmpty(b)); // True
Console.WriteLine(string.IsNullOrEmpty(c)); // True
Console.WriteLine(string.IsNullOrEmpty(d)); // False
Console.WriteLine(string.IsNullOrWhiteSpace(d)); // True
```

Katalog B’deki metafor (“bardak yok” vs “boş bardak var”) pedagojik olarak isabetlidir; teknik doğrusu yukarıdaki tablodadır.

2.4.10 Nullable Değer Tipleri ve Nullable Reference Types

Değer tipleri normalde null alamaz. Nullable<T> veya kısa yazımı T? ile bu kısıtlama gevşetilir:

```
int a = null; // HATA
int? b = null; // OK
int? c = 5;

if (c.HasValue) Console.WriteLine(c.Value);
Console.WriteLine(c ?? 0); // null-coalescing: c null ise 0
```

.NET 6+ ile **Nullable Reference Types** özelliği varsayılan olarak açıktır. `string s = null;` derleme uyarısı üretir (CS8600). Amaç, “referansı null olabilir mi olamaz mı” bilgisini tip sistemine taşımaktır.

```
string ad = "Ali"; // null olamaz
string? soyad = null; // null olabilir

Console.WriteLine(ad.Length); // güvenli
Console.WriteLine(soyad!.Length); // '!' ile null-forgiving; runtime'da riskli
```

2.4.11 Kapsam (Scope)

Bir değişkenin görünürlüğü, bildirildiği süslü parantez blokuyla sınırlıdır.

```
void Metot()
{
    int a = 10;
    {
        int b = 20;
        Console.WriteLine(a + b); // her ikisi görünür
    }
    // Console.WriteLine(b); // HATA – b bloğun dışında
}
```

Katalog B'deki “Scope” slaytında belirtildiği gibi, değişken ve fonksiyonların erişilebilirlik sınırlarını belirleyen alan süslü parantezlerdir. İç bloktan dış bloğa erişilir, ters yönde erişilmez.

2.4.12 Yaygın Hatalar

Hatalı Kod	Hata Mesajı	Neden	Düzeltilmiş
<code>int x = 3.14;</code>	CS0266: cannot implicitly convert type 'double' to 'int'	Bilgi kaybı olabilir	<code>int x = (int)3.14;</code> veya tipi <code>double</code> 'a çevir
<code>int x = 2000000000000;</code> <code>decimal p = 199.99;</code>	CS0266 veya taşma CS0664: cannot implicitly convert 'double' to 'decimal'	Değer <code>int</code> aralığı dışında Suffix eksik	<code>long x = 2000000000000L;</code> <code>199.99m</code>
<code>var x;</code> (tip çıkarılamaz)	CS0818: implicitly-typed variables must be initialized	Başlangıç değeri yok	<code>var x = 0;</code>
<code>int y;</code> <code>Console.WriteLine(y);</code>	CS0165: use of unassigned local variable 'y'	Yerel değişkenler otomatik başlatılmaz	<code>int y = 0;</code>

2.4.13 Mini Sözlük

- **Değer tipi:** Kendi değerini doğrudan taşıyan tip (`int`, `bool`, `struct`).
- **Referans tipi:** Bir nesneye işaret eden tip (`class`, `string`, dizi).
- **Stack:** Yerel değişkenlerin LIFO sırayla tutulduğu bellek bölgesi.
- **Heap:** `new` ile üretilen nesnelerin yaşadığı GC-yönetimli bellek bölgesi.
- **Garbage Collector (GC):** Referans edilmeyen heap nesnelerini otomatik toplayan mekanizma.
- **Immutable:** Oluşturulduktan sonra durumu değiştirilemeyen; `string`, `DateTime` gibi.
- **Nullable:** `null` alabilme yeteneği. Değer tipleri için `?` ile açılır (`int?`).

2.4.14 Bölüm Özeti

- Değişkenler ya değer ya referans tipi taşır; atama semantiği farklıdır.
- Stack = yerel değişkenler + referanslar; heap = nesneler.
- Para hesaplarında decimal, bilimsel hesapta double.
- string referans tipi ama immutable — değer semantiğiyle davranır.
- null ≠ string.Empty ≠ " ".
- .NET 6+’da nullable reference types varsayılan olarak açıktır; string? ile açıkça null-erişimli tip belirtilir.
- Değişken kapsamı bloğuyla ({ }) sınırlıdır.

İlgili konular: §2.5 (Tip dönüşümü), §2.1.7 (Nesne bellek yerleşimi), §2.11 (String detayı).

2.5 Tip Dönüşümü, Boxing ve Unboxing

2.5.1 Genel Bakış

Tip dönüşümü (type conversion / casting), bir değer bir tipten diğerine çevrilmesidir. C#’ta üç ana yaklaşım vardır:

1. **Implicit conversion** (örtük): Bilgi kaybı olmadığı garanti edilen durumlarda derleyici otomatik yapar. `int` → `long` gibi.
2. **Explicit conversion** (açık, cast): Bilgi kaybı olabilir; programcının niyetini (tip)değer sözdizimiyle belirtmesi gerekir.
3. **Parse / TryParse / Convert**: Farklı tipler arası — özellikle `string` ↔ `sayı` — çevirim için kütüphane metotları.

Ayrıca bir alt konu **boxing ve unboxing** vardır: değer tiplerinin referans tipleri arasında (özellikle object içinde) taşınması.

2.5.2 Implicit Conversion

Küçük aralıklı bir tipten daha büyük aralıklı bir tipe geçerken bilgi kaybı yoktur; derleyici otomatik dönüştürür.

```
int i = 42;
long l = i;           // int → long, implicit
float f = i;          // int → float
double d = f;         // float → double

// char → int implicit'tir (ASCII/Unicode kod noktası)
char c = 'A';
int kod = c;          // 65
```

Çıktı:

```
42
65
```

Implicit conversion “her zaman güvenli” olarak derleyici tarafından kabul edilen dönüşümlerdir. Genişleyen dönüşüm (widening conversion) olarak da bilinir.

2.5.3 Explicit Conversion (Cast)

Bilgi kaybı olabilecek durumlarda programcı (tip) cast operatörüyle niyeti belirtir. Bilgi kaybının sonucu (yuvarlama, taşma) programcının sorumluluğundadır.

```
double d = 3.7;
int i = (int)d;    // 3 – ondalık kısım atılır (truncate)

long l = 2147483648L; // int aralığının bir üstü
int x = (int)l;      // taşma: sonuç -2147483648
```

```
int k = 65;
char ch = (char)k; // 'A' – Unicode kod noktası
```

Çıktı:

```
3
-2147483648
A
```

(int)3.7 yuvarlamaz, keser (truncate). Yuvarlamak için Math.Round, Math.Floor, Math.Ceiling.

2.5.4 Parse ve TryParse

Bir string'in içindeki sayıyı tipe çevirmek için Parse veya TryParse kullanılır.

```
string s1 = "42";
int a = int.Parse(s1); // 42

string s2 = "abc";
// int b = int.Parse(s2); // FormatException!

// Güvenli versiyon: TryParse
if (int.TryParse(s2, out int b))
    Console.WriteLine($"Sayı: {b}");
else
    Console.WriteLine("Geçerli sayı değil.");

// Kültür duyarlı
double f = double.Parse("3,14", new System.Globalization.CultureInfo("tr-TR"));
```

Kural: Kullanıcıdan gelen verilerde asla Parse, her zaman TryParse. Parse başarısızlığında exception fırlatır; TryParse sadece false döner.

Metot	Başarısızlıkta Davranış
int.Parse(s)	FormatException fırlatır
int.TryParse(s, out x)	false döner; x = 0 olur
Convert.ToInt32(s)	s == null ise 0, geçersizse FormatException

2.5.5 Convert Sınıfı

Convert, sistematik dönüşüm için tasarlanmış statik metotlar koleksiyonudur. Özellikle null değerleri güvenli işler.

```
int a = Convert.ToInt32("42"); // 42
int b = Convert.ToInt32(null); // 0 (Parse null'da exception atardı)
string s = Convert.ToString(42); // "42"
bool x = Convert.ToBoolean("true"); // true
double d = Convert.ToDouble("3,14"); // kültüre göre
```

Parse vs Convert seçimi:

- Kaynak kesin string'se, Parse veya TryParse.
- Kaynak object veya null olabilirse, Convert.

2.5.6 ToString() — Her Şeyi String'e

Her tipin ToString() metodu vardır (System.Object'ten kalıtım). Sayısal tipler için biçimlendirme dizesi alabilir:

```
int a = 42;
double d = 3.14159;

Console.WriteLine(a.ToString()); // "42"
```

```

Console.WriteLine(a.ToString("D5")); // "00042" - 5 haneli
Console.WriteLine(d.ToString("F2")); // "3,14" - 2 ondalık
Console.WriteLine(d.ToString("P")); // "%314,16" - yüzde
Console.WriteLine(19.99m.ToString("C")); // "₺19,99" - para

```

Çıktı:

```

42
00042
3,14
%314,16
₺19,99

```

Kültürlendirme (tr-TR vs en-US), IFormatProvider parametresiyle kontrol edilir. Web uygulamalarında çoğu zaman CultureInfo.InvariantCulture (nokta ondalık, virgöl binlik) tercih edilir.

2.5.7 Boxing ve Unboxing

Boxing: Bir değer tipini (int, double, struct) object (veya object'ten türeyen bir interface) referansına dönüştürmek. Değer heap'e kopyalanır ve referans döner.

Unboxing: Boxing edilmiş bir object referansından tekrar orijinal değer tipini çıkarmak. Cast gerekir.

```

int a = 28;
object kutulu = a; // Boxing - int değeri heap'e kopyalandı
int b = (int)kutulu; // Unboxing - cast zorunlu

```

Bellek görünümü:

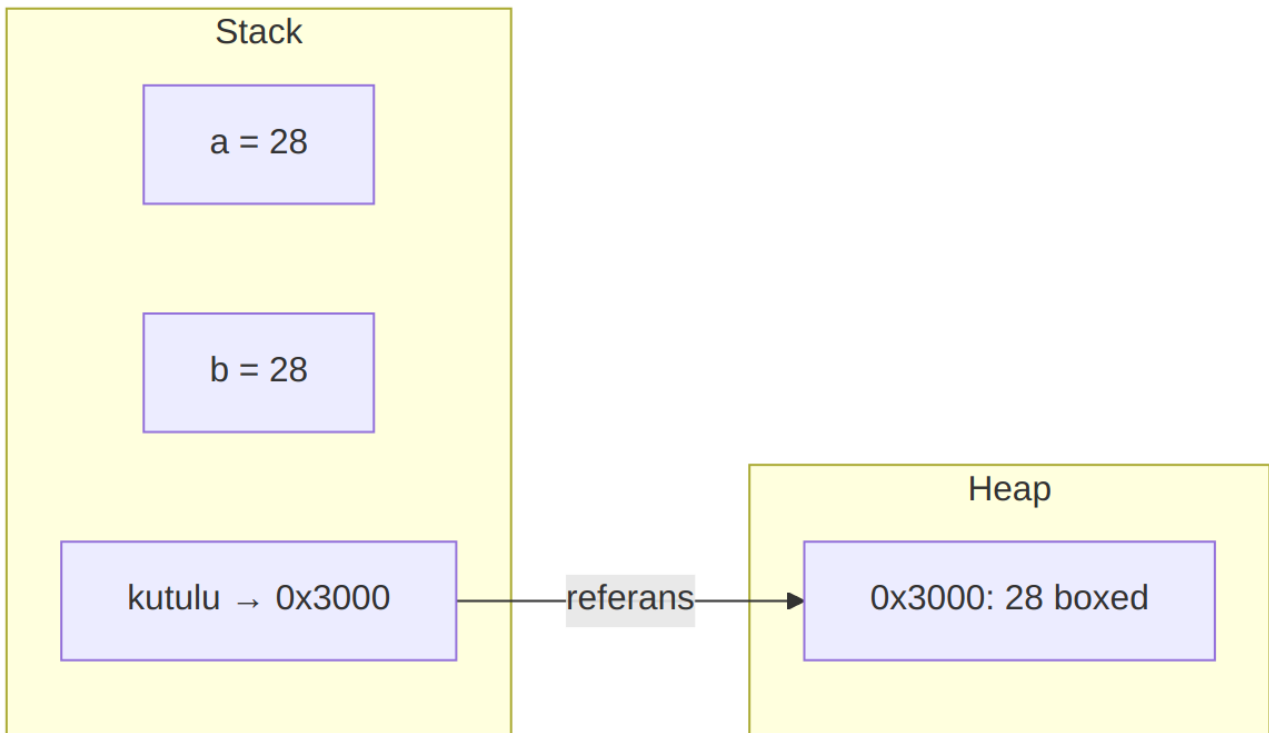


Figure 3: Diyagram 3

Performans notu: Boxing/unboxing heap tahsisi doğurur; sıkı döngülerde performansı düşürebilir. Generic koleksiyonlar (List<int>) tam da bunu önlemek için tasarlandı; eski ArrayList her int eklemesinde boxing yapardı.

Unboxing kuralları:

- Sadece **birebir aynı** tipe unbox edilebilir. int boxed nesneden long unbox edilemez.
- Yanlış tip → InvalidCastException.

```
object o = 42; // int boxed
long l = (long)o; // HATA: InvalidCastException – önce int'e cast, sonra long'a
long l2 = (long)(int)o; // OK
```

2.5.8 Shallow Copy ve Deep Copy

Referans tipler için “kopyalama” sorunu iki farklı seviyede tanımlıdır:

- **Shallow copy (sığ kopya)**: Nesnenin alan değerleri kopyalanır ama iç referanslar aynı nesneleri paylaşır. `object.MemberwiseClone()` bunu yapar.
- **Deep copy (derin kopya)**: Nesnenin ve içinde barındırdığı tüm referans nesnelerin ayrı kopyaları üretilir. Manuel yazmak veya serileştirme (`System.Text.Json`) ile yapılır.

Örnek 2.5.1 — Shallow copy’nin sorunu

```
public class Adres { public string Sehir { get; set; } }
public class Kisi
{
    public string Ad { get; set; }
    public Adres Adres { get; set; }

    public Kisi ShallowKopya() => (Kisi)this.MemberwiseClone();
}

var k1 = new Kisi { Ad = "Ali", Adres = new Adres { Sehir = "İstanbul" } };
var k2 = k1.ShallowKopya();

k2.Ad = "Veli";
k2.Adres.Sehir = "Ankara";

Console.WriteLine($"{k1.Ad} – {k1.Adres.Sehir}");
Console.WriteLine($"{k2.Ad} – {k2.Adres.Sehir}");
```

Çıktı:

```
Ali – Ankara
Veli – Ankara
```

`k2.Ad` bağımsız değişti (string immutable olduğu için ayrı referans), ama `Adres.Sehir` değişikliği `k1`’i de etkiledi çünkü ikisinin `Adres` alanı **aynı** heap nesnesine işaret ediyor.

Örnek 2.5.2 — Deep copy düzeltmesi

```
public Kisi DeepKopya()
{
    return new Kisi
    {
        Ad = this.Ad,
        Adres = new Adres { Sehir = this.Adres?.Sehir }
    };
}

var k1 = new Kisi { Ad = "Ali", Adres = new Adres { Sehir = "İstanbul" } };
var k2 = k1.DeepKopya();

k2.Adres.Sehir = "Ankara";

Console.WriteLine($"{k1.Ad} – {k1.Adres.Sehir}"); // İstanbul (etkilenmedi)
Console.WriteLine($"{k2.Ad} – {k2.Adres.Sehir}");
```

Çıktı:

Ali – İstanbul
Ali – Ankara

Genel amaçlı deep copy için ICloneable interface'i vardır ama önerilmez (sözleşme muğlaktır); modern C# projelerinde ya elle özel DeepClone metodu yazılır ya da JSON serileştirme kullanılır:

```
using System.Text.Json;
```

```
Kisi DeepCloneJson(Kisi kaynak)
{
    var json = JsonSerializer.Serialize(kaynak);
    return JsonSerializer.Deserialize<Kisi>(json);
}
```

Bu yaklaşım genel amaçlıdır ama performansı manuel klondan düşüktür ve [JsonIgnore] gibi işaretlemelere dikkat gerektirir.

2.5.9 Yaygın Hatalar

Hatalı Kod	Hata / Sonuç	Neden	Düzeltilmiş
int x = "42";	CS0029: cannot implicitly convert 'string' to 'int'	Otomatik dönüşüm yok	int.Parse("42") veya int.TryParse
int x = int.Parse(user); (user null)	ArgumentNullException	Parse null kabul etmez	int.TryParse
object o = 42; long l = (long)o; Convert.ToInt32("abc")	InvalidCastException FormatException	Unboxing tipe tam uyumluluk ister Geçersiz format	long l = (long)(int)o; TryParse ile önce kontrol
Para hesabında double kullanmak	Yuvarlama hataları (0.1 + 0.2 == 0.3 false)	Binary floating point sınırı	decimal kullan

2.5.10 Anti-Pattern ve İdiyomatik Kullanım

Anti-Pattern	İdiyomatik	Neden Daha İyi
Kullanıcı input'unda int.Parse	int.TryParse + fallback	Exception maliyetinden kurtulma
Convert.ToInt32 string'de	Kaynak string ise Parse/TryParse	Niyet açık
Sıkı döngüde boxing (object koleksiyona int atma)	List<int> gibi generic koleksiyon	Heap tahsisi yok
Deep copy için ICloneable	Elle DeepClone metodu veya JSON	Sözleşme netliği

2.5.11 Mini Sözlük

- **Implicit conversion:** Derleyici otomatik yapan, güvenli genişletici dönüşüm.
- **Explicit conversion (cast):** Bilgi kaybı olabilecek dönüşüm; (tip) sözdizimi.
- **Boxing:** Değer tipini object içinde heap'e koymak.
- **Unboxing:** Boxed değeri geri çıkarmak; tam tip uyumu ister.
- **Shallow copy:** Alan-alan kopya; iç referanslar paylaşılır.
- **Deep copy:** Tüm iç nesnelerin de ayrı kopyaları.
- **MemberwiseClone:** object'in her sınıfta bulunan protected metodu; shallow copy üretir.

2.5.12 Bölüm Özeti

- Implicit dönüşüm güvenli, derleyici otomatik; explicit dönüşüm cast'lı, sorumluluk programcıda.
- Parse exception atar, TryParse bool döner; kullanıcı verisinde TryParse tercih edilir.
- Boxing/unboxing performans maliyeti getirir; generic koleksiyonlarla önlenir.
- MemberwiseClone shallow copy üretir; deep copy manuel veya serileştirme ile yapılır.
- Para hesapları decimal, bilimsel hesap double.

İlgili konular: §2.4 (Veri tipleri), §2.10 (Generic koleksiyonlar), §2.11 (String biçimlendirme).

2.6 Kontrol Yapıları ve Pattern Matching

2.6.1 Genel Bakış

Bir C# programı, ardışık ifadelerin yanı sıra kararlar üzerine yol ayrımları kurar. Kontrol yapıları bu kararların ifade edildiği yerdir: if / else, switch, üçlü operatör (?:), null-koşullu operatör (?.), null-birleştirme (??) ve modern C#'ın **pattern matching** ailesi. Pattern matching, switch deyimini “eşitlik karşılaştırma”nın ötesine taşıyarak tip, alan değeri, tuple yapısı gibi zengin desenleri tek satırda ifade etmeye imkân tanır.

2.6.2 if / else if / else

```
int not = 78;
string harf;

if (not >= 90)    harf = "AA";
else if (not >= 80) harf = "BB";
else if (not >= 70) harf = "CC";
else if (not >= 60) harf = "DD";
else             harf = "FF";

Console.WriteLine($"Not: {not}, Harf: {harf}");
```

Çıktı:

Not: 78, Harf: CC

JavaScript karşılığı §1.1.5'te; iki dilin sözdizimsel yakınlığı if / else if yapısında belirgindir. Fark: C#'ta koşul ifadesi mutlaka bool üretmeli; if (5) derleme hatası verir.

2.6.3 Üçlü Operatör

```
int yas = 20;
string durum = yas >= 18 ? "Yetişkin" : "Çocuk";
```

?: üçlü operatörü, kısa değer atamalarında if / else bloğunun yerini alır. Yan etkili işlerde (metot çağırısı, atama) tercih edilmez; sadece “değer seçimi” için kullanılır.

2.6.4 Null-Koşullu ve Null-Birleştirme

```
string ad = null;
int? uzunluk = ad?.Length;           // null (crash yok)
string gorunen = ad ?? "(bilinmiyor)"; // "(bilinmiyor)"

// Zincirlenebilir
var soyad = musteri?.Adres?.Sehir ?? "yok";
```

- ?. — null-koşullu erişim: solu null'sa tümü null'a döner, hata atmaz.
- ?? — null-birleştirme: solu null değilse solu, null'sa sağı verir.
- ??= — null-coalescing atama: x ??= y; yalnızca x null'sa y atar.

2.6.5 Klasik switch

```
string not = "BB";
switch (not)
{
    case "AA":
    case "BA":
        Console.WriteLine("Pekiyi"); break;
    case "BB":
    case "CB":
        Console.WriteLine("İyi"); break;
    case "CC":
    case "DC":
        Console.WriteLine("Orta"); break;
    default:
        Console.WriteLine("Başarısız"); break;
}
```

Çıktı:

İyi

Klasik switch, sabit değer karşılaştırmasıyla sınırlıdır. Her case bir break, return veya throw ile bitmelidir; C# fall-through (bir sonraki case'e geçme) izin vermez — birden çok değer aynı gövdeyi paylaşacaksa üst üste case yazılır (yukarıdaki örnekte olduğu gibi).

2.6.6 Pattern Matching (Modern switch)

C# 7'den itibaren switch bir "eşitlik seçici"nden çıkıp bir "desen eşleştirici"ye evrildi. C# 8 ile birlikte gelen **switch expression** (=> sözdizimli) modern kodun standart aracıdır.

Örnek 2.6.1 — Switch expression

```
int not = 85;
string harf = not switch
{
    >= 90 => "AA",
    >= 80 => "BB",
    >= 70 => "CC",
    >= 60 => "DD",
    _ => "FF"
};
Console.WriteLine(harf);
```

Çıktı:

BB

- >= gibi relational pattern'lar (C# 9) doğrudan yazılır.
- _ (discard) tüm eşleşmeyen durumları yakalar; klasik default yerine geçer.
- Switch expression bir **ifadedir** (değer üretir); klasik switch bir **deyimdir** (yan etki üretir).

2.6.7 Tip Pattern'i

```
object nesne = "Merhaba";

string aciklama = nesne switch
{
    int i when i > 100 => $"Büyük tam sayı: {i}",
    int i              => $"Küçük tam sayı: {i}",
    string s           => $"String uzunluğu: {s.Length}",
    null               => "Null!",
    _                  => $"Bilinmeyen tip: {nesne.GetType().Name}"
};
```

```
Console.WriteLine(aciklama);
```

Çıktı:

String uzunluğu: 7

int i when i > 100 deseni, hem tipi hem koşulu tek satırda birleştirir. when guard'ı ek koşul ekler.

2.6.8 Property Pattern

Nesnenin alanlarına göre eşleştirme yapar.

```
public record Kisi(string Ad, int Yas);
```

```
Kisi k = new("Ayşe", 25);
```

```
string kategori = k switch
{
    { Yas: >= 65 } => "Yaşlı",
    { Yas: >= 18 } => "Yetişkin",
    { Yas: >= 13 } => "Genç",
    { Yas: >= 0 }  => "Çocuk",
    -             => "Geçersiz"
};
```

```
Console.WriteLine(kategori);
```

Çıktı:

Yetişkin

{ Yas: >= 18 } deseni, “objenin Yas property’si 18 veya üzeri” anlamına gelir. İç içe property pattern’lar da mümkündür: { Adres: { Sehir: "İstanbul" } }.

2.6.9 Tuple Pattern

Birden çok değeri birlikte eşleştirir.

```
static string TahminEt(bool guclu, bool hizli) => (guclu, hizli) switch
{
    (true, true) => "Süper kahraman",
    (true, false) => "Ağır ama etkili",
    (false, true) => "Çevik",
    (false, false) => "Sıradan"
};
```

```
Console.WriteLine(TahminEt(true, false));
```

Çıktı:

Ağır ama etkili

Tuple pattern, “çok değişkenli karar tablosu” senaryolarında zincirlenmiş if’lerin çok daha okunaklı alternatifidir.

2.6.10 Var Pattern

Eşleşen değeri yeni bir isme bağlar; her zaman eşleşir, filtre when ile eklenir.

```
int? deger = 42;
```

```
string mesaj = deger switch
{
    var x when x is null => "Yok",
    var x when x % 2 == 0 => $"{x} çift",
};
```

```
var x => $"{x} tek";
};
```

Bu, “her değeri bir isme al ve when ile filtrele” desenidir. Karmaşık hesaplamayı when içinde önceden yapmak isteyen durumlarda tercih edilir.

2.6.11 is ile Pattern Matching

switch dışında da desen eşleştirme kullanılır:

```
object x = "abc";

if (x is string s && s.Length > 0)
    Console.WriteLine($"Boş olmayan string: {s}");

if (x is not null)
    Console.WriteLine("null değil");

// C# 9+ mantıksal desen operatörleri
if (x is int and (>= 1 and <= 10))
    Console.WriteLine("1-10 arası int");
```

2.6.12 Yaygın Hatalar

Hatalı Kod	Hata Mesajı	Neden	Düzeltilmiş
if (5) { }	CS0029: cannot implicitly convert 'int' to 'bool'	C#'ta if koşulu bool zorunlu	if (5 != 0) { }
switch case'inde break unutulmuş	CS0163: control cannot fall through from one case label	Fall-through yasak	break, return veya throw ekle
Switch expression'ında _ yok, eşleşmeyen değer geldi	SwitchExpressionException (runtime)	Non-exhaustive	_ => ... ekle
x?.Length + 1 (x null olabilir)	Beklenmedik null	?. zinciri null döndürdü, +1 null'a uygulandı	(x?.Length ?? 0) + 1

2.6.13 Anti-Pattern ve İdiyomatik Kullanım

Anti-Pattern	İdiyomatik	Neden Daha İyi
Uzun if / else if zincirleri	Switch expression	Okunabilirlik, exhaustive check
Tip kontrolü + cast ayrı ayrı Klasik switch + break'lerle uzun blok	is Tip x pattern Switch expression =>	Tek adım, güvenli Kısa, ifade tabanlı
if (x != null) { x.M(); }	x?.M()	Kısa, atlanmayan null-güvenlik

2.6.14 Bölüm Özeti

- if / else, üçlü operatör, klasik switch temel kararlar.
- Switch expression (=>) modern kodun standart aracı.
- Tip pattern, property pattern, tuple pattern, var pattern zengin desenler sunar.
- is operatörü tek satırda tip kontrolü + değişken bağlama yapar.
- ?. , ??, ??= null-güvenli akış için üç anahtar operatördür.
- _ (discard) exhaustive default'tır.

İlgili konular: §2.13 (Tuple), §2.4 (Nullable), §4.7 (Controller action return switch).

2.7 Döngüler

2.7.1 Genel Bakış

Döngüler, aynı kod bloğunun birden fazla kez çalıştırılmasını sağlar. C#'ta dört yapı vardır: for, while, do-while, foreach. Seçim, elimizdeki bilgiye göre yapılır:

Yapı	Ne zaman	Kaç kez
for	İterasyon sayısı önceden biliniyor	Sayaç kontrollü
while	Koşul geçerli olduğu sürece	Bilinmiyor, önce koşul
do-while	En az bir kez çalışmalı	Bilinmiyor, önce blok
foreach	Bir koleksiyonun tüm elemanları üzerinde	Koleksiyon boyutu kadar

C#'ta "iterasyon" ile "döngü" arasında pratik bir eşdeğerlik vardır: her ikisi de aynı bloğun tekrar tekrar çalıştırılmasını ifade eder. Teknik ayrımıyla iterasyon, bir kaynağın (koleksiyon, iterator) üzerinden geçme kavramıdır; döngü, bunu yapan dilsel yapıdır. Master defter boyunca "döngü" kullanılır.

2.7.2 for Döngüsü

```
for (int i = 0; i < 5; i++)  
    Console.WriteLine($"i = {i}");
```

Çıktı:

```
i = 0  
i = 1  
i = 2  
i = 3  
i = 4
```

Üç bölüm: **başlangıç** (int i = 0), **koşul** (i < 5), **artım** (i++). Her döngü adımında koşul kontrol edilir; true ise gövde çalışır, sonra artım işletilir.

Ters yönde:

```
for (int i = 10; i > 0; i -= 2)  
    Console.WriteLine(i);
```

Çıktı:

```
10  
8  
6  
4  
2
```

2.7.3 while Döngüsü

Koşul önce kontrol edilir; koşul en başından false ise gövde hiç çalışmaz.

```
int sayi = 1;  
while (sayi < 5)  
{  
    Console.WriteLine(sayi);  
    sayi++;  
}
```

Çıktı:

```
1
2
3
4
```

Kullanıcı girişi doğrulama senaryosu:

```
string cevap;
do
{
    Console.Write("Devam etmek istiyor musunuz? (E/H): ");
    cevap = Console.ReadLine();
} while (cevap != "E" && cevap != "H");
```

2.7.4 do-while Döngüsü

Gövde en az bir kez çalışır, sonra koşul kontrol edilir.

```
int i = 0;
do
{
    Console.WriteLine("En az bir kez.");
    i++;
} while (i < 3);
```

Çıktı:

```
En az bir kez.
En az bir kez.
En az bir kez.
```

while ile temel fark: koşul en başta false olsa bile blok bir kere çalışır. Kullanım alanı dar; en yaygın olarak menü döngülerinde ve kullanıcı girişi doğrulamada tercih edilir.

2.7.5 foreach Döngüsü

Bir koleksiyonun her elemanını sıra ile ziyaret eder. Sayaç kullanmaz.

```
string[] adlar = { "Ayşe", "Mehmet", "Fatma" };
foreach (var ad in adlar)
    Console.WriteLine(ad);
```

Çıktı:

```
Ayşe
Mehmet
Fatma
```

Kritik kural: foreach sırasında koleksiyonun içeriği değiştirilemez. Ekleme/silme yapılırsa InvalidOperationException: Collection was modified fırlar.

Yanlış:

```
var list = new List<int> { 1, 2, 3, 4, 5 };
foreach (var x in list)
    if (x % 2 == 0) list.Remove(x); // HATA (runtime)
```

Doğru — tersten for ile:

```
for (int i = list.Count - 1; i >= 0; i--)
    if (list[i] % 2 == 0)
        list.RemoveAt(i);
```

Ya da LINQ ile Where:

```
list = list.Where(x => x % 2 != 0).ToList();
```

Teknik olarak foreach, arka planda koleksiyonun IEnumerator nesnesini alır (GetEnumerator().MoveNext()) çağırarak ilerler. Bu **iterator pattern**'in dil düzeyindeki karşılığıdır.

2.7.6 break ve continue

- break: İçinde bulunulan döngüden anında çıkar.
- continue: İçinde bulunulan iterasyonu atlar, döngü kontrolüne (koşul/artım) döner.

```
for (int i = 0; i < 10; i++)
{
    if (i == 5) break;
    if (i % 2 == 0) continue;
    Console.Write($"{i} ");
}
// Çıktı: 1 3
```

Çıktı:

1 3

2.7.7 İç İçe Döngüler

```
// Çarpım tablosu
for (int i = 1; i <= 3; i++)
{
    for (int j = 1; j <= 3; j++)
        Console.Write($"{i * j,4}");
    Console.WriteLine();
}
```

Çıktı:

```
1  2  3
2  4  6
3  6  9
```

{i * j,4} biçimlendirmesi, değeri 4 karakter genişliğinde sağa hizalar.

2.7.8 Yaygın Hatalar

Hatalı Kod	Sonuç	Neden	Düzeltilmiş
for (int i = 0; i < 10; i--)	Sonsuz döngü	Artım yanlış yönde	i++ veya koşulu i > 0 yap
foreach içinde koleksiyon değiştirme	InvalidOperationException	Enumerator geçersiz olur	Ters for veya LINQ Where
while (x = 5) (tek =)	CS0029 (bool bekleniyor)	Atama, karşılaştırma değil	while (x == 5)
Sayacın foreach içinde artırılması ve i beklentisi	Sayaç yok	foreach sayaç sağlamaz	Sayaç istiyorsan for veya .Select((x,i) => ...)

2.7.9 Bölüm Özeti

- Dört döngü: for, while, do-while, foreach.
- İterasyon sayısı belliyse for; koleksiyon üzerinde foreach.
- foreach içinde koleksiyon değiştirilmez.
- break çıkar, continue iterasyonu atlar.
- Arka planda foreach, IEnumerator sözleşmesini kullanır (iterator pattern).

İlgili konular: §2.10 (Koleksiyonlar ve IEnumerable), §2.9 (Diziler).

2.8 Try-Catch (Hata Yönetimi)

2.8.1 Genel Bakış

Bir program çalışırken beklenmeyen durumlar (dosya bulunamadı, ağ hatası, null referans, geçersiz format) exception (istisna) olarak fırlatılır. Exception yakalanmazsa program çöker. try-catch-finally yapısı bu tür durumları kontrollü biçimde yakalar, kullanıcıya anlamlı geri bildirim verir ve gerekirse temizlik işlerini yapar.

C#'taki tüm exception'lar System.Exception tipinden türer. Yaygın alt tipler:

- `NullReferenceException`: null referans üzerinde metot/property çağrısı.
- `IndexOutOfRangeException`: dizi/koleksiyon sınır dışı erişim.
- `ArgumentException`, `ArgumentNullException`: parametre validasyonu.
- `InvalidCastException`: hatalı cast.
- `FormatException`: Parse başarısız.
- `DivideByZeroException`: sıfıra bölme (yalnızca tam sayılarda; double sıfıra bölme Infinity üretir).
- `FileNotFoundException`, `IOException`: dosya/G-Ç.

2.8.2 Sözdizimi

```
try
{
    // riskli kod
}
catch (SpesifikException ex)
{
    // sadece bu tipteki hataları yakala
}
catch (Exception ex)
{
    // her hatayı yakalayan genel yakalayıcı
}
finally
{
    // hata olsa da olmasa da çalışır; temizlik
}
```

- catch bloğu, en spesifikten en genele sıralanır. Genel Exception'ı önce yazmak, altındaki spesifik yakalayıcıları erişilmez kılar (CS0160).
- finally her durumda çalışır; hatta try içinde return bile olsa finally önce koşular.

2.8.3 Örnek 2.8.1 — Sayıya Çevirme

```
Console.Write("Bir sayı gir: ");
string girdi = Console.ReadLine();

try
{
    int sayi = int.Parse(girdi);
    Console.WriteLine($"Kare: {sayi * sayi}");
}
catch (FormatException)
{
    Console.WriteLine("Bu geçerli bir sayı değil.");
}
catch (OverflowException)
{
    Console.WriteLine("Sayı çok büyük veya çok küçük.");
}
catch (Exception ex)
```

```

{
    Console.WriteLine($"Beklenmeyen hata: {ex.Message}");
}
finally
{
    Console.WriteLine("İşlem tamamlandı.");
}

```

Örnek girdilere göre çıktı:

Bir sayı gir: 12
Kare: 144
İşlem tamamlandı.

Bir sayı gir: abc
Bu geçerli bir sayı değil.
İşlem tamamlandı.

Bir sayı gir: 99999999999999999999
Sayı çok büyük veya çok küçük.
İşlem tamamlandı.

2.8.4 Örnek 2.8.2 — Sıfıra Bölme

```

try
{
    int a = 10, b = 0;
    int c = a / b;    // DivideByZeroException
    Console.WriteLine(c);
}
catch (DivideByZeroException ex)
{
    Console.WriteLine($"Sıfıra bölme: {ex.Message}");
}

```

Çıktı:

Sıfıra bölme: Attempted to divide by zero.

double için aynı işlem hata fırlatmaz; Infinity üretir:

```

double d = 10.0 / 0.0;
Console.WriteLine(d);           // Infinity
Console.WriteLine(double.IsInfinity(d)); // True

```

2.8.5 Exception Fırlatma — throw

Kendi hatanızı yaratmak için throw:

```

public void ParaCek(decimal miktar)
{
    if (miktar <= 0)
        throw new ArgumentException("Miktar pozitif olmalı.", nameof(miktar));
    if (miktar > Bakiye)
        throw new InvalidOperationException("Bakiye yetersiz.");
    Bakiye -= miktar;
}

```

nameof(miktar) derleme zamanında parametrenin adını ("miktar") string olarak üretir; refactor-ing'de sağlam bir tekniktir.

Yeniden fırlatma:

```
try { /* ... */ }
catch (Exception ex)
{
    Log(ex);
    throw; // stack trace korunur (throw ex; yazma!)
}
```

throw ex; yazmak stack trace'i sıfırlar; throw; orijinal fırlatma yerini korur.

2.8.6 Exception Filtresi (when)

C# 6+ ile catch bloğuna koşul eklenebilir:

```
try { /* ... */ }
catch (HttpRequestException ex) when (ex.StatusCode == HttpStatusCode.NotFound)
{
    Console.WriteLine("Bulunamadı.");
}
catch (HttpRequestException ex)
{
    Console.WriteLine($"HTTP hatası: {ex.Message}");
}
```

when koşulu yakalanmadan önce değerlendirilir; koşul false ise catch atlanır ve stack unwind devam eder.

2.8.7 Yaygın Hatalar

Hatalı Kod	Sonuç	Neden	Düzeltilmiş
Boş catch { }	Hata sessizce yutulur	Görünürlük yok	En azından logla, ya sadece bilinen tipleri yakala
throw ex;	Stack trace kayboluyor	Yeniden yaratılıyor	throw;
catch (Exception) { } ilk sırada	Alt catch'lere ulaşılmıyor	Genel önce yakalar	Spesifikten genele sırala
Kullanıcı input'unda try/catch ile Parse finally içinde exception atılırsa dış exception yutulur	Exception maliyeti yüksek Orijinal hata kaybolur	Kontrol akışı için exception Yeni exception dış olanı ezer	int.TryParse kullan finally'de exception fırlatma

2.8.8 Anti-Pattern ve İdiyomatik Kullanım

Anti-Pattern	İdiyomatik	Neden Daha İyi
Her metot etrafında try/catch	Yalnız gerçek risk noktalarında + üstte tek "global handler"	Kod okunabilir kalır
catch (Exception) her yerde Exception'ı akış kontrolü için kullanma	Spesifik exception tipleri Öncesinde TryParse, Contains, TryGetValue	Beklenen hata net Performans + niyet açık
İşlem başarısızlığında null dönme	throw new BusinessException(...)	Çağırın açık geri bildirim alır

2.8.9 Bölüm Özeti

- Exception'lar System.Exception'tan türer; spesifik tiplerle yakalanır.
- Sıralama: spesifikten genele.
- finally her durumda çalışır; kaynak temizliği için idealdir (using bunu otomatikleştirir).

- throw; (parametresiz) stack trace'i korur; throw ex; yazma.
- when ile catch'e koşul eklenir.
- Kontrol akışı için exception kullanma — TryParse gibi güvenli alternatifleri tercih et.

İlgili konular: §2.5.4 (TryParse), §4.17 (Middleware ile global exception handler).

2.9 Diziler ve Çok Boyutlu Diziler

2.9.1 Genel Bakış

Dizi (array), aynı tipteki elemanların sabit boyutlu, ardışık koleksiyonudur. Boyut oluşturma anında belirlenir ve sonradan değiştirilemez. Referans tipi olarak heap'te bir bütün olarak tahsis edilir; her elemana sıfır tabanlı indeksle erişilir.

Dizi vs koleksiyon (List<T> gibi):

- Dizi: sabit boyut, en hızlı erişim, indeksleme.
- Koleksiyon: dinamik boyut, ekleme/silme, LINQ desteği.

Çoğu uygulama düzeyi kod artık List<T> kullanır; diziler daha çok performans-kritik yerlerde ve API imzalarında görülür.

2.9.2 Tek Boyutlu Diziler

Bildirim ve başlatma:

```
// 1) Boyutla üret, elemanları sonra ata
int[] sayilar = new int[5];           // {0, 0, 0, 0, 0}
sayilar[0] = 10;
sayilar[1] = 20;

// 2) Doğrudan literal
int[] notlar = { 90, 85, 78, 66, 55 };

// 3) new + literal
string[] renkler = new string[] { "kırmızı", "mavi", "yeşil" };

// 4) new + boyut + literal
double[] fiyatlar = new double[3] { 1.5, 2.5, 3.5 };
```

Erişim ve iterasyon:

```
int[] notlar = { 90, 85, 78, 66, 55 };
Console.WriteLine($"Boyut: {notlar.Length}");
Console.WriteLine($"İlk: {notlar[0]}");
Console.WriteLine($"Son: {notlar[notlar.Length - 1]}"); // veya notlar[^1] (C# 8+)

for (int i = 0; i < notlar.Length; i++)
    Console.Write($"{notlar[i]} ");

Console.WriteLine();

foreach (var n in notlar)
    Console.Write($"{n} ");
```

Çıktı:

```
Boyut: 5
İlk: 90
Son: 55
90 85 78 66 55
90 85 78 66 55
```

C# 8+ ile **index from end** operatörü ^ ve **range** operatörü .. diziler ve Span<T> üzerinde çalışır:

```
int[] a = { 10, 20, 30, 40, 50 };
Console.WriteLine(a[^1]); // 50 (sondan 1.)
Console.WriteLine(a[^2]); // 40
int[] orta = a[1..4]; // {20, 30, 40}
```

2.9.3 Dizi Metotları — System.Array

Array sınıfının statik metotları:

Metot	Görevi
Array.Sort(dizi)	Yerinde artan sırada sıralar
Array.Reverse(dizi)	Yerinde tersine çevirir
Array.IndexOf(dizi, deger)	Değerin ilk indeksini döner; yoksa -1
Array.Copy(src, dst, uzunluk)	Öğeleri kopyalar
Array.Clear(dizi, ilk, adet)	Belirli aralığı tipin default değeriyle doldurur
Array.Resize(ref dizi, yeniBoyut)	Yeni boyutta yeni dizi üretir, referansı günceller

Örnek 2.9.1 — Sıralama, tersine çevirme, arama

```
int[] sayilar = { 5, 3, 8, 1, 4 };

Array.Sort(sayilar);
Console.WriteLine(string.Join(", ", sayilar));

Array.Reverse(sayilar);
Console.WriteLine(string.Join(", ", sayilar));

int indeks = Array.IndexOf(sayilar, 3);
Console.WriteLine($"3'ün indeksi: {indeks}");
```

Çıktı:

```
1, 3, 4, 5, 8
8, 5, 4, 3, 1
3'ün indeksi: 3
```

string.Join(", ", dizi) bir dizinin elemanlarını virgülle birleştirir; log ve debug için sıklıkla kullanılır.

2.9.4 Çok Boyutlu Diziler

C# iki tür çok boyutlu dizi sunar:

1. **Dikdörtgen (rectangular) dizi:** int[,], int[,,]. Bellekte tek blok.
2. **Jagged (dişli) dizi:** int[][]. Dizilerden oluşan bir dizi; her iç dizinin boyu farklı olabilir.

Dikdörtgen 2 boyutlu:

```
// 3x3 matris
int[,] matris = new int[3, 3]
{
    { 1, 2, 3 },
    { 4, 5, 6 },
    { 7, 8, 9 }
};

Console.WriteLine(matris[0, 0]); // 1
Console.WriteLine(matris[2, 2]); // 9
Console.WriteLine(matris.GetLength(0)); // 3 satır
Console.WriteLine(matris.GetLength(1)); // 3 sütun

for (int i = 0; i < matris.GetLength(0); i++)
```

```
{
    for (int j = 0; j < matris.GetLength(1); j++)
        Console.Write($"{matris[i, j],3}");
    Console.WriteLine();
}
```

Çıktı:

```
1
9
3
3
  1  2  3
  4  5  6
  7  8  9
```

Jagged (dişli):

```
int[][] jag = new int[3][];
jag[0] = new int[] { 1, 2 };
jag[1] = new int[] { 3, 4, 5, 6 };
jag[2] = new int[] { 7 };

for (int i = 0; i < jag.Length; i++)
{
    for (int j = 0; j < jag[i].Length; j++)
        Console.Write($"{jag[i][j]} ");
    Console.WriteLine();
}
```

Çıktı:

```
1 2
3 4 5 6
7
```

Ne zaman hangisi?

- Dikdörtgen: matris hesapları, sabit boyutlu tablolar.
- Jagged: satır boyutları farklıysa (kelime listeleri, gruplu veri) veya iç diziler bağımsız işleniyorsa.

2.9.5 3 Boyutlu Dizi

```
int[, ,] kup = new int[2, 3, 4]; // 2 katman x 3 satır x 4 sütun
kup[0, 1, 2] = 42;
Console.WriteLine(kup[0, 1, 2]); // 42
```

Nadiren gerekir; genelde nesne modellemesi (List<Katman> içeren sınıf) daha okunabilir.

2.9.6 Yaygın Hatalar

Hatalı Kod	Hata / Sonuç	Neden	Düzeltilmiş
int[] a = new int[3] { 1, 2 };	CS0847: array initializer size mismatch	Boyut ile literal uzunluk uyumsuz	Ya boyutu düşür ya literal genişlet
a[3] (boyut 3 olan dizide)	IndexOutOfRangeException	Sıfır tabanlı, geçerli indeksler 0-2	a[a.Length - 1] veya a[^1]
foreach ile diziyi değiştirmek	foreach değişkeni readonly	Iterator yapısı	for ile indeksle değiştir
Dikdörtgen ve jagged'ın karıştırılması	int[,] a = new int[3][]; derlenmez	İki farklı tip	int[,] için new int[3,3]

2.9.7 Bölüm Özeti

- Dizi = sabit boyutlu, sıfır tabanlı, aynı tipte elemanlar.
- Boyut oluşturma anında belirlenir; `Array.Resize` yeni dizi üretir.
- `Length`, `GetLength(dim)`, `[^1]`, `[i..j]` sık kullanılan sözdizimleri.
- İki çok boyutlu tip: dikdörtgen (`[,]`) ve jagged (`[[]]`).
- Dinamik boyut ve ekleme/silme için `List<T>` (bkz. §2.10).

İlgili konular: §2.10 (Koleksiyonlar), §2.11 (`String.Join`), §3.11 (LINQ dizi üzerinde çalışır).

2.10 Koleksiyonlar

2.10.1 Genel Bakış

Koleksiyonlar, birden çok elemanı gruplayan generic tiplerdir. .NET'in koleksiyon dünyası bir hiyerarşi üzerine kuruludur: en yukarıda soyut sözleşmeler (`IEnumerable<T>`, `ICollection<T>`, `IList<T>`), altta somut sınıflar (`List<T>`, `Dictionary<TKey, TValue>`, `HashSet<T>`, `Queue<T>`, `Stack<T>`).

Bu hiyerarşinin başındaki üç interface, gündelik kodda en çok karşılaşılanlardır. Master defter boyunca özellikle LINQ (§3.11) ve EF Core (§3.13) bağlamında bu tiplerle karşılaşılacaktır.

2.10.2 `IEnumerable<T>`, `ICollection<T>`, `IList<T>` Hiyerarşisi

Katalog C'deki benzetmeleri temel alan sezgisel eşleme:

- **`IEnumerable<T>`** — “kaydıran tren”. Elemanları **tek yönde** ve **teker teker** ziyaret edebilirsiniz. Kaç eleman olduğunu, ortadaki bir elemana rastgele erişimi bilemezsiniz. Sadece `foreach` çalıştırabilirsiniz. LINQ sorgularının çıkış tipi çoğunlukla `IEnumerable<T>`'dir.
- **`ICollection<T>`** — “bir kova”. Elemanları sayabilir (`Count`), ekleyip çıkarabilir (`Add`, `Remove`), koleksiyonun tümüyle karıştırıp temizleyebilirsiniz (`Clear`). Ancak sırayla erişim garantisi yoktur; belirli bir indeks yoktur.
- **`IList<T>`** — “sıra numarası olan dolap”. Elemanların sıralı bir düzeni vardır; belirli bir indeksle (`list[3]`) erişebilirsiniz, araya ekleyebilirsiniz (`Insert`), indeksle çıkarabilirsiniz (`RemoveAt`).

Kalıtım zinciri:

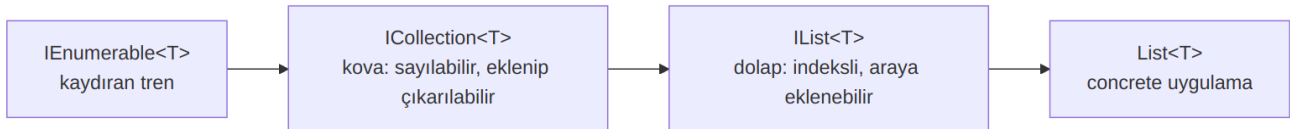


Figure 4: Diyagram 4

`List<T>`, `IList<T>` interface'ini uygular. `IList<T>` `ICollection<T>`'i uygular. `ICollection<T>` `IEnumerable<T>`'yi uygular. Yani `List<T>` bir `IList<T>`'tir, aynı zamanda bir `ICollection<T>`'tür ve bir `IEnumerable<T>`'dir.

2.10.3 Üye Karşılaştırması

Üye	<code>IEnumerable<T></code>	<code>ICollection<T></code>	<code>IList<T></code>
<code>GetEnumerator()</code> (<code>foreach</code>)	☐	☐	☐
<code>Count</code>	☐	☐	☐
<code>Add(item)</code> , <code>Remove(item)</code> , <code>Clear()</code>	☐	☐	☐
<code>Contains(item)</code>	☐	☐	☐
<code>CopyTo(array, index)</code>	☐	☐	☐
<code>this[i]</code> — indeksleyici	☐	☐	☐
<code>IndexOf(item)</code> , <code>Insert(i, item)</code> , <code>RemoveAt(i)</code>	☐	☐	☐

Kural: **imza en gevşek olanı iste, en zengin olanı döndür**. Bir metod parametresi olarak IEnumerable<T> almak, çağırana en fazla esneklik verir. Ancak koleksiyona ekleme veya sayım gerektiriyorsa ICollection<T> iste.

2.10.4 List<T> — En Yaygın Somut Koleksiyon

Dinamik boyutlu, indeksleyicili, generic bir dizi olarak düşünülebilir. Arka planda büyüyen bir iç dizi (T[]) tutar; kapasite dolunca yeni ve daha büyük bir diziye kopyalanır.

```
var isimler = new List<string>();
isimler.Add("Ali");
isimler.Add("Veli");
isimler.Add("Ayşe");
isimler.Insert(1, "Mehmet");           // 1. indekse ekle

Console.WriteLine(isimler.Count);      // 4
Console.WriteLine(isimler[0]);          // Ali
Console.WriteLine(isimler[1]);          // Mehmet

isimler.Remove("Veli");                 // değere göre sil
isimler.RemoveAt(0);                   // indekse göre sil

foreach (var i in isimler)
    Console.WriteLine(i);
```

Çıktı:

```
4
Ali
Mehmet
Mehmet
Ayşe
```

Yaygın List<T> metotları:

Metot	Görevi
Add(item)	Sona ekler
Insert(i, item)	i. konuma ekler, sonrakiler kayar
Remove(item)	İlk eşleşen elemanı çıkarır (bool döner)
RemoveAt(i)	i. indeksteki elemanı çıkarır
Contains(item)	Var mı?
IndexOf(item)	İlk indeks; yoksa -1
Sort(), Sort(comparer)	Yerinde sıralar
Reverse()	Yerinde tersine çevirir
ToArray()	Diziye kopyalar
Clear()	Tüm elemanları silir
Find(pred), FindAll(pred)	Predikat ile arama
AddRange(collection)	Tümünü sona ekler
Any(pred), All(pred), Count(pred)	LINQ üzerinden (using System.Linq)

2.10.5 Dictionary<TKey, TValue> — Anahtar-Değer

Her elemanı bir **anahtar** ve **değer** çiftidir. Anahtar benzersizdir; hash tabanlı olduğu için erişim O(1) hızındadır.

```
var telefonlar = new Dictionary<string, string>
{
    ["Ali"]    = "0555 111 22 33",
    ["Veli"]   = "0555 222 33 44"
};

telefonlar["Ayşe"] = "0555 333 44 55";
```

```

telefonlar["Ali"] = "0555 000 00 00"; // Ali'nin numarası güncellendi

if (telefonlar.ContainsKey("Veli"))
    Console.WriteLine(telefonlar["Veli"]);

// Güvenli erişim
if (telefonlar.TryGetValue("Fatma", out var num))
    Console.WriteLine(num);
else
    Console.WriteLine("Fatma yok");

foreach (var kayıt in telefonlar)
    Console.WriteLine($"{kayıt.Key}: {kayıt.Value}");

```

Çıktı:

```

0555 222 33 44
Fatma yok
Ali: 0555 000 00 00
Veli: 0555 222 33 44
Ayşe: 0555 333 44 55

```

Kritik: dict["Fatma"] yazmak, anahtar yoksa KeyNotFoundException fırlatır. Güvenli okuma için TryGetValue tercih edilir.

2.10.6 Diğer Somut Koleksiyonlar (Kısa)

Tip	Ne için
HashSet<T>	Benzersiz elemanlar, hash tabanlı; hızlı Contains
Queue<T>	FIFO (ilk giren ilk çıkar); Enqueue, Dequeue
Stack<T>	LIFO (son giren ilk çıkar); Push, Pop
SortedDictionary<K,V>	Anahtara göre sıralı dictionary
LinkedList<T>	Bağlı liste; araya ekleme O(1) ama indeksleme O(n)

2.10.7 Koleksiyon Başlatıcı ve Koleksiyon İfadesi

```

// Koleksiyon başlatıcı (C# 3+)
var sayilar = new List<int> { 1, 2, 3, 4, 5 };
var telefonlar = new Dictionary<string, string>
{
    { "Ali", "0555..." },
    { "Veli", "0555..." }
};

// Koleksiyon ifadesi (C# 12+) – birleşik sözdizim
List<int> baska = [10, 20, 30];
int[] dizi = [1, 2, 3];

```

C# 12'nin **collection expressions** özelliği, dizi ve List<T> başlatmayı ortak bir sözdizime bağlar. Şu anki proje kodlarında geleneksel { 1, 2, 3 } sözdizimi hâlâ yaygındır; her ikisi de derleyicide aynı IL'e derlenir.

2.10.8 Yaygın Hatalar

Hatalı Kod	Sonuç	Neden	Düzeltilmiş
IEnumerable<T> üzerinde .Count çağırma	Derleme hatası	Count property'si IEnumerable'da yok	.Count() (LINQ) veya ICollection<T>

Hatalı Kod	Sonuç	Neden	Düzeltilmiş
foreach içinde list.Remove(x) dict["yok"] yazma List<T> boyutunu Count yerine Length sanma	InvalidOperationException KeyNotFoundException Derleme hatası	Enumerator geçersiz olur Anahtar yok Length diziler için, Count list'ler için	Tersten for veya LINQ Where TryGetValue list.Count
list.Add(x) sonra foreach içinde ekleme	Değişiklik yakalanmaz veya exception	Iterator geçersiz	Önce liste değişikliğini bitir

2.10.9 Anti-Pattern ve İdiyomatik Kullanım

Anti-Pattern	İdiyomatik	Neden Daha İyi
new ArrayList() (non-generic) Metot imzasında List<T> param	new List<T>() IEnumerable<T> param	Tip güvenliği, boxing yok Çağırana esneklik
Metot imzasında IEnumerable<T> return, hemen ardından .ToList().ToArray()	Sadece bir tanesi	Gereksiz enumerate
dict.ContainsKey(k) ? dict[k] : default	dict.TryGetValue(k, out var v) ? v : default	Tek sorgulama
Sıralı benzersiz veri için List<T> + Contains döngüsü	HashSet<T>	O(1) Contains

2.10.10 Bölüm Özeti

- Koleksiyon hiyerarşisi: IEnumerable<T> → ICollection<T> → IList<T> → List<T>.
- Benzetmeler: tren (sadece iler), kova (sayılıp değişir), dolap (indeksli).
- List<T> en yaygın; dinamik dizi + indeks + ekleme/silme.
- Dictionary<K,V> hash tabanlı anahtar-değer; TryGetValue güvenli erişim.
- Parametre imzasında en gevşek tipi tercih et (IEnumerable<T>).
- C# 12 koleksiyon ifadesi ([1, 2, 3]) sözdizimini birleştirir.

İlgili konular: §3.11 (LINQ IEnumerable<T> üzerinde), §3.12 (EF Core IQueryable<T>), §4.14 (View-Model'in koleksiyon property'leri).

2.11 String İşlemleri

2.11.1 Genel Bakış

string (System.String), Unicode karakterlerden oluşan **immutable** bir referans tipidir. Bir string oluşturulduktan sonra içeriği değiştirilemez; tüm “değiştirme” görünümlü metotlar (Replace, ToUpper, Trim, + ile birleştirme) yeni bir string üretir. Bu davranış, string'lerin tahmin edilebilir ve thread-safe olmasını sağlar; ancak yoğun manipülasyon senaryolarında StringBuilder tercih edilir.

Web uygulamalarında string, formdan gelen veriden HTML çıktısına, log kayıtlarından JSON serileştirmeye kadar her yerdedir. Bu bölüm günlük kullanımda karşılaşılan tüm operasyonları toplar.

2.11.2 Temel Özellikler ve Metotlar

```
string s = "Merhaba Dünya";

Console.WriteLine(s.Length);           // 13
Console.WriteLine(s.ToUpper());         // MERHABA DÜNYA
Console.WriteLine(s.ToLower());         // merhaba dünya
Console.WriteLine(s.Substring(0, 7));   // Merhaba
```

```

Console.WriteLine(s.Substring(8)); // Dünya
Console.WriteLine(s.IndexOf("D")); // 8
Console.WriteLine(s.Contains("bab")); // True
Console.WriteLine(s.Replace("Dünya", "Ali")); // Merhaba Ali
Console.WriteLine(s.StartsWith("Mer")); // True
Console.WriteLine(s.EndsWith("ünya")); // True

```

Çıktı:

```

13
MERHABA DÜNYA
merhaba dünya
Merhaba
Dünya
8
True
Merhaba Ali
True
True

```

Ana metot referansı:

Metot	Görevi	Örnek
Length	Karakter sayısı (property)	"abc".Length → 3
ToUpper(), ToLower()	Büyük/küçük harfe çevirir	"Ali".ToUpper() → "ALI"
Substring(baslangic)	Belirli konumdan sonuna	"abcdef".Substring(2) → "cdef"
Substring(baslangic, uzunluk)	Belirli aralık	"abcdef".Substring(1, 3) → "bcd"
IndexOf(deger)	İlk konum; yoksa -1	"Ali Veli".IndexOf("V") → 4
LastIndexOf(deger)	Son konum	"aba".LastIndexOf("a") → 2
Contains(deger)	İçeriyor mu?	"Merhaba".Contains("hab") → True
Replace(eski, yeni)	Tüm eşleşmeleri değiştirir	"aba".Replace("a", "x") → "xbx"
Trim()	Baştaki/sondaki boşlukları temizler	" x ".Trim() → "x"
TrimStart(), TrimEnd()	Sadece bir taraftan	" x ".TrimEnd() → "x"
Split(karakter)	Ayrıcıya göre parçalara böler	"a,b,c".Split(',') → ["a","b","c"]
PadLeft(n, karakter)	Sol dolgu	"5".PadLeft(3, '0') → "005"
PadRight(n, karakter)	Sağ dolgu	"5".PadRight(3, '.') → "5.."
StartsWith(x), EndsWith(x)	Prefix / suffix kontrolü	"file.txt".EndsWith(".txt") → True
string.IsNullOrEmpty(s)	null veya "" mı?	(statik metot)
string.IsNullOrWhiteSpace(s)	null, "" veya sadece boşluk mu?	(statik metot)
string.Join(ayirac, dizi)	Elemanları birleştirir	string.Join(", ", ["a","b"]) → "a, b"
string.Format(sablon, ...)	Formatlı string üretir	string.Format("{0}+{1}", 2, 3) → "2+3"
string.Concat(...)	Birleştirir (+ gibi)	string.Concat("a","b") → "ab"

2.11.3 String Birleştirme

Dört ana yol:

```

string ad = "Ali", soyad = "Yılmaz";

// 1) + operatörü

```

```
string a = ad + " " + soyad;

// 2) string.Concat
string b = string.Concat(ad, " ", soyad);

// 3) string.Format
string c = string.Format("{0} {1}", ad, soyad);

// 4) String interpolation ($) – modern C# tercihi
string d = $"{ad} {soyad}";
```

Hepsi aynı sonucu üretir. Modern kodda \$"..." interpolation en okunaklı ve tercih edilendir.

2.11.4 String Interpolation Detayları

```
int yas = 30;
double pi = 3.14159;
decimal fiyat = 19.99m;
DateTime bugun = new DateTime(2024, 6, 15);

Console.WriteLine($"Yaş: {yas}");
Console.WriteLine($"Pi: {pi:F2}"); // 2 ondalık
Console.WriteLine($"Fiyat: {fiyat:C}"); // para formatı
Console.WriteLine($"Tarih: {bugun:yyyy-MM-dd}");
Console.WriteLine($"Hizalı: [{yas,5}]"); // 5 karakter sağa hizalı
Console.WriteLine($"Sola hizalı: [{yas,-5}]"); // 5 karakter sola hizalı
```

Çıktı:

```
Yaş: 30
Pi: 3,14
Fiyat: ₺19,99
Tarih: 2024-06-15
Hizalı: [  30]
Sola hizalı: [30  ]
```

Format özellikleri iki nokta üst üste ile eklenir: {deger:format}. Hizalama virgülle: {deger,genislik} veya birlikte {deger,genislik:format}.

2.11.5 Split ve Join

```
string csv = "Ali,Veli,Ayşe,Fatma";
string[] adlar = csv.Split(',');
// ["Ali", "Veli", "Ayşe", "Fatma"]

foreach (var a in adlar)
    Console.WriteLine(a);

string birlestirilmis = string.Join(" | ", adlar);
// "Ali | Veli | Ayşe | Fatma"
Console.WriteLine(birlestirilmis);
```

Çıktı:

```
Ali
Veli
Ayşe
Fatma
Ali | Veli | Ayşe | Fatma
```

Boş elemanları atlamak için StringSplitOptions:

```
"a,,b,,c".Split(',', StringSplitOptions.RemoveEmptyEntries);
// ["a", "b", "c"]
```

2.11.6 Verbatim (@) ve Raw String (""") Literal'leri

Verbatim string (@): Kaçış karakterleri işlenmez; çok satırlı yazılabilir.

```
string yol = @"C:\Users\Ali\Documents\notlar.txt";
string sql = @"
    SELECT *
    FROM Kitaplar
    WHERE Fiyat > 100";
```

Raw string literal (C# 11+, """): Tırnak kaçırma sorunu olmayan büyük metin blokları için.

```
string json = """
{
    "ad": "Ali",
    "yas": 30
}
""";
```

Üç veya daha çok tırnakla açıp aynı sayıda tırnakla kapatılır; ilk satır boş bırakılabilir. HTML/JSON şablonları, SQL sorguları için idealdir.

2.11.7 StringBuilder — Yoğun Manipülasyon İçin

Bir string'i çok kez birleştirmek her seferinde yeni nesne üretir; bu, döngü içinde performansı düşürür. StringBuilder bir değiştirilebilir arabellek tutar.

```
using System.Text;

var sb = new StringBuilder();
for (int i = 0; i < 1000; i++)
    sb.Append($"Satır {i}\n");

string sonuc = sb.ToString();
```

sb.Append, Insert, Remove, Replace, Clear metotları vardır. Büyük string üretimi (log, CSV, HTML raporu) için tercih edilir; birkaç birleştirme için \$"..." daha okunaklıdır.

2.11.8 Karşılaştırma

```
string a = "Ali";
string b = "ali";

Console.WriteLine(a == b); // False
Console.WriteLine(a.Equals(b, StringComparison.OrdinalIgnoreCase)); // True
Console.WriteLine(string.Equals(a, b, StringComparison.CurrentCultureIgnoreCase)); // True

// Sıralama
int k = string.Compare("Ali", "Veli"); // negatif - "Ali" önce
```

StringComparison seçenekleri:

- Ordinal / OrdinalIgnoreCase: byte bazlı karşılaştırma, en hızlı, kültürsüz. Kod, dosya adı, key gibi karşılaştırmalar için.
- CurrentCulture / CurrentCultureIgnoreCase: kullanıcının kültürüne göre. UI'da görünen metinler için.
- InvariantCulture: kültürden bağımsız ama linguistic; kalıcı formatlar için.

2.11.9 Yaygın Hatalar

Hatalı Kod	Sonuç	Neden	Düzeltilmiş
s.Replace("a","b"); (dönüş atlanmış)	Değişiklik yok	String immutable	s = s.Replace(...)
Döngüde sonuc += yeni (binlerce kez)	Yavaş, GC baskısı	Her adımda yeni string	StringBuilder
s == null ? "" : s.Trim()	Uzun	Null-koşullu var	s?.Trim() ?? ""
s == "" (null olabilir)	NullPointerException	Null üzerinde == çalışır ama içeride crash riski	string.IsNullOrEmpty(s)
"Türkçe".ToLower() "TR" kültürde beklenmedik	'İ' → 'i' ama 'I' → 'i'	Kültür fark eder	ToLowerInvariant() veya StringComparison. OrdinalIgnoreCase

2.11.10 Anti-Pattern ve İdiyomatik Kullanım

Anti-Pattern	İdiyomatik	Neden Daha İyi
"a" + b + "c" + d + ... (uzun zincirler)	\$"a{b}c{d}"	Okunabilir
.ToLower().Contains(x.ToLower())	Contains(x, StringComparison. OrdinalIgnoreCase)	Ekstra tahsis yok
if (s != null && s != "") { }	if (!string.IsNullOrEmpty(s)) { }	Tek çağrı
string.Format("{0}...", x) her yerde	\$"..." interpolation	Modern sözdizim

2.11.11 Bölüm Özeti

- string immutable; tüm işlem yeni string üretir.
- Interpolation (\$"...") modern kodun standart birleştirilmesi.
- Substring, IndexOf, Split, Join, Replace, Trim en sık kullanılanlar.
- Verbatim (@) kaçış kapatır; raw string ("") büyük bloklar için.
- Yoğun manipülasyon için StringBuilder.
- Kültürsüz karşılaştırma için StringComparison.OrdinalIgnoreCase.

İlgili konular: §2.5.6 (ToString formatları), §4.11 (Tag Helper output), §4.19 (JSON string).

2.12 Metotlar ve Parametre Türleri

2.12.1 Genel Bakış

Metot (method), sınıfın davranışını ifade eden isimlendirilmiş kod bloğudur. Parametreler girdiyi, dönüş tipi çıktıyı belirtir. Metot imzası (signature) = ad + parametre tipleri; iki metot aynı isimli olsa da farklı parametre imzasıyla aynı sınıfta bir arada bulunabilir (overload).

2.12.2 Sözdizimi

```
[erişim] [modifier] donusTipi MetotAdi(parametreListesi)
{
    // gövde
    return sonuc;    // dönüş tipi void değilse
}
```

Örnekler:

```
public int Topla(int a, int b) => a + b; // expression-bodied

public void Selamla(string ad)
{
    Console.WriteLine($"Merhaba {ad}");
}

public static double DaireAlani(double r) => Math.PI * r * r; // static
```

2.12.3 Parametre Türleri

C#'ta bir parametre şu şekillerde tanımlanabilir:

Modifier	Anlam
(yok)	Değer ile geçirilir (referans tiplerde referansın kopyası)
ref	Referans ile geçirilir; çağrıdan önce başlatılmış olmalı
out	Çıkış parametresi; metot içinde mutlaka atanmalı, dönüşte değere sahip olur
in	Salt-okunur referans ile geçirilir (büyük struct'lar için performans)
params	Değişken sayıda argüman (varargs)

2.12.3.a Normal Parametre (Değer Geçişi) Değer tipleri için doğrudan kopya; referans tipleri için "referansın kopyası" geçirilir. Referansın işaret ettiği nesne dışarıdan da değiştirilebilir; ancak parametreyi yeni bir nesneye atamak dışarıyı etkilemez.

```
public void Sifirla(int x) { x = 0; }
int a = 5;
Sifirla(a);
Console.WriteLine(a); // 5 (değişmedi)

public void AdiDegistir(Kisi k) { k.Ad = "Yeni"; } // içerik değişir
public void KisiyiDegistir(Kisi k) { k = new Kisi(); } // dışarı etkilemez
```

2.12.3.b ref Parametresi Çağırılan değişkenin kendisini geçirir. Metot içinde yapılan atama dışarıya yansır.

```
public static void Takas(ref int x, ref int y)
{
    (x, y) = (y, x);
}

int a = 10, b = 20;
Takas(ref a, ref b);
Console.WriteLine($"{a}, {b}");
```

Çıktı:

20, 10

Kural: hem tanım hem çağrıda ref yazılır; çağrıdan önce değişken atanmış olmalıdır.

2.12.3.c out Parametresi Metotun çıkış parametresidir. Çağırmadan önce atanmış olması gerekmez; metot dönmeden mutlaka atanmalıdır. Metotun dönüş tipini birden fazla değere genişletmenin klasik yoludur.

```
public static bool BolmeGüvenli(int a, int b, out int sonuc)
{
    if (b == 0) { sonuc = 0; return false; }
```

```

        sonuc = a / b;
        return true;
    }

    if (BolmeGuvanlı(20, 4, out int r))
        Console.WriteLine($"Sonuç: {r}");
    else
        Console.WriteLine("Sıfıra bölme");

```

Çıktı:

Sonuç: 5

`int.TryParse(s, out int x)` bu desenin en tanınmış örneğidir. Modern C#'ta `out` değişkeni satır içinde bildirilir (`out int r`).

2.12.3.d params Parametresi Değişken sayıda argümanı bir dizi olarak toplar. Bir metodun sadece son parametresi olabilir.

```

public static int Topla(params int[] sayilar)
{
    int toplam = 0;
    foreach (var s in sayilar) toplam += s;
    return toplam;
}

```

```

Console.WriteLine(Topla(1, 2, 3));           // 6
Console.WriteLine(Topla(1, 2, 3, 4, 5));     // 15
Console.WriteLine(Topla(new int[] { 10, 20 })); // 30 – dizi doğrudan geçirilebilir

```

`string.Format("{0} {1}", ad, soyad)` ve `Console.WriteLine("{0}", ...)` çağrılarının arka planındaki `params object[]` bu mekanizmayı kullanır.

2.12.4 Metot Overload (Aşırı Yükleme)

Aynı isimli birden fazla metod, farklı parametre imzalarıyla:

```

public class Hesaplayici
{
    public int Topla(int a, int b) => a + b;
    public double Topla(double a, double b) => a + b;
    public string Topla(string a, string b) => a + b;
    public int Topla(int a, int b, int c) => a + b + c;
}

```

Derleyici, çağrındaki argüman tiplerine bakarak doğru overload'u seçer. Yalnızca dönüş tipi farklı overload **derlenmez** — imzada sayılmaz.

2.12.5 Varsayılan (Default) ve İsimlendirilmiş Parametreler

```

public void Kaydet(string ad, bool yedekle = true, int deneme = 3)
{
    // ...
}

Kaydet("veri.txt");           // yedekle=true, deneme=3
Kaydet("veri.txt", false);    // yedekle=false, deneme=3
Kaydet("veri.txt", deneme: 5); // yedekle=true, deneme=5
Kaydet("veri.txt", yedekle: false, deneme: 1);

```

İsimlendirilmiş parametreler, opsiyonelleri seçici geçme olanağı verir.

2.12.6 Expression-Bodied Members

Tek ifadelik metot ve property'ler için kısa sözdizim:

```
public double Alan() => Math.PI * Yaricap * Yaricap;
public string Ad => $"{IlkAd} {Soyad}";
public override string ToString() => $"{Ad} ({Yas})";
```

=> sonrası tek bir ifade yer alır; metot bunu döndürür. void metotlar için de kullanılabilir (void Yaz() => Console.WriteLine("x"));).

2.12.7 Yaygın Hatalar

Hatalı Kod	Hata Mesajı	Neden	Düzeltilmiş
Sifirla(a) (ref parametrelili metoda ref'siz çağrı)	CS1615: '...' cannot be passed as 'ref' argument	Çağrıda ref eksik	Sifirla(ref a);
out parametresi atanmadan return	CS0177: '...' must be assigned before control leaves the current method	Zorunlu atama yapılmadı	Metot çıkışından önce ata
Sirala(1,2,3,4) — params yok	Overload bulunamaz	Fazla argüman	params int[] x ekle
Sadece dönüş tipi farklı overload	CS0111: already defined a member '...'	İmzada dönüş tipi sayılmaz	Parametre imzasını değiştir

2.12.8 Bölüm Özeti

- Metot = imza + gövde. İmza: ad + parametre tipleri.
- Değer geçişi varsayılan; ref, out, in, params özel durumlar.
- out = çoklu dönüş için klasik desen; TryParse bunun örneğidir.
- params = varargs; sadece son parametrede.
- Overload = aynı isim, farklı imza. Dönüş tipi imzada değildir.
- Opsiyonel + isimlendirilmiş parametreler API'yi rahatlatır.
- => expression-bodied members tek ifadelik üyeler için kısayoldur.

İlgili konular: §2.5.4 (TryParse'ın out kullanımı), §2.13 (Tuple ile çoklu dönüş), §4.7 (Action metotları).

2.13 Tuple

2.13.1 Genel Bakış

Tuple, farklı tipteki değerleri geçici olarak birlikte taşıyan hafif bir veri yapısıdır. Bir metot tek bir dönüş değeri döndürebilir; birden fazla değer döndürmek için üç seçenek vardır: (a) out parametreleri (§2.12.3.c), (b) özel bir sınıf yazmak, (c) tuple. Küçük, iç kullanımlı durumlarda tuple; kalıcı model gerektiren durumlarda sınıf tercih edilir.

C# 7 ile birlikte gelen **value tuple** ((T1, T2, ...) sözdizimi), eski Tuple<T1, T2> sınıfının yerini almıştır. Value tuple bir struct'tır (değer tipi); daha hızlı, daha okunaklı ve **isimlendirilebilir**.

2.13.2 Sözdizimi

```
// Adsız tuple
(int, string) kayıt = (1, "Ali");
Console.WriteLine(kayıt.Item1); // 1
Console.WriteLine(kayıt.Item2); // Ali

// İsimlendirilmiş tuple
```

```
(int Id, string Ad) kisi = (1, "Ali");
Console.WriteLine(kisi.Id);           // 1
Console.WriteLine(kisi.Ad);           // Ali

// Metot dönüşü
(int min, int max) MinMax(int[] dizi) => (dizi.Min(), dizi.Max());

var mm = MinMax(new[] { 5, 2, 8, 1, 4 });
Console.WriteLine($"min={mm.min}, max={mm.max}");
```

Çıktı:

```
1
Ali
1
Ali
min=1, max=8
```

2.13.3 Deconstruction (Ayrıştırma)

Bir tuple'ı ayrı değişkenlere açmak:

```
var (min, max) = MinMax(new[] { 5, 2, 8, 1, 4 });
Console.WriteLine($"min={min}, max={max}");

// Tip belirtme
(int a, int b) = (10, 20);

// Discard (yoksay)
var (_, sadeceMax) = MinMax(dizi);
```

Sınıflar da deconstruction destekleyebilir; sınıf içinde Deconstruct metodu yazılır:

```
public class Nokta
{
    public int X { get; set; }
    public int Y { get; set; }
    public void Deconstruct(out int x, out int y) { x = X; y = Y; }
}

var n = new Nokta { X = 3, Y = 4 };
var (x, y) = n;
Console.WriteLine($"({x}, {y})"); // (3, 4)
```

2.13.4 Tuple Değişkeni Örneği (Katalog B “Tuples” slaytı)

Farklı tipteki değerleri tek değişkende taşıma:

```
(int Yas, string Ad, bool AktifMi) kullanıcı = (30, "Zeynep", true);

Console.WriteLine($"Ad: {kullanici.Ad}, Yaş: {kullanici.Yas}, Aktif: {kullanici.AktifMi}");
```

Çıktı:

```
Ad: Zeynep, Yaş: 30, Aktif: True
```

2.13.5 Tuple Karşılaştırma

Value tuple'lar bileşen-bileşen karşılaştırılır:

```
var t1 = (1, "Ali");
var t2 = (1, "Ali");
var t3 = (2, "Ali");
```

```
Console.WriteLine(t1 == t2); // True
Console.WriteLine(t1 == t3); // False
```

Bu, birden çok değerin toplu karşılaştırılması için pratik bir araçtır (özellikle pattern matching ile birlikte, §2.6.9).

2.13.6 Ne Zaman Tuple, Ne Zaman Sınıf?

Durum	Tercih	Gerekçe
Metotun iç kullanımlı çoklu dönüşü	Tuple	Ek tip tanımlamamak
Domain modeli, kalıcı veri DTO / ViewModel	Sınıf / record Sınıf	Anlamlı isim, validasyon Data annotations, model binding
LINQ ara sonuç API dönüş tipi (public)	Tuple / anonim tip Sınıf	Geçici Sözleşme netliği

record (C# 9+), tuple ile sınıf arası bir orta yoldur: sözdizimi kısa, semantiği değer eşitliği:

```
public record Nokta(int X, int Y);
var n1 = new Nokta(3, 4);
var n2 = new Nokta(3, 4);
Console.WriteLine(n1 == n2); // True (record değer eşitliği)
```

2.13.7 Yaygın Hatalar

Hatalı Kod	Sonuç	Neden	Düzeltilmiş
Public API dönüş tipi olarak tuple	Sözleşme muğlak	Item1 çağırını üzer	Sınıf veya record kullan
Tuple<int, string> (eski, referans tipi)	GC baskısı	Class-based	(int, string) value tuple
Named tuple isimlerinde tutarsızlık	Kafa karıştırıcı	(int Id, string Ad) vs sadece (int, string)	Tutarlı yaz

2.13.8 Bölüm Özeti

- Tuple, birden fazla değeri geçici olarak birlikte taşır.
- Value tuple (int, string) modern C#'ta klasik Tuple<> sınıfının yerini aldı.
- İsimlendirilmiş tuple, Item1/Item2 yerine anlamlı erişim sağlar.
- Deconstruction ile tek satırda çoklu değişken ataması yapılır.
- Kısa ömürlü iç kullanım için ideal; kalıcı model için sınıf/record.

İlgili konular: §2.12.3.c (out parametreleri), §2.6.9 (Tuple pattern), §4.14 (ViewModel — record alternatif).

2.14 Debug ve Watch Penceresi

2.14.1 Genel Bakış

Debug, çalışan bir programı adım adım izleyerek değişkenlerin nasıl değiştiğini görmeyi ve mantık hatalarını tespit etmeyi sağlar. Visual Studio'nun debug motoru, kod satırlarında **breakpoint** (kesme noktası) koyup çalıştırmayı orada durdurmayı, sonra tek satır tek satır ilerlemeyi mümkün kılar.

Bir web isteği (Controller action) debug ederken, request pipeline durur, ilgili breakpoint'e gelindiğinde IDE öne geçer ve tüm scope değişkenleri incelenebilir. Bu, MVC projelerinde en sık kullanılan hata bulma yöntemidir.

2.14.2 Anahtar Klavye Kısayolları

Tuş	İşlem	Türkçe Karşılığı
F5	Debug ile çalıştır / devam et	Continue
Ctrl + F5	Debug'sız çalıştır	Start without debugging
Shift + F5	Debug'ı durdur	Stop
F9	Aktif satıra breakpoint koy/kaldır	Toggle breakpoint
F10	Bir sonraki satıra geç (metot içine girmeden)	Step Over
F11	Bir sonraki satıra geç (metot içine gir)	Step Into
Shift + F11	Mevcut metottan çık	Step Out
Ctrl + Shift + F5	Debug'ı yeniden başlat	Restart

2.14.3 Breakpoint

Kırmızı daire ile işaretlenen bir kod satırı. Program o satıra geldiğinde durur; o ana kadar oluşmuş tüm değişken değerleri incelenebilir.

Koşullu breakpoint: Sağ tık → Conditions. Bir koşul (ör. $i > 100$) veya bir "hit count" (100. çalışmada dursun) belirlenebilir. Yalnız belirli bir kullanıcı için tetiklenmesi gereken hatalarda çok değerlidir.

Log point: Kod değiştirmeden output penceresine mesaj basmak için — dursun değil, sadece bildirsin.

2.14.4 Watch Penceresi

Debug modunda Debug → Windows → Watch → Watch 1 ile açılır. İzlemek istenen değişken adları veya ifadeler eklenir; program her durduğunda güncel değerleri görüntülenir.

Watch penceresinde:

- Basit değişken: x, kullanıcı
- Property zinciri: kullanıcı.Adres.Sehir
- İfade: sayılar.Count, list[0].Ad
- LINQ: list.Where(x => x.Yas > 18).Count() — dikkat, yan etki üretebilir; sadece güvenli sorgular yazın.

2.14.5 Diğer Debug Pencereleeri

Pencere	Ne gösterir
Locals	Mevcut scope'daki tüm yerel değişkenler
Autos	Otomatik: son iki satırda kullanılan değişkenler
Call Stack	Mevcut metoda nasıl geldiğinizin çağrı zinciri
Immediate Window	Debug ederken tek satır C# ifadesi yürütmek (? kullanıcı.Ad)
Output	Console.WriteLine çıktıları + IDE mesajları

2.14.6 Adım Adım Örnek

```
public int FaktoriyelHesapla(int n)
{
    int sonuc = 1; // F9 ile breakpoint bu satıra
    for (int i = 1; i <= n; i++)
    {
        sonuc *= i; // Watch'a: sonuc, i
    }
    return sonuc;
}
```

Debug adımları:

1. F9 — Breakpoint kur.
2. F5 — Debug'la başlat; FaktoriyelHesapla(5) çağrısı gelince ilk satırda dur.
3. **Watch'a ekle:** sonuc, i, n.
4. F10 — Bir satır ileri git. sonuc = 1, i görünmez henüz (döngüye girmedik).
5. F10 tekrar — döngü girişi.
6. F10 her tuşa bir adım; sonuc ve i değerlerinin nasıl evrildiği izlenir.

Alternatif: F5 ile durmadan koştur, sonraki breakpoint'e (varsa) atla.

2.14.7 ASP.NET Core MVC Debug İpuçları

- Controller action'ında breakpoint koyup IIS Express ile başlatın; formu submit ettiğinizde debugger devreye girer.
- ViewModel binding hatalarını tespit için `if (!ModelState.IsValid)` satırına breakpoint koyun ve `ModelState.Values.SelectMany(v => v.Errors)` ifadesini Watch'a ekleyin.
- EF Core sorgusunun ürettiği SQL'i görmek için, DbContext yapılandırmasına `LogTo(Console.WriteLine)` ekleyin (bkz. §3.10) veya debug ederken sorguyu Immediate window'da çalıştırıp `.ToString()` çağırın (EF Core 5+).

2.14.8 Bölüm Özeti

- Breakpoint = kırmızı daire, durak. F9 ile kur/kaldır.
- F5 başlat/devam, F10 step over, F11 step into, Shift+F11 step out.
- Watch penceresi = seçili değişken/ifadeleri her durumda tekrar değerlendirir.
- Locals, Autos, Call Stack, Immediate en sık kullanılan pencerelerdir.
- Web'de action'a breakpoint + ModelState incelemesi en yaygın MVC debug akışıdır.

İlgili konular: §4.7 (Controller action debug), §4.13 (Model binding hatalarını izleme).

2.15 Top-Level Statements, Random, using Direktifleri

2.15.1 Top-Level Statements

.NET 6+ ile birlikte bir konsol uygulamasının Program.cs dosyası artık Main metodu ve sınıf tanımı gerektirmez. Kod doğrudan dosyanın en üst seviyesine yazılır:

```
// Program.cs (.NET 6+ konsol)
Console.WriteLine("Merhaba dünya!");

int x = 5;
int y = 10;
Console.WriteLine($"Toplam: {x + y}");
```

Derleyici arka planda otomatik olarak Program sınıfını ve Main metodunu üretir; komut satırı argümanlarına args ile, awaitable dönüş return ile erişilebilir:

```
// args mevcut
if (args.Length > 0) Console.WriteLine($"İlk argüman: {args[0]}");

// async top-level de destekli
await Task.Delay(100);
return 0; // exit code
```

Bir projede en fazla bir dosya top-level statement kullanabilir; bu dosya "giriş noktası" olur. ASP.NET Core MVC şablonu Program.cs dosyasını bu şekilde kullanır (bkz. §4.3).

2.15.2 Klasik vs Top-Level Karşılaştırması

Klasik (.NET 5 ve öncesi):

```
using System;

namespace HelloWorld
```

```

{
    public class Program
    {
        public static void Main(string[] args)
        {
            Console.WriteLine("Merhaba dünya!");
        }
    }
}

```

Top-level (.NET 6+):

```
Console.WriteLine("Merhaba dünya!");
```

Derleme sonucu tıpatıp aynıdır; sadece boilerplate azalır.

2.15.3 using Direktifleri

using, bir dosyanın kullandığı namespace'leri (isim alanlarını) bildirir. Böylece System.Console.WriteLine yerine Console.WriteLine yazılabilir.

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;

```

Static using (C# 6+): Bir sınıfın statik üyelerine sınıf adı yazmadan erişim.

```

using static System.Console;
using static System.Math;

WriteLine("Merhaba");           // Console.WriteLine yerine
double r = Sqrt(2) * PI;         // Math.Sqrt(2) * Math.PI

```

Global using (C# 10+): Bir kez global using yazılırsa tüm projede geçerli. Genelde GlobalUsings.cs gibi bir dosyada toplanır.

```

// GlobalUsings.cs
global using System;
global using System.Collections.Generic;
global using System.Linq;
global using System.Threading.Tasks;

```

.NET 6+ SDK projelerinde <ImplicitUsings>enable</ImplicitUsings> proje ayarı, bu sık kullanılan namespace'leri otomatik ekler; MVC projelerinde varsayılan olarak açıktır.

using deyimi (IDisposable ile): Daha aşağıda §2.15.5'te.

2.15.4 Random Sınıfı

System.Random, sözde rastgele sayı üretir. Bir örnek üzerinden Next metotları çağrılır.

```

Random rnd = new Random();

int a = rnd.Next();           // 0 ... int.MaxValue
int b = rnd.Next(100);        // 0 ... 99
int c = rnd.Next(10, 21);      // 10 ... 20 (üst dahil değil)
double d = rnd.NextDouble();   // 0.0 ... 1.0

Console.WriteLine($"{a}, {b}, {c}, {d:F2}");

```

Beklenen çıktı (doğrulanmadı — her çalıştırmada farklı):

```
1893452817, 42, 15, 0,73
```

Kritik uyarı: `new Random()` çağrısını sıkı bir döngüde kullanmayın; aynı milisaniyede aynı seed'i alır ve aynı sayıyı üretebilir. Bir örneği paylaşın veya `Random.Shared` kullanın (.NET 6+):

```
int deger = Random.Shared.Next(1, 100); // thread-safe, tek örnek
```

Güvenlik notu: Kriptografik amaçlarla (parola, token, session ID) `Random` kullanılmaz. `Random-NumberGenerator` (`System.Security.Cryptography`) tercih edilir.

2.15.5 using Deyimi (IDisposable)

`IDisposable` interface'ini uygulayan tipler (dosya, ağ soketi, DB bağlantısı) yönetilmeyen kaynak tutabilir. `using` deyimi/bildirimi, blok bittiğinde `Dispose()` metodunu otomatik çağırır.

Klasik using bloğu:

```
using (var writer = new StreamWriter("dosya.txt"))
{
    writer.WriteLine("Merhaba");
} // burada writer.Dispose() otomatik çağrılır
```

Using declaration (C# 8+):

```
{
    using var writer = new StreamWriter("dosya.txt");
    writer.WriteLine("Merhaba");
} // scope bitince Dispose
```

EF Core `DbContext` de `IDisposable`'dır; ancak MVC projelerinde DI container `DbContext`'i **scoped** yaşamla yönetir ve `dispose`'u kendisi yapar. Elle `using` yazmak istisnadır; konsol projeleri veya arka plan işlerinde kullanılır.

2.15.6 Yaygın Hatalar

Hatalı Kod	Sonuç	Neden	Düzeltilmiş
Sıkı döngüde <code>var r = new Random(); r.Next();</code>	Aynı sayı tekrar tekrar	Aynı seed	<code>Random.Shared</code> veya bir kez oluştur
Kriptografik ID için <code>Random</code>	Tahmin edilebilir	Non-cryptographic	<code>RandomNumberGenerator.GetByte</code>
Bir projede iki dosyada top-level statements	CS8802: Only one compilation unit can have top-level statements	Tek giriş noktası olur	Yalnız bir dosyada
<code>IDisposable</code> üzerine <code>using</code> unutulmuş	Kaynak sızıntısı	<code>Dispose</code> çağrılmamış	<code>using var</code> veya <code>using (...)</code>

2.15.7 Bölüm Özeti

- Top-level statements: Program sınıfı ve Main metodu olmadan doğrudan üst seviyede kod. Proje tek dosya.
- `using` (direktifi): namespace import; static ve global varyantları vardır.
- ASP.NET Core MVC projelerinde global `using` ve implicit `using` varsayılan olarak açıktır.
- `Random` = sözde rastgele; kriptografide kullanılmaz. .NET 6+ ile `Random.Shared` thread-safe kolaylık.
- `using` (deyimi): `IDisposable` kaynaklarının otomatik temizliği.

İlgili konular: §4.3 (Program.cs top-level yapısı), §3.6 (`DbContext` yaşam yönetimi).

Bölüm III — Veritabanı ve Entity Framework Core

Bu bölüm, ilişkisel veritabanı temellerinden başlayıp ORM kavramına ve Entity Framework Core'un pratik kullanımına doğru ilerler. Amaç, bir MVC controller'ının `_context.Doktorlar.Include(d => d.Randevular).ToList()` gibi bir satır yazdığında arka planda neyin nasıl çalıştığını anlaşılır kılmaktır.

3.1 İlişkisel Veritabanı Temelleri

3.1.1 Genel Bakış

İlişkisel veritabanı, veriyi **tablolarda** saklayan bir modeldir. Her tablo, satır (row / kayıt) ve sütun (column / alan) yapısına sahiptir. Tablolar birbirine **anahtarlar** (key) ve **ilişkiler** üzerinden bağlanır.

Terim eşleştirmesi (Katalog C'deki ORM tablosu):

Veritabanı Dünyası	C# / OOP Dünyası
Tablo	Sınıf (Entity)
Satır	Nesne (Instance)
Sütun	Property
Foreign Key	Navigation Property
Primary Key	Id property'si (konvansiyon)
Veri Tipi (int, nvarchar, ...)	C# Tipi (int, string, ...)

Bir ORM'nin (§3.3) yaptığı iş, bu iki dünya arasında otomatik köprü kurmaktır.

3.1.2 Primary Key ve Foreign Key

- **Primary Key (birincil anahtar):** Bir tabloda her satırı benzersiz olarak tanımlayan sütun(lar). Genellikle Id (int) veya Guid. Boş olamaz.
- **Foreign Key (yabancı anahtar):** Başka bir tablonun birincil anahtarına referans veren sütun. İki tablo arasındaki bağlantıyı kurar.

Örnek — bir randevu tablosu:

Doktor tablosu:

```
+-----+
| Id | Ad      | Uzmanlik|
+-----+
|  1 | Dr. Ali | Dahiliye|
|  2 | Dr. Ayşe| Cildiye |
+-----+
```

Randevu tablosu:

```
+-----+-----+-----+-----+
| Id | Tarih   | HastaAdi | DoktorId|
+-----+-----+-----+-----+
|  1 | 2024-06-10 | Mehmet  | 1        | ← Dr. Ali'ye
|  2 | 2024-06-11 | Fatma   | 2        | ← Dr. Ayşe'ye
|  3 | 2024-06-12 | Zeynep  | 1        | ← Dr. Ali'ye
+-----+-----+-----+-----+
```

Randevu.DoktorId, Doktor.Id'ye referans veren foreign key'dir. Bu sayede bir doktora ait tüm randevular kolayca sorgulanır.

3.1.3 CRUD ve SQL

Her ilişkisel veritabanı üzerinde dört temel işlem yapılır — **CRUD** (Create, Read, Update, Delete). SQL karşılıkları:

İşlem	SQL
Create (ekle)	INSERT INTO Randevu (Tarih, HastaAdi, DoktorId) VALUES ('2024-06-10', 'Mehmet', 1)
Read (oku)	SELECT * FROM Randevu WHERE DoktorId = 1
Update (güncelle)	UPDATE Randevu SET HastaAdi = 'Ahmet' WHERE Id = 1
Delete (sil)	DELETE FROM Randevu WHERE Id = 3

EF Core, C# kodundan (`_context.Randevular.Add(r); await _context.SaveChangesAsync();`) bu SQL cümlelerini otomatik üretir. §3.13'te CRUD işlemleri detaylıca ele alınır.

3.1.4 Bölüm Özeti

- İlişkisel DB, veriyi tablolarda tutar; tablolar Primary/Foreign Key ile ilişkilendirir.
- Tablo ↔ sınıf, satır ↔ nesne, sütun ↔ property.
- CRUD dört temel işlem — SQL'de INSERT/SELECT/UPDATE/DELETE.
- ORM (§3.3), C# tarafından SQL üreterek bu ikiliği köprüler.

İlgili konular: §3.2 (İlişki türleri), §3.11 (LINQ = SQL'in C# karşılığı), §3.13 (CRUD).

3.2 İlişkiler ve Foreign Key Davranışları

3.2.1 Genel Bakış

Bir ilişkisel veritabanının gücü, tabloların yalıtık olmayışından — birbiriyle **ilişki** kurmasından — gelir. Üç ana ilişki türü vardır: **1-1** (bire bir), **1-N** (bire çok), **N-M** (çoka çok). Her ilişki, tablolar arasında foreign key(ler) üzerinden ifade edilir. EF Core, entity sınıflarındaki navigation property'lere ve [ForeignKey] etikitlerine (veya Fluent API'ye) bakarak bu ilişkileri otomatik üretir.

Bir de foreign key üzerindeki referans bütünlüğü kuralı vardır: bir doktor silinirse ona bağlı randevulara ne olur? İlgili tablo satırları da silinsin mi (CASCADE), silme reddedilsin mi (RESTRICT/NO ACTION), yoksa foreign key NULL'a mı çekilsin (SET NULL)?

3.2.2 Bire Bir (1-1) İlişki

Bir tarafın her satırı, karşı tarafın en fazla bir satırıyla eşleşir. En yaygın örnek: User ↔ UserProfile — her kullanıcının tam olarak bir profil kartı olur, o kart başka bir kullanıcıya ait değildir.

```
// Models/User.cs
public class User
{
    public int Id { get; set; }
    public string KullaniciAdi { get; set; }

    // Navigation
    public UserProfile Profil { get; set; }
}

// Models/UserProfile.cs
public class UserProfile
{
    public int Id { get; set; } // hem PK hem FK olabilir
    public string Bio { get; set; }
    public string ProfilResmi { get; set; }

    // Foreign key + navigation
    public int UserId { get; set; }
    public User User { get; set; }
}
```

Bu sıklıkla iki neden için kullanılır: (a) tabloları dikey bölme (nadir kullanılan alanları ayrı tabloya), (b) opsiyonel alan grubu — bir kullanıcının profili henüz oluşturulmamış olabilir.

3.2.3 Bire Çok (1-N) İlişki

En yaygın ilişki tipidir. Bir tarafın bir satırı, karşı tarafın birden fazla satırıyla eşleşebilir; ama karşı taraftan bakınca her satır tek bir “üst” satıra bağlıdır. Örnek: bir doktorun birden çok randevusu vardır; her randevu tek bir doktora aittir.

```
// Models/Doktor.cs
public class Doktor
{
    public int Id { get; set; }
    public string Ad { get; set; }
    public string Uzmanlik { get; set; }

    // Bir doktorun birden çok randevusu var
    public ICollection<Randevu> Randevular { get; set; } = new List<Randevu>();
}

// Models/Randevu.cs
public class Randevu
{
    public int Id { get; set; }
    public DateTime Tarih { get; set; }
    public string HastaAdi { get; set; }

    // Foreign key
    public int DoktorId { get; set; }

    // Navigation
    public Doktor Doktor { get; set; }
}
```

Diyagram:

Doktor'da `ICollection<Randevu>`, “birden çok” tarafını temsil eder. Randevu'da `DoktorId` foreign key'i ve Doktor navigation'ı, “tek” tarafın karşılığıdır. EF Core konvansiyonel olarak `<EntityAdi>Id` isimli property'yi foreign key olarak tanır (Katalog A'nın “DoktorID” adlandırma tavsiyesi).

SQL karşılığı:

```
CREATE TABLE Doktor (
    Id INT IDENTITY PRIMARY KEY,
    Ad NVARCHAR(100) NOT NULL,
    Uzmanlik NVARCHAR(100) NOT NULL
);

CREATE TABLE Randevu (
    Id INT IDENTITY PRIMARY KEY,
    Tarih DATETIME NOT NULL,
    HastaAdi NVARCHAR(100) NOT NULL,
    DoktorId INT NOT NULL,
    CONSTRAINT FK_Randevu_Doktor FOREIGN KEY (DoktorId) REFERENCES Doktor(Id)
);
```

3.2.4 Çoka Çok (N-M) İlişki

Her iki taraftan da “birden çok” ilişkilenebilir. Örnek: Öğrenci ↔ Ders — bir öğrenci birden çok derse yazılabilir; bir ders birden çok öğrenciye açık olabilir. İlişkisel modelde bu, bir **ara tablo** ile temsil edilir.

EF Core 5+ ile “join entity'siz” çoka çok:

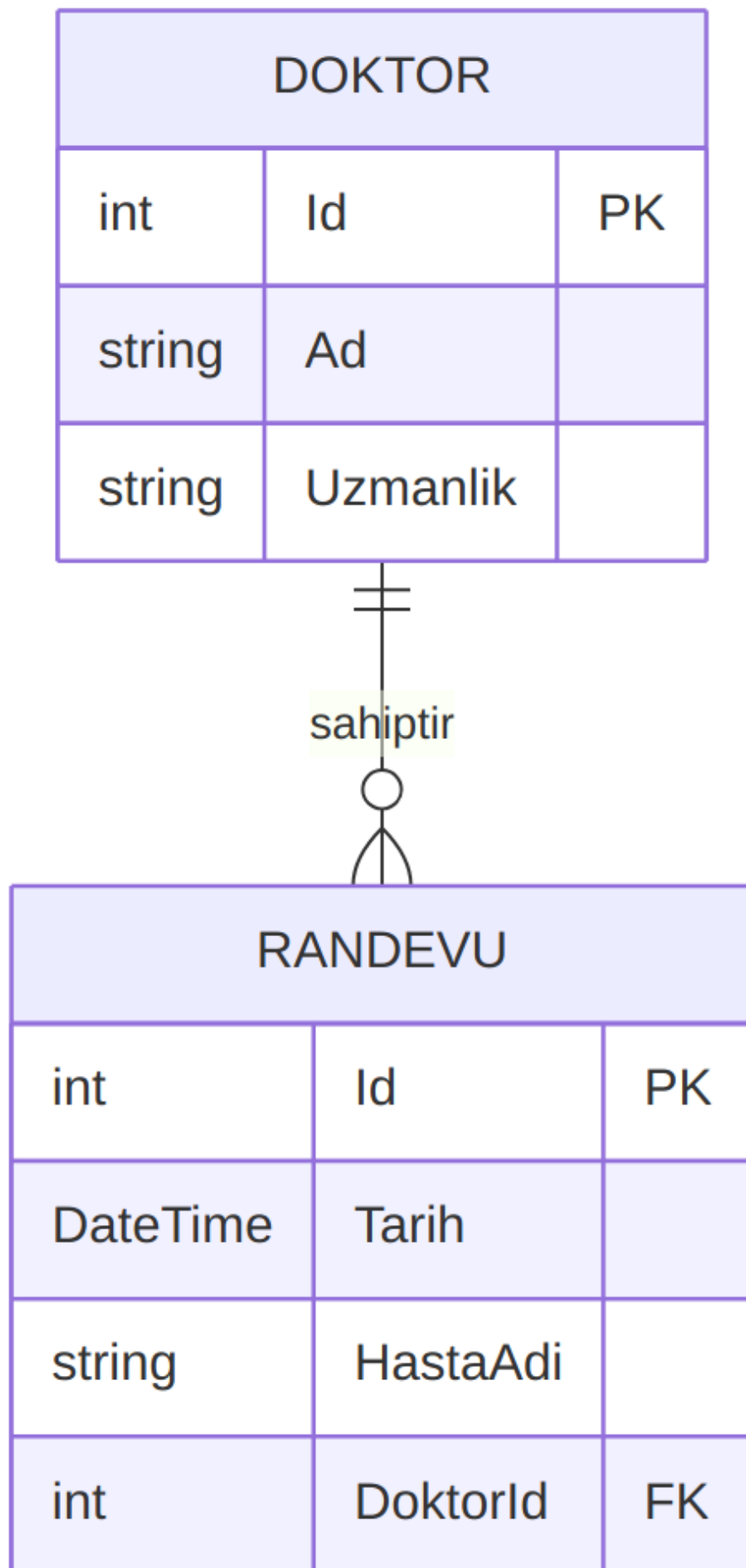


Figure 5: Diyagram 5

```
// Models/Ogrenci.cs
public class Ogrenci
{
    public int Id { get; set; }
    public string Ad { get; set; }
    public ICollection<Ders> Dersler { get; set; } = new List<Ders>();
}

// Models/Ders.cs
public class Ders
{
    public int Id { get; set; }
    public string Kod { get; set; }
    public string Ad { get; set; }
    public ICollection<Ogrenci> Ogrenciler { get; set; } = new List<Ogrenci>();
}
```

EF Core 5+ otomatik olarak OgrenciDers adlı ara tabloyu üretir (OgrenciId, DersId sütunları). Ek alan gerekmiyorsa bu kısa sözdizim yeterlidir.

Ara tablo alan taşıyorsa (kayıt tarihi, not gibi), açık join entity yazılır:

```
public class OgrenciDers
{
    public int OgrenciId { get; set; }
    public Ogrenci Ogrenci { get; set; }

    public int DersId { get; set; }
    public Ders Ders { get; set; }

    public DateTime KayitTarihi { get; set; }
    public int Not { get; set; }
}

// Ogrenci ve Ders sınıflarında
public ICollection<OgrenciDers> OgrenciDersleri { get; set; } = new List<OgrenciDers>();
```

Bu şekilde Ogrenci → OgrenciDers → Ders çift 1-N ilişkisiyle çoka çok modellenir.

Diyagram:

3.2.5 Foreign Key Silme Davranışları

Bir doktor silindiğinde randevularına ne olacağı, foreign key üzerindeki **on-delete** kuralıyla belirlenir. SQL Server'da dört seçenek vardır:

Kural	Davranış	Ne zaman
CASCADE	Parent silinirse child da otomatik silinir	Child parent'a mantıksal olarak bağımlıysa (sipariş kalemleri sipariş silinince)
RESTRICT / NO ACTION	Child varsa parent silinemez, hata verilir	Referans bütünlüğünü koruma; genel varsayılan güvenli seçim
SET NULL	Parent silinirse child'ın FK sütunu NULL olur	FK opsiyonelse; "kayıt kalsın ama bağlantı kopsun"
SET DEFAULT	Parent silinirse child'ın FK sütunu varsayılan değere döner	Nadir kullanılır

EF Core'da varsayılan davranış:

- **FK required (non-nullable)** ise: CASCADE

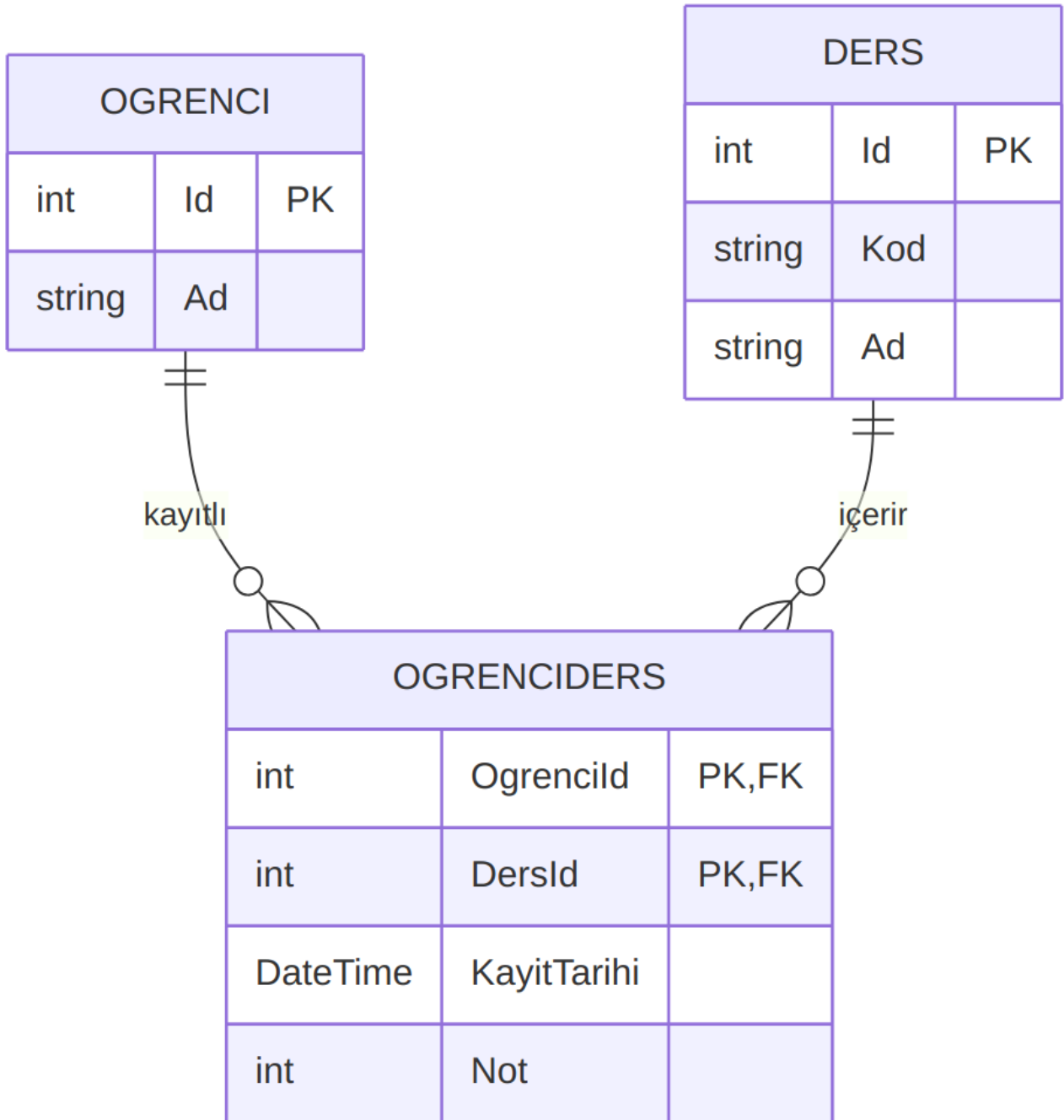


Figure 6: Diyagram 6

- FK **optional (nullable)** ise: SET NULL

Yani `public int DoktorId { get; set; }` (int, nullable değil) yazıldığında EF Core, “doktor silinirse randevular da silinsin” der. Bu istenmediğinde Fluent API ile değiştirilir:

```
// OnModelCreating içinde
modelBuilder.Entity<Randevu>()
    .HasOne(r => r.Doktor)
    .WithMany(d => d.Randevular)
    .HasForeignKey(r => r.DoktorId)
    .OnDelete(DeleteBehavior.Restrict); // veya SetNull, Cascade
```

3.2.6 Örnek 3.2.1 — Çift Yönlü Navigation ile 1-N

Tam bir hastane senaryosu:

```
public class Doktor
{
    public int Id { get; set; }
    public string Ad { get; set; }
    public string Uzmanlik { get; set; }
    public ICollection<Randevu> Randevular { get; set; } = new List<Randevu>();
}

public class Randevu
{
    public int Id { get; set; }
    public DateTime Tarih { get; set; }
    public string HastaAdi { get; set; }
    public int DoktorId { get; set; }
    public Doktor Doktor { get; set; }
}
```

Sorgular:

```
// Belirli doktorun randevuları – Include ile eager load
var dr = _context.Doktorlar
    .Include(d => d.Randevular)
    .FirstOrDefault(d => d.Id == 1);

foreach (var r in dr.Randevular)
    Console.WriteLine($"{r.Tarih:yyyy-MM-dd} - {r.HastaAdi}");

// Ters yön – bir randevunun doktoru
var rnd = _context.Randevular
    .Include(r => r.Doktor)
    .FirstOrDefault(r => r.Id == 5);

Console.WriteLine($"Doktor: {rnd.Doktor.Ad}");
```

Üretilen SQL (özet):

```
SELECT d.*, r.*
FROM Doktor d
LEFT JOIN Randevu r ON r.DoktorId = d.Id
WHERE d.Id = 1;
```

3.2.7 Yaygın Hatalar

Hatalı Kod	Sonuç	Neden	Düzeltilmiş
Navigation ICollection<T> başlatılmamış	NullReferenceException	Koleksiyon null	= new List<T>();
FK ve navigation isimleri uyumsuz	EF Core FK'yi bulamaz, shadow property üretir	Konvansiyon dışında	<Entity>Id konvansiyonu ya da [ForeignKey]
Required FK ama child'lar var iken parent silinmiyor	Microsoft.EntityFrameworkCore.DeleteException: The DELETE statement conflicted with the REFERENCE constraint	OnDelete(DeleteBehavior.Restrict) (veya tersi)	OnDelete(DeleteBehavior.Cascade) veya önce child'ları sil
N-M ara tabloya elle FK yazmak eskisi (EF 3.x tarzı)	Modern EF 5+'ta gereksiz	Konvansiyon değişti	Basit N-M için sadece iki ICollection

3.2.8 Anti-Pattern ve İdiyomatik Kullanım

Anti-Pattern	İdiyomatik	Neden Daha İyi
Her yerde CASCADE	Kritik silmelerde RESTRICT + kod tarafında kontrol	Yanlışlıkla toplu silmeleri önler
İlişki için elle join sütunları	Navigation property + Include	LINQ okunaklı
N-M için iki List<int> (ID koleksiyonu) tutma	Navigation property olarak ICollection<T>	EF izlemesi (change tracking) çalışır
FK'yi nullable yapıp SET NULL istememek	Non-nullable yap veya explicit OnDelete(Restrict)	Niyet net

3.2.9 Bölüm Özeti

- Üç ana ilişki: 1-1, 1-N (en yaygın), N-M.
- FK sütunu + navigation property = EF Core'un ilişkiyi anladığı yol.
- Konvansiyon: <EntityAdi>Id sütunu foreign key olarak tanınır.
- Silme davranışı: CASCADE / RESTRICT / SET NULL. Required FK için varsayılan CASCADE.
- EF Core 5+ ile çoka çok, join entity olmadan iki ICollection ile yazılabilir.
- Fluent API ile davranış özelleştirilir (bkz. §3.8).

İlgili konular: §3.6 (DbContext), §3.8 (Fluent API), §3.12 (Include / eager loading), §5.2 (Has-taneMVC ilişkileri).

3.3 ORM Kavramı ve Entity Framework Core'a Genel Bakış

3.3.1 Genel Bakış

ORM (Object-Relational Mapping / Nesne-İlişkisel Eşleme), C# gibi bir OOP dilinde çalışan uygulama ile SQL tabanlı ilişkisel veritabanı arasında **otomatik köprü** kuran bir yazılım katmanıdır. Uygulama tarafında sınıflar (entity), veritabanı tarafında tablolar bulunur; ORM bu iki dünyanın "sözlüğünü" tutar ve gerekli SQL komutlarını üretir.

ORM olmadan bir kayıt eklemek şu adımları gerektirir: SQL cümlesini string olarak yazmak, SqlConnection açmak, SqlCommand üretmek, parametreler bağlamak, ExecuteNonQuery çağırmak, DataReader'la sonucu okuyup elle nesneye eşlemek, bağlantıyı kapatmak. ORM ile aynı iş üç satır C#'a düşer:

```
var doktor = new Doktor { Ad = "Dr. Ali", Uzmanlik = "Dahiliye" };
_context.Doktorlar.Add(doktor);
await _context.SaveChangesAsync();
```

Arkada üretilen SQL:

```
INSERT INTO Doktor (Ad, Uzmanlik) VALUES (@p0, @p1);
SELECT SCOPE_IDENTITY();
```

3.3.2 ORM'in Katmanları

Bir ORM üç ana sorumluluk üstlenir:

1. **Mapping (Eşleme)**: Sınıf ↔ Tablo, property ↔ sütun, navigation ↔ FK eşlemesi.
2. **Query Translation (Sorgu Çevirisi)**: LINQ ifadelerini SQL'e çevirmek.
3. **Change Tracking (Değişiklik İzleme)**: Bellekteki entity'lerin durum değişikliklerini izleyip SaveChanges çağrısında uygun INSERT/UPDATE/DELETE üretmek.

.NET ekosisteminde başlıca ORM'ler: **Entity Framework Core** (Microsoft, tam ORM), **Dapper** (Stack Overflow, mikro-ORM — sadece nesne-veri eşlemesi, SQL elle yazılır), **NHibernate** (klasik ORM). Master defterin odağı EF Core'dur.

3.3.3 Nesne-Tablo Eşlemesi

Katalog C'deki eşleme tablosunun tam ve doğrulanmış hali:

İlişkisel DB	ORM (EF Core)
Tablo (Table)	Sınıf / Entity
Satır (Row)	Nesne (Instance)
Sütun (Column)	Property
Primary Key	Id property'si (konvansiyon) veya [Key]
Foreign Key	<Entity>Id property'si veya [ForeignKey]
İlişki (Relationship)	Navigation Property
SQL Sorgusu	LINQ Sorgusu
Veri Tipi (nvarchar, int, bit)	C# Tipi (string, int, bool)
NULL	Nullable tip (int?, string?)
Index	[Index] veya Fluent API HasIndex()
Unique Constraint	HasIndex().IsUnique()

3.3.4 EF Core Nedir?

Entity Framework Core, .NET için Microsoft'un modern, açık kaynak (MIT lisanslı), cross-platform ORM'idir. .NET 5 ile birlikte "EF Core" adını almış, önceki nesil "Entity Framework 6" (bkz. §3.4) ile farklı bir kod tabanı olarak devam eder.

Temel yetenekleri:

- **Kod öncelikli modelleme** (Code First — sınıflar önce, DB oluştur): §3.9.
- **Veritabanı öncelikli modelleme** (Database First — DB var, sınıfları scaffold et): §3.10.
- **Migration**: Şema evrimini kod olarak sürdürme.
- **LINQ sorguları**: C# ifadelerinin SQL'e otomatik çevirisi.
- **Change tracking**: Bellekteki değişikliklerin toplu commit'i.
- **Provider modeli**: SQL Server, PostgreSQL, MySQL, SQLite, Oracle, ve daha fazlası.

3.3.5 EF Core Mimarisi (Yüksek Seviye)

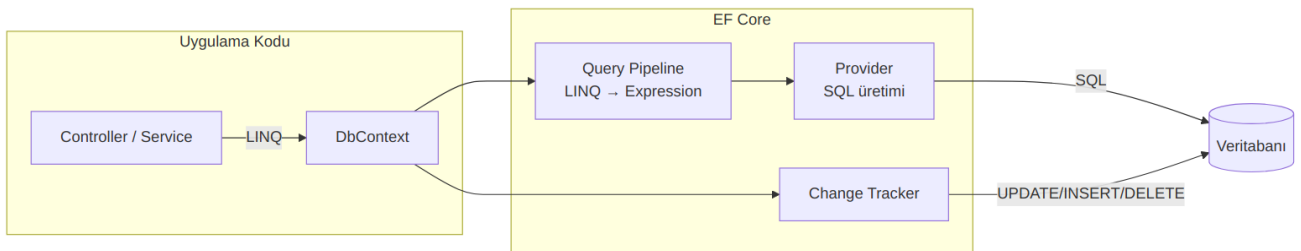


Figure 7: Diyagram 7

Uygulama kodu `_context.Doktorlar.Where(d => d.Uzmanlik == "Dahiliye")` yazar. EF Core:

1. Bu ifadeyi bir Expression Tree'ye (soyut sözdizim ağacı) çevirir.
2. Provider (SqlServer için Microsoft.EntityFrameworkCore.SqlServer) bu ağacı SQL'e derler.
3. SQL veritabanına gönderilir, sonuç SqlDataReader üzerinden okunur, entity nesnelere materyalize edilir.
4. Change tracker, döndürülen nesnelerin ilk durumunu tutar; sonraki değişiklikler SaveChanges'te UPDATE olarak yansır.

3.3.6 Bölüm Özeti

- ORM, OOP'nin nesneleri ile ilişkisel DB'nin tabloları arasında otomatik köprüdür.
- Üç sorumluluk: mapping, query translation, change tracking.
- EF Core, .NET için Microsoft'un modern, açık kaynak, cross-platform ORM'idir.
- Code First (§3.9) ve Database First (§3.10) iki ana çalışma modu.
- Uygulama LINQ yazar, EF Core SQL üretir.

İlgili konular: §3.4 (EF vs EF Core), §3.5 (Kurulum), §3.11 (LINQ), §3.13 (CRUD).

3.4 EF vs EF Core Karşılaştırması

3.4.1 Genel Bakış

.NET dünyasında iki farklı Entity Framework vardır ve karıştırılırlar:

- **Entity Framework 6 (EF6)**: 2008'de başlayan, .NET Framework (Windows-only) üzerinde çalışan klasik EF'in son sürümü. Bakımdadır, yeni özellik almaz.
- **Entity Framework Core (EF Core)**: 2016'da sıfırdan yazılan, cross-platform (.NET Core / .NET 5+), modern EF. Aktif geliştirilir.

Bir projede ikisi bir arada kullanılmaz. .NET 8 ile MVC projesi = EF Core kullanır. "EF" jargon olarak çoğu zaman EF Core'u kasteder.

3.4.2 Öne Çıkan Farklar

Özellik	EF 6	EF Core
Platform	Sadece .NET Framework (Windows)	Cross-platform (.NET Core, .NET 5+, Windows/Linux/macOS)
Model yaklaşımı	Code First, Database First, Model First (EDMX)	Code First, Database First (EDMX yok)
Performans	İyi	Genelde daha hızlı, hafif
Lisans	Microsoft (kapalı geliştirme dönemi vardı)	Açık kaynak, MIT
Konfigürasyon	XML tabanlı (edmx), Data Annotations, Fluent API	Data Annotations, Fluent API
Migration komutu (PMC)	Add-Migration, Update-Database	Add-Migration, Update-Database (benzer)
Migration komutu (CLI)	Yok	dotnet ef migrations add, dotnet ef database update
Bulk operation	Yok (extension gerekir)	ExecuteUpdate, ExecuteDelete (EF 7+)
Global query filter	Yok	Var (soft delete için ideal)
Alternate keys	Yok	Var
Shadow properties	Yok	Var
JSON kolon eşleme	Yok	Var (EF 7+)

3.4.3 Ne Zaman Hangisi?

- Yeni bir projede: **EF Core**. .NET 8 zaten cross-platform, EF Core en uygun EF sürümü.
- Var olan bir .NET Framework projesinde: **EF 6**, ta ki tüm proje .NET 8'e taşınana kadar.

3.4.4 Bölüm Özeti

- İki farklı ürün: EF6 (klasik, .NET Framework) ve EF Core (modern, cross-platform).
- .NET 8 + MVC = EF Core.
- Master defterin geri kalanı EF Core'a dayanır.

İlgili konular: §3.5 (EF Core kurulumu), §3.11 (LINQ).

3.5 EF Core Kurulumu ve NuGet Paketleri

3.5.1 Genel Bakış

EF Core, birden fazla NuGet paketinden oluşur. Minimum bir web MVC projesi için üç paket gerekir: **provider** (hedef veritabanı için), **design** (migration için tasarım araçları), ve **tools** (PMC komutları). Yerleşik ASP.NET Core proje şablonu bunları çoğunlukla otomatik ekler.

3.5.2 Zorunlu Paketler (SQL Server)

Package Manager Console'dan kurulum:

```
Install-Package Microsoft.EntityFrameworkCore.SqlServer
Install-Package Microsoft.EntityFrameworkCore.Tools
Install-Package Microsoft.EntityFrameworkCore.Design
```

.NET CLI ile kurulum:

```
dotnet add package Microsoft.EntityFrameworkCore.SqlServer
dotnet add package Microsoft.EntityFrameworkCore.Tools
dotnet add package Microsoft.EntityFrameworkCore.Design
```

Paketlerin sürümleri, projedeki .NET sürümü ile uyumlu olmalıdır (.NET 8 → EF Core 8.x). Sürümü açıkça belirtmek için -Version parametresi kullanılır.

3.5.3 Provider Paketleri (Alternatif Veritabanları)

Veritabanı	Paket
SQL Server / LocalDB	Microsoft.EntityFrameworkCore.SqlServer
PostgreSQL	Npgsql.EntityFrameworkCore.PostgreSQL
MySQL	Pomelo.EntityFrameworkCore.MySql
SQLite	Microsoft.EntityFrameworkCore.Sqlite
Oracle	Oracle.EntityFrameworkCore
In-Memory (test)	Microsoft.EntityFrameworkCore.InMemory

Master defterin geri kalanında SQL Server / LocalDB varsayılan alınır.

3.5.4 appsettings.json — Connection String

Veritabanı bağlantı bilgisi kod içinde değil, konfigürasyon dosyasında tutulur.

```
// appsettings.json
{
  "ConnectionStrings": {
    "DefaultConnection": "Server=(localdb)\\mssqllocaldb;Database=YOUR_DATABASE;Trusted_Connection=true;Password=;"
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  }
}
```

```
"AllowedHosts": "*"
}
```

Anahtar adı: DefaultConnection. EF Core ve ASP.NET Core Identity şablonları bu adı bekler. Farklı bir ad kullanılabilir, ama tutarlılık için DefaultConnection tercih edilir. Uygulamanın Program.cs’inde builder.Configuration.GetConnectionString("DefaultConnection") bu değeri okur.

Ortama özel değerler: appsettings.Development.json, appsettings.Production.json dosyaları ortama göre override eder. Üretim connection string’i genellikle Azure Key Vault, environment variable veya secrets manager’da tutulur — repository’ye commit edilmez.

Yaygın connection string bileşenleri (SQL Server):

Bileşen	Anlamı
Server=	SQL Server adresi. (localdb)\mssqllocaldb, .\SQLEXPRESS, tcp:my.host,1433
Database=	Kullanılacak veritabanı
Trusted_Connection=True	Windows Authentication
User Id=;Password=;	SQL Server Authentication
MultipleActiveResultSets=true	Aynı bağlantıdan çok sonuç okuma (MARS)
Encrypt=True;TrustServerCertificate=True	SSL bağlantı zorunluluğu (SQL Server 2022+)
Integrated Security=SSPI	Trusted_Connection eşdeğeri

3.5.5 Program.cs’ye DbContext Kaydı

```
// Program.cs
var builder = WebApplication.CreateBuilder(args);

// EF Core DbContext'i DI container'a kaydet
builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlServer(
        builder.Configuration.GetConnectionString("DefaultConnection")));

builder.Services.AddControllersWithViews();

var app = builder.Build();

// ... (middleware ve routing, bkz. §4.3)
app.Run();
```

AddDbContext<T> extension metodu, DbContext sınıfını **scoped** ömürle (bkz. §4.16) container’a ekler. Her HTTP request’i kendi DbContext örneğini alır; request bitince dispose edilir.

UseSqlServer(connectionString) provider’ı seçer. Alternatif provider’lar için UseNpgsql, UseSqlite vb.

3.5.6 Migration Araçları (dotnet-ef)

.NET CLI ile migration çalıştırabilmek için global araç kurulumu:

```
dotnet tool install --global dotnet-ef
# Güncelleme:
dotnet tool update --global dotnet-ef
```

Bu, dotnet ef ... komutlarını her yerden çağırabilmeyi sağlar. Package Manager Console (Visual Studio) kullanıcıları için ayrı bir kurulum gerekmez; Microsoft.EntityFrameworkCore.Tools paketi PMC komutlarını (Add-Migration, Update-Database) aktifleştirir.

3.5.7 Kurulum Sonrası Doğrulama

Package Manager Console’da:

Get-Help entityframework

.NET CLI'da:

```
dotnet ef --version
```

Beklenen çıktı:

```
Entity Framework Core .NET Command-line Tools  
8.0.x
```

3.5.8 Bölüm Özeti

- Üç zorunlu paket: EntityFrameworkCore.SqlServer (provider) + Tools (PMC) + Design (migration).
- Sürümler .NET sürümü ile uyumlu olmalı (.NET 8 ↔ EF Core 8.x).
- Connection string appsettings.json altında ConnectionStrings:DefaultConnection anahtarına yazılır.
- Program.cs'de AddDbContext<T> ile DI container'a kaydedilir; scoped ömür.
- dotnet ef global tool, CLI migration'ları için gereklidir.

İlgili konular: §3.6 (DbContext), §3.9 (Migration), §4.3 (Program.cs), §4.4 (appsettings.json detayı).

3.6 DbContext ve DbSet

3.6.1 Genel Bakış

DbContext, EF Core'un giriş noktasıdır. Bir DbContext örneği:

- Bir veritabanı bağlantısını temsil eder.
- Uygulamadaki tüm entity setlerinin (DbSet<T>) sahibidir.
- Change tracker'ı barındırır (bellekteki entity durumlarını izler).
- SaveChanges / SaveChangesAsync metotlarıyla toplu commit yapar.

Her uygulama, DbContext'ten türeyen kendi context sınıfını yazar (AppDbContext, HastaneDbContext, BulkyDbContext gibi). Bu sınıf, uygulamanın veri modelini toplu olarak tanıtır.

3.6.2 Sözdizimi

```
using Microsoft.EntityFrameworkCore;  
  
public class AppDbContext : DbContext  
{  
    public AppDbContext(DbContextOptions<AppDbContext> options)  
        : base(options)  
    {  
    }  
  
    // Her DbSet, bir tabloya karşılık gelir  
    public DbSet<Doktor> Doktorlar { get; set; }  
    public DbSet<Randevu> Randevular { get; set; }  
  
    // Fluent API konfigürasyonu (bkz. §3.8)  
    protected override void OnModelCreating(ModelBuilder modelBuilder)  
    {  
        base.OnModelCreating(modelBuilder);  
        // opsiyonel model yapılandırması  
    }  
}
```

- Constructor DbContextOptions<T> parametresi alır; DI container bu parametreyi AddDbContext çağrısındaki UseSqlServer(...) üzerinden doldurur.

- Her `DbSet<T>` property'si, T tipinin veritabanı tablosunu temsil eder. Property adı (Doktorlar) tablo adı olur (konvansiyon: çoğul olarak yazılır; EF Core adı olduğu gibi kullanır).

3.6.3 DbContext'in Yaşam Döngüsü

ASP.NET Core MVC'de DbContext yaşamı **scoped**'tir — her HTTP request'i kendi DbContext örneğini alır, request bitince dispose edilir.

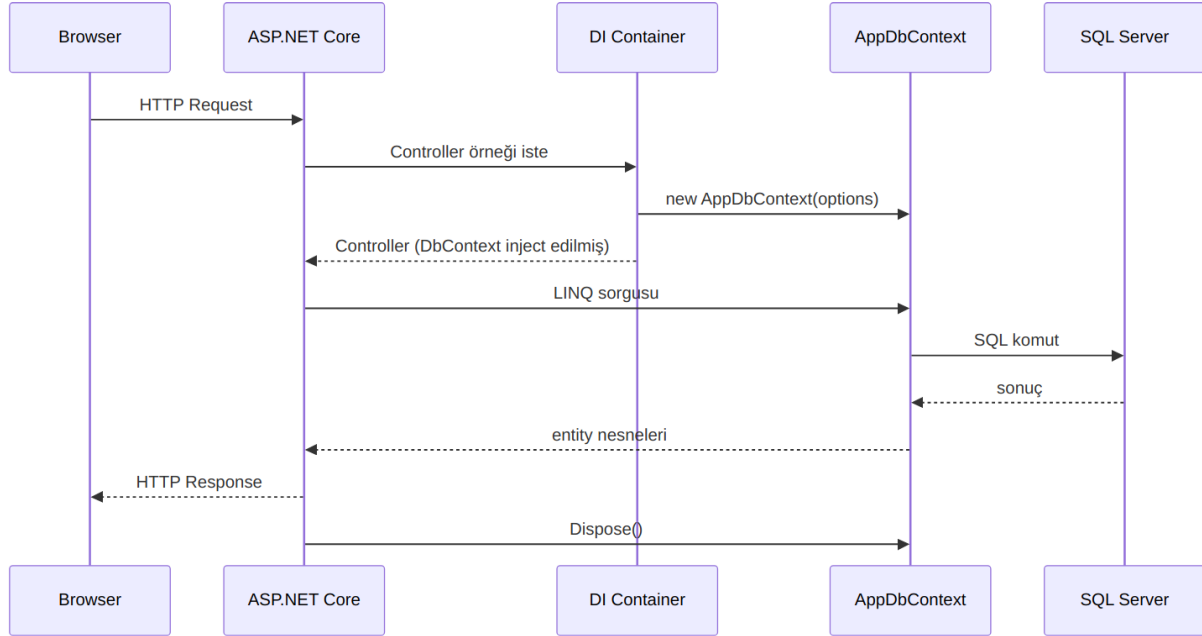


Figure 8: Diyagram 8

Kritik sonuçlar:

- **DbContext thread-safe değildir.** Aynı örneği iki thread'de paralel kullanmak `InvalidOperationException` üretir.
- **DbContext singleton olmaz.** Statik olarak tutulan bir DbContext, bellek sızıntısı ve stale data'ya yol açar.
- **DbContext manuel dispose edilmez** (MVC'de). DI dispose'u yönetir.

3.6.4 Örnek 3.6.1 — Basit DbContext ve Kullanım

Model:

```
// Models/Kategori.cs
public class Kategori
{
    public int Id { get; set; }
    public string Ad { get; set; }
    public int GosterimSirasi { get; set; }
}
```

DbContext:

```
// Data/AppDbContext.cs
public class AppDbContext : DbContext
{
    public AppDbContext(DbContextOptions<AppDbContext> options) : base(options) { }

    public DbSet<Kategori> Kategoriler { get; set; }
}
```

Program.cs kaydı:

```
builder.Services.AddDbContext<AppDbContext>(o =>
    o.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));
```

Controller'da constructor injection:

```
// Controllers/KategoriController.cs
public class KategoriController : Controller
{
    private readonly AppDbContext _context;

    public KategoriController(AppDbContext context)
    {
        _context = context;
    }

    public IActionResult Index()
    {
        var liste = _context.Kategoriler.ToList();
        return View(liste);
    }
}
```

`_context.Kategoriler.ToList()` çağrısında EF Core şu SQL'i üretir:

```
SELECT [k].[Id], [k].[Ad], [k].[GosterimSirasi]
FROM [Kategoriler] AS [k]
```

3.6.5 Change Tracker Durumları

Bellekteki her entity, DbContext tarafından şu beş durumdan birinde tutulur:

EntityState	Anlam	Ne zaman
Detached	DbContext izlemiyor	new ile üretilmiş, henüz Add çağrılmamış
Added	Yeni; SaveChanges'te INSERT	Add sonrası
Unchanged	Değişiklik yok	Veritabanından yüklenmiş, dokunulmamış
Modified	Değiştirilmiş; SaveChanges'te UPDATE	Property'si değiştirilmiş
Deleted	Silinecek; SaveChanges'te DELETE	Remove sonrası

```
var k = new Kategori { Ad = "Yeni" };
Console.WriteLine(_context.Entry(k).State); // Detached

_context.Kategoriler.Add(k);
Console.WriteLine(_context.Entry(k).State); // Added

await _context.SaveChangesAsync();
Console.WriteLine(_context.Entry(k).State); // Unchanged

k.Ad = "Güncel";
Console.WriteLine(_context.Entry(k).State); // Modified

await _context.SaveChangesAsync();
_context.Kategoriler.Remove(k);
Console.WriteLine(_context.Entry(k).State); // Deleted

await _context.SaveChangesAsync();
```

Change tracker, entity yüklenirken bir “orijinal değerler” fotoğrafı çeker. SaveChanges çağrısında mevcut durumla karşılaştırıp değişen alanlar için UPDATE üretir.

3.6.6 AsNoTracking() — İzlemesiz Sorgular

Sadece okuma yapılacaksa, change tracking’i devre dışı bırakmak performansı artırır (özellikle liste sayfalarında).

```
var liste = _context.Kategoriler
    .AsNoTracking()
    .ToList();
```

Kural: **UPDATE / DELETE yapmayacaksan AsNoTracking kullan.** Read-only bir listenin sadece view’a gösterilmesi durumunda idealdir.

3.6.7 Tablo İsimlendirme

Varsayılan davranış: DbSet<Doktor> Doktorlar { get; set; } → tablo adı **Doktorlar**. EF Core, DbSet property adını olduğu gibi kullanır; pluralization otomatik yapılmaz (EF6’nın aksine).

Tablo adını değiştirmek için:

Data Annotation (§3.7):

```
[Table("tblDoktor")]
public class Doktor { ... }
```

Fluent API (§3.8):

```
modelBuilder.Entity<Doktor>().ToTable("tblDoktor");
```

3.6.8 Birden Fazla DbContext

Bir projede birden fazla DbContext olabilir — örneğin domain’i modüllere bölmüşse (SalesDbContext, IdentityDbContext). Her biri kendi migration klasörüne sahiptir:

```
Add-Migration Init -Context SalesDbContext
Add-Migration Init -Context IdentityDbContext
```

Basit projelerde tek DbContext yaygındır.

3.6.9 Yaygın Hatalar

Hatalı Kod	Hata Mesajı	Neden	Düzeltilmiş
AddDbContext çağrılmamış, controller’da inject bekleniyor	Unable to resolve service for type 'AppDbContext'	DI kaydı eksik	Program.cs’e builder.Services.AddDbContext ekle
Aynı DbContext’i iki thread’de paralel kullanmak	InvalidOperationException: A second operation was started on this context instance before a previous operation completed	Thread-safe değil	Her thread için ayrı örnek (scoped)
SaveChanges sonrası entity’nin ID’si 0	Provider desteklemiyor veya Id PK olarak tanınmamış	Konvansiyon dışı isim	[Key] ekle
Farklı DbContext örneklerinden entity’yi karıştırmak	InvalidOperationException: The instance of entity type '...' cannot be tracked because another instance	İki context aynı entity’yi izlemeye çalışıyor	Tek context içinde çalış

Hatalı Kod	Hata Mesajı	Neden	Düzeltilmiş
Constructor'da new ApplicationDbContext()	Options yok	DbContext parametresiz constructor beklemez	DI ile inject al

3.6.10 Anti-Pattern ve İdiyomatik Kullanım

Anti-Pattern	İdiyomatik	Neden Daha İyi
Statik DbContext Her sorgu için new ApplicationDbContext() Read-only listede tracking Elle Connection.Open / Close SaveChanges'i her satır sonrası	DI ile scoped Constructor injection AsNoTracking() using gerekmez, EF yönetir Toplu SaveChanges	Thread safety, GC Aynı request'te aynı context Bellek + hız Kaynak sızıntısı yok Tek transaction, hızlı

3.6.11 Mini Sözlük

- **DbContext**: EF Core'un giriş noktası; bir bağlantıyı ve entity setlerini temsil eder.
- **DbSet**: Bir entity tipinin tablo karşılığı; sorgu ve komut kaynağıdır.
- **Change tracker**: Bellekteki entity durumlarını izleyen bileşen.
- **AsNoTracking**: Change tracking'i kapatan sorgu extension'ı.
- **EntityState**: Bir entity'nin change tracker'daki mevcut durumu (Added, Modified, ...).
- **Scoped lifetime**: DI'nin "her request için tek örnek" ömür politikası.

3.6.12 Bölüm Özeti

- DbContext, EF Core'un ana bileşeni; entity setlerini, change tracker'ı ve bağlantıyı tutar.
- ASP.NET Core MVC'de scoped ömürlü; her request bir DbContext alır.
- Thread-safe değildir; statik veya singleton yapılmaz.
- DbSet<T>, tabloyu temsil eder; adı tablo adı olur.
- Change tracker beş durum izler: Detached, Added, Unchanged, Modified, Deleted.
- Read-only sorgularda AsNoTracking() tercih edilir.

İlgili konular: §3.7 (Data Annotations), §3.8 (Fluent API), §3.13 (CRUD), §4.16 (DI ile inject), §5.1 (BulkDbContext), §5.2 (HastaneDbContext).

3.7 Data Annotations

3.7.1 Genel Bakış

Data Annotations, entity sınıflarındaki property'lerin **hem veritabanı** şemasını **hem de** model validation kurallarını **attribute'lar** ile bildiriminin yoludur. System.ComponentModel.DataAnnotations ve System.ComponentModel.DataAnnotations.Schema namespace'lerinde tanımlıdır. EF Core, migration üretirken bu attribute'ları okur ve şemayı ona göre şekillendirir; ASP.NET Core MVC ise model binding sonrasında validasyonu bunlar üzerinden çalıştırır (bkz. §4.15).

Bu ikili rol — hem şema hem doğrulama — annotations'ı çok ekonomik bir araç yapar; ancak karmaşık kural gerektiğinde Fluent API (§3.8) tercih edilir. Master defterin standardı: **basit kısıtlar** için Data Annotations, **karmaşık ilişki ve konfigürasyon** için Fluent API.

3.7.2 Sık Kullanılan Annotations

Şema ile ilgili (System.ComponentModel.DataAnnotations.Schema):

Attribute	Görevi
[Table("ad")]	Tablo adı belirler

Attribute	Görevi
[Column("ad", TypeName = "decimal(18,2)")]	Sütun adı ve/veya SQL tipi
[Key]	Primary key olarak işaretler (isim Id değilse)
[ForeignKey("NavAdi")] veya [ForeignKey("FKAdi")]	Navigation ↔ FK eşlemesini açıkça belirtir
[DatabaseGenerated(...)]	Identity, Computed, None
[NotMapped]	Bu property'yi tabloda oluşturma
[Index] (EF Core 5+)	Index üretir

Validasyon ile ilgili (System.ComponentModel.DataAnnotations):

Attribute	Görevi
[Required]	Zorunlu; NULL olamaz. Şemada NOT NULL, validasyonda zorunlu alan
[StringLength(max, MinimumLength = min)]	Karakter uzunluğu aralığı
[MaxLength(n)]	Sadece üst sınır
[MinLength(n)]	Sadece alt sınır
[Range(min, max)]	Sayısal aralık
[EmailAddress]	E-posta formatı
[Phone]	Telefon formatı (temel kontrol)
[RegularExpression("pattern")]	Regex ile format
[Compare("DigerProperty")]	Başka bir property ile eşitlik (parola tekrarı)
[Url]	URL formatı
[CreditCard]	Kredi kartı formatı (Luhn algoritması)
[DisplayName("Etiket")] veya [Display(Name = "Etiket")]	UI'da gösterilecek etiket
[DataType(DataType.Date)] vb.	HTML5 input tipi ipucu

3.7.3 Örnek 3.7.1 — Katalog Entity'si

```
// Models/Urun.cs
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

[Table("tblUrun")]
public class Urun
{
    [Key]
    public int Id { get; set; }

    [Required(ErrorMessage = "Ürün adı zorunlu.")]
    [StringLength(100, MinimumLength = 2, ErrorMessage = "2-100 karakter arası.")]
    [Display(Name = "Ürün Adı")]
    public string Ad { get; set; }

    [Required]
    [Range(0.01, 100000, ErrorMessage = "Fiyat 0,01 - 100.000 arasında olmalı.")]
    [Column(TypeName = "decimal(18,2)")]
    [Display(Name = "Birim Fiyatı")]
    public decimal Fiyat { get; set; }

    [Range(0, int.MaxValue)]
    public int Stok { get; set; }

    [DataType(DataType.Date)]
    [Display(Name = "Eklenme Tarihi")]
    public DateTime EklemeTarihi { get; set; } = DateTime.Now;
}
```

```
// Navigation + FK
public int KategoriId { get; set; }
[ForeignKey(nameof(KategoriId))]
public Kategori Kategori { get; set; }

[NotMapped]
public string EkOzet { get; set; } // veritabanına yazılmaz
}
```

Bu sınıftan üretilcek SQL Server tablosu (özet):

```
CREATE TABLE tblUrun (
    Id INT IDENTITY PRIMARY KEY,
    Ad NVARCHAR(100) NOT NULL,
    Fiyat DECIMAL(18,2) NOT NULL,
    Stok INT NOT NULL,
    EklemeTarihi DATETIME2 NOT NULL,
    KategoriId INT NOT NULL,
    CONSTRAINT FK_tblUrun_Kategoriler FOREIGN KEY (KategoriId) REFERENCES Kategoriler(Id)
);
```

EkOzet sütunu üretilmez çünkü [NotMapped] işaretlidir.

3.7.4 [Required] — Şema + Validasyon Etkisi

Bu attribute iki dünyada birden çalışır:

- **Şema:** EF Core, sütunu NOT NULL olarak üretir.
- **Validasyon:** ASP.NET Core MVC model binding sonrası, alan gelmemişse ModelState hata ekler.

Değer tipleri (int, DateTime, bool) zaten NOT NULL'dır; [Required] onlara **validasyon** amaçlıdır (ör. HTML form input'un boş bırakılmasını engellemek).

Referans tipleri (string, class) için hem şema hem validasyon etkisi vardır.

Nullable reference types (NRT) açıkken (bkz. §2.4.10):

- `public string Ad { get; set; }` — otomatik NOT NULL gibi davranır, [Required] beklenir.
- `public string? Ad { get; set; }` — NULL alabilir.

3.7.5 [StringLength] vs [MaxLength]

Sık karışan iki attribute:

Attribute	Etki
[StringLength(100)]	Şemada NVARCHAR(100); validasyonda maks 100 karakter
[MaxLength(100)]	Şemada NVARCHAR(100); validasyonda maks 100 karakter
[StringLength(100, MinimumLength = 5)] [MinLength(5)]	Ayrıca min uzunluk validasyonu Sadece alt sınır (validasyon)

Pratikte ikisi de benzer şema üretir; StringLength daha eski, MaxLength/MinLength daha yeni ve semantik olarak nettir. Modern kodda [MaxLength] + [MinLength] kombinasyonu tercih edilir.

3.7.6 [ForeignKey] — İki Yönlü Bildirim

Konvansiyon dışı isim kullanıldığında navigation'ın hangi FK sütununa karşılık geldiğini açıkça belirtmek için:

FK üzerinde:

```
public int DrId { get; set; }
[ForeignKey(nameof(DrId))]
public Doktor Doktor { get; set; }
```

Navigation üzerinde (aynı etki):

```
public int DrId { get; set; }
[ForeignKey(nameof(Doktor))]
public Doktor Doktor { get; set; }
```

nameof kullanmak, refactoring'te (property adı değişirse) hata görünmesini sağlar.

3.7.7 [Column(TypeName = "...")]

SQL tipini elle belirtmek için:

```
[Column(TypeName = "decimal(18,2)")]
public decimal Fiyat { get; set; }

[Column(TypeName = "date")]
public DateTime DogumTarihi { get; set; }

[Column(TypeName = "nvarchar(max)")]
public string Aciklama { get; set; }
```

Para tutarları için decimal(18,2) (16 tam basamak + 2 ondalık) standart tercihtir. Kısa metin: nvarchar(N); sınırsız/uzun metin: nvarchar(max); tarih (saat gerekmezse): date.

3.7.8 [DatabaseGenerated]

Sütun değerinin nasıl üretileceğini belirler:

Değer	Anlam
Identity	Kayıt oluşturulurken veritabanı otomatik değer atar (IDENTITY sütunu). Int PK'ler için varsayılan.
Computed	Bir başka sütuna göre hesaplanır (computed column).
None	Uygulama tarafından atanır.

```
[DatabaseGenerated(DatabaseGeneratedOption.Identity)]
public int Id { get; set; }

[DatabaseGenerated(DatabaseGeneratedOption.None)]
public string HesapNumarasi { get; set; } // uygulama üretir (örn. Guid)
```

3.7.9 Örnek 3.7.2 — Login Form Modeli (Validasyon Odaklı)

```
// ViewModels/LoginVM.cs
public class LoginVM
{
    [Required(ErrorMessage = "E-posta zorunlu.")]
    [EmailAddress(ErrorMessage = "Geçerli e-posta girin.")]
    [Display(Name = "E-posta")]
    public string Email { get; set; }

    [Required(ErrorMessage = "Parola zorunlu.")]
    [DataType(DataType.Password)]
    [StringLength(50, MinimumLength = 6, ErrorMessage = "Parola 6-50 karakter olmalı.")]
    public string Parola { get; set; }
```

```
[Display(Name = "Beni Hatırla")]
public bool BeniHatirla { get; set; }
}
```

Bu bir “entity” değil, sadece form model — bu nedenle NotMapped’e gerek yok; DbSet<LoginVM> de olmayacak. ViewModel’lerde şema attribute’ları (Table, Column) yer almaz; sadece validasyon attribute’ları anlamlıdır.

3.7.10 Yaygın Hatalar

Hatalı Kod	Sonuç	Neden	Düzeltilmiş
[Required] public int Yas { get; set; }	Şema değişmez (int zaten NOT NULL) ama validasyon HTML5 required tetiklenmez	Değer tipi zaten zorunlu	Nullable yap (int?) veya kaldır
[StringLength(int.MaxValue)]	nvarchar(2147483647) ve migration hatası	Uzun metin için yanlış	[Column(TypeName = "nvarchar(max)")]
[Range(0, int.MaxValue)] decimal’e	Runtime tip uyumsuzluğu	Range int overload	[Range(0, (double)decimal.MaxValue)] veya Fluent API
[ForeignKey] sadece bir tarafta ama tarafındaki isim yanlış	EF shadow property üretir	Konvansiyon bozuk	İki tarafın da adları uyumlu

3.7.11 Bölüm Özeti

- Data Annotations, entity property’lerine attribute ile hem şema hem validasyon bildiriminin yolu.
- İki namespace: Schema (tablo/sütun/FK) ve validasyon (Required, StringLength, Range, ...).
- [Required] string üzerinde şema + validasyon; değer tipinde sadece validasyon.
- [Column(TypeName = "decimal(18,2)")] para için standart.
- [NotMapped] sütun oluşturmaz.
- Karmaşık kurallar Fluent API’ye (§3.8) taşınır.

İlgili konular: §3.8 (Fluent API), §4.15 (Model Validation), §3.14 (Unique Constraint), §5.1 (BulkWeb Kategori annotation’ları).

3.8 OnModelCreating ve Fluent API

3.8.1 Genel Bakış

Fluent API, DbContext.OnModelCreating(ModelBuilder modelBuilder) metodunun içinde, method chaining sözdizimiyle model konfigürasyonu yazmanın yoludur. Data Annotations’ın (§3.7) tüm yeteneklerini üstlenir ve fazlasını sağlar:

- Çoklu sütun index’i
- Karmaşık ilişki konfigürasyonları
- Query filter (global soft-delete filtresi)
- Seed data (HasData)
- Composite (birleşik) primary key
- Value converter’lar

Data Annotations vs Fluent API — hangisi ne zaman:

Durum	Tercih
Basit Required, MaxLength, Range	Data Annotations (entity içinde yakın)
Composite key, çoklu index, karmaşık ilişki	Fluent API

Durum	Tercih
Entity POCO'nun EF Core'a bağımlı olmasını istememek	Fluent API
ViewModel/DTO validasyonu	Data Annotations
Seed data	Fluent API

3.8.2 OnModelCreating Şablonu

```
public class AppDbContext : DbContext
{
    public AppDbContext(DbContextOptions<AppDbContext> options) : base(options) { }

    public DbSet<Doktor> Doktorlar { get; set; }
    public DbSet<Randevu> Randevular { get; set; }

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);

        // Entity konfigürasyonları
        modelBuilder.Entity<Doktor>(entity =>
        {
            entity.ToTable("tblDoktor");
            entity.HasKey(d => d.Id);
            entity.Property(d => d.Ad).IsRequired().HasMaxLength(100);
            entity.Property(d => d.Uzmanlik).IsRequired().HasMaxLength(50);

            entity.HasMany(d => d.Randevular)
                .WithOne(r => r.Doktor)
                .HasForeignKey(r => r.DoktorId)
                .OnDelete(DeleteBehavior.Restrict);
        });

        modelBuilder.Entity<Randevu>(entity =>
        {
            entity.ToTable("tblRandevu");
            entity.HasKey(r => r.Id);
            entity.Property(r => r.HastaAdi).IsRequired().HasMaxLength(100);
        });
    }
}
```

Her Entity<T> çağrısı, o tipin konfigürasyon başlangıcıdır. Chained çağrılarla property'ler, ilişkiler, index'ler, ve tablo düzeyi seçenekler bildirilir.

3.8.3 Property Konfigürasyonu

Metot	Görevi
HasKey(x => x.Id)	Primary key belirler
Property(x => x.Ad).IsRequired()	NOT NULL yapar
.HasMaxLength(100)	Maks uzunluk
.HasColumnName("kolonAdi")	Sütun adı
.HasColumnType("nvarchar(max)")	SQL tipi
.HasDefaultValue(...)	Varsayılan değer (şemada)
.HasDefaultValueSql("GETDATE()")	Varsayılan değer SQL ifadesiyle
.HasComputedColumnSql("[Fiyat] * [Miktar]")	Hesaplanmış sütun
.ValueGeneratedOnAdd()	Sadece INSERT'te üretilir
.HasConversion<string>()	Value converter (enum → string gibi)

3.8.4 İlişki Konfigürasyonu

Bir doktor-randevu ilişkisini tamamen Fluent API ile:

```
modelBuilder.Entity<Randevu>()
    .HasOne(r => r.Doktor)           // Randevu'nun bir Doktor'u var
    .WithMany(d => d.Randevular)     // Doktor'un çok Randevu'su
    .HasForeignKey(r => r.DoktorId)  // FK sütunu
    .OnDelete(DeleteBehavior.Restrict); // silme davranışı
```

Diğer ilişki türleri:

- **1-1**: .HasOne(...).WithOne(...)
- **N-M** (join entity): .HasMany(...).WithMany(...).UsingEntity<TJoin>(...)

OnDelete seçenekleri:

- DeleteBehavior.Cascade — parent silinirse child'lar da silinir
- DeleteBehavior.Restrict — child varsa parent silinemez
- DeleteBehavior.SetNull — child'ın FK'sı NULL olur (FK nullable olmalı)
- DeleteBehavior.NoAction — DB tarafında hiçbir işlem, sorumluluk uygulamada

3.8.5 Index ve Unique Constraint

```
// Tek sütun index
modelBuilder.Entity<Doktor>()
    .HasIndex(d => d.Ad);

// Unique index
modelBuilder.Entity<Kullanici>()
    .HasIndex(k => k.Email)
    .IsUnique();

// Birleşik index
modelBuilder.Entity<Randevu>()
    .HasIndex(r => new { r.DoktorId, r.Tarih });

// İsim + filtre
modelBuilder.Entity<Urun>()
    .HasIndex(u => u.Kod)
    .HasDatabaseName("IX_Urun_Kod")
    .HasFilter("[Kod] IS NOT NULL");
```

Katalog B'de gösterilen kullanıcı adı benzersiz kısıtı bu şekilde ifade edilir. Unique index, aynı değere sahip ikinci bir kayıt eklenmesini SQL Server düzeyinde engeller (bkz. §3.14).

3.8.6 Global Query Filter (Soft Delete İçin İdeal)

Bir entity'nin **her sorgusuna** otomatik eklenecek filtreyi tanımlar. En yaygın kullanım: silinen kayıtları listelerde göstermemek.

```
modelBuilder.Entity<Doktor>()
    .HasQueryFilter(d => !d.IsDeleted);
```

Bu sonrasında `_context.Doktorlar.ToList()` çağrısı otomatik olarak `WHERE IsDeleted = 0` filtresini SQL'e ekler. Silinmiş kayıtları görmek için `IgnoreQueryFilters()` kullanılır:

```
var hepsi = _context.Doktorlar.IgnoreQueryFilters().ToList();
```

Soft delete deseni §4.21'de tam olarak ele alınır.

3.8.7 Seed Data — HasData

Migration çalıştığında tabloya sabit satırlar eklemek için:

```
modelBuilder.Entity<Kategori>().HasData(
    new Kategori { Id = 1, Ad = "Kitap",      GosterimSirasi = 1 },
    new Kategori { Id = 2, Ad = "Dergi",      GosterimSirasi = 2 },
    new Kategori { Id = 3, Ad = "Ders Kitabı", GosterimSirasi = 3 }
);
```

- Id değerleri elle verilir; migration bunları sabit değer olarak INSERT eder.
- Sonradan HasData içeriğini değiştirmek, yeni bir migration üretir; ekleme/silme farkı otomatik hesaplanır.

3.8.8 Composite Key (Birleşik Anahtar)

Bir tabloda PK birden fazla sütundan oluşuyorsa (klasik ara tablolarda):

```
modelBuilder.Entity<OgrenciDers>()
    .HasKey(od => new { od.OgrenciId, od.DersId });
```

Anonim tip (new { ... }) sözdizimi birleşik anahtarın standart ifadesidir.

3.8.9 Değer Dönüştürücü (Value Converter)

Örneğin bir enum'u tabloda string olarak saklamak:

```
public enum Durum { Beklemede, Onaylandi, Iptal }

public class Randevu
{
    public int Id { get; set; }
    public Durum Durum { get; set; }
}

// OnModelCreating
modelBuilder.Entity<Randevu>()
    .Property(r => r.Durum)
    .HasConversion<string>(); // int yerine "Beklemede", "Onaylandi", "Iptal"
```

Diğer yaygın dönüşümler: JSON serialize edip nvarchar(max) sütunda saklamak, DateTime ↔ long (Unix timestamp).

3.8.10 IEntityConfiguration İle Ayırma

Büyük projelerde OnModelCreating şişer. Her entity'nin konfigürasyonunu ayrı sınıfa çıkarmak için IEntityConfiguration<T> uygulanır:

```
// Data/Configurations/DoktorConfiguration.cs
public class DoktorConfiguration : IEntityConfiguration<Doktor>
{
    public void Configure(EntityTypeBuilder<Doktor> builder)
    {
        builder.ToTable("tblDoktor");
        builder.Property(d => d.Ad).IsRequired().HasMaxLength(100);
        builder.HasIndex(d => d.Ad);
    }
}

// AppDbContext.cs
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.ApplyConfigurationsFromAssembly(typeof(AppDbContext).Assembly);
}
```

ApplyConfigurationsFromAssembly, assembly'deki tüm IEntityConfiguration<T> uygulayıcılarını bulur ve otomatik uygular. Endüstriyel projelerin standart yaklaşımıdır.

3.8.11 Bölüm Özeti

- Fluent API, OnModelCreating içinde method chaining ile konfigürasyon.
- Data Annotations'ın süperseti; karmaşık senaryolarda tercih edilir.
- İlişkiler HasOne / HasMany / WithOne / WithMany / HasForeignKey / OnDelete ile.
- HasIndex().IsUnique() unique index üretir.
- HasQueryFilter global filtre — soft delete için ideal.
- HasData seed migration üretir.
- Büyük projelerde konfigürasyon IEntityConfiguration<T> ile ayrılır.

İlgili konular: §3.7 (Data Annotations), §3.14 (Unique), §4.21 (Soft Delete), §5.2 (HastaneMVC konfigürasyonları).

3.9 Code First ve Migration

3.9.1 Genel Bakış

Code First yaklaşımında sınıflar (entity'ler) önce yazılır; veritabanı, bu sınıflardan otomatik olarak türetilir. Yeni bir alan eklendiğinde, ilişki değiştiğinde şema evrimini yönetmek için EF Core **migration** mekanizmasını kullanır: her değişikliği versiyonlanmış bir “değişiklik dosyası” olarak üretir, üst üste ekleyerek veritabanını hedef sürüme getirir.

Migration üç dosya üretir:

- <Zaman>_<AdiniVerdigin>.cs — Up (uygula) ve Down (geri al) metotları.
- <Zaman>_<AdiniVerdigin>.Designer.cs — model snapshot bilgisi.
- ApplicationDbContextModelSnapshot.cs — mevcut modelin durumu.

3.9.2 Temel İş Akışı



Figure 9: Diyagram 9

3.9.3 PMC (Package Manager Console) Komutları

Visual Studio → **Tools** → **NuGet Package Manager** → **Package Manager Console**. Bir proje varsayılan olarak seçili olmalı (üstteki Default project dropdown).

Yeni migration üret:

Add-Migration InitialCreate

Bu, uygulanmamış tüm model değişikliklerini bir migration dosyası olarak Migrations/ klasörüne yazar. InitialCreate isim örneğidir; anlamlı isim koyulmalı (AddDoktorTable, ChangeFiyatDecimal18_2, SeedKategoriData).

Veritabanına uygula:

Update-Database

Uygulanmamış tüm migration'ları sırasıyla çalıştırır. Veritabanı yoksa oluşturur, varsa __EFMigrationsHistory tablosuna bakarak son uygulanan migration'ı bulur.

Belirli bir sürüme geri gitmek:

Update-Database -Migration InitialCreate

Belirtilen migration'ı hedef alır. Şu anki durumdan hedefe göre ileri veya geri gider (Down metotlarını çalıştırır).

Tüm migration'ları geri al (veritabanını sıfırla):

```
Update-Database -Migration 0
```

Bunun ardından Remove-Migration ile son migration dosyası silinebilir.

Son migration'ı silmek (henüz DB'ye uygulanmadıysa):

```
Remove-Migration
```

Zaten DB'ye uygulanmışsa önce Update-Database -Migration <önceki> ile geri alınmalı, sonra dosya silinir.

Migration'dan SQL script üretmek (DB'ye direkt gitmeden):

```
Script-Migration
```

```
Script-Migration -From MigrationX -To MigrationY
```

Bu, production ortamına manuel uygulanacak SQL'i üretir.

3.9.4 .NET CLI Komutları

Alternatif olarak dotnet ef global tool ile:

```
# Yeni migration
```

```
dotnet ef migrations add InitialCreate
```

```
# Uygula
```

```
dotnet ef database update
```

```
# Belirli sürüme
```

```
dotnet ef database update InitialCreate
```

```
# Sıfırla
```

```
dotnet ef database update 0
```

```
# Migration sil
```

```
dotnet ef migrations remove
```

```
# SQL script üret
```

```
dotnet ef migrations script
```

```
dotnet ef migrations script MigrationX MigrationY
```

PMC ve CLI aynı motoru çağırır; ekip tercihinine göre seçilir.

3.9.5 Örnek 3.9.1 — İlk Migration Akışı

Adım 1 — Model:

```
// Models/Doktor.cs
```

```
public class Doktor
```

```
{
```

```
    public int Id { get; set; }
```

```
    [Required, MaxLength(100)]
```

```
    public string Ad { get; set; }
```

```
    [MaxLength(50)]
```

```
    public string Uzmanlik { get; set; }
```

```
}
```

Adım 2 — DbContext:

```
// Data/AppDbContext.cs
```

```
public class AppDbContext : DbContext
```

```
{
```

```
    public AppDbContext(DbContextOptions<AppDbContext> options) : base(options) { }
```

```
    public DbSet<Doktor> Doktorlar { get; set; }
```

```
}
```

Adım 3 — Program.cs kaydı (§3.5.5).

Adım 4 — Migration üret:

Add-Migration InitialCreate

Üretilen dosya (özet):

```
// Migrations/20240610120000_InitialCreate.cs
public partial class InitialCreate : Migration
{
    protected override void Up(MigrationBuilder migrationBuilder)
    {
        migrationBuilder.CreateTable(
            name: "Doktorlar",
            columns: table => new
            {
                Id = table.Column<int>(nullable: false)
                    .Annotation("SqlServer:Identity", "1, 1"),
                Ad = table.Column<string>(maxLength: 100, nullable: false),
                Uzmanlik = table.Column<string>(maxLength: 50, nullable: true)
            },
            constraints: table =>
            {
                table.PrimaryKey("PK_Doktorlar", x => x.Id);
            });
    }

    protected override void Down(MigrationBuilder migrationBuilder)
    {
        migrationBuilder.DropTable(name: "Doktorlar");
    }
}
```

Adım 5 — Uygula:

Update-Database

Beklenen çıktı (özet):

```
Build started...
Build succeeded.
Applying migration '20240610120000_InitialCreate'.
Done.
```

3.9.6 Örnek 3.9.2 — Yeni Alan Ekleme

Doktor sınıfına TelefonNo eklenecek:

```
public class Doktor
{
    public int Id { get; set; }
    [Required, MaxLength(100)]
    public string Ad { get; set; }
    [MaxLength(50)]
    public string Uzmanlik { get; set; }

    [MaxLength(20)]
    public string TelefonNo { get; set; } // YENİ
}
```

Migration:

Add-Migration AddTelefonNoToDoktor

Update-Database

Üretilen (özet):

```
protected override void Up(MigrationBuilder migrationBuilder)
{
    migrationBuilder.AddColumn<string>(
        name: "TelefonNo",
        table: "Doktorlar",
        maxLength: 20,
        nullable: true);
}

protected override void Down(MigrationBuilder migrationBuilder)
{
    migrationBuilder.DropColumn(name: "TelefonNo", table: "Doktorlar");
}
```

Down metodu, Update-Database -Migration <önceki> çağrıldığında sütunu kaldırır.

3.9.7 Migration Geri Alma İş Akışı

Uygulanan bir migration'ı geri almak istiyorsun:

1. Hangi migration'a döneceğini belirle (bir önceki).
2. Update-Database -Migration <hedef> çalıştır — Down metodları çalışır, DB o sürüme döner.
3. Silmek istiyorsan Remove-Migration ile migration dosyalarını sil.

Uygulanmadan üretilmiş bir migration'ı silmek için doğrudan Remove-Migration yeterli — DB dokunulmamıştır.

Kritik: production'da migration geri almak çoğu zaman veri kaybı riskidir (sütun silme, tablo silme). Kural: **ileri git, geri gitme; gerekirse yeni bir "düzeltici" migration üret.**

3.9.8 __EFMigrationsHistory Tablosu

EF Core, uygulanan migration'ları veritabanında __EFMigrationsHistory isimli bir tabloda tutar:

MigrationId	ProductVersion
20240610120000_InitialCreate	8.0.0
20240611130000_AddTelefonNoToDoktor	8.0.0

Yeni bir migration çalıştırıldığında, sadece bu tabloda olmayan migration'lar uygulanır. Bu tabloyu manuel değiştirmek EF Core'un tutarsız duruma düşmesine yol açar.

3.9.9 Yaygın Hatalar

Hatalı Kod / Durum	Hata Mesajı	Neden	Düzeltilmiş
Add-Migration — Unable to create an object of type 'AppDbContext'	Options gelmiyor	DbContext DI'siz üretilemez	Program.cs'de AddDbContext kaydı yapılmış olmalı
Update-Database — Login başarısız	Bağlantı hatası	ConnectionString yanlış	appsettings.json düzelt
Aynı isimli migration üretme	A migration with the name '...' already exists	Migration adı unique olmalı	Farklı isim ver
Down çalışmıyor / veri kayboldu	Sütunda veri vardı, drop yapıldı	Migration destructive	Önce veri yedeği
İki geliştirici aynı anda migration eklemiş, çakışıyor	Snapshot birleşmez	Git conflict	Sadece biri branch'te tutulsun, diğer merge sonrası re-üret

3.9.10 Anti-Pattern ve İdiyomatik Kullanım

Anti-Pattern	İdiyomatik	Neden Daha İyi
Migration dosyalarını Git'e commit etmeme	Migration dosyaları da kaynak kod	Ekip tutarlılığı
Elle DB'de sütun ekleyip modele yazmak	Migration üzerinden	__EFMigrationsHistory senkron kalır
Her küçük değişiklikte migration	Anlamli grup migration	History okunabilir
Migration dosyasını sonradan düzenlemek (uygulandıktan sonra)	Yeni migration üretmek	Snapshot bozulmaz

3.9.11 Bölüm Özeti

- Code First: entity önce, DB sonra.
- Migration = versiyonlanmış şema değişikliği.
- İki komut: Add-Migration <ad> üret, Update-Database uygula.
- CLI karşılıkları: dotnet ef migrations add, dotnet ef database update.
- Remove-Migration uygulanmamış migration'ı siler; uygulanmışsa önce geri al.
- __EFMigrationsHistory tablosu neyin uygulandığını izler.
- Production'da destructive migration'lardan kaçın.

İlgili konular: §3.7 (Annotations → şema), §3.8 (Fluent API → şema), §5.1 (BulkyBookWeb migration).

3.10 Database First ve Scaffold-DbContext

3.10.1 Genel Bakış

Database First yaklaşımında veritabanı önceden vardır (tablolar, ilişkiler, view'lar tanımlanmış); EF Core, bunları okuyup C# sınıflarını (entity + DbContext) otomatik üretir. Kurumsal ortamlarda yaygındır: DBA'ler şemayı ve saklı yordamları yönetir, uygulama geliştiricileri EF Core aracılığıyla bu şemayı tüketir.

Karar rehberi:

- Yeni proje, saha yok: **Code First**.
- Var olan bir DB var, ondan uygulama üretilecek: **Database First**.
- İkisi karıştırılabilir mi? Teorik olarak evet, pratikte migration mekanizması karışır — projede tek yaklaşım tercih edilir.

3.10.2 Scaffold-DbContext Komutu

PMC'de:

```
Scaffold-DbContext "Server=(localdb)\mssqllocaldb;Database=YOUR_DATABASE;Trusted_Connection=True;
Microsoft.EntityFrameworkCore.SqlServer
-OutputDir Models
```

CLI'da:

```
dotnet ef dbcontext scaffold "Server=(localdb)\mssqllocaldb;Database=YOUR_DATABASE;Trusted_Connection=True;
Microsoft.EntityFrameworkCore.SqlServer \
--output-dir Models
```

Parametreler:

Parametre	Açıklama
<connectionString>	Veritabanı bağlantı dizesi
<provider>	Provider paket adı

Parametre	Açıklama
-OutputDir / --output-dir	Üretilen dosyaların klasörü
-Context <Ad>	DbContext sınıfının adı
-Tables T1,T2	Sadece belirtilen tabloları scaffold et
-DataAnnotations	Data Annotations kullan (varsayılan: Fluent API)
-Force	Var olan dosyaların üzerine yaz
-UseDatabaseNames	Sütun adlarını olduğu gibi kullan (aksi halde EF Core PascalCase yapar)
-NoOnConfiguring	OnConfiguring içinde connection string yazma (Program.cs'ten alınacak)

3.10.3 Tipik Kullanım (Katalog C — AspNet7Hastane)

Var olan bir SQL Server veritabanından tüm tabloları scaffold etmek:

```
Scaffold-DbContext "Server=(localdb)\mssqllocaldb;Database=hastanedb;Trusted_Connection=True"
Microsoft.EntityFrameworkCore.SqlServer
-OutputDir Models
-Context HastaneContext
-Force
```

Üretilen dosyalar:

- Models/HastaneContext.cs — DbContext, tüm DbSet'ler, OnModelCreating (Fluent API).
- Models/Bolum.cs, Models/Doktor.cs, Models/Hasta.cs, Models/Randevu.cs, Models/Personel.cs, Models/Tetkik.cs — entity sınıfları.

Üretilen entity örneği (özet):

```
// Models/Doktor.cs
public partial class Doktor
{
    public Doktor()
    {
        Randevular = new HashSet<Randevu>();
    }

    public int Id { get; set; }
    public string Ad { get; set; }
    public string Soyad { get; set; }
    public int BolumId { get; set; }

    public virtual Bolum Bolum { get; set; }
    public virtual ICollection<Randevu> Randevular { get; set; }
}
```

partial işaretlemesi kritik: geliştirici, aynı sınıfı ayrı bir dosyada (Doktor.Partial.cs) genişletebilir, scaffold yeniden çalıştığında elle yazılan kısım kaybolmaz.

3.10.4 Şema Değişince Ne Olur?

DBA tabloya yeni sütun ekledi. İki seçenek:

1. **Yeniden scaffold** (-Force ile) — tüm entity sınıfları yeniden üretilir, elle yaptığın değişiklikler kaybolur (partial extension'lar korunur).
2. **Elle senkronla** — yeni sütuna karşılık gelen property'yi entity'ye elle ekle.

Genelde kısmi tablo değişikliklerinde 2. yol, büyük şema değişikliklerinde 1. yol tercih edilir. Kritik nokta: **scaffold ile üretilen dosyaları elle çok değiştirme**, aksi halde bir sonraki scaffold hepsini yer.

3.10.5 Database First'te Migration Kullanımı?

Database First projede genellikle **migration kullanılmaz**; şema DBA tarafından SQL script'lerle yönetilir. Ancak EF Core, Add-Migration çağrıldığında mevcut şemayı entity'lerle karşılaştırıp fark almak isteyebilir. Bu karışıklığı önlemek için proje net bir tercih yapar.

3.10.6 Örnek 3.10.1 — AspNet7Hastane Şema İş Akışı

Katalog C'deki senaryo:

```
-- SQL Server Management Studio üzerinde hastanedb adında bir veritabanı oluşturulur
-- Tablolar SQL script ile yaratılır (Bolum, Doktor, Hasta, Randevu, Personel, Tetkik)
-- İlişkiler (FK) tanımlanır
```

Sonra Visual Studio'da yeni bir ASP.NET Core MVC projesi oluşturulur. NuGet paketleri kurulur (§3.5.2). PMC'de:

```
Scaffold-DbContext "Server=(localdb)\mssqllocaldb;Database=hastanedb;Trusted_Connection=True"
Microsoft.EntityFrameworkCore.SqlServer
-OutputDir Models
```

Sonuç: Models/ klasöründe tüm entity'ler ve hastanedbContext sınıfı hazır. Program.cs'e DI kaydı eklenir:

```
builder.Services.AddDbContext<hastanedbContext>(o =>
    o.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));
```

appsettings.json'a DefaultConnection yazılır. Artık controller'lar hastanedbContext üzerinden LINQ ile sorgu yapabilir.

3.10.7 Bölüm Özeti

- Database First: DB önce, sınıflar Scaffold-DbContext ile üretilir.
- Komut PMC ve CLI'da mevcuttur; ana parametreler -OutputDir, -Context, -Tables, -Force.
- Üretilen sınıflar partial'dır — elle genişletme mümkün.
- Şema değişince genelde yeniden scaffold; elle değişiklikler bir sonraki scaffold'da kaybolabilir.
- Database First'te migration mekanizması genellikle kullanılmaz.

İlgili konular: §5.3 (HastaneWeb / AspNet7Hastane projesi).

3.11 LINQ Sorguları

3.11.1 Genel Bakış

LINQ (Language Integrated Query), koleksiyonlar ve veritabanı üzerinde tek tip bir sorgulama dili sunar. C# ifadeleri olarak yazılır; hedef kaynağa göre (bellek koleksiyonu, EF Core, XML) uygun sağlayıcı devreye girer.

İki temel sözdizim vardır:

- **Method syntax (extension method chaining):** .Where(x => x.Yas > 18).Select(x => x.Ad).ToList(). Modern kodda tercih edilir.
- **Query syntax (SQL-benzeri):** from x in liste where x.Yas > 18 select x.Ad. Bazı senaryolarda (group by, join) daha okunaklı.

Master defter method syntax'ı standart alır; query syntax gerekirse yanında verir.

3.11.2 IEnumerable<T> vs IQueryable<T>

Kritik bir ayrım:

Tip	Nerede yürür	Ne zaman
IEnumerable<T>	Bellekte (LINQ to Objects)	Veri zaten yüklenmiş; List, dizi üzerinde
IQueryable<T>	Veritabanında (LINQ to SQL / EF Core)	Sorgu SQL'e çevrilip DB'ye gönderilecek

Aynı LINQ metodu (Where, Select) her iki tipe uygulansa da, IQueryable üzerindeki metotlar bir **expression tree** oluşturur; sonuç materialize edildiğinde (.ToList(), .First(), foreach) SQL üretilir. IEnumerable üzerinde ise metot direkt delegate olarak çalışır.

Uygulamada:

- `_context.Doktorlar` → IQueryable<Doktor> (SQL çıkar).
- `list.Where(...)` → listin tipine göre; List<T> IEnumerable üzerinde çalışır (bellekte).

Yaygın hata:

```
// KÖTÜ – tüm doktorları belleğe alır, sonra filtreler (ToList erken çağrılmış)
var d = _context.Doktorlar.ToList().Where(x => x.Uzmanlik == "Dahiliye");

// İYİ – filtre SQL'de çalışır
var d = _context.Doktorlar.Where(x => x.Uzmanlik == "Dahiliye").ToList();
```

ToList() çağrısı bir “materialization” noktasıdır — sorunun ne zaman çalıştırılacağını belirler.

3.11.3 En Sık Kullanılan Metotlar

Metot	Görevi
Where(pred)	Koşul filtresi
Select(sel)	Projeksiyon (biçim değiştir)
SelectMany(sel)	Flatten (iç içe koleksiyonu tek düze)
OrderBy(key) / OrderByDescending	Sıralama
ThenBy(key) / ThenByDescending	İkinci seviye sıralama
First() / FirstOrDefault()	İlk eleman (yoksa exception / null)
Single() / SingleOrDefault()	Tek eleman (birden çoksa exception)
Any(pred)	Var mı? (boolean)
All(pred)	Hepsi mi?
Count() / Count(pred)	Sayı
Sum(sel) / Average(sel) / Min(sel) / Max(sel)	Agrega
GroupBy(key)	Gruplama
Take(n) / Skip(n)	Sayfalama
Distinct()	Tekrarları kaldır
Contains(item)	İçeriyor mu?
Include(nav) (EF Core)	Eager load
ToList() / ToArray() / ToDictionary(...)	Materialize (sorgu çalıştır)

3.11.4 Örnek 3.11.1 — Temel Sorgu

```
var liste = _context.Doktorlar
    .Where(d => d.Uzmanlik == "Dahiliye")
    .OrderBy(d => d.Ad)
    .Select(d => new { d.Id, d.Ad })
    .ToList();

foreach (var d in liste)
    Console.WriteLine($"{d.Id} - {d.Ad}");
```

Üretilen SQL (özet):

```
SELECT [d].[Id], [d].[Ad]
FROM [Doktorlar] AS [d]
WHERE [d].[Uzmanlik] = N'Dahiliye'
ORDER BY [d].[Ad]
```

Select içindeki anonim tip, sadece istenen sütunları çeker; tam entity yüklenmesini önler.

3.11.5 Örnek 3.11.2 — Gruplama ve Agrega

Uzmanlığa göre doktor sayısı:

```
var gruplar = _context.Doktorlar
    .GroupBy(d => d.Uzmanlik)
    .Select(g => new { Uzmanlik = g.Key, Sayi = g.Count() })
    .OrderByDescending(x => x.Sayi)
    .ToList();
```

Üretilen SQL:

```
SELECT [d].[Uzmanlik] AS [Uzmanlik], COUNT(*) AS [Sayi]
FROM [Doktorlar] AS [d]
GROUP BY [d].[Uzmanlik]
ORDER BY COUNT(*) DESC
```

3.11.6 Örnek 3.11.3 — Join (İlişki Zaten Navigation ile)

EF Core, navigation property'ler tanımlıysa Join yazmaya gerek bırakmaz:

```
// Include ile eager load – bkz. §3.12
var randevular = _context.Randevular
    .Include(r => r.Doktor)
    .Where(r => r.Tarih >= DateTime.Today)
    .OrderBy(r => r.Tarih)
    .ToList();

foreach (var r in randevular)
    Console.WriteLine($"{r.Tarih} - {r.HastaAdi} → {r.Doktor.Ad}");
```

Navigation kullanılmayan durumlarda klasik Join:

```
var q = from r in _context.Randevular
        join d in _context.Doktorlar on r.DoktorId equals d.Id
        where r.Tarih >= DateTime.Today
        select new { r.Tarih, r.HastaAdi, DoktorAd = d.Ad };
```

3.11.7 Örnek 3.11.4 — Sayfalama (Paging)

10'ar kayıt sayfalar halinde:

```
int sayfa = 2;
int sayfaBoyut = 10;

var kayitlar = _context.Doktorlar
    .OrderBy(d => d.Ad)
    .Skip((sayfa - 1) * sayfaBoyut)
    .Take(sayfaBoyut)
    .ToList();
```

Üretilen SQL:

```
SELECT [d].[Id], [d].[Ad], [d].[Uzmanlik]
FROM [Doktorlar] AS [d]
ORDER BY [d].[Ad]
OFFSET 10 ROWS FETCH NEXT 10 ROWS ONLY
```

3.11.8 First vs FirstOrDefault vs Single

Metot	0 sonuç	1 sonuç	2+ sonuç
First()	InvalidOperationException	Döner	İlkini döner
FirstOrDefault()	null / default	Döner	İlkini döner
Single()	Exception	Döner	Exception
SingleOrDefault()	null	Döner	Exception

Seçim kuralı:

- Sonuç garanti tek: Single() (Id ile bakma).
- Sonuç tek olabilir, 0 da olabilir: SingleOrDefault().
- Sonuç birden çok olabilir, ilkini istiyorsun: First() veya FirstOrDefault().
- Sadece varlık kontrolü: Any(pred) (daha hızlı, COUNT(*) çekmez).

3.11.9 Query Syntax Örneği

```
var q = from d in _context.Doktorlar
        where d.Uzmanlik == "Dahiliye"
        orderby d.Ad
        select new { d.Id, d.Ad };
```

Method syntax ile birebir aynı — derleyici query syntax'ı method syntax'a çevirir. Karmaşık grup/join işlemlerinde bazen daha okunaklı; günlük kodda method syntax standarttır.

3.11.10 Yaygın Hatalar

Hatalı Kod	Sonuç	Neden	Düzeltilmiş
.ToList().Where(...)	Tüm tablo belleğe iner, sonra filtre	Erken materialization	Önce Where, sonra ToList
.Where(d => Fonksiyon(d)) (kendi statik metodun)	InvalidOperationException	EF Core özel metodu SQL'e çeviremez	Basit ifade veya materialize sonrası
.First() boş sonuç üzerinde	InvalidOperationException	First en az 1 bekler	FirstOrDefault()
.Count() > 0 yerine .Any()	Yavaş (COUNT(*)), Any EXISTS üretir	Performans	.Any()
Sayfalama Skip/Take ama sıralama yok	Belirsiz sıra	ORDER BY olmadan OFFSET/FETCH güvenilmez	Önce OrderBy

3.11.11 Anti-Pattern ve İdiyomatik Kullanım

Anti-Pattern	İdiyomatik	Neden Daha İyi
.ToList().Count()	.Count() (int direkt SQL COUNT)	Bellek + hız
Tüm entity yükleyip belleğe filtreleme	Sadece gereken alanları	Az veri transferi
Foreach içinde her elemanda ayrı sorgu (N+1)	.Select ile	Trip sayısı 1'e iner
Where(x => x.Id == id).First()	First(x => x.Id == id)	Kısa
Include(x => x.Alt).Include(x => x.Alt2) uzun zincirler	Split query (§3.12)	Kartezyen patlamayı önler

3.11.12 Mini Sözlük

- **LINQ**: Language Integrated Query; C#'ın entegre sorgulama motoru.
- **IQueryable<T>**: Sorgunun SQL'e çevrileceği tip; expression tree oluşturur.
- **IEnumerable<T>**: Bellek üzerinde iterasyon; delegate ile çalışır.
- **Deferred execution**: Sorgu, materialize (ToList, foreach) edilene kadar çalışmaz.
- **Materialization**: Sorgu çalıştırılıp bellekte nesneye dönüşür.
- **Expression tree**: Kodu veri olarak temsil eden ağaç yapısı; SQL çeviricinin ham girdisi.
- **Projection**: Select ile veri şeklini değiştirme.

3.11.13 Bölüm Özeti

- LINQ, koleksiyon ve DB için tek tip sorgu dili.
- IQueryable DB'de, IEnumerable bellekte çalışır — sıralama önemli.
- Method syntax standart; query syntax alternatif.
- Where → OrderBy → Select → ToList sıralaması klasik.
- Materialization (ToList, First, foreach) sorguyu çalıştırır — deferred execution.
- Any var mı diye sormanın en hızlı yolu; Count > 0 yerine Any kullan.
- Sayfalama için OrderBy + Skip + Take üçlüsü.

İlgili konular: §3.12 (Include, projection), §3.6 (DbSet — IQueryable kaynağı), §2.10 (IEnumerable).

3.12 Eager Loading, Lazy Loading, Projection, Materialization

3.12.1 Genel Bakış

EF Core, ilişkili entity'leri (navigation property'leri) veritabanından çekme konusunda üç stratejiye sahiptir:

- **Eager loading** (Include): İlk sorguda ilişkileri de birlikte yükle. En yaygın ve önerilen.
- **Explicit loading** (.Load()): Ana entity zaten belleğe alınmış; sonradan elle “şunu da yükle” demek.
- **Lazy loading**: Navigation property'ye erişildiği anda otomatik ek sorgu çalıştır. Varsayılan olarak kapalı; açılırsa “N+1 problem”ine kolayca yol açar.

3.12.2 Eager Loading — Include

En sık kullanılan yaklaşım.

```
var doktor = _context.Doktorlar
    .Include(d => d.Randevular)
    .FirstOrDefault(d => d.Id == 1);

foreach (var r in doktor.Randevular)
    Console.WriteLine($"{r.Tarih} - {r.HastaAdi}");
```

Üretilen SQL (özet):

```
SELECT d.*, r.*
FROM Doktorlar d
LEFT JOIN Randevular r ON r.DoktorId = d.Id
WHERE d.Id = 1
```

Zincirleme ThenInclude:

```
var randevular = _context.Randevular
    .Include(r => r.Doktor)
    .ThenInclude(d => d.Bolum)
    .Include(r => r.Hasta)
    .ToList();
```

- Include bir sonraki seviyeye geçer.
- ThenInclude en son include edilen seviyeden bir aşağı iner.
- Farklı navigation dallarına gitmek için tekrar Include (üstten başlar).

3.12.3 Split Query — Kartezyen Patlamayı Önlemek

Birden çok Include çekildiğinde, tek JOIN sorgusu satır sayısını çarpımsal büyütebilir. EF Core 5+ ile AsSplitQuery bunu çözer:

```
var doktorlar = _context.Doktorlar
    .Include(d => d.Randevular)
    .Include(d => d.Tetkikler)
    .AsSplitQuery() // her Include ayrı SQL sorgusuyla
    .ToList();
```

Üretilen SQL — üç ayrı sorgu:

```
SELECT * FROM Doktorlar;
SELECT * FROM Randevular WHERE DoktorId IN (...);
SELECT * FROM Tetkikler WHERE DoktorId IN (...);
```

Her koleksiyon için ayrı sorgu, çarpımsal büyümeyi önler; buna karşılık DB trip sayısı artar. Küçük veri setlerinde single query daha hızlıdır.

3.12.4 Explicit Loading

Ana entity zaten çekilmiş; sonradan ilişkiyi yüklemek:

```
var doktor = _context.Doktorlar.First(d => d.Id == 1);

// Navigation koleksiyonu yükle
_context.Entry(doktor).Collection(d => d.Randevular).Load();

// Tek referans navigation
_context.Entry(randevu).Reference(r => r.Doktor).Load();
```

Explicit loading, koşullu ilişki yüklemede işe yarar: “eğer kullanıcı ‘Detaylar’a tıklarsa randevuları da yükle”.

3.12.5 Lazy Loading

Varsayılan olarak kapalıdır. Açmak için:

1. Microsoft.EntityFrameworkCore.Proxies paketini kur.
2. Navigation property’leri virtual yap.
3. Program.cs’de UseLazyLoadingProxies() ekle.

```
builder.Services.AddDbContext<AppDbContext>(o =>
    o.UseLazyLoadingProxies()
    .UseSqlServer(...));
```

```
public class Doktor
{
    public int Id { get; set; }
    public virtual ICollection<Randevu> Randevular { get; set; }
}
```

Artık doktor.Randevular ilk erişildiği anda arka planda ekstra SQL sorgusu tetiklenir. Kolaylık gibi görünse de **N+1 problem**’ini kolayca doğurur:

```
foreach (var d in _context.Doktorlar.ToList()) // 1 sorgu
    Console.WriteLine(d.Randevular.Count); // her doktor için ayrı sorgu
```

100 doktor varsa 101 sorgu çalışır. Master defterin önerisi: **lazy loading kullanma**. Include ile açık yükleme her zaman daha öngörülebilirdir.

3.12.6 Projection — Sadece Gereken Alanları Çek

En verimli yaklaşım, tam entity yüklemeyip yalnızca ihtiyaç duyulan alanları anonim tip veya DTO’ya projekte etmektir.

```
var liste = _context.Doktorlar
    .Where(d => d.Uzmanlik == "Dahiliye")
    .Select(d => new
    {
        d.Id,
        d.Ad,
        RandevuSayisi = d.Randevular.Count
    })
    .ToList();
```

Üretilen SQL:

```
SELECT d.Id, d.Ad, (
    SELECT COUNT(*) FROM Randevular r WHERE r.DoktorId = d.Id
) AS RandevuSayisi
FROM Doktorlar d
WHERE d.Uzmanlik = N'Dahiliye'
```

Projection'ın kazançları:

- Sadece istenen sütunlar taşınır (network ve bellek).
- Change tracking devre dışı (anonim tip tracking'e girmez).
- SQL çevirisi doğrudan hesap yapabilir (Count() sorgu içinde).

3.12.7 DTO'ya Projeksiyon

Anonim tip yerine record veya sınıf kullanmak, sonucu bir metot dışına döndürebilmek için gereklidir:

```
public record DoktorOzet(int Id, string Ad, int RandevuSayisi);

public List<DoktorOzet> Ozetler() =>
    _context.Doktorlar
        .Select(d => new DoktorOzet(d.Id, d.Ad, d.Randevular.Count))
        .ToList();
```

3.12.8 Materialization — Sorguyu Çalıştıran Metotlar

IQueryable<T> bir "plan"dır; aşağıdaki metotlardan biri çağrılana kadar SQL çalışmaz:

- ToList(), ToArray(), ToDictionary(...) — koleksiyonlaştırma
- First(), FirstOrDefault(), Single(), SingleOrDefault(), Last(), LastOrDefault() — tek eleman
- Count(), Any(), All(), Sum(), Min(), Max(), Average() — agrega
- foreach içinde yürüme
- .AsEnumerable() sonrası devam eden ifadeler (dikkat: buradan sonrası bellekte çalışır)

3.12.9 Yaygın Hatalar

Hatalı Kod	Sonuç	Neden	Düzeltilmiş
Lazy loading açık, koleksiyon iterasyonu Include sonrası	N+1 sorgu	Her navigation ayrı SQL	Include ile eager
Select yeni tipe geçince navigation kayboluyor	Include etkisiz	Projection sonrası Include EF tarafından atlanıyor	Projection'a alanları elle ekle
Zincirleme Include (5+)	Kartezyen patlaması	JOIN sonucu çok satır	AsSplitQuery()
_context.Doktorlar.ToList() içinde Include(...)	Derleme hatası	Include IQueryable'a uygulanır	ToList() öncesinde Include

3.12.10 Bölüm Özeti

- Üç strateji: eager (Include), explicit (Load), lazy (varsayılan kapalı).
- Eager loading standart; öngörülebilir SQL üretir.
- AsSplitQuery çoklu Include senaryolarında kartezyen patlamayı önler.
- Projection (Select), sadece gereken alanları çeker; en verimli yol.
- Materialization noktaları (ToList, First, ...) sorguyu çalıştırır.
- Lazy loading N+1 riskini doğurur; genelde kaçınılır.

İlgili konular: §3.11 (LINQ), §3.6 (AsNoTracking), §4.14 (ViewModel'e projeksiyon).

3.13 CRUD Operasyonları ve SaveChanges

3.13.1 Genel Bakış

CRUD (Create, Read, Update, Delete), EF Core'un günlük kullanımının %90'ıdır. Read için LINQ (§3.11) ve loading stratejileri (§3.12) ele alındı; bu bölüm Create/Update/Delete akışlarına ve SaveChanges mekanizmasına odaklanır.

Temel yaklaşım:

1. Entity'nin durumunu değiştir (yeni ekle / property güncelle / işaretle sil).
2. SaveChanges() veya SaveChangesAsync() çağır — EF Core, change tracker'daki tüm değişiklikleri tek transaction'da DB'ye yansıtır.

3.13.2 Create — Ekleme

```
var yeni = new Doktor
{
    Ad = "Dr. Ayşe",
    Uzmanlik = "Cildiye"
};

_context.Doktorlar.Add(yeni);
await _context.SaveChangesAsync();

Console.WriteLine($"Oluşturulan Id: {yeni.Id}");
```

Üretilen SQL:

```
INSERT INTO Doktorlar (Ad, Uzmanlik) VALUES (N'Dr. Ayşe', N'Cildiye');
SELECT SCOPE_IDENTITY();
```

SaveChanges sonrası yeni.Id, veritabanının atadığı IDENTITY değeriyle dolar. Bu, INSERT sonrası dönüş değerinin otomatik geri okunmasıdır.

Toplu ekleme:

```
_context.Doktorlar.AddRange(new[]
{
    new Doktor { Ad = "A", Uzmanlik = "X" },
    new Doktor { Ad = "B", Uzmanlik = "Y" }
});
await _context.SaveChangesAsync();
```

EF Core 8, INSERT cümlesini tek bir INSERT INTO ... VALUES (...), (...) şeklinde batch'ler; birçok satır tek trip'te gider.

3.13.3 Read — Okuma

Detaylı LINQ örnekleri §3.11'de. Özet:

```
// Tek kayıt
var d = _context.Doktorlar.Find(1);
```

```
// veya
var d = _context.Doktorlar.FirstOrDefault(x => x.Id == 1);

// Liste
var hepsi = _context.Doktorlar.ToList();

// Filtreli
var dahiliye = _context.Doktorlar.Where(x => x.Uzmanlik == "Dahiliye").ToList();
```

Find(id) primary key'e göre optimize edilmiştir: change tracker'da varsa DB'ye gitmez, doğrudan bellekten döner.

3.13.4 Update — Güncelleme

En yaygın desen: **tracked entity üzerinde property değişikliği**.

```
var d = _context.Doktorlar.Find(1);
d.Uzmanlik = "Kardiyooloji";
await _context.SaveChangesAsync();
```

Üretilen SQL:

```
UPDATE Doktorlar
SET Uzmanlik = N'Kardiyooloji'
WHERE Id = 1;
```

Change tracker "orijinal Uzmanlik = 'Dahiliye'" bilgisini tutar; SaveChanges'te sadece değişen alanları UPDATE'ler. Değişmeyen sütun SQL'e girmez.

Detached entity güncelleme (ör. formdan gelen model):

```
// Controller içinde
[HttpPost]
public IActionResult Edit(Doktor form)
{
    _context.Doktorlar.Update(form); // durumu Modified yapar
    _context.SaveChanges();
    return RedirectToAction("Index");
}
```

Update, entity'nin tüm alanlarını Modified olarak işaretler; SQL'de tüm sütunlar UPDATE'e girer. Alternatif: entity'yi bul, alanları elle kopyala (bkz. §5.2 controller örnekleri).

3.13.5 Delete — Silme

```
var d = _context.Doktorlar.Find(5);
_context.Doktorlar.Remove(d);
await _context.SaveChangesAsync();
```

Üretilen SQL:

```
DELETE FROM Doktorlar WHERE Id = 5;
```

Bulmadan silme (EF Core 7+ ile):

```
await _context.Doktorlar
    .Where(d => d.Id == 5)
    .ExecuteDeleteAsync();
```

Bu SELECT yapmadan doğrudan DELETE FROM ... WHERE Id = 5 üretir; change tracker'a hiç girmez. Toplu silme için idealdir.

Soft delete (§4.21 için ön hazırlık): silmek yerine IsDeleted = true işaretlemek.

3.13.6 SaveChanges Detayları

SaveChanges() çağrısı:

1. Change tracker'daki tüm entity'leri gezer.
2. Added → INSERT, Modified → UPDATE, Deleted → DELETE komutları üretir.
3. Tek bir transaction içinde hepsini sırayla çalıştırır.
4. Herhangi biri başarısızsa tümü rollback (otomatik).
5. Başarılıysa commit; entity durumları Unchanged'e alınır; IDENTITY değerleri geri yansır.

Etkilenen satır sayısı döner:

```
int sayi = _context.SaveChanges();
Console.WriteLine($"{sayi} kayıt işlendi.");
```

Async versiyon (web'de tercih edilir):

```
await _context.SaveChangesAsync();
```

Web request'inde async SaveChanges, DB I/O sırasında thread'i serbest bırakır — ölçeklenmeyi iyileştirir.

3.13.7 Örnek 3.13.1 — MVC Controller CRUD

```
// Controllers/DoktorController.cs
public class DoktorController : Controller
{
    private readonly AppDbContext _context;
    public DoktorController(AppDbContext context) => _context = context;

    // GET: /Doktor
    public IActionResult Index()
    {
        var liste = _context.Doktorlar.AsNoTracking().ToList();
        return View(liste);
    }

    // GET: /Doktor/Create
    public IActionResult Create() => View();

    // POST: /Doktor/Create
    [HttpPost]
    [ValidateAntiForgeryToken]
    public async Task<IActionResult> Create(Doktor doktor)
    {
        if (!ModelState.IsValid) return View(doktor);
        _context.Doktorlar.Add(doktor);
        await _context.SaveChangesAsync();
        return RedirectToAction(nameof(Index));
    }

    // GET: /Doktor/Edit/5
    public IActionResult Edit(int id)
    {
        var d = _context.Doktorlar.Find(id);
        if (d == null) return NotFound();
        return View(d);
    }

    // POST: /Doktor/Edit/5
    [HttpPost]
    [ValidateAntiForgeryToken]
    public async Task<IActionResult> Edit(int id, Doktor form)
    {

```

```

        if (id != form.Id) return BadRequest();
        if (!ModelState.IsValid) return View(form);
        _context.Doktorlar.Update(form);
        await _context.SaveChangesAsync();
        return RedirectToAction(nameof(Index));
    }

    // POST: /Doktor/Delete/5
    [HttpPost, ActionName("Delete")]
    [ValidateAntiForgeryToken]
    public async Task<IActionResult> DeleteConfirmed(int id)
    {
        var d = await _context.Doktorlar.FindAsync(id);
        if (d != null)
        {
            _context.Doktorlar.Remove(d);
            await _context.SaveChangesAsync();
        }
        return RedirectToAction(nameof(Index));
    }
}

```

Bu şablon, master defterin standart CRUD controller'ıdır; §5.1 (BulkyBookWeb Kategori) ve §5.2 (HastaneMVC) projelerinde bu deseni izler.

3.13.8 Yaygın Hatalar

Hatalı Kod	Hata	Neden	Düzeltilmiş
Add sonrası SaveChanges unutulmuş Update(entity) — entity'nin Id'si 0	Hiçbir kayıt DB'ye gitmez INSERT olur, UPDATE değil	Değişiklikler bellekte Yeni gibi görülür	SaveChanges çağır Id'yi doğru gir veya Find + property atama
Remove unutulup boş SaveChanges Concurrency olmadan uzun süreli entity düzenlemek SaveChanges her bir kayıt için ayrı çağırma (döngü içinde)	Hiçbir şey olmaz Diğer kullanıcının değişiklikleri kaybolabilir Yavaş	Change tracker'da Deleted yok Optimistic concurrency yok Her seferinde transaction	Remove çağır [ConcurrencyCheck] veya RowVersion Toplu ekle, tek SaveChanges

3.13.9 Bölüm Özeti

- Ekleme: Add / AddRange → SaveChanges.
- Güncelleme: tracked entity property değiştir → SaveChanges. Detached için Update.
- Silme: Remove → SaveChanges. Toplu için ExecuteDeleteAsync.
- SaveChanges tek transaction, tüm değişiklikler; başarısızsa rollback.
- Web'de SaveChangesAsync tercih edilir.
- MVC controller CRUD şablonu master defterin proje bölümlerinde tekrarlanır.

İlgili konular: §3.6 (Change tracker), §4.7 (Controller action), §5.1, §5.2.

3.14 Unique Constraint ve HasIndex

3.14.1 Genel Bakış

Bir tabloda belirli bir sütunun (veya sütun grubunun) tüm satırlar arasında **benzersiz** olması gerekebilir — kullanıcı adı, e-posta, ürün kodu gibi. Bu, veritabanı düzeyinde **unique index** ile garanti edilir. EF Core’da Fluent API’nin `HasIndex().IsUnique()` çağrısı ile ifade edilir.

Neden veritabanı düzeyinde? Uygulama kodu her seferinde “önce bak, sonra ekle” yapsa da, iki paralel request aynı anda bunu yapabilir (race condition). Sadece DB düzeyinde unique constraint, hangi thread’in kazanacağını kesin çözer.

3.14.2 Tek Sütun Unique Index

```
// Models/Kullanici.cs
public class Kullanici
{
    public int Id { get; set; }
    public string KullaniciAdi { get; set; }
    public string Email { get; set; }
}

// OnModelCreating
modelBuilder.Entity<Kullanici>()
    .HasIndex(k => k.KullaniciAdi)
    .IsUnique();

modelBuilder.Entity<Kullanici>()
    .HasIndex(k => k.Email)
    .IsUnique();
```

Migration üretir:

```
migrationBuilder.CreateIndex(
    name: "IX_KullaniciAdi",
    table: "Kullanici",
    column: "KullaniciAdi",
    unique: true);
```

SQL:

```
CREATE UNIQUE INDEX IX_KullaniciAdi
ON Kullanici (KullaniciAdi);
```

Artık aynı kullanıcı adıyla ikinci INSERT başarısız olur.

3.14.3 Data Annotation İle (EF Core 5+)

```
[Index(nameof(KullaniciAdi), IsUnique = true)]
public class Kullanici
{
    public int Id { get; set; }
    public string KullaniciAdi { get; set; }
}
```

[Index] attribute sınıf seviyesindedir (property seviyesinde değil), çünkü birden fazla sütun içerebilir.

3.14.4 Birleşik (Composite) Unique Index

Aynı sınıfa aynı öğretmen atanamaz — (SınıfId, ÖğretmenId) beraber benzersiz olmalı:

```
modelBuilder.Entity<SinifOgretmen>()
    .HasIndex(so => new { so.SinifId, so.OgretmenId })
    .IsUnique();
```

Bir öğretmen farklı sınıflara atanabilir; bir sınıfta farklı öğretmenler olabilir; ama aynı çift tekrarlanamaz.

3.14.5 Filtreli Unique Index (Nullable Sütunlar İçin)

Bir sütun nullable ise ve sadece null olmayan değerlerin benzersiz olmasını istiyorsan:

```
modelBuilder.Entity<Urun>()
    .HasIndex(u => u.Barkod)
    .IsUnique()
    .HasFilter("[Barkod] IS NOT NULL");
```

Bu, “barkodu olan ürünler benzersizdir, barkodsuzlar (NULL) sınırsız olabilir” anlamına gelir. SQL Server’da filtered index olarak üretilir.

3.14.6 Constraint İhlali Yakalama

Bir INSERT unique constraint’i ihlal ederse DbUpdateException fırlar. İçindeki inner exception SQL sağlayıcısına özgüdür (SQL Server’da SqlException — hata numarası 2601 veya 2627 unique constraint ihlali).

```
try
{
    _context.Kullanicilar.Add(yeni);
    await _context.SaveChangesAsync();
}
catch (DbUpdateException ex) when (ex.InnerException is SqlException sql && (sql.Number == 2601
{
    ModelState.AddModelError("KullaniciAdi", "Bu kullanıcı adı zaten kayıtlı.");
    return View(yeni);
}
```

Kullanıcı dostu yaklaşım: INSERT’ten önce .Any(k => k.KullaniciAdi == yeni.KullaniciAdi) ile ön kontrol yap; race condition için catch bloğunu da bırak.

3.14.7 Non-Unique Index (Performans İçin)

Sık aranan sütunlar için non-unique index performansı artırır — sıklıkla Uzmanlik, KategoriId, Tarih gibi FK ve sık kullanılan sütunlarda:

```
modelBuilder.Entity<Doktor>()
    .HasIndex(d => d.Uzmanlik); // IsUnique() yok
```

EF Core, foreign key sütunları için otomatik olarak index üretir; ek bir şey yapmaya gerek yoktur.

3.14.8 Yaygın Hatalar

Hatalı Kod	Sonuç	Neden	Düzeltilmiş
Unique kontrolü sadece uygulama kodunda	Race condition doğar	Paralel request aynı kontrolü geçer	DB’de unique index
Nullable sütunda basit unique	İkinci NULL değer INSERT’i fail eder (SQL Server)	Standart unique NULL’ı da benzersiz sayar	Filtered index (WHERE ... IS NOT NULL)
Constraint hatası yakalanmıyor	500 hata sayfası	Kullanıcıya anlamlı mesaj yok	try-catch + ModelState
Migration yerine elle CREATE INDEX	Snapshot senkron değil	EF farkı sayar	Fluent API + migration

3.14.9 Bölüm Özeti

- Unique constraint DB düzeyinde `HasIndex().IsUnique()` ile kurulur.
- `[Index(nameof(...), IsUnique = true)]` attribute alternatifi.
- Composite index `new { A, B }` sözdizimi ile.
- Nullable sütunlarda “sadece dolu değerler benzersiz” için `HasFilter`.
- Uygulama kodu ön kontrol yapabilir; ama son sözü DB söyler — `DbUpdateException` yakalanmalı.
- FK sütunları için EF Core otomatik non-unique index üretir.

İlgili konular: §3.8 (Fluent API), §3.9 (Migration), §4.15 (Validation ile birlikte kullanım).

Bölüm IV — ASP.NET Core MVC

ASP.NET Core MVC, .NET üzerinde web uygulaması geliştirmek için Microsoft’un modern, cross-platform framework’ünün MVC (Model-View-Controller) tabanlı sunum katmanıdır. Bu bölüm, bir HTTP request’inin browser’dan controller action’ına, oradan Razor view’una ve tekrar browser’a dönme yolculuğunun her durağını ele alır. §4.1’den §4.10’a temeller, §4.11–§4.15 form ve validasyon, §4.16–§4.22 platform servisleri ve pratik desenler işlenir.

4.1 MVC Tasarım Deseni

4.1.1 Genel Bakış

MVC (Model-View-Controller), 1970’lerin sonlarında Trygve Reenskaug tarafından formüle edilen bir yazılım mimarisi desendir. Amaç, bir uygulamayı üç bağımsız sorumluluğa ayırarak bakım ve testi kolaylaştırmaktır:

- **Model:** Verinin şekli ve iş kuralları. Domain entity’leri, ViewModel’ler, servis sınıfları.
- **View:** Kullanıcıya gösterilen arayüz. HTML, CSS ve Razor sözdizimi.
- **Controller:** İki tarafın arasındaki aracı. Gelen HTTP isteğini işler, Model’i günceller ve uygun View’ı döndürür.

Bu ayırım, “bir domain sınıfı doğrudan HTML yazmasın, bir view SQL sorgusu çalıştırmassın, bir controller HTML template üretmesin” prensibinin uygulama seviyesindeki karşılığıdır. Her bileşenin kendi sorumluluğu vardır ve diğerini bilmeden test edilebilir.

4.1.2 Bir HTTP Request’in Yolculuğu

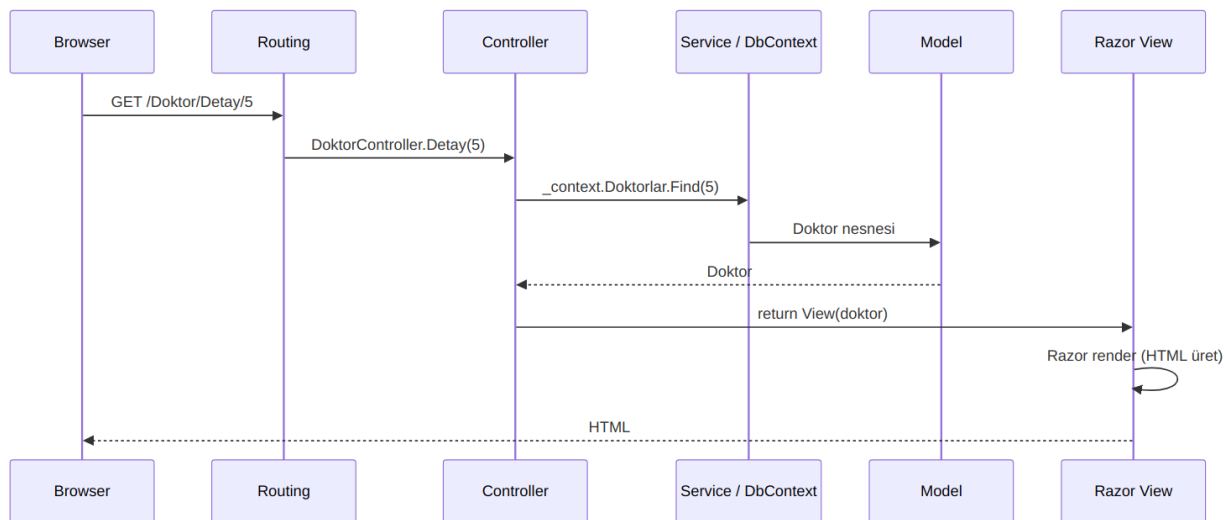


Figure 10: Diyagram 10

Adımlar:

1. Browser bir URL gönderir.
2. Routing motoru URL'yi bir Controller ve Action'a eşler.
3. Controller action, gerekli veriyi bir servis veya DbContext üzerinden ister.
4. Model (entity/ViewModel) alınır.
5. Controller, model'i view'a geçer (return View(model)).
6. Razor engine HTML üretir.
7. Response browser'a döner.

Bu yolun her adımı sonraki bölümlerde ayrıntılı ele alınır: §4.6 Routing, §4.7 Controller, §4.8 Razor.

4.1.3 Model — Verinin ve Kuralın Yeri

Bir MVC uygulamasında “Model” kelimesi bağlama göre üç farklı şeyi kastedebilir:

- **Domain Model / Entity:** Veritabanına eşlenmiş kalıcı veri sınıfları. Doktor, Randevu, Urun. EF Core entity'leri (§3.6).
- **ViewModel:** Belirli bir view'a özgü, sunum için biçimlendirilmiş model. Domain entity'lerinden projekte edilir, ek UI alanları içerebilir (§4.14).
- **DTO (Data Transfer Object):** Katmanlar arası veri taşımada kullanılan sınıflar. ViewModel ile örtüşür ama genellikle web API bağlamında geçer.

Master defterin standardı: **Controller'a entity dönme, ViewModel kullan.** Bu, view'ın entity iç yapısına bağımlı kalmasını önler ve doğrulama attribute'larını sunum ihtiyaçlarına göre ayarlama serbestliği verir.

4.1.4 View — Kullanıcının Gördüğü

View, HTML iskelet üzerine Razor sözdiziminin (@) serpiştirildiği dosyalardır: .cshtml uzantılı. Bir view:

- Model'in içinden veriyi okur (@Model.Ad).
- Kontrol akışı yürütür (@if, @foreach).
- Alt bileşenlere delegasyon yapar (@Html.Partial(...)).
- HTML üretir; hiçbir zaman veri kaydetmez.

View'lar Views/<Controller>/<Action>.cshtml konvansiyonuyla organize edilir. Ortak parçalar Views/Shared/'da yer alır (§4.9 Layout).

4.1.5 Controller — Orkestra Şefi

Controller, gelen HTTP isteğini alan, model'i (veri) üzerinden iş yapan ve bir sonuç (view, redirect, JSON, dosya) döndüren sınıftır. Adı <Ad>Controller konvansiyonuna uyar (örn. DoktorController), Controllers/ klasörüne konur, Controller taban sınıfından türer.

Bir controller içindeki her public metot, potansiyel bir **action**'dır — routing motoru URL'lerden action'lara eşleme yapar. Action, IActionResult (veya alt tipi) döndürür (§4.7).

Kural: **controller ince olur.** İş mantığını servis sınıflarına devreder, sadece “iste, hesapla, döndür” akışını yönetir.

4.1.6 Neden MVC?

MVC deseninin sağladığı somut kazanımlar:

- **Test edilebilirlik:** Controller'lar unit test edilebilir; view render'a bağımlı değildirler.
- **Paralel çalışma:** Frontend geliştirici view üzerinde, backend geliştirici controller ve model üzerinde ayrı ayrı çalışabilir.
- **Kod tekrarının azalması:** Ortak layout, partial view, servis katmanı sayesinde aynı işi tekrar yazma azalır.
- **Katman izolasyonu:** Bir katmandaki değişiklik diğerini kırmaz (view teması değişebilir, controller sabit kalır).

4.1.7 ASP.NET Core MVC vs Diğer Sunum Modelleri

.NET dünyasında birden fazla sunum katmanı seçeneği vardır:

Model	Açıklama	Ne zaman
MVC	Klasik controller + view; URL bazlı routing	Kurumsal uygulama, çoklu ekran
Razor Pages	Sayfa merkezli; view dosyasının yanında kod dosyası	Basit CRUD, sayfa başına tek "controller-benzeri"
Blazor (Server / WASM) Web API	C# ile SPA; component tabanlı JSON dönen controller; view yok	Zengin UI, gerçek zamanlı Backend, mobil, SPA
Minimal API	Sınıf yok, doğrudan <code>app.MapGet(...)</code>	Küçük servisler, mikroservis

Master defterin konusu MVC'dir. Aynı proje içinde birkaç MVC controller ile birkaç Web API controller birlikte var olabilir (Controllers/Api/UserApiController).

4.1.8 Bölüm Özeti

- MVC üç sorumluluğu ayırır: Model (veri), View (görüntü), Controller (koordinasyon).
- Bir istek: URL → Routing → Controller Action → Model işleme → View render → HTML.
- Domain entity ≠ ViewModel — controller ideally ViewModel döner.
- View sadece görüntüler; veri değiştirmez.
- Controller ince, servis kalın; iş kuralı controller'da değil, servis / model'de.

İlgili konular: §4.6 (Routing), §4.7 (Controller), §4.14 (ViewModel), §5.1–§5.4 (Proje uygulamaları).

4.2 Proje Yapısı ve Anahtar Dosyalar

4.2.1 Genel Bakış

Yeni bir ASP.NET Core MVC projesi ("ASP.NET Core Web App (Model-View-Controller)" şablonu) belirli bir klasör iskeletiyle gelir. Bu iskelet konvansiyona dayanır — controller'ları belirli bir yerde bulur, view'ları belirli bir yolda arar. Konvansiyonu bilmek, projede gezinmenin ve yeni parçalar eklerken doğru yerlere koymanın anahtarıdır.

4.2.2 Şablon Klasör Ağacı

```
MyProject/
├── Properties/
│   └── launchSettings.json          # Debug profiller, port, ortam
├── wwwroot/                        # Statik dosyalar (kullanıcıya sunulur)
│   ├── css/
│   ├── js/
│   ├── lib/                        # jQuery, Bootstrap (LibMan/npm)
│   └── favicon.ico
├── Controllers/
│   └── HomeController.cs
├── Models/                         # Entity, ViewModel sınıfları
│   └── ErrorViewModel.cs
├── Views/
│   ├── Home/
│   │   ├── Index.cshtml
│   │   └── Privacy.cshtml
│   └── Shared/
│       ├── _Layout.cshtml
│       └── _ValidationScriptsPartial.cshtml
```

```

|   |   |   | Error.cshtml
|   |   |   | _ViewStart.cshtml
|   |   |   | _ViewImports.cshtml
|   |   | Data/                                # DbContext (elle eklenir)
|   |   |   | AppDbContext.cs
|   |   | Migrations/                          # EF Core migration dosyaları
|   |   | appsettings.json
|   |   | appsettings.Development.json
|   |   | Program.cs
|   |   | MyProject.csproj

```

wwwroot: Web sunucusunun doğrudan sunduğu tek klasör. CSS, JS, resim burada yer alır. Kullanıcı /css/site.css yazınca sunucu bu dosyayı wwwroot/css/site.css'ten bulur. Sunucu-tarafı .cshtml dosyaları burada olmaz.

Controllers: Tüm controller sınıfları. <Ad>Controller.cs konvansiyonu — sınıf adı Controller sonekiyle biter.

Models: Entity, ViewModel, DTO sınıfları. Bazı büyük projelerde Entities/, ViewModels/, DTOs/ şeklinde ayrılır.

Views: Razor view dosyaları. Alt klasör isimleri controller adlarıyla eşleşir (HomeController → Views/Home/). Shared/ klasörü tüm controller'lara açıktır (layout, partial, error).

Data: DbContext ve migration dosyaları. Şablonda gelmez; elle oluşturulur.

appsettings.json: Konfigürasyon (connection string, log ayarları). Ortama özel override için appsettings.<Environment>.json.

Program.cs: Uygulamanın giriş noktası. Servis kaydı, middleware pipeline, Run() (§4.3).

MyProject.csproj: Proje dosyası; SDK sürümü, hedef framework, paket referansları.

4.2.3 Konvansiyon Detayları

Controller-View eşleşmesi:

- HomeController.Index() → Views/Home/Index.cshtml aranır.
- Bulamazsa Views/Shared/Index.cshtml denenir.
- Yoksa InvalidOperationException fırlar ("view was not found").

Alt klasör konvansiyonu:

- View adı controller adıyla eşleşen alt klasörde aranır.
- Ortak partial'lar Views/Shared/'da.
- Views/_ViewStart.cshtml her view'ın en üstünde otomatik çalışır (§4.9).
- Views/_ViewImports.cshtml tüm view'lara import ekler.

Statik dosya konvansiyonu:

- Sadece wwwroot altındakiler URL'den erişilebilir.
- Views/, Models/ klasörleri kullanıcıya kapalıdır (sunucu-tarafı kod).

4.2.4 launchSettings.json

Debug profillerini tutar. Sürüm kontrolüne girer ama kişisel port tercihleri karışabilir.

```

{
  "profiles": {
    "http": {
      "commandName": "Project",
      "dotnetRunMessages": true,
      "launchBrowser": true,
      "applicationUrl": "http://localhost:5000",
      "environmentVariables": {
        "ASPNETCORE_ENVIRONMENT": "Development"
      }
    }
  }
}

```

```

    },
    "https": {
      "commandName": "Project",
      "launchBrowser": true,
      "applicationUrl": "https://localhost:5001;http://localhost:5000",
      "environmentVariables": {
        "ASPNETCORE_ENVIRONMENT": "Development"
      }
    },
    "IIS Express": {
      "commandName": "IISExpress",
      "launchBrowser": true,
      "environmentVariables": {
        "ASPNETCORE_ENVIRONMENT": "Development"
      }
    }
  }
}

```

ASPNETCORE_ENVIRONMENT ortam değişkeni “Development”, “Staging” veya “Production” olabilir. Bu değer `builder.Environment.IsDevelopment()` gibi kontrol noktalarını etkiler.

4.2.5 Proje Dosyası (.csproj)

```

<Project Sdk="Microsoft.NET.Sdk.Web">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
  </PropertyGroup>

  <ItemGroup>
    <PackageReference Include="Microsoft.EntityFrameworkCore.SqlServer" Version="8.0.0" />
    <PackageReference Include="Microsoft.EntityFrameworkCore.Tools" Version="8.0.0" />
    <PackageReference Include="Microsoft.EntityFrameworkCore.Design" Version="8.0.0" />
  </ItemGroup>
</Project>

```

- `Sdk="Microsoft.NET.Sdk.Web"`: Web SDK’sı; MVC dahil default’ları getirir.
- `TargetFramework`: .NET sürümü.
- `Nullable`: Nullable reference types açık (bkz. §2.4.10).
- `ImplicitUsings`: Sık namespace’ler otomatik using edilir (bkz. §2.15.3).

4.2.6 Bölüm Özeti

- MVC proje şablonu konvansiyon üzerine kuruludur.
- `wwwroot` = tarayıcıya açık; diğer klasörler sunucu-tarafı.
- Controller adı → alt view klasörü ile eşleşir.
- `Views/Shared/` her yerden erişilebilir.
- `Program.cs` giriş noktası; `appsettings.json` konfigürasyon; `.csproj` proje tanımı.
- `launchSettings.json` debug profilleri (repo’ya girer ama kişisel port tercihi çıkarılabilir).

İlgili konular: §4.3 (`Program.cs`), §4.4 (`appsettings.json`), §4.6 (Routing), §4.8 (Razor view’ları).

4.3 Program.cs (Minimal Hosting)

4.3.1 Genel Bakış

.NET 6+ ile birlikte, ASP.NET Core uygulaması iki dosyaya bölünmüş `Startup.cs` + `Program.cs` şablonundan **tek Program.cs**’ye taşındı. Bu, top-level statements (§2.15.1) üzerine kurulu **mini-**

mal hosting model'dir. Uygulama başlatma iki bölümden oluşur:

1. **Servis kaydı** — `builder.Services.Add...` çağrıları. DI container'a hangi servislerin eklendiği burada bildirilir.
2. **Middleware pipeline** — `app.Use...` çağrıları. Her isteğin geçeceği katmanlar burada tanımlanır.

Bu ayrım, servis kaydı ile pipeline yapılandırmasının farklı zaman dilimlerinde çalıştığını yansıtır: önce container inşa edilir (`builder.Build()`), sonra pipeline kurulur.

4.3.2 Klasik MVC Program.cs (.NET 8)

```
// Program.cs
using Microsoft.EntityFrameworkCore;
using MyProject.Data;

var builder = WebApplication.CreateBuilder(args);

// 1) Servis kaydı
builder.Services.AddControllersWithViews();
builder.Services.AddDbContext<AppDbContext>(o =>
    o.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));

var app = builder.Build();

// 2) Middleware pipeline
if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Home/Error");
    app.UseHsts();
}

app.UseHttpsRedirection();
app.UseStaticFiles();

app.UseRouting();

app.UseAuthorization();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");

app.Run();
```

4.3.3 Satır Satır Anlamı

var builder = WebApplication.CreateBuilder(args);

`WebApplicationBuilder` üretir. `builder.Services` (DI container), `builder.Configuration` (appsettings.json + env variables), `builder.Environment` (Development/Production), `builder.Logging` gibi bileşenleri barındırır.

builder.Services.AddControllersWithViews();

MVC hizmetlerini DI container'a kaydeder — controller keşfi, model binder, tag helper, Razor view engine. Bu çağrı yapılmazsa `MapControllerRoute` çalışmaz.

builder.Services.AddDbContext<AppDbContext>(...)

EF Core `DbContext`'i scoped ömürle ekler (§3.5.5).

var app = builder.Build();

DI container inşa edilir; servis kaydı burada kapanır. Bu satırdan sonra `builder.Services.Add...` çağırmak `InvalidOperationException` üretir. Middleware pipeline yapılandırması burada başlar.

```
if (!app.Environment.IsDevelopment()) { app.UseExceptionHandler("/Home/Error");  
app.UseHsts(); }
```

Üretim ortamında (Production) global hata yakalayıcı devreye girer; hatalar `/Home/Error`'a yönlendirilir. HSTS (HTTP Strict Transport Security) HTTPS'i zorlar. Development'ta bunlar açılmaz; geliştirici tam stack trace görsün diye.

```
app.UseHttpsRedirection();
```

HTTP istekleri otomatik olarak HTTPS'e yönlendirilir.

```
app.UseStaticFiles();
```

`wwwroot` altındaki dosyaları (css, js, resim) sunmak için middleware. Bu çağrı olmazsa `/css/site.css` 404 döner.

```
app.UseRouting();
```

Routing middleware — gelen URL'yi endpoint'e eşler. Kesin eşleştirme henüz yapılmaz, sadece hazırlanır.

```
app.UseAuthorization();
```

Yetkilendirme middleware'i. Authentication ile birlikte kullanılır (`app.UseAuthentication()` önce gelir eğer eklenmişse).

```
app.MapControllerRoute(name: "default", pattern: "{controller=Home}/{action=Index}/{id?}");
```

Convention-based route tanımı. URL kalıbını controller/action'a eşler. Detay §4.6'da.

```
app.Run();
```

Uygulamayı ayağa kaldırır ve request beklemeye başlar. Bu satır bloke edicidir; program burada bekler.

4.3.4 Klasik Startup.cs Karşılaştırması

.NET 5 ve öncesindeki iki dosyalı model:

```
// Program.cs (.NET 5)  
public class Program  
{  
    public static void Main(string[] args) => CreateHostBuilder(args).Build().Run();  
  
    public static IHostBuilder CreateHostBuilder(string[] args) =>  
        Host.CreateDefaultBuilder(args)  
            .ConfigureWebHostDefaults(webBuilder => webBuilder.UseStartup<Startup>());  
}  
  
// Startup.cs (.NET 5)  
public class Startup  
{  
    public IConfiguration Configuration { get; }  
    public Startup(IConfiguration configuration) { Configuration = configuration; }  
  
    public void ConfigureServices(IServiceCollection services)  
    {  
        services.AddControllersWithViews();  
        services.AddDbContext<AppDbContext>(...);  
    }  
  
    public void Configure(IApplicationBuilder app, IWebHostEnvironment env)  
    {  
        if (!env.IsDevelopment()) app.UseExceptionHandler("/Home/Error");  
        app.UseHttpsRedirection();  
        app.UseStaticFiles();  
    }  
}
```

```

app.UseRouting();
app.UseAuthorization();
app.UseEndpoints(endpoints =>
{
    endpoints.MapControllerRoute(
        name: "default",
        pattern: "{controller=Home}/{action=Index}/{id?}");
    }
});

```

- ConfigureServices → builder.Services.Add... (Program.cs, ilk yarı).
- Configure → app.Use... (Program.cs, ikinci yarı).
- Uygulama davranışı özdeş; kod dağıtımı farklı.

Master defterin standardı: **.NET 6+ minimal hosting (tek Program.cs)**. Router tanımları ve endpoint mapping bu dosyada yapılır, ayrı bir Startup.cs yoktur.

4.3.5 Sık Yapılan Ek Servis Kayıtları

```

// Session
builder.Services.AddDistributedMemoryCache();
builder.Services.AddSession();

// Newtonsoft.Json (System.Text.Json yerine)
builder.Services.AddControllersWithViews()
    .AddNewtonsoftJson();

// Razor Runtime Compilation (view'ları restart etmeden değiştirebilmek için)
builder.Services.AddControllersWithViews()
    .AddRazorRuntimeCompilation();

// Identity / Authentication
builder.Services.AddDefaultIdentity<IdentityUser>(o => o.SignIn.RequireConfirmedAccount = true)
    .AddEntityFrameworkStores<AppDbContext>();

// Custom service
builder.Services.AddScoped<IDoktorService, DoktorService>();

```

Middleware pipeline'a da eşlik eden Use çağrılar eklenir:

```

app.UseSession();
app.UseAuthentication();
app.UseAuthorization();

```

Middleware sırası kritiktir — bir middleware, kendisinden sonrakileri etkiler (§4.17).

4.3.6 Yaygın Hatalar

Hatalı Kod	Sonuç	Neden	Düzeltilmiş
builder.Services.Add... InvalidOperationException builder.Build() sonrasında	Container kapalı	Container kapalı	Tüm servis kayıtları Build öncesi
UseStaticFiles unutulmuş	CSS/JS 404	Middleware yok	Ekle
UseRouting'ten sonra UseAuthorization, önce middleware'ler	Yetki kontrolü çalışmaz	Sıra yanlış	Routing → Authentication → Authorization → endpoints
MapControllerRoute çağrılmamış	Controller URL'leri 404	Endpoint mapping yok	Ekle

Hatalı Kod	Sonuç	Neden	Düzeltilmiş
Environment kontrolü yanlış	Prod'da dev sayfası veya tersi	ASPNETCORE_ENVIRONMENT değeri kontrol edilmemiş	App.Environment.IsDevelopment

4.3.7 Bölüm Özeti

- .NET 6+ minimal hosting: tek Program.cs, top-level statements.
- İki faz: (1) servis kaydı (builder.Services.Add...), (2) middleware pipeline (app.Use...).
- builder.Build() container'ı kapatır; sonrası pipeline.
- Middleware sırası önemli: static → routing → authentication → authorization → endpoints.
- Klasik Startup.cs = eski model; .NET 8'de tek Program.cs standarttır.
- MVC için AddControllersWithViews + MapControllerRoute zorunlu ikilidir.

İlgili konular: §4.4 (appsettings), §4.6 (Routing detay), §4.16 (DI), §4.17 (Middleware pipeline detay).

4.4 appsettings.json ve ConnectionStrings

4.4.1 Genel Bakış

appsettings.json, uygulamanın konfigürasyon değerlerini tutan JSON dosyasıdır. Connection string, log seviyeleri, API anahtarları (üretimde), özel uygulama ayarları burada yer alır. Kod, bu değerlere IConfiguration üzerinden erişir.

Ortama özel değişkenler için ek dosyalar vardır: appsettings.Development.json, appsettings.Production.json. Aynı anahtar birden fazla dosyada varsa, ortam dosyası temel dosyayı override eder.

4.4.2 Şablon Yapı

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Server=(localdb)\\mssqllocaldb;Database=YOUR_DATABASE;Trusted_Connection=true;Password=yourpassword;"
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning",
      "Microsoft.EntityFrameworkCore.Database.Command": "Information"
    }
  },
  "AllowedHosts": "*"
}
```

ConnectionStrings: EF Core ve Identity özel olarak buraya bakar. Anahtar adı DefaultConnection konvansiyoneldir; başka bir ad kullanılırsa GetConnectionString("digerAd") ile açıkça belirtilir.

Logging.LogLevel: Her namespace veya category için log seviyesi. "Default" genel değeri, spesifik namespace'ler override eder. Microsoft.EntityFrameworkCore.Database.Command seviyesi Information yapılırsa EF Core her SQL sorgusunu log'lar — debug için değerli.

AllowedHosts: Host header validation. * her host'u kabul eder; üretimde spesifik host'lar yazılır.

4.4.3 IConfiguration ile Erişim

Program.cs'te:

```
var connString = builder.Configuration.GetConnectionString("DefaultConnection");
var appAyar = builder.Configuration["AppSettings:SiteAdi"];
var maxDeneme = builder.Configuration.GetValue<int>("AppSettings:MaxDeneme");
```

Controller'da (constructor injection):

```
public class HomeController : Controller
{
    private readonly IConfiguration _config;

    public HomeController(IConfiguration config) => _config = config;

    public IActionResult Index()
    {
        string siteAdi = _config["AppSettings:SiteAdi"];
        return View(model: siteAdi);
    }
}
```

Options Pattern (tercih edilen yöntem):

```
// appsettings.json
{
  "AppSettings": {
    "SiteAdi": "HastaneMVC",
    "MaxDeneme": 3
  }
}

// Models/AppSettings.cs
public class AppSettings
{
    public string SiteAdi { get; set; }
    public int MaxDeneme { get; set; }
}

// Program.cs
builder.Services.Configure<AppSettings>(builder.Configuration.GetSection("AppSettings"));

// Controller
public class HomeController : Controller
{
    private readonly AppSettings _settings;
    public HomeController(IOptions<AppSettings> options) => _settings = options.Value;
}
```

Options pattern, konfigürasyonu tip-güvenli bir sınıfa bind eder; string anahtar aramanın önüne geçer.

4.4.4 Ortama Özel Dosyalar

appsettings.Development.json:

```
{
  "Logging": {
    "LogLevel": {
      "Microsoft.EntityFrameworkCore.Database.Command": "Information"
    }
  }
}
```

appsettings.Production.json:

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Server=prod-sql.internal;Database=hastane;Encrypt=True;..."
  }
}
```

Load sırası (öncelik en yüksekte en düşüğe):

1. Environment variables (ConnectionStrings__DefaultConnection=...).
2. User Secrets (Development'ta) — dotnet user-secrets set "ConnectionStrings:DefaultConnection" "...".
3. appsettings.<Environment>.json.
4. appsettings.json.

4.4.5 Secret Yönetimi

Bir kaç kural:

- Production connection string'i, API anahtarı, parola **asla** appsettings.json'a yazılmaz; bu dosya repository'ye commit'lenir.
- Development ortamında **User Secrets** kullanılır — proje ile ilişkilendirilen ama repo dışında bir dosya.
- Production'da environment variable, Azure Key Vault, AWS Secrets Manager gibi çözümler tercih edilir.

User Secrets kullanımı:

```
dotnet user-secrets init
dotnet user-secrets set "ConnectionStrings:DefaultConnection" "Server=...;User Id=...;Password=..."
```

Bu değerler %APPDATA%\Microsoft\UserSecrets\<projectId>\secrets.json altında tutulur; Configuration otomatik olarak yükler.

4.4.6 Örnek 4.4.1 — Tam Connection String Kullanımı

```
// Program.cs
var builder = WebApplication.CreateBuilder(args);

// ConnectionStrings:DefaultConnection okunuyor
builder.Services.AddDbContext<AppDbContext>(o =>
    o.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));

builder.Services.AddControllersWithViews();

var app = builder.Build();
```

GetConnectionString("DefaultConnection") çağrısı, IConfiguration["ConnectionStrings:DefaultConnection"] kısayoludur; ikisi de aynı değeri döner.

Alternatif adlar ("SqlCon" gibi) kullanılabilir, ancak DefaultConnection konvansiyonu Identity ve şablonlar tarafından beklenir. Master defterin standardı DefaultConnection'dır.

4.4.7 Log Level Seviyeleri

Seviye	Kullanım
Trace	Çok detaylı iç akış; nadiren açılır
Debug	Debug bilgisi; development'ta yaygın
Information	Normal akış olayları
Warning	Beklenmeyen ama fatal olmayan durumlar
Error	Hata; işlem tamamlanmadı
Critical	Ciddi hata; uygulama devam edemeyebilir
None	Log'u kapat

Konvansiyonel dağılım:

- Development: Information (varsayılan).
- Production: Warning — sadece anlamlı olaylar.
- EF Core SQL trace için Microsoft.EntityFrameworkCore.Database.Command'ı geliştirme ortamında Information yap.

4.4.8 Bölüm Özeti

- appsettings.json = uygulama konfigürasyonu.
- IConfiguration üzerinden erişim; Options pattern tip-güvenli tercihtir.
- appsettings.<Environment>.json ortama özel override.
- Öncelik: env variables > user secrets > env-specific > base.
- DefaultConnection konvansiyonel ad.
- Secret'lar repo'ya yazılmaz — user secrets veya vault kullan.

İlgili konular: §3.5.4 (Connection string bileşenleri), §4.3 (Program.cs kullanımı), §4.16 (Options pattern ile DI).

4.5 SDK vs Runtime

4.5.1 Genel Bakış

.NET yükleme paketleri iki türdür: **SDK** ve **Runtime**. İkisi arasındaki farkı bilmek, hem geliştirme makinesi hem üretim sunucusu için doğru kurulumu yapmak açısından temeldir.

- **Runtime**: Derlenmiş .NET uygulamalarını **çalıştırmak** için gerekli minimum bileşenler. CLR, temel kütüphaneler.
- **SDK**: Runtime'ı **içerir** + geliştirici araçları (dotnet CLI, derleyici, project templates, MS-Build).

Kural: geliştirici makinesinde **SDK** kur; kullanıcı / üretim sunucusunda **Runtime** yeter.

4.5.2 Karşılaştırma

Bileşen	SDK	Runtime
Uygulama çalıştırma	☐	☐
dotnet build	☐	☐
dotnet run	☐	☐
dotnet publish	☐	☐
dotnet ef (migration)	☐	☐
Roslyn (C# derleyici)	☐	☐
Boyut	~700 MB	~100 MB

4.5.3 Sürüm Kontrolü

Kurulu sürümleri görmek için:

```
dotnet --version      # aktif SDK sürümü
dotnet --list-sdks    # tüm kurulu SDK'ler
dotnet --list-runtimes # tüm kurulu runtime'lar
dotnet --info         # tam bilgi (SDK + runtime + platform)
```

Beklenen çıktı (örnek):

```
$ dotnet --version
8.0.100

$ dotnet --list-sdks
6.0.418 [C:\Program Files\dotnet\sdk]
7.0.404 [C:\Program Files\dotnet\sdk]
```

```
8.0.100 [C:\Program Files\dotnet\sdk]
```

```
$ dotnet --list-runtimes
Microsoft.AspNetCore.App 8.0.0 [C:\Program Files\dotnet\shared\Microsoft.AspNetCore.App]
Microsoft.NETCore.App 8.0.0 [C:\Program Files\dotnet\shared\Microsoft.NETCore.App]
```

4.5.4 Runtime Türleri

Runtime kendi içinde üç varyanta ayrılır:

Runtime	İçerik	Ne zaman
.NET Runtime	Sadece temel CLR + BCL	Konsol uygulaması
ASP.NET Core Runtime	.NET Runtime + web framework	Web uygulaması (MVC dahil)
.NET Desktop Runtime	.NET Runtime + WPF, WinForms	Windows masaüstü

Bir ASP.NET Core MVC uygulaması için üretim sunucusuna **ASP.NET Core Runtime** gerekir.

4.5.5 Framework-Dependent vs Self-Contained Publish

Uygulama yayınlarken iki mod vardır:

Framework-dependent (varsayılan):

```
dotnet publish -c Release
```

Küçük çıktı; sunucuda ilgili runtime'ın kurulu olmasını gerektirir. Bakım kolay (runtime güncellemesi merkezi).

Self-contained:

```
dotnet publish -c Release -r win-x64 --self-contained true
```

Uygulama kendi runtime'ını yanında taşır; sunucuda .NET yüklü olmasına gerek yok. Çıktı boyutu ~60-100 MB büyür. Runtime güncellemesi her uygulama için ayrı yapılır.

4.5.6 Bölüm Özeti

- SDK: geliştirme için (build/run/publish).
- Runtime: sadece çalıştırma için.
- Geliştirici → SDK, üretim → Runtime yeter.
- Sürüm kontrolü: `dotnet --info`.
- Runtime üç türlü: .NET, ASP.NET Core, Desktop.
- Publish: framework-dependent (küçük, runtime gerekli) veya self-contained (büyük, self-yeterli).

İlgili konular: §4.3 (Program.cs — SDK ile derlenir, Runtime'la çalışır), Ek B (Migration komutları — SDK gerektirir).

4.6 Routing

4.6.1 Genel Bakış

Routing, gelen HTTP request URL'sinin hangi controller ve action'a gideceğini belirleyen bileşendir. ASP.NET Core'da iki yaklaşım vardır:

1. **Convention-based routing** (varsayılan): Program.cs'de tek bir kalıp — `{controller}/{action}/{id?}` — tüm URL'lerin şablonu.
2. **Attribute routing**: Controller ve action'ın üstüne `[Route("...")]` yazarak URL'yi orada belirtme. Web API'ler ve karmaşık URL'ler için tercih edilir.

Master defterin MVC uygulamaları çoğunlukla convention-based routing kullanır; attribute routing tekil özel durumlarda araya girer.

4.6.2 Convention-Based Routing

Program.cs içinde:

```
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");
```

Kalıbın parçaları:

- {controller} — URL'nin ilk segmenti, controller adına eşlenir. Varsayılan Home.
- {action} — İkinci segment, action metoduna eşlenir. Varsayılan Index.
- {id?} — Üçüncü segment, action parametresi. ? opsiyonel işareti.

Örnekler:

URL	Eşleşen
/	HomeController.Index()
/Home	HomeController.Index()
/Home/Privacy	HomeController.Privacy()
/Doktor	DoktorController.Index()
/Doktor/Detay/5	DoktorController.Detay(5)
/Doktor/Create	DoktorController.Create() (GET veya POST)

id parametresi, action metodunun aynı isimli parametresine geçer:

```
public IActionResult Detay(int id) { ... }
// URL: /Doktor/Detay/5 → id = 5
```

4.6.3 Attribute Routing

Sözdizimi:

```
[Route("api/[controller]")]
public class DoktorApiController : Controller
{
    [HttpGet]
    public IActionResult GetAll() { ... }

    [HttpGet("{id}")]
    public IActionResult GetById(int id) { ... }

    [HttpGet("uzmanlik/{uzmanlik}")]
    public IActionResult ByUzmanlik(string uzmanlik) { ... }

    [HttpPost]
    public IActionResult Create([FromBody] Doktor doktor) { ... }
}
```

- [controller] token'ı controller adına genişler (sınıf adından Controller çıkarılır).
- [action] action adına.
- HTTP method attribute'ları (HttpGet, HttpPost) hem method'u hem şablonu ifade edebilir.

Üretilen URL'ler:

Attribute	URL	HTTP Method
[HttpGet] (sınıf üstü api/[controller])	/api/DoktorApi	GET
[HttpGet("{id}")]	/api/DoktorApi/5	GET

Attribute	URL	HTTP Method
[HttpGet("uzmanlik/{uzmanlik}")]	/api/DoktorApi/uzmanlik/Dahiliye	GET
[HttpPost]	/api/DoktorApi	POST

4.6.4 Route Constraints (Kısıtları)

URL parçalarına tip / format kısıtı eklemek:

```
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id:int?}");
```

{id:int?} — id opsiyonel ve int olmalı; /Doktor/Detay/abc eşleşmez, /Doktor/Detay/5 eşleşir.

Yaygın constraint'ler:

Constraint	Anlam	Örnek
int	Tam sayı	{id:int}
long, bool, datetime, decimal, double, float, guid	Belirtilen tip	{tarih:datetime}
alpha	Sadece harf	{ad:alpha}
length(n) / length(min,max)	Karakter uzunluğu	{kod:length(6)}
min(n), max(n), range(min,max)	Sayısal aralık	{yas:range(0,120)}
regex(pattern)	Regex	{kod:regex(^[A-Z]{3}\d{4}\$)}

4.6.5 Örnek 4.6.1 — Katalog Uygulaması

```
public class UrunController : Controller
{
    // /Urun → Index
    public IActionResult Index() { ... }

    // /Urun/Detay/5 → id = 5
    public IActionResult Detay(int id) { ... }

    // /Urun/Ara?kelime=kalem → kelime = "kalem"
    public IActionResult Ara(string kelime) { ... }

    // /Urun/Kategori/Kitap → kategori = "Kitap"
    // Convention-based route ile id yerine kategori parametresi
    public IActionResult Kategori(string kategori) { ... }
}
```

/Urun/Ara?kelime=kalem çağrısında kelime **query string**'ten gelir, {id?} opsiyonel kısmı boş kalır. Query string parametreleri her zaman action parametresine bind edilir (isim uyumu şartıyla).

4.6.6 URL Üretme — Reverse Routing

View veya controller'dan URL üretmek için:

```
// Controller içinde
string url = Url.Action("Detay", "Doktor", new { id = 5 });
// "/Doktor/Detay/5"

// Razor view içinde
@Url.Action("Detay", "Doktor", new { id = Model.Id })
<a asp-controller="Doktor" asp-action="Detay" asp-route-id="@Model.Id">Detay</a>
```

Elle URL string'i yazmak yerine `Url.Action / Tag Helper` kullanmak, route şeması değişince tüm bağlantıların otomatik güncellenmesini sağlar.

4.6.7 Areas – Modüler URL'ler

Büyük projelerde bir grup controller'ı ayrı bir "alan"a ayırmak için:

```
Areas/
├── Admin/
│   ├── Controllers/
│   │   └── DoktorController.cs
│   └── Views/
│       ├── Doktor/
│       │   └── Index.cshtml
│       └── 
└── Musteri/
    ├── Controllers/
    └── Views/
```

Controller'a [Area("Admin")] attribute'u eklenir; route buna göre /Admin/Doktor/Index olur. Program.cs'de area route'u:

```
app.MapControllerRoute(
    name: "areas",
    pattern: "{area:exists}/{controller=Home}/{action=Index}/{id?}");
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");
```

4.6.8 URL'nin Anatomisi

Bir URL'nin farklı kaynaklardan gelen parametrelerini ayırmak önemlidir:

https://ornek.com/Doktor/Detay/5?filter=aktif&sayfa=2

└── scheme + host └── controller/action/route data └── query string ──>

- **Route data** (5): URL kalıbındaki {id?} slotu.
- **Query string** (filter, sayfa): ? sonrası, isteğe bağlı.
- **Form data**: POST isteğinde body içinde. Route ile aynı isimli anahtarlar üzerine yazılabilir.

Action parametresi doldurma önceliği model binding'in sorumluluğunda (§4.13).

4.6.9 Yaygın Hatalar

Hatalı Kod	Sonuç	Neden	Düzeltilmiş
MapControllerRoute çağırısı yok	Tüm controller URL'leri 404	Endpoint yok	Ekle
Attribute route şablonunda [controller] bozuk yazılmış	Route bulunmaz	Token yanlış	[controller] küçük harfle
Action parametre adı URL slot adıyla uyuşmuyor	Parametre 0/null gelir	Model binder eşleştiremez	İsimleri uyumla
[HttpPost] yok, POST istekleri GET olarak eşleşiyor	Yanlış action tetiklenir	HTTP method attribute eksik	[HttpGet] / [HttpPost]
Convention + Attribute route karışıklığı	Öngörülemez URL	Karma tanımlar	Ya tam attribute, ya tam convention

4.6.10 Anti-Pattern ve İdiyomatik Kullanım

Anti-Pattern	İdiyomatik	Neden Daha İyi
View'da elle URL string yazmak	asp-action / asp-controller Tag Helper	Refactoring güvenliği
Her action'a [Route("...")] yazmak (basit MVC'de)	Convention-based {controller}/{action}/{id?}	Kısa, standart
Route kısıtsız {id} — action int bekliyor	{id:int}	404 ile hatalı format ayrılır
Aynı URL için birden fazla action	Attribute veya farklı HTTP method	Belirsizlik yok

4.6.11 Mini Sözlük

- **Route:** URL şablonu ve controller/action eşlemesi.
- **Route data:** URL'nin route slot'larından çıkarılan değerler.
- **Query string:** URL'nin ? sonrası anahtar-değer parçası.
- **Route constraint:** Bir slot'un uyacağı format/tip kuralı.
- **Reverse routing:** Kod içinden URL üretmek (Url.Action, Tag Helper).
- **Area:** Alt-modül; kendi route prefix'i ve kendi klasör yapısı.

4.6.12 Bölüm Özeti

- Convention-based: {controller=Home}/{action=Index}/{id?} — MVC şablonu.
- Attribute routing: [Route(...)], HTTP method attribute'ları — Web API idiyomatik.
- Route constraint (:int, :alpha, :regex) ile format sınırlaması.
- /A/B/C?x=1&y=2 → A=controller, B=action, C=route data, x/y=query string.
- URL üretimi için Url.Action veya Tag Helper (asp-controller, asp-action, asp-route-...).
- Büyük projelerde Areas ile modüler URL yapısı.

İlgili konular: §4.3 (Program.cs — MapControllerRoute), §4.7 (Controller / Action), §4.11 (asp- Tag Helper), §4.13 (Model binding).

4.7 Controller ve ActionResult Tipleri

4.7.1 Genel Bakış

Controller, gelen HTTP isteğini karşılayan sınıftır. Adı <Ad>Controller konvansiyonuna uyar, Controllers/ klasörüne konur ve Microsoft.AspNetCore.Mvc.Controller taban sınıfından türer. Her public metot bir potansiyel **action**'dır; routing motoru URL'yi bu action'lardan birine eşler.

Bir action metodu, HTTP request'i işleyip bir **IActionResult** (veya alt tipi) döner. IActionResult, "bir HTTP cevabı üret" sözleşmesidir; farklı alt tipleri farklı türde cevaplar üretir (HTML view, JSON, dosya indirme, yönlendirme, hata kodu).

4.7.2 Controller Sözdizimi

```
using Microsoft.AspNetCore.Mvc;

public class HomeController : Controller
{
    // GET: /Home veya /
    public IActionResult Index()
    {
        return View();
    }

    // GET: /Home/Privacy
    public IActionResult Privacy()
```

```

{
    return View();
}

// GET: /Home/Contact?id=5
public IActionResult Contact(int id)
{
    var model = new ContactVM { Id = id };
    return View(model);
}
}

```

- Controller taban sınıfı, View(), Redirect(), Json() gibi yardımcı metotları getirir.
- Constructor injection ile DI'dan servisler (DbContext, IConfiguration, custom service) alınır.
- [HttpGet] / [HttpPost] attribute'larıyla HTTP method kısıtlanır.

4.7.3 Constructor Injection ile Bağımlılıklar

```

public class DoktorController : Controller
{
    private readonly AppDbContext _context;
    private readonly ILogger<DoktorController> _logger;

    public DoktorController(AppDbContext context, ILogger<DoktorController> logger)
    {
        _context = context;
        _logger = logger;
    }

    public IActionResult Index()
    {
        _logger.LogInformation("Doktor listesi istendi.");
        var liste = _context.Doktorlar.ToList();
        return View(liste);
    }
}

```

Constructor injection, controller'ın bağımlılıklarını dışa açık ve test edilebilir yapar (§4.16 DI detayı).

4.7.4 IActionResult Alt Tipleri — Tam Liste

Katalog C'deki tablonun genişletilmiş ve doğrulanmış hali:

Tip	Yardımcı Metot	Ne yapar
ViewResult	View(), View(model), View("Ad"), View("Ad", model)	Razor view render eder, HTML döner
PartialViewResult	PartialView(), PartialView("Ad", model)	Layout olmadan view render eder (AJAX için)
ContentResult	Content("metin"), Content("x", "text/html")	Ham metin/HTML döner
JsonResult	Json(nesne)	JSON döner (AJAX response için)
FileResult	File(byteDizisi, contentType, fileName), PhysicalFile(...)	Dosya indirme
EmptyResult	new EmptyResult()	Boş 200 OK

Tip	Yardımcı Metot	Ne yapar
RedirectResult	Redirect("/farkli-url")	302 yönlendirme (mutlak/relatif URL)
LocalRedirectResult	LocalRedirect("~/Home/Index")	Sadece aynı site içi yönlendirme (güvenlik)
RedirectToActionResult	RedirectToAction("Index", "Doktor", new { id = 5 })	Controller/action'a yönlendirme
RedirectToRouteResult	RedirectToRoute("routeAdi", new { ... })	Named route'a yönlendirme
RedirectToPageResult	RedirectToPage("...")	Razor Page'e yönlendirme
NotFoundResult	NotFound(), NotFound(mesaj)	404
BadRequestResult	BadRequest(), BadRequest(ModelState)	400
UnauthorizedResult	Unauthorized()	401
ForbiddenResult	Forbidden()	403
StatusCodeResult	StatusCode(500)	Belirli HTTP status kodu
OkResult / OkObjectResult	Ok(), Ok(nesne)	200 OK; parametreliliyse nesneyi serialize eder
CreatedResult	Created(uri, nesne)	201 Created
NoContentResult	NoContent()	204 No Content
AcceptedResult	Accepted(...)	202 Accepted

Bu metotların çoğu Controller taban sınıfının üyesidir; kısaltılmış çağrıyla kullanılır: `return View(model);`.

4.7.5 Örnek 4.7.1 — Farklı ActionResult'lar

```
public class OrneklerController : Controller
{
    // View döner (Razor)
    public IActionResult View1()
    {
        var kisi = new { Ad = "Ali", Yas = 30 };
        return View(kisi);
    }

    // JSON döner (AJAX için)
    public IActionResult Json1()
    {
        var veri = new { Basarili = true, Mesaj = "Kayıt eklendi", Id = 42 };
        return Json(veri);
    }

    // Ham HTML/metin
    public IActionResult Content1()
    {
        return Content("<h1>Merhaba</h1>", "text/html");
    }

    // Dosya indirme
    public IActionResult File1()
    {
        byte[] icerik = System.IO.File.ReadAllBytes("wwwroot/rapor.pdf");
        return File(icerik, "application/pdf", "rapor.pdf");
    }

    // Yönlendirme
    public IActionResult Yonlendir()
    {

```

```

        return RedirectToAction("Index", "Home");
    }

    // 404
    public IActionResult YokBul(int id)
    {
        var kayit = _context.Doktorlar.Find(id);
        if (kayit == null) return NotFound();
        return View(kayit);
    }
}

```

4.7.6 View Yükleme Konvansiyonu

return View() çağrısında:

1. **View adı:** Action metodunun adıyla aynı (Index() → Index.cshtml). View("BaskaAd") ile override edilir.
2. **Aranan yollar** (sırayla):
 - Views/<ControllerAd>/<ViewAd>.cshtml
 - Views/Shared/<ViewAd>.cshtml
3. Bulunamazsa InvalidOperationException: The view '...' was not found.

View(model) çağrısında model, view'a Model property'si olarak geçer:

```

// Controller
public IActionResult Detay(int id)
{
    var d = _context.Doktorlar.Find(id);
    return View(d); // Views/Doktor/Detay.cshtml çağrılır, @Model = d
}

```

```

<!-- Views/Doktor/Detay.cshtml -->
@model Doktor
<h2>@Model.Ad</h2>

```

4.7.7 HTTP Method Attribute'ları

Bir action'ın hangi HTTP method'una cevap vereceğini belirlemek için:

```

[HttpGet] // sadece GET
public IActionResult Create() { return View(); }

[HttpPost] // sadece POST
[ValidateAntiForgeryToken] // CSRF koruması (§4.18)
public IActionResult Create(Doktor form) { ... }

[HttpDelete] // Web API'ler için
public IActionResult Delete(int id) { ... }

// Herhangi biri
[AcceptVerbs("Get", "Post")]
public IActionResult Karma() { ... }

```

Aynı isimde iki action varsa (GET Create + POST Create), farklı HTTP method attribute'larıyla ayırmak zorunludur. Aksi halde routing "iki eşleşen action" görür ve AmbiguousMatchException fırlatır.

4.7.8 GET-POST-Redirect Deseni (PRG)

Form gönderdikten sonra doğrudan view döndürmek yerine yönlendirme yapmak, "F5 refresh formu iki kez gönderir" sorununu önler:

```
[HttpPost]
public IActionResult Create(Doktor form)
{
    if (!ModelState.IsValid) return View(form);

    _context.Doktorlar.Add(form);
    _context.SaveChanges();

    TempData["Success"] = "Kayıt eklendi.";
    return RedirectToAction(nameof(Index)); // ← view değil, redirect
}
```

Bu, “Post-Redirect-Get” (PRG) desenidir; modern MVC uygulamalarının standardıdır. TempData ile mesaj (§4.10) yönlendirilen sayfaya taşınır.

4.7.9 Action Parametre Türleri

Bir action’a parametre üç kaynaktan gelebilir:

Kaynak	Örnek	Attribute (opsiyonel)
Route data	/Detay/5 → int id	[FromRoute]
Query string	?ad=Ali&yas=30 → string ad, int yas	[FromQuery]
Form body	POST <form> alanları	[FromForm]
JSON body	API request body	[FromBody]
Header	Authorization: ...	[FromHeader]
Services (DI)	Servis nesnesi	[FromServices]

Model binding, action parametre isimlerine bakarak uygun kaynaklardan doldurur (§4.13). Attribute’lar sadece belirsizlik durumunda gerekir.

4.7.10 Async Action Metotları

I/O bekleyen (DB, ağ) işler için async metotlar tercih edilir; thread’ler DB beklerken serbest kalır, ölçeklenme iyileşir.

```
public async Task<IActionResult> Index()
{
    var liste = await _context.Doktorlar.ToListAsync();
    return View(liste);
}

[HttpPost]
public async Task<IActionResult> Create(Doktor form)
{
    if (!ModelState.IsValid) return View(form);
    _context.Doktorlar.Add(form);
    await _context.SaveChangesAsync();
    return RedirectToAction(nameof(Index));
}
```

EF Core ToListAsync, FirstOrDefaultAsync, SaveChangesAsync gibi async metotlar Microsoft.EntityFrameworkCore namespace’inde bulunur.

4.7.11 [NonAction] Attribute

Bir public metodun action olarak sayılmasını istememek için:

```
[NonAction]
public string HesaplaSlug(string ad) => ad.ToLower().Replace(" ", "-");
```

Bu, controller içi yardımcı metotlar için gerekir. Alternatif: metodu private yap.

4.7.12 Yaygın Hatalar

Hatalı Kod	Hata	Neden	Düzeltilmiş
<code>return View("Ad", model, extra)</code>	Derlenmez	View overload'ı yok	<code>View("Ad", model)</code> — üçüncü parametre yok
Aynı imzalı iki action (GET)	<code>AmbiguousMatchException</code>	Routing ayıramıyor	Farklı HTTP method veya farklı route
<code>View(model)</code> — model yanlış tipte	Runtime: model cast hatası	<code>View @model X</code> bildirimi ile uyuşmuyor	Tipleri hizala
Async metotta void	<code>SynchronizationContext</code> uyarısı	Async void yakalanamaz	<code>async Task<IActionResult></code>
POST action'da <code>[ValidateAntiForgeryToken]</code> yok	CSRF açığı	Token doğrulanmıyor	<code>Attribute</code> 'u ekle veya global filter

4.7.13 Bölüm Özeti

- Controller <Ad>Controller — Controllers/ altında, Controller'dan türer.
- Her public metod bir action; `IActionResult` döndürür.
- Farklı sonuç tipleri: View, PartialView, Json, File, Redirect*, NotFound, BadRequest, ...
- View konvansiyonu: Views/<Controller>/<Action>.cshtml.
- HTTP method attribute'ları: `[HttpGet]`, `[HttpPost]`, ...
- POST sonrası `RedirectToAction` — PRG deseni.
- I/O yoğun action'lar `async Task<IActionResult>`.

İlgili konular: §4.8 (Razor view), §4.10 (Model geçme), §4.13 (Model binding), §4.16 (Constructor injection), §4.18 (Anti-forgery).

4.8 Razor View Engine

4.8.1 Genel Bakış

Razor, ASP.NET Core'un view template motorudur. HTML iskelet üzerine @ karakteriyle işaretlenmiş C# ifadelerini gömerek dinamik sayfa üretir. .cshtml uzantılı dosyalarda çalışır. Klasik `Response.Write` veya string birleştirme yerine, HTML ve C#'ı harmanlanmış biçimde okumaya doğal bir sözdizim sunar.

Razor kodu server'da derlenir; browser'a yalnızca üretilen HTML gider. Yani `@Model.Ad` yazımı server-side'da çözülür, browser bunu görmez.

4.8.2 Temel @ Sözdizimleri

Değer ifadesi:

```
<p>Hoş geldin, @Model.Ad!</p>
<p>Yaş: @Model.Yas</p>
```

`@Model.Ad` çıktıyı otomatik olarak HTML-escape eder (XSS koruması); `<script>` içerse `<script>` olarak yazılır.

Ham HTML (escape etmeden):

```
<div>@Html.Raw(Model.HtmlIcerik)</div>
```

Sadece güvendiğin veri (kendi ürettiğin HTML) için kullan; kullanıcı girdisi için asla.

Bloc ifadesi (@{ ... }):

```
@{
    var isim = "Ali";
    var yasEklenmis = Model.Yas + 1;
    ViewData["Title"] = "Detay";
}
```

```
}  
<p>@isim - @yasEklenmis</p>
```

Bloklar C# kodu yürütür; içeriği HTML üretmez (aksine belirtilmedikçe).

Kontrol akışı:

```
@if (Model.Aktif)  
{  
    <span class="badge bg-success">Aktif</span>  
}  
else  
{  
    <span class="badge bg-secondary">Pasif</span>  
}  
  
@foreach (var d in Model)  
{  
    <li>@d.Ad</li>  
}  
  
@for (int i = 0; i < 5; i++)  
{  
    <span>Adım @i</span>  
}  
  
@switch (Model.Tip)  
{  
    case "A": <p>Tip A</p> break;  
    case "B": <p>Tip B</p> break;  
    default: <p>Bilinmiyor</p> break;  
}
```

C#'ta olduğu gibi { } blokları kullanılır; her { } bir HTML/text içeriği sarabilir.

4.8.3 @model Direktifi

View'ın çalışacağı model tipini belirler:

```
@model Doktor  
  
<h1>@Model.Ad</h1>  
<p>Uzmanlık: @Model.Uzmanlik</p>
```

Küçük harfli @model (direktif) ile büyük harfli @Model (nesne) karışmaz:

- @model Doktor — tip bildirimi (dosya başında bir kez).
- @Model.Ad — nesnenin property'sine erişim.

Koleksiyon için:

```
@model IEnumerable<Doktor>  
  
<ul>  
    @foreach (var d in Model)  
    {  
        <li>@d.Ad</li>  
    }  
</ul>
```

Anonymous type modeli için tip belirtilemez (tip runtime'da bilinir); onun yerine dynamic veya ViewBag kullanılır.

4.8.4 Örnek 4.8.1 — Tam View

```
@* Views/Doktor/Detay.cshtml *@
@model Doktor

@{
    ViewData["Title"] = $"Doktor Detay: {Model.Ad}";
}

<h1>@Model.Ad</h1>
<p><strong>Uzmanlık:</strong> @Model.Uzmanlik</p>

<h3>Randevular</h3>
@if (Model.Randevular?.Any() == true)
{
    <ul>
        @foreach (var r in Model.Randevular.OrderBy(x => x.Tarih))
        {
            <li>@r.Tarih.ToString("dd.MM.yyyy HH:mm") - @r.HastaAdi</li>
        }
    </ul>
}
else
{
    <p class="text-muted">Randevu bulunmuyor.</p>
}

<a asp-action="Index" class="btn btn-secondary">Geri</a>
<a asp-action="Edit" asp-route-id="@Model.Id" class="btn btn-primary">Düzenle</a>
```

Yorum: @* *@ Razor yorumudur; HTML çıktısına yazılmaz. <!-- --> HTML yorumudur; browser'a gider.

4.8.5 Karakter Kaçırma

Razor @ karakterini takip eden şeyi C# olarak yorumlar. Bir e-posta veya sabit @ yazmak için:

```
<p>İletişim: ali@ornek.com</p>          <!-- @@ ile kaç -->

<p>@("example")</p>                    <!-- literal string olarak -->
```

Çıktı:

```
İletişim: ali@ornek.com
@example
```

4.8.6 Explicit Expression @()

Bir ifadenin nerede bittiğini derleyiciye açıkça belirtmek gerekirse:

```
<p>Fiyat: @(Model.Fiyat * 1.20)</p>
<p>@(Model.Ad).uzantı</p>
```

@Model.Fiyat * 1.20 yazımı da çalışır ama okunabilirlik için parantez tercih edilebilir.

4.8.7 @await — Asenkron Çağrı

Razor içinde async partial veya component render'ı için:

```
@await Html.PartialAsync("_Menu")
@await Component.InvokeAsync("Sepet", new { userId = User.Identity.Name })
```

Bunlar view boyunca async akışı bozmadan işler.

4.8.8 View’da Kullanılan Yerleşik Property’ler

Razor view’ında Controller taban sınıfının değil, view’a özel bir `RazorPage<TModel>` sınıfının içindedir. Ondan gelen property’ler:

Property	Ne verir
Model	Controller’den geçen model
ViewData / ViewBag	Sözlük-tabanlı veri (§4.10)
TempData	Yönlendirmede taşınan veri (§4.10)
Url	URL üretim helper’ı
Html	HTML helper’ı (klasik)
User	Mevcut kullanıcı (ClaimsPrincipal)
Context / HttpContext	Request/response bağlamı
IsSectionDefined("SectionAd")	Section var mı kontrolü

4.8.9 Razor Kodu Nerede Kalıyor?

Bir `.cshtml` dosyası derleme sırasında C# sınıfına çevrilir (`_Layout.cshtml` → `_Layout` sınıfı, `RazorPage<TModel>`’den türer). Her HTML parçası `WriteLiteral(...)`, her `@` ifadesi `Write(...)` çağrısına döner. Çıktı, gelen response stream’ine yazılır.

Development ortamında `AddRazorRuntimeCompilation()` ile view’lar restart etmeden yeniden derlenir (§Ek D):

```
builder.Services.AddControllersWithViews()
                .AddRazorRuntimeCompilation();
```

Bu paketi (`Microsoft.AspNetCore.Mvc.Razor.RuntimeCompilation`) proje bağımlılığı olarak eklemek gerekir.

4.8.10 Yaygın Hatalar

Hatalı Kod	Sonuç	Neden	Düzeltilmiş
<code>@Model = ...</code>	Derleme hatası	Model read-only	Controller’da ata, view’da okumaya kal
<code>@Html.Raw(kullaniciGirdisi)</code>	XSS zafiyeti	HTML işlenir	<code>@kullaniciGirdisi</code> (auto-escape)
<code>@model</code> yerine <code>@Model</code> (direktif olarak)	Derleme hatası	Küçük harfle	<code>@model</code> Doktor
Model null, <code>@Model.Ad</code>	<code>NullReferenceException</code>	Null-koruma yok	<code>@Model?.Ad ?? "(yok)"</code>
Runtime’da view bulunamıyor (<code>InvalidOperationException</code>)	View path yanlış	Klasör/isim uyumsuz	<code>Views/<Ctrl>/<Action>.cshtml</code>

4.8.11 Anti-Pattern ve İdiyomatik Kullanım

Anti-Pattern	İdiyomatik	Neden Daha İyi
View’da veritabanı sorgusu Uzun if / for blokları içinde HTML	Controller’da yap, model’i geç Partial view’a çıkar	Katman izolasyonu Okunabilirlik
<code>@Html.Raw</code> kullanıcı verisiyle String interpolation’la HTML üretmek (<code>\$"{x}"</code>)	Auto-escape (<code>@x</code>) Ayrı <code>@x</code> etiketi	XSS güvenliği Escape korunur

4.8.12 Bölüm Özeti

- Razor = HTML + @ ile C# gömme.

- @degisken, @method(), @{ ... }, @if, @foreach en temel biçimler.
- @model X view'ın çalışacağı tipi belirtir; @Model.Prop erişim.
- @ otomatik HTML-escape eder; @Html.Raw bunu bypass eder — kullanıcı verisiyle asla.
- Razor derlenip C# sınıfına döner; response stream'e HTML yazılır.
- Runtime compilation, restart olmadan view değişikliklerini görmeye izin verir.

İlgili konular: §4.9 (Layout / Sections / Partial), §4.10 (ViewBag / ViewData / Model), §4.11 (Tag Helpers), Ek D (Runtime Compilation).

4.9 Layout, _ViewStart, _ViewImports, Sections, Partial View

4.9.1 Genel Bakış

Bir MVC uygulamasındaki tüm view'lar aynı üst yapıyı (header, nav, footer) paylaşır. Bu ortak iskelet **Layout** dosyasında toplanır; her view yalnız kendine özgü içeriği yazar, gerisini Layout üstlenir.

Bunun yanında Razor birkaç özel dosya türü tanır:

- **_ViewStart.cshtml**: Her view'ın en üstünde otomatik çalışan kod.
- **_ViewImports.cshtml**: Tüm view'lara import edilecek using'ler, Tag Helper kayıtları.
- **Partial view** (_Ad.cshtml): Küçük, tekrar kullanılabilir view parçası.
- **@section**: Layout'un ayrılmış bir bölgesine view'dan içerik enjekte etme.

Konvansiyon: Layout ve partial dosya adları alt çizgiyle başlar (_Layout, _ViewStart, _ValidationScriptsPartial) — bu, bunların tek başına bir URL'e karşılık gelmediğinin işaretidir.

4.9.2 _Layout.cshtml

Konum: Views/Shared/_Layout.cshtml.

```
<!DOCTYPE html>
<html lang="tr">
<head>
  <meta charset="utf-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>@ViewData["Title"] - MyProject</title>
  <link rel="stylesheet" href="~/lib/bootstrap/dist/css/bootstrap.min.css" />
  <link rel="stylesheet" href="~/css/site.css" asp-append-version="true" />
  @await RenderSectionAsync("Styles", required: false)
</head>
<body>
  <header>
    <nav class="navbar navbar-expand-sm navbar-light bg-white border-bottom">
      <div class="container-fluid">
        <a class="navbar-brand" asp-controller="Home" asp-action="Index">MyProject</a>
        <ul class="navbar-nav flex-grow-1">
          <li class="nav-item">
            <a class="nav-link" asp-controller="Home" asp-action="Index">Ana Sayfa</a>
          </li>
          <li class="nav-item">
            <a class="nav-link" asp-controller="Home" asp-action="Privacy">Gizlilik</a>
          </li>
        </ul>
      </div>
    </nav>
  </header>

  <div class="container">
    <main role="main" class="pb-3">
      @RenderBody()
    </main>
  </div>
```

```

<footer class="border-top footer text-muted">
  <div class="container">
    &copy; @DateTime.Now.Year - MyProject
  </div>
</footer>

<script src="~/lib/jquery/dist/jquery.min.js"></script>
<script src="~/lib/bootstrap/dist/js/bootstrap.bundle.min.js"></script>
<script src="~/js/site.js" asp-append-version="true"></script>
@await RenderSectionAsync("Scripts", required: false)
</body>
</html>

```

Anahtar Razor çağrıları:

- @RenderBody() — child view'ın içeriği buraya yerleşir. Her Layout'ta bir kez.
- @RenderSectionAsync("Ad", required) — child view'ın named section'ı buraya yerleşir. required: false ise section yoksa hata vermez.
- @ViewData["Title"] — child view'ın set ettiği başlık.
- asp-append-version="true" — dosyaya cache-busting hash ekler (§4.11).

4.9.3 _ViewStart.cshtml

Konum: Views/_ViewStart.cshtml. Her view'ın en üstünde otomatik çalışır.

```

@{
    Layout = "_Layout";
}

```

Bu tek satır, tüm view'ların varsayılan Layout'unun _Layout.cshtml olduğunu söyler. Belirli bir view'da farklı Layout kullanmak için:

```

@* Views/Doktor/Print.cshtml *@
@{
    Layout = "_PrintLayout"; // veya null yaparak layoutsuz
}

```

Layout aramada aynı konvansiyon çalışır: önce controller'ın klasörü, sonra Views/Shared/.

_ViewStart.cshtml her klasör seviyesinde ayrı ayrı olabilir; en yakın olan geçer. Örnek: Areas/Admin/Views/_ViewStart.cshtml Admin area'sının view'larında geçerlidir.

4.9.4 _ViewImports.cshtml

Konum: Views/_ViewImports.cshtml. Tüm view'lar için ortak using ve Tag Helper kayıtlarını tutar.

```

@using MyProject
@using MyProject.Models
@using MyProject.ViewModels
@addTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers

```

- @using — C# namespace import'u (view'da tam yol yazmaya gerek kalmaz).
- @addTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers — yerleşik Tag Helper'ları aktifler.

_ViewImports de klasör hiyerarşisine göre birleştirilir; alt klasördeki üstündekiyle çakışırsa alt olanı üzerine yazar.

4.9.5 Section'lar — Layout'un Ayrılmış Bölgeleri

Layout birden fazla RenderSection çağrısı içerebilir. Her section, child view'dan içerik almak için özel bir "delik"tir.

Layout'ta:

```

<head>
    ...
    @await RenderSectionAsync("Styles", required: false)
</head>
<body>
    ...
    @RenderBody()
    ...
    @await RenderSectionAsync("Scripts", required: false)
</body>

```

Child view’da:

```

@model Doktor

@section Styles {
    <link rel="stylesheet" href="~/css/doktor.css" />
}

<h1>@Model.Ad</h1>
<div id="grafik"></div>

@section Scripts {
    <script src="~/lib/chartjs/dist/chart.min.js"></script>
    <script>
        $(function () {
            // sayfa-özel grafik kodu
        });
    </script>
}

```

Section içeriği, layout’un ilgili yerine enjekte edilir. Section adı büyük-küçük harfe duyarlı değildir; ama konvansiyon PascalCase’dir.

required: false demek section yoksa hata verme demektir; required: true (varsayılan) section yoksa InvalidOperationException fırlatır. Master defterin standardı: Scripts ve Styles section’ları required: false.

4.9.6 Partial View

Küçük, tekrar kullanılabilir view parçaları. Konum: alt çizgiyle başlar; genellikle Views/Shared/ veya çağırın controller’ın klasöründe.

Örnek — Views/Shared/_DoktorKart.cshtml:

```

@model Doktor

<div class="card mb-2">
    <div class="card-body">
        <h5 class="card-title">@Model.Ad</h5>
        <p class="card-text text-muted">@Model.Uzmanlik</p>
        <a asp-action="Detay" asp-route-id="@Model.Id" class="btn btn-primary btn-sm">Detay</a>
    </div>
</div>

```

Ana view’dan çağırım:

```

@* Views/Doktor/Index.cshtml *@
@model IEnumerable<Doktor>

<h1>Doktorlar</h1>
<div class="row">
    @foreach (var d in Model)
    {

```

```

        <div class="col-md-4">
            <partial name="_DoktorKart" model="d" />
        </div>
    }
</div>

```

<partial> Tag Helper'ı (§4.11) partial view'ı render eder. Klasik alternatifler:

```

@await Html.PartialAsync("_DoktorKart", d)
@Html.Partial("_DoktorKart", d)           <!-- senkron, önerilmez -->

```

Tag Helper (<partial>) modern kodun tercihidir.

Layout'suz olarak render eder — sadece partial view'ın kendi HTML'i çıkar. AJAX ile parçalı yükleme için de aynı partial return PartialView("_DoktorKart", d) ile controller'dan döndürülebilir.

4.9.7 View Component'ler (Kısa Not)

Partial view veri geçmeyi çağırana bırakır (<partial name="X" model="d" />). **View Component** ise kendi verisini kendi çeker; ayrı bir sınıf + view çifti:

```

// ViewComponents/SepetOzetiViewComponent.cs
public class SepetOzetiViewComponent : ViewComponent
{
    private readonly ISepetServis _servis;
    public SepetOzetiViewComponent(ISepetServis s) => _servis = s;

    public async Task<IViewComponentResult> InvokeAsync()
    {
        var ozet = await _servis.OzetGetir();
        return View(ozet); // Views/Shared/Components/SepetOzeti/Default.cshtml
    }
}

```

Kullanım:

```
@await Component.InvokeAsync("SepetOzeti")
```

Karar rehberi:

- Partial: Aynı verinin farklı yerlerde tekrar kullanımı.
- View component: Sayfaya bağımsız (kendi veri kaynağı olan) küçük bileşen — header'daki sepet sayısı, kullanıcı menüsü, son ziyaretler.

4.9.8 Layout Zincirleme

Bir Layout başka bir Layout'u kalıtabilir:

```

@* Views/Shared/_AdminLayout.cshtml *@
@{
    Layout = "_Layout";
}
<div class="admin-panel">
    <aside>...</aside>
    <main>
        @RenderBody()
    </main>
</div>

```

Bu, admin panelin ana site içine yerleşmesini sağlar. Nadiren kullanılır; daha çok Area'lar tercih edilir (§4.6.7).

4.9.9 Yaygın Hatalar

Hatalı Kod	Hata	Neden	Düzeltilmiş
<code>_ViewStart.cshtml</code> 'de <code>Layout = null</code> unutulmuş, <code>print</code> view'ında hâlâ layout görünüyor	Ana layout görünür	Ortak <code>_ViewStart</code> geçerli	<code>Print view</code> 'da <code>Layout = null</code> ; veya <code>Layout = "_PrintLayout"</code>
Section tanımlı ama layout'ta <code>RenderSection</code> yok	Section ignore ediliyor	Layout section'ı tanımıyor	Layout'a <code>@await RenderSectionAsync(...)</code> ekle
<code>RenderSection("X")</code> (<code>required: true</code> default) çağırıp view'da section yok	<code>InvalidOperationException</code>	Zorunlu section yok	<code>required: false</code> yap
Partial view'a model geçmeden çağırma, <code>partial @model Doktor</code> bekliyor	<code>NullReferenceException</code> Model uyumsuz veya cast hatası		<code><partial name="_X" model="d" /></code>
<code>_ViewImports.cshtml</code> 'de <code>@addTagHelper</code> unutulmuş	Tag Helper'lar çalışmıyor	Kayıt yok	Ekle

4.9.10 Bölüm Özeti

- Layout: ortak sayfa iskeleti; `@RenderBody` ile içerik, `@RenderSection` ile bölge.
- `_ViewStart.cshtml`: her view'da otomatik çalışır; Layout ataması burada.
- `_ViewImports.cshtml`: ortak using + Tag Helper kayıtları.
- Section: view'dan layout'un ayrılmış bölgesine içerik.
- Partial view: tekrarlanan HTML parçası; `<partial name="_X" model="d" />`.
- View component: kendi verisi olan yeniden kullanılabilir mini-controller + view.
- Tüm bu dosyalar alt çizgi ile başlar; bağımsız URL'leri yoktur.

İlgili konular: §4.8 (Razor sözdizim), §4.10 (ViewData Title), §4.11 (Tag Helpers), §4.19 (AJAX ile partial dönüş).

4.10 Controller'dan View'a Veri Taşıma

4.10.1 Genel Bakış

Bir controller'dan view'a veri geçirmenin dört yolu vardır:

1. **Strongly-typed model** (`return View(model)`) — tip güvenli, önerilen.
2. **ViewBag** — dinamik property; runtime'da yazılır ve okunur.
3. **ViewData** — sözlük; string anahtar ile veri saklar.
4. **TempData** — sadece bir sonraki request'e taşınan veri; PRG deseninde kullanılır.

Her birinin ömrü, tip güvenliği ve kullanım senaryosu farklıdır. Karar tablosuyla başlayıp her birini detaylandırıyoruz.

4.10.2 Dört Yol Karşılaştırması

Özellik	Model	ViewBag	ViewData	TempData
Tip güvenliği	□ (derleme zamanı)	□ (dynamic)	□ (object)	□ (object)
Ömrü	Tek request	Tek request	Tek request	İki request (bir sonraki taşıyıcı)
Kullanım şekli	<code>@Model.Prop</code>	<code>ViewBag.Prop</code>	<code>ViewData["anahtar"]</code>	<code>TempData["anahtar"]</code>
Ana kullanım	Ana veri	Küçük ek veriler	Layout'ta paylaşılan veri	Yönlendirme sonrası mesaj

Özellik	Model	ViewBag	ViewData	TempData
Depolama	Method parametresi	dynamic (ViewData wrapper'ı)	Dictionary<string,Session tabanlı object>	
IntelliSense	□	□	□	□

4.10.3 Model — Strongly-Typed (Önerilen)

Controller'dan tipli bir nesne geçir; view onu @model direktifiyle bildirir.

Controller:

```
public IActionResult Detay(int id)
{
    var doktor = _context.Doktorlar.Find(id);
    return View(doktor); // Model parametresi
}
```

View:

@model Doktor

```
<h1>@Model.Ad</h1>
<p>Uzmanlık: @Model.Uzmanlik</p>
```

Model, controller ile view arasındaki resmi sözleşmedir. IntelliSense çalışır, refactoring güvenlidir, derleme zamanı hata yakalar. Master defter uygulamalarının varsayılan tavsiyesi budur.

Koleksiyon modeli:

```
public IActionResult Index()
{
    var liste = _context.Doktorlar.ToList();
    return View(liste);
}
```

@model IEnumerable<Doktor>

```
<table>
    @foreach (var d in Model) { <tr><td>@d.Ad</td></tr> }
</table>
```

4.10.4 ViewBag

ViewBag, dynamic bir wrapper'dır; runtime'da istediğin property'yi yazabilir, okuyabilirsin.

Controller:

```
public IActionResult Index()
{
    ViewBag.SiteAdi = "HastaneMVC";
    ViewBag.Yil = DateTime.Now.Year;
    ViewBag.DoktorSayisi = _context.Doktorlar.Count();
    return View(_context.Doktorlar.ToList());
}
```

View:

@model IEnumerable<Doktor>

```
<h1>@ViewBag.SiteAdi</h1>
<p>© @ViewBag.Yil</p>
<p>Toplam doktor: @ViewBag.DoktorSayisi</p>
```

- Property adı yazım hatası runtime'da fark edilir; ViewBag.SiteAdi derlenir, null döner.

- Küçük, tekil ek veriler için pratiktir.
- Ana veri değil, “ekstra ek” gibi düşün.

4.10.5 ViewData

ViewData, sözlük (Dictionary<string, object>) tabanlı bir yapıdır. Aynı depolama alanını ViewBag ile paylaşır; ikisi aynı şeyin iki yüzüdür.

```
public IActionResult Index()
{
    ViewData["SiteAdi"] = "HastaneMVC";
    ViewData["Yil"] = DateTime.Now.Year;
    return View();
}
```

```
<h1>@ViewData["SiteAdi"]</h1>
<p>© @ViewData["Yil"]</p>
```

ViewBag.SiteAdi yazmak, ViewData["SiteAdi"] yazmakla aynıdır — tek fark sözdizimi.

ViewData ne zaman ViewBag’a tercih edilir?

- Anahtar adı bir değişken (string) ise: ViewData[key] =
- Anahtar boşluk içeriyorsa: ViewData["Kullanıcı Adı"]. Bu ViewBag ile olmaz.

4.10.6 TempData

TempData, bir request’te set edilen değer **bir sonraki request’te** okunması için tasarlanmıştır. Session tabanlıdır ve bir kez okunduktan sonra otomatik silinir.

Post-Redirect-Get (PRG) mesajı için ideal:

```
[HttpPost]
public IActionResult Create(Doktor form)
{
    if (!ModelState.IsValid) return View(form);
    _context.Doktorlar.Add(form);
    _context.SaveChanges();

    TempData["Success"] = "Doktor başarıyla eklendi.";
    return RedirectToAction(nameof(Index)); // → yeni request
}

public IActionResult Index()
{
    // İlk okumada TempData["Success"] görülür, sonra silinir
    return View(_context.Doktorlar.ToList());
}
```

Layout’ta gösterim:

```
@* Views/Shared/_Layout.cshtml *@
@if (TempData["Success"] != null)
{
    <div class="alert alert-success">@TempData["Success"]</div>
}
@if (TempData["Error"] != null)
{
    <div class="alert alert-danger">@TempData["Error"]</div>
}
```

TempData, Peek() metoduyla silmeden okunabilir; Keep(anahtar) ile silinmeyi bir sonraki request’e ertelenebilir.

Konfigürasyon: TempData varsayılan olarak Cookie provider’ı kullanır; büyük veri için Session provider’ına geçilebilir:

```
builder.Services.AddSession();
builder.Services.AddControllersWithViews()
    .AddSessionStateTempDataProvider();
```

4.10.7 Model + ViewBag Birlikte

En yaygın desen — ana veri model’de, sayfa başlığı ve küçük ekler ViewBag’de:

```
public IActionResult Detay(int id)
{
    var doktor = _context.Doktorlar.Find(id);
    ViewBag.Baslik = $"Doktor: {doktor.Ad}";
    ViewBag.SonRandevuTarihi = _context.Randevular
        .Where(r => r.DoktorId == id)
        .Max(r => (DateTime?)r.Tarih);
    return View(doktor);
}
```

@model Doktor

```
@{
    ViewData["Title"] = ViewBag.Baslik;
}
```

```
<h1>@Model.Ad</h1>
<p>Son randevu: @ViewBag.SonRandevuTarihi?.ToString("dd.MM.yyyy") ?? "yok"</p>
```

ViewData["Title"], Layout’un <title> tag’ini doldurmak için konvansiyoneldir.

4.10.8 Dropdown / Select Liste Verisi Nasıl Geçirilir?

Bir form’da dropdown göstermek için select liste iki şekilde geçirilebilir:

Kötü yaklaşım — ViewBag (tip güvenli değil):

```
public IActionResult Create()
{
    ViewBag.Uzmanliklar = new SelectList(
        new[] { "Dahiliye", "Cildiye", "Kardiyoloji" });
    return View();
}
```

İyi yaklaşım — ViewModel (§4.14):

```
public class DoktorCreateVM
{
    public Doktor Doktor { get; set; } = new();
    public IEnumerable<SelectListItem> UzmanlikListesi { get; set; } = new List<SelectListItem>()

public IActionResult Create()
{
    var vm = new DoktorCreateVM
    {
        UzmanlikListesi = new[]
        {
            new SelectListItem { Value = "Dahiliye", Text = "Dahiliye" },
            new SelectListItem { Value = "Cildiye", Text = "Cildiye" }
        }
    };
    return View(vm);
}
```

Karışıklığı önlemek için: **koleksiyon veriler ViewModel'e**. §4.14'te ViewModel deseni detaylıca ele alınır.

4.10.9 Karar Rehberi

- **Ana sayfa verisi** → `return View(model);` — daima model.
- **Sayfa başlığı** → `ViewData["Title"] = "..."` — konvansiyonel.
- **Dropdown / SelectList** → ViewModel property'si (ViewBag'e koyma).
- **Başarı/hata mesajı sonrası redirect** → `TempData["Success"]`.
- **Küçük, tekil ek** (yıl, sayaç) → ViewBag/ViewData.
- **Layout'ta gösterilecek global veri** (kullanıcı adı, mesaj) → view component (§4.9.7) veya `_ViewImports` + injected service.

4.10.10 Yaygın Hatalar

Hatalı Kod	Hata	Neden	Düzeltilmiş
<code>@ViewBag.Ad.Length</code> (Ad null)	<code>RuntimeBinderException</code>	ViewBag runtime resolved, null'da patlar	<code>@(ViewBag.Ad?.ToString())?.Length ?? 0</code>
<code>ViewBag.Kayit = doktor</code> — view'da property'lere IntelliSense yok	Yazım hatası runtime'da bulunur	dynamic	<code>return View(doktor);</code> (model)
<code>TempData</code> 'yı ikinci kez oku	null döner	İlk okumada silindi	<code>TempData.Peek(key)</code>
Model tipi uyuşmuyor: controller Doktor, view @model <code>List<Doktor></code>	Runtime cast hatası	İki tarafın da hizası	View'ı @model Doktor yap veya controller'dan koleksiyon geçir
<code>TempData</code> 'ya karmaşık nesne (List, class) atma, cookie provider	Cookie limit aşımı	Cookie ~4KB	Session TempData provider veya küçük string

4.10.11 Anti-Pattern ve İdiyomatik Kullanım

Anti-Pattern	İdiyomatik	Neden Daha İyi
Her şeyi ViewBag ile geçirmek	Model + gerektiğinde ViewBag	IntelliSense + refactoring
Dropdown verileri ViewBag'de Model'e boş nesne + ViewBag ekleri	ViewModel property'si ViewModel — tek sınıf hepsini toplar	Tip güvenli, tek nesne Şeffaf sözleşme
Success mesajını query string'de taşıma	TempData	URL temiz kalır

4.10.12 Bölüm Özeti

- Dört yol: Model (tip güvenli, standard), ViewBag (dynamic), ViewData (dictionary), TempData (bir sonraki request'e taşınır).
- Ana veri **daima Model** ile geçer; ViewModel deseni idealidir (§4.14).
- ViewBag ↔ ViewData aynı depoyu paylaşır; syntactic tercih farkı.
- TempData PRG için tasarlandı — POST sonrası redirect sırasında mesaj taşımak.
- `ViewData["Title"]` layout başlığı için konvansiyoneldir.
- Dropdown / SelectList verileri ViewModel'e; ViewBag'e değil.

İlgili konular: §4.7.8 (PRG pattern), §4.14 (ViewModel), §4.11 (asp-items ile dropdown), §4.22 (Toastr — TempData ile).

4.11 Tag Helpers

4.11.1 Genel Bakış

Tag Helper'lar, HTML etiketlerine asp-* prefix'li attribute'lar ekleyerek Razor kodunu server-side işlem yapılmış HTML'e çeviren bileşenlerdir. Klasik `Html.ActionLink(...)`, `Html.BeginForm(...)`, `Html.LabelFor(...)` gibi HTML Helper'ların yerine, tarayıcının gördüğü HTML'e yakın bir sözdizim getirir:

Klasik HTML Helper:

```
@Html.ActionLink("Detay", "Detay", "Doktor", new { id = Model.Id }, new { @class = "btn btn-primary">
```

Tag Helper:

```
<a asp-controller="Doktor" asp-action="Detay" asp-route-id="@Model.Id" class="btn btn-primary">
```

Tag Helper hem daha okunaklı hem de standart HTML'e çok yakın — front-end geliştirici kod tabanına daha kolay dahil olur. .NET Core 1.0'dan beri modern MVC'nin standart yaklaşımıdır.

4.11.2 Kayıt: `_ViewImports.cshtml`

Tag Helper'lar `_ViewImports.cshtml` içinde aktive edilir (§4.9.4):

```
@addTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers
```

Bu satır olmadan asp-* attribute'ları çalışmaz; sadece düz metin olarak kalır. Yerleşik şablon zaten bu satırı içerir.

4.11.3 Anchor Tag Helper (<a>)

URL üretmek için <a> üzerinde:

Attribute	Anlamı
asp-controller	Hedef controller adı
asp-action	Hedef action adı
asp-area	Alan (area) adı
asp-route- <code><param></code>	Route parametresi
asp-route	Named route adı
asp-fragment	URL sonuna #fragment
asp-host	Host override
asp-protocol	http / https

Örnekler:

```
<!-- /Doktor/Detay/5 -->
<a asp-controller="Doktor" asp-action="Detay" asp-route-id="5">Detay</a>

<!-- Aynı controller içindeyken -->
<a asp-action="Edit" asp-route-id="@Model.Id">Düzenle</a>

<!-- Area ile -->
<a asp-area="Admin" asp-controller="Doktor" asp-action="Index">Yönetim</a>

<!-- Fragment (URL'ye #section ekler) -->
<a asp-action="Detay" asp-route-id="@Model.Id" asp-fragment="randevular">Randevular</a>
```

Üretilen HTML:

```
<a href="/Doktor/Detay/5">Detay</a>
<a href="/Home/Edit/3">Düzenle</a>
<a href="/Admin/Doktor">Yönetim</a>
<a href="/Doktor/Detay/5#randevular">Randevular</a>
```

Yol tekilse ~/Doktor/Detay/5 gibi elle URL yazmak yerine tag helper kullanmak, route şeması değişince tüm bağlantıların otomatik güncellenmesini sağlar.

4.11.4 Form Tag Helper (<form>)

Form için <form> üzerinde:

```
<form asp-controller="Doktor" asp-action="Create" method="post">
    <!-- input alanları -->
    <button type="submit" class="btn btn-primary">Kaydet</button>
</form>
```

Tag Helper otomatik olarak <input name="__RequestVerificationToken" type="hidden" value="..." /> ekler (anti-forgery token, §4.18). Manuel eklemeye gerek yoktur.

Route parametreleri:

```
<form asp-controller="Doktor" asp-action="Edit" asp-route-id="@Model.Id" method="post">
    ...
</form>
```

4.11.5 Input Tag Helper — asp-for

Bir property'ye bağlı input üretmek için:

@model Doktor

```
<div class="mb-3">
    <label asp-for="Ad" class="form-label"></label>
    <input asp-for="Ad" class="form-control" />
    <span asp-validation-for="Ad" class="text-danger"></span>
</div>

<div class="mb-3">
    <label asp-for="Uzmanlik"></label>
    <input asp-for="Uzmanlik" class="form-control" />
    <span asp-validation-for="Uzmanlik" class="text-danger"></span>
</div>
```

asp-for="Ad" iki iş yapar:

1. name="Ad" ve id="Ad" üretir.
2. Modelin Ad property'sinin mevcut değerini value="..." olarak atar.
3. Property'nin C# tipine göre uygun type seçer (int → number, DateTime → datetime-local, bool → checkbox).
4. Data annotation'lardan ([Required], [StringLength]) data-val-* attribute'ları ekler — client-side validation için (§4.15).

Label Tag Helper <label asp-for="Ad"> boş olsa da, [Display(Name = "Doktor Adı")] varsa onu; yoksa property adını (Ad) yazar.

Üretilen HTML (örnek):

```
<div class="mb-3">
    <label for="Ad" class="form-label">Doktor Adı</label>
    <input class="form-control" type="text" id="Ad" name="Ad"
        data-val="true"
        data-val-required="The Ad field is required."
        data-val-length="Ad, maksimum 100 karakter olmalı."
        data-val-length-max="100"
        value="@Model.Ad" />
    <span class="text-danger field-validation-valid" data-valmsg-for="Ad" data-valmsg-replace="t
</div>
```

4.11.6 Select Tag Helper — asp-items

Dropdown için:

```
<select asp-for="UzmanlikId" asp-items="Model.UzmanlikListesi" class="form-select">
    <option value="">-- Seçiniz --</option>
</select>
```

asp-items bir IEnumerable<SelectListItem> bekler. Her SelectListItem'da Value ve Text vardır; opsiyonel Selected işaretlenerek başlangıç seçimi belirtilir.

Controller'da (§4.10.8 karşılığı):

```
public IActionResult Create()
{
    var vm = new DoktorCreateVM
    {
        UzmanlikListesi = _context.Uzmanliklar
            .Select(u => new SelectListItem
            {
                Value = u.Id.ToString(),
                Text = u.Ad
            }).ToList()
    };
    return View(vm);
}
```

SelectList sınıfı üzerinden kısayol:

```
vm.UzmanlikListesi = new SelectList(
    _context.Uzmanliklar,
    nameof(Uzmanlik.Id),
    nameof(Uzmanlik.Ad),
    seciliDeger: null);
```

4.11.7 Textarea Tag Helper

```
<textarea asp-for="Aciklama" class="form-control" rows="5"></textarea>
```

Aynı asp-for iş yapar; sadece etiket farklı.

4.11.8 Validation Tag Helper'ları

```
<div asp-validation-summary="ModelOnly" class="text-danger"></div>
<span asp-validation-for="Ad" class="text-danger"></span>
```

- asp-validation-summary: Formun genel hata özeti. Değerler: None, ModelOnly (property'ye bağlı olmayan hatalar), All (tümü).
- asp-validation-for: Belirli bir property'nin hata mesajı için placeholder. Client-side validation çalıştığında JavaScript bu span'i doldurur.

4.11.9 Environment Tag Helper

Ortama göre HTML üretmek için:

```
<environment include="Development">
    <link rel="stylesheet" href="~/lib/bootstrap/dist/css/bootstrap.css" />
</environment>
<environment exclude="Development">
    <link rel="stylesheet" href="~/lib/bootstrap/dist/css/bootstrap.min.css" asp-append-version=
</environment>
```

Development'ta normal CSS, Production'da minified CSS kullanılır. include / exclude attribute'ları ASPNETCORE_ENVIRONMENT değerine bakar.

4.11.10 Cache-Busting: asp-append-version

Statik dosyanın cache'ini kırmak için, sürüm hash'ini URL'e ekler:

```
<link rel="stylesheet" href="/css/site.css" asp-append-version="true" />
<script src="/js/site.js" asp-append-version="true"></script>
```

Üretilen HTML:

```
<link rel="stylesheet" href="/css/site.css?v=Kl4X2M8..." />
<script src="/js/site.js?v=abc123..."></script>
```

Dosya değiştiğinde hash de değişir; browser yeni sürümü çeker. CSS güncellemeleri hemen görünür — F5 refresh yeterli.

4.11.11 Image Tag Helper

 üzerinde asp-append-version çalışır:

```

```

4.11.12 Cache Tag Helper

Bir HTML bloğunu server-side cache'lemek için:

```
<cache expires-after="@TimeSpan.FromMinutes(5)">
  <div>Şu anki zaman: @DateTime.Now</div>
</cache>
```

Blok içeriği 5 dakika boyunca cache'lenir; her request'te yeniden hesaplanmaz. Kritik senaryolarda daha ince kontrol için vary-by-* attribute'ları kullanılır.

4.11.13 Partial Tag Helper

Partial view render'ı için modern söz dizim:

```
<partial name="_DoktorKart" model="d" />
<partial name="_Filtre" model="Model.Filtre" view-data="ViewData" />
```

Klasik @await Html.PartialAsync("_X", d) çağrısının yerine geçer.

4.11.14 Script ve Link Tag Helper'ları

Fallback desteği ile:

```
<script src="https://cdn.example.com/jquery/3.6.0/jquery.min.js"
  asp-fallback-src="/lib/jquery/dist/jquery.min.js"
  asp-fallback-test="window.jQuery"
  crossorigin="anonymous"></script>
```

CDN erişilemezse local dosya devreye girer. asp-fallback-test bir JS ifadesi — falsy değer dön-erse fallback yüklenir.

4.11.15 Örnek 4.11.1 — Tam CRUD Form'u

```
@* Views/Doktor/Create.cshtml *@
@model DoktorCreateVM
```

```
@{
  ViewData["Title"] = "Yeni Doktor";
}
```

```
<h1>Yeni Doktor</h1>
```

```
<form asp-controller="Doktor" asp-action="Create" method="post">
```

```

<div asp-validation-summary="ModelOnly" class="text-danger"></div>

<div class="mb-3">
  <label asp-for="Doktor.Ad" class="form-label"></label>
  <input asp-for="Doktor.Ad" class="form-control" />
  <span asp-validation-for="Doktor.Ad" class="text-danger"></span>
</div>

<div class="mb-3">
  <label asp-for="Doktor.UzmanlikId" class="form-label"></label>
  <select asp-for="Doktor.UzmanlikId" asp-items="Model.UzmanlikListesi" class="form-select">
    <option value="">-- Uzmanlık Seç --</option>
  </select>
  <span asp-validation-for="Doktor.UzmanlikId" class="text-danger"></span>
</div>

<button type="submit" class="btn btn-primary">Kaydet</button>
<a asp-action="Index" class="btn btn-secondary">İptal</a>
</form>

@section Scripts {
  <partial name="_ValidationScriptsPartial" />
}

```

Bu şablon, master defterin standart Create form’udur; §5.1 ve §5.2 projelerinde tekrar tekrar karşılaşılr.

4.11.16 Ek A ile İlişki

Tüm Tag Helper’ların bir “kısa referans tablosu” için Ek A’ya bakılabilir.

4.11.17 Yaygın Hatalar

Hatalı Kod	Sonuç	Neden	Düzeltilmiş
_ViewImports’ta @addTagHelper unutulmuş	asp-* attribute’ları HTML çıktısında görünüyor	Kayıt yok	Ekle
asp-route-id="5" yerine id="5" (HTML attribute)	Route data’ya gitmiyor	HTML id, route değil	asp-route-id prefix’i doğru
asp-for="Ad" — model tipi property’yi bulamıyor	Runtime cast hatası	Yanlış path	Model tipi kontrol et
asp-append-version build sırasında ortam yok	Version hash boş	IWebHostEnvironment eksik	Standart proje şablonu bunu getirir

4.11.18 Bölüm Özeti

- Tag Helper = asp-* prefix ile server-side işlem yapılmış HTML.
- _ViewImports.cshtml’de @addTagHelper *, ... ile aktif.
- <a asp-controller asp-action asp-route-...> URL üretir.
- <form>, <input>, <select> üzerinde asp-for, asp-items model bağlama.
- asp-append-version cache-busting hash ekler.
- <partial>, <environment>, <cache> yardımcı tag’ler.
- HTML Helper’lar hâlâ çalışır ama modern MVC standardı Tag Helper.

İlgili konular: §4.15 (Validation Tag Helper), §4.14 (ViewModel + SelectListItem), §4.18 (Antiforgery token), Ek A (Tag Helper referansı).

4.12 HTML Form Elemanları

4.12.1 Genel Bakış

Bir MVC formu, HTML form elemanları (<input>, <select>, <textarea>, <button>) ile kurulur. Tag Helper (§4.11) bunları server-side'a bağlar. Bu bölüm her form elemanının HTML semantiği + Tag Helper karşılığını verir.

Bir formun bel kemiği:

```
<form action="/Doktor/Create" method="post">
  <input type="text" name="Ad" />
  <button type="submit">Kaydet</button>
</form>
```

- action: hedef URL.
- method: HTTP method (get veya post).
- name: form alanının anahtar adı; model binding bunu kullanır.

4.12.2 Metin Girişi

Text:

```
<input asp-for="Ad" class="form-control" />
```

asp-for string property'ye bağlıysa üretilen: <input type="text" name="Ad" value="@Model.Ad" ... />.

Password:

```
<input asp-for="Parola" type="password" class="form-control" />
```

Password için [DataType(DataType.Password)] ile property'de belirtilebilir; Tag Helper type="password" üretir.

Email:

```
<input asp-for="Email" class="form-control" />
```

[EmailAddress] veya [DataType(DataType.EmailAddress)] varsa type="email" üretilir.

Number:

```
<input asp-for="Yas" class="form-control" />
```

int, decimal, double tipli property'ler için type="number" otomatik.

Date:

```
<input asp-for="DogumTarihi" class="form-control" />
```

[DataType(DataType.Date)] varsa type="date"; yoksa datetime-local üretilir. HTML5 date picker gösterilir.

4.12.3 Checkbox

```
<div class="form-check">
  <input asp-for="AktifMi" class="form-check-input" />
  <label asp-for="AktifMi" class="form-check-label"></label>
</div>
```

bool tipli property için Tag Helper type="checkbox" üretir + arka planda gizli bir <input type="hidden"> de ekler (checkbox işaretlenirken form'a "false" gelmesi için). Bu davranış özeldir: HTML'de işaretlenmiş checkbox form'a hiçbir şey göndermez; hidden ile "false" gelmesi sağlanır.

4.12.4 Radio Button

```

<div class="form-check">
    <input class="form-check-input" type="radio" asp-for="Cinsiyet" value="E" />
    <label class="form-check-label" for="Cinsiyet_E">Erkek</label>
</div>
<div class="form-check">
    <input class="form-check-input" type="radio" asp-for="Cinsiyet" value="K" />
    <label class="form-check-label" for="Cinsiyet_K">Kadın</label>
</div>

```

Radio grubunda name aynı olmalı; value her seçenek için farklı. Model'in Cinsiyet değeri hangi radio'ya eşitse o seçili gelir.

4.12.5 Textarea (Uzun Metin)

```

<textarea asp-for="Aciklama" class="form-control" rows="4"></textarea>

```

<input>'un aksine textarea çok satırlı. [DataType(DataType.MultilineText)] attribute'u yardımcı olur.

4.12.6 Select (Dropdown)

Tekil seçim:

```

<select asp-for="KategoriId" asp-items="Model.KategoriListesi" class="form-select">
    <option value="">-- Seçiniz --</option>
</select>

```

Çoklu seçim:

```

<select asp-for="RolIds" asp-items="Model.RolListesi" multiple class="form-select">
</select>

```

asp-for bir koleksiyon tipi (List<int>) ise multiple otomatik açılır; form'dan gelen değerler koleksiyon olarak bind edilir.

4.12.7 Hidden

```

<input type="hidden" asp-for="Id" />

```

Formdan gitmesi gereken ama kullanıcıya görünmesi istenmeyen değerler (Id, geldiği sayfa, gizli token) için. **Kritik güvenlik notu:** hidden görünmez, ama browser tools'tan değiştirilebilir. Güvenlik için sadece hidden'a dayanma; server-side'da her zaman doğru.

4.12.8 File Upload

Dosya yükleme için form'a enctype attribute'u:

```

<form asp-action="Upload" method="post" enctype="multipart/form-data">
    <input type="file" name="dosya" class="form-control" />
    <button type="submit" class="btn btn-primary">Yükle</button>
</form>

```

Controller tarafı:

```

[HttpPost]
public async Task<IActionResult> Upload(IFormFile dosya)
{
    if (dosya == null || dosya.Length == 0)
    {
        ModelState.AddModelError("dosya", "Lütfen dosya seçin.");
        return View();
    }

    var yol = Path.Combine("wwwroot/uploads", Path.GetFileName(dosya.FileName));

```

```

using var fs = new FileStream(yol, FileMode.Create);
await dosya.CopyToAsync(fs);

TempData["Success"] = "Dosya yüklendi.";
return RedirectToAction(nameof(Index));
}

```

IFormFile — .NET Core’un dosya upload arayüzü. Property’leri: FileName, Length, ContentType; metodları: OpenReadStream(), CopyToAsync(...).

Kritik: Kullanıcıdan gelen dosya adını **doğrudan disk yoluna koyma**; ../../etc/passwd gibi path traversal saldırısına yol açar. Path.GetFileName ile yalnız dosya adı bileşenini çıkar, ve/veya Guid.NewGuid() ile yeniden adlandır.

4.12.9 Buton Türleri

```

<button type="submit" class="btn btn-primary">Kaydet</button>
<button type="reset" class="btn btn-secondary">Sıfırla</button>
<button type="button" class="btn btn-info" onclick="fnk()">İşlem</button>

```

- type="submit" (varsayılan): formu gönderir.
- type="reset": formu ilk değerlere döndürür.
- type="button": hiçbir şey yapmaz; JS ile bağlanır.

Buton içinde HTML de olabilir: <button type="submit"><i class="bi bi-save"></i> Kaydet</button>.

4.12.10 GET vs POST

- method="get": form verileri URL’de query string olarak gider (?ad=Ali). Arama, filtre için ideal. Yer imi tutulabilir; ama parola gibi hassas veri asla GET ile gönderilmez.
- method="post": veriler request body’de gider. Kayıt, güncelleme, silme için standart. Tarayıcı geri butonu genelde “formu yeniden gönder?” diye sorar.

4.12.11 Örnek 4.12.1 — Karma Form

```

<form asp-action="Kaydet" method="post" enctype="multipart/form-data">
  <input type="hidden" asp-for="Id" />

  <div class="mb-3">
    <label asp-for="Ad" class="form-label"></label>
    <input asp-for="Ad" class="form-control" />
    <span asp-validation-for="Ad" class="text-danger"></span>
  </div>

  <div class="mb-3">
    <label asp-for="DogumTarihi" class="form-label"></label>
    <input asp-for="DogumTarihi" class="form-control" />
  </div>

  <div class="mb-3">
    <label asp-for="Cinsiyet" class="form-label"></label>
    <div class="form-check form-check-inline">
      <input class="form-check-input" type="radio" asp-for="Cinsiyet" value="E" />
      <label class="form-check-label" for="Cinsiyet_E">Erkek</label>
    </div>
    <div class="form-check form-check-inline">
      <input class="form-check-input" type="radio" asp-for="Cinsiyet" value="K" />
      <label class="form-check-label" for="Cinsiyet_K">Kadın</label>
    </div>
  </div>
</form>

```

```

<div class="mb-3">
    <label asp-for="Sehir" class="form-label"></label>
    <select asp-for="Sehir" asp-items="Model.Sehirler" class="form-select">
        <option value="">-- Seçiniz --</option>
    </select>
</div>

<div class="mb-3 form-check">
    <input asp-for="Onay" class="form-check-input" />
    <label asp-for="Onay" class="form-check-label"></label>
</div>

<div class="mb-3">
    <label asp-for="Aciklama" class="form-label"></label>
    <textarea asp-for="Aciklama" class="form-control" rows="3"></textarea>
</div>

<div class="mb-3">
    <label class="form-label">Profil Resmi</label>
    <input type="file" name="resim" class="form-control" />
</div>

<button type="submit" class="btn btn-primary">Kaydet</button>
</form>

```

Controller karşılığı (§4.13 model binding ile):

```

[HttpPost]
public async Task<IActionResult> Kaydet(KullaniciVM vm, IFormFile resim)
{
    if (!ModelState.IsValid) { /* dropdown listeleri yeniden yükle */ return View(vm); }

    // ...form işleme + resim kaydetme
    return RedirectToAction(nameof(Index));
}

```

4.12.12 Yaygın Hatalar

Hatalı Kod	Sonuç	Neden	Düzeltilmiş
File input olan form'da enctype="multipart/form-data" yok	Dosya server'a ulaşmaz	Enctype eksik	enctype="multipart/form-data" ekle
Checkbox işaretli form gönderiliyor, model'de false beklenmiyor	Bool alan bind olmaz gibi görünür	Tag Helper hidden ekliyor, elle yazılınca eksik	asp-for Tag Helper kullan
Radio button'lar farklı name ile	Grup çalışmaz	Tek bir property için tüm radio'lar aynı name	asp-for ile aynı property'ye bind
GET ile parola gönderme	Loglar/URL'de parola görünür	Method yanlış	POST kullan

4.12.13 Bölüm Özeti

- HTML form elemanları: input (metin/sayı/tarih/parola/dosya), textarea, select, checkbox, radio, hidden, button.
- Tag Helper (asp-for, asp-items) form elemanlarını model'e bağlar.
- Checkbox arka planda hidden ekler; işaretli "true" gelir.
- Dosya upload için enctype="multipart/form-data" + IFormFile parametresi.
- POST hassas veri için, GET arama/filtre için.

İlgili konular: §4.11 (Tag Helper detayları), §4.13 (Model Binding), §4.15 (Validation), §4.19 (AJAX form gönderimi).

4.13 Model Binding

4.13.1 Genel Bakış

Model Binding, gelen HTTP request'inin farklı kaynaklarındaki verileri (route, query string, form, JSON body) action metodunun parametrelerine ve model property'lerine otomatik olarak dönüştürüp doldurma sürecidir. Bu, MVC'nin en güçlü kolaylıklarından biridir: bir controller yazan geliştirici, `Request.Form["Ad"]` gibi düşük seviye erişimlere hiç girmez.

Örnek — bir form POST edildiğinde:

```
<form action="/Doktor/Create" method="post">
  <input name="Ad" value="Dr. Ali" />
  <input name="Uzmanlik" value="Dahiliye" />
  <input name="Yas" value="42" />
</form>
```

Controller:

```
[HttpPost]
public IActionResult Create(Doktor doktor)
{
    // doktor.Ad      = "Dr. Ali"   (form'dan)
    // doktor.Uzmanlik = "Dahiliye" (form'dan)
    // doktor.Yas      = 42         (string'den int'e otomatik dönüştürüldü)
    ...
}
```

MVC, Doktor sınıfının property'lerine bakar, form'daki name'leri onlarla eşleştirir, tip dönüşümü yapar, ModelState doldurulur.

4.13.2 Veri Kaynakları ve Öncelik

Bir action parametresi doldurulurken kaynaklar sırayla denenir:

1. **Form values** (POST body — application/x-www-form-urlencoded veya multipart/form-data).
2. **Route values** ({controller}/{action}/{id} gibi URL parçaları).
3. **Query string** (?ad=Ali&yas=30).

Birden fazla kaynak aynı isimde değer taşıyorsa yukarıdaki sıra geçerlidir — form önce, sonra route, sonra query. JSON body ise ayrıca [FromBody] ile açık belirtilir.

4.13.3 Kaynak Kısıtlayıcı Attribute'lar

Belirli bir parametrenin **sadece** bir kaynaktan gelmesini istemek için:

Attribute	Kaynak
[FromRoute]	Route data
[FromQuery]	Query string
[FromForm]	Form body
[FromBody]	JSON/XML body (Web API için)
[FromHeader]	HTTP header
[FromServices]	DI container'dan servis

```
[HttpPost]
public IActionResult Sil([FromRoute] int id, [FromForm] string neden)
{
}
```

```

    ...
}

[HttpGet("api/urunler/{id}")]
public IActionResult Getir([FromRoute] int id, [FromQuery] bool detay)
{
    ...
}

```

4.13.4 Basit Tipli Parametreler

Route veya query'den gelen int, string, DateTime, bool, Guid, enum — otomatik parse edilir. Başarısız olursa parametre default (int → 0, string → null) kalır, ModelState hata ekler.

```

// GET /Urun/Detay/5
public IActionResult Detay(int id)
{
    // id = 5
}

// GET /Urun?ara=kalem&sayfa=2
public IActionResult Liste(string ara, int sayfa = 1)
{
    // ara = "kalem", sayfa = 2
}

```

4.13.5 Karmaşık Model Bağlama

Bir sınıf parametresi verildiğinde, tüm public property'leri sırayla doldurmaya çalışır. Kaynak isimleri property adlarıyla eşleşmeli (case-insensitive).

```

public class DoktorForm
{
    public int Id { get; set; }
    public string Ad { get; set; }
    public string Uzmanlik { get; set; }
    public DateTime BaslangicTarihi { get; set; }
}

[HttpPost]
public IActionResult Kaydet(DoktorForm form)
{
    // form.Id, form.Ad, form.Uzmanlik, form.BaslangicTarihi otomatik doldurulur
}

```

4.13.6 Nested Model Bağlama

İç içe nesneleri bağlamak için isimlendirme "dot" veya "index" notasyonu kullanır:

Nested object:

```

<input name="Doktor.Ad" value="Dr. Ali" />
<input name="Doktor.Uzmanlik" value="Dahiliye" />

public class DoktorCreateVM
{
    public Doktor Doktor { get; set; }
}

[HttpPost]
public IActionResult Create(DoktorCreateVM vm)
{
}

```

```
// vm.Doktor.Ad ve vm.Doktor.Uzmanlik doldurulur
}
```

Tag Helper (asp-for="Doktor.Ad") doğru isim şablonunu otomatik üretir.

Koleksiyon:

```
<input name="Randevular[0].HastaAdi" value="Mehmet" />
<input name="Randevular[0].Tarih" value="2024-06-10" />
<input name="Randevular[1].HastaAdi" value="Fatma" />
<input name="Randevular[1].Tarih" value="2024-06-11" />
```

```
public class DoktorCreateVM
{
    public Doktor Doktor { get; set; }
    public List<Randevu> Randevular { get; set; }
}
```

Model binder, Randevular[0], Randevular[1] isimlerini görüp koleksiyona iki eleman ekler.

4.13.7 [Bind] ve [BindNever]

Bazen bir property'nin bağlama sürecine dahil olmasını istemezsin (özellikle güvenlik için — IsAdmin gibi bir alan formdan gelmesin).

[Bind] — sınıfa veya action parametresine, hangi property'lerin bağlanacağını kısıtlar:

```
[HttpPost]
public IActionResult Create([Bind("Ad,Uzmanlik")] Doktor doktor)
{
    // Sadece Ad ve Uzmanlik doldurulur; diğer property'ler default kalır
}
```

[BindNever] — bir property üzerinde:

```
public class Kullanici
{
    public int Id { get; set; }
    public string KullaniciAdi { get; set; }

    [BindNever]
    public bool IsAdmin { get; set; }
}
```

IsAdmin form'dan gelse bile model binder tarafından yok sayılır.

[BindRequired] — property mutlaka form'da olmalı:

```
[BindRequired]
public int Id { get; set; }
```

Form'da yoksa ModelState hata ekler.

4.13.8 Tip Dönüşüm ve Kültür

Model binder, string değerini hedef tipe çevirmek için TypeConverter altyapısını kullanır. Sayı ve tarih formatı **kültür duyarlıdır**:

- Türkçe kültür: 3,14 (virgül) doğru; 3.14 (nokta) hata.
- Invariant / English kültür: 3.14 doğru; 3,14 hata.

Web sunucusu kültürü, Program.cs'te ayarlanır:

```
var supportedCultures = new[] { new CultureInfo("tr-TR"), new CultureInfo("en-US") };
app.UseRequestLocalization(new RequestLocalizationOptions
{
    DefaultRequestCulture = new RequestCulture("tr-TR"),
```

```
SupportedCultures = supportedCultures,
SupportedUICultures = supportedCultures
});
```

Form/AJAX'ta çoğu zaman **invariant** format tercih edilir (client-server tutarlılığı için):

```
$.ajax({
  data: { fiyat: 19.99 } // dot decimal, invariant
});
```

4.13.9 [FromBody] — JSON API'ler İçin

Web API senaryolarında, request body JSON içerir:

```
fetch("/api/doktorlar", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ ad: "Dr. Ali", uzmanlik: "Dahiliye" })
});
```

```
[HttpPost]
public IActionResult Create([FromBody] DoktorDto dto)
{
  // dto.Ad, dto.Uzmanlik JSON'dan doldurulur
}
```

[ApiController] attribute'u varsa, karmaşık tip parametreler otomatik olarak [FromBody] sayılır — explicit yazmaya gerek kalmaz.

4.13.10 ModelState

Model binding sırasında oluşan tüm dönüşüm hataları ve validasyon sonuçları ModelState sözlüğünde toplanır (§4.15). Controller action bunu kontrol eder:

```
[HttpPost]
public IActionResult Create(Doktor form)
{
  if (!ModelState.IsValid)
  {
    return View(form); // hata mesajlarıyla form'a döner
  }

  _context.Doktorlar.Add(form);
  _context.SaveChanges();
  return RedirectToAction(nameof(Index));
}
```

ModelState.AddModelError("propertyAdi", "mesaj") ile manuel hata eklenebilir.

4.13.11 Yaygın Hatalar

Hatalı Kod	Sonuç	Neden	Düzeltilmiş
Form input name property adıyla uyuşmuyor	Property null/default gelir	Eşleşme kurulamıyor	Aynı isim; Tag Helper asp-for kullan
Fiyat 3,14 girildi, kültür en-US	Bind fail, ModelState hata	Kültür uyumsuz	Kültürü ayarla veya invariant kullan
IsAdmin gibi güvenlik alanı formdan bind edildi	Yetki yükseltme saldırısı	Aşırı bağlama	[BindNever] veya ViewModel'de bulunmasın
POST'a gelen JSON, action [FromBody] yok	Property'ler null	Kaynak seçilmemiş	[FromBody] ekle veya [ApiController]

Hatalı Kod	Sonuç	Neden	Düzeltilmiş
Nested Doktor.Ad yerine Ad gönderildi	Doktor property'si null	İsimlendirme yanlış	Tag Helper asp-for="Doktor.Ad"

4.13.12 Anti-Pattern ve İdiyomatik Kullanım

Anti-Pattern	İdiyomatik	Neden Daha İyi
Entity'yi doğrudan form'a bind (mass assignment riski)	ViewModel arasında	Sadece gereken alanlar
Request.Form["Ad"] ile elle okumak	Model parametresi	Kolay, testlenebilir
[Bind] string listesiyle koruma	ViewModel + entity dönüşüm	Refactoring güvenli
Client'ta virgül/nokta ondalık karışıklığı	Invariant format + data-* bind	Server bağımsız

4.13.13 Bölüm Özeti

- Model binding = HTTP request → action parametreleri otomatik.
- Kaynak sırası: form → route → query. [FromBody] JSON için.
- Basit tipler (int, DateTime, bool) auto-parse; tip dönüşümü kültür duyarlı.
- Nested nesne için Doktor.Ad; koleksiyon için Randevular[0].Alan.
- [BindNever] güvenlik alanlarını dışarıda tut.
- ModelState.IsValid ile bind + validasyon sonucunu kontrol et.
- Entity yerine ViewModel'e bind — mass assignment saldırısına karşı standart savunma.

İlgili konular: §4.11 (asp-for isim üretimi), §4.14 (ViewModel), §4.15 (Model validation), §4.19 (AJAX ile JSON bind).

4.14 ViewModel ve DTO

4.14.1 Genel Bakış

ViewModel, bir view'a özel biçimlendirilmiş, view'ın ihtiyaç duyduğu tüm veriyi tek bir nesnede toplayan sınıftır. Domain entity'sinden (Doktor) farklıdır: view'ın ihtiyacı olan ek alanları (dropdown listeleri, biçimlenmiş metinler) taşır, entity'nin görünmesini istemediğin alanlarını gizler.

DTO (Data Transfer Object), katmanlar arası veri taşıyan sınıftır. ViewModel ile örtüşür — teknik olarak ViewModel bir DTO'dur. Ayrım:

- **DTO**: Genellikle Web API bağlamında; JSON serialize/deserialize edilen “veri paketi”.
- **ViewModel**: MVC bağlamında; bir view'a bağlı, form binding ve validation attribute'ları taşıyan sunum modeli.

Master defterin standardı: ViewModel her form ve liste view'ında; entity'yi doğrudan view'a asla geçirme.

4.14.2 Neden ViewModel?

Entity'yi doğrudan view'a geçirmenin sorunları:

1. **Aşırı bağlama (over-posting)**: Kullanıcı formdan IsDeleted, RolId, CreatedBy gibi güvenlik alanlarını gönderebilir; binder onları da doldurur ([BindNever] unutulmuşsa).
2. **View'ın ihtiyacı olmayan alanlar**: PasswordHash, SecurityStamp gibi hassas alanlar view'a gitmemeli — ama entity'yi geçince gider (Razor render edilmese bile HTTP response'da hazırdr).
3. **View'ın ekstra ihtiyaçları**: Dropdown listeleri, tarih format string'i, hesaplanmış alanlar entity'ye eklemek kirlidir.
4. **Validation ayrımı**: Entity DB kısıtları için validation'a sahipken, view farklı validation ister (parola tekrar alanı, checkbox onayı).

ViewModel bu kaygıları çözer.

4.14.3 Tipik ViewModel Yapısı

```
// ViewModels/DoktorCreateVM.cs
public class DoktorCreateVM
{
    public Doktor Doktor { get; set; } = new();

    // Dropdown verileri
    public IEnumerable<SelectListItem> UzmanlikListesi { get; set; } = new List<SelectListItem>();
    public IEnumerable<SelectListItem> BolumListesi { get; set; } = new List<SelectListItem>();

    // Ek UI durumları
    public bool IsEdit { get; set; }
}
```

Controller kullanımı:

```
public IActionResult Create()
{
    var vm = new DoktorCreateVM
    {
        UzmanlikListesi = _context.Uzmanliklar
            .Select(u => new SelectListItem { Value = u.Id.ToString(), Text = u.Ad })
            .ToList(),
        BolumListesi = _context.Bolumler
            .Select(b => new SelectListItem { Value = b.Id.ToString(), Text = b.Ad })
            .ToList()
    };
    return View(vm);
}

[HttpPost]
public IActionResult Create(DoktorCreateVM vm)
{
    if (!ModelState.IsValid)
    {
        // Dropdown listelerini yeniden yükle çünkü postback'te kayboldu
        vm.UzmanlikListesi = _context.Uzmanliklar.Select(...).ToList();
        vm.BolumListesi = _context.Bolumler.Select(...).ToList();
        return View(vm);
    }

    _context.Doktorlar.Add(vm.Doktor);
    _context.SaveChanges();
    return RedirectToAction(nameof(Index));
}
```

Kritik ayrıntı: Form gönderiminde sadece Doktor alanları geri gelir; UzmanlikListesi boş döner (browser'a giden bilgi form input'undakiyle sınırlıdır). Validation başarısızsa dropdown verilerini yeniden yüklemek zorundasın.

4.14.4 Liste ViewModel'i

Bir liste sayfası için:

```
public class DoktorListeVM
{
    public IEnumerable<Doktor> Doktorlar { get; set; }
    public string AramaMetni { get; set; }
    public int SayfaNo { get; set; }
    public int ToplamSayfa { get; set; }
    public IEnumerable<SelectListItem> UzmanlikFiltreleri { get; set; }
}
```

Filtreleme + sayfalama + arama, tek sınıfta toplandığı için view sadece @Model.Doktorlar, @Model.SayfaNo gibi okumalar yapar.

4.14.5 Entity → ViewModel Dönüşümü

Manuel dönüşüm:

```
public class DoktorDetayVM
{
    public int Id { get; set; }
    public string Ad { get; set; }
    public string UzmanlikAd { get; set; }
    public int RandevuSayisi { get; set; }
    public DateTime? SonRandevu { get; set; }
}

public IActionResult Detay(int id)
{
    var vm = _context.Doktorlar
        .Where(d => d.Id == id)
        .Select(d => new DoktorDetayVM
        {
            Id = d.Id,
            Ad = d.Ad,
            UzmanlikAd = d.Uzmanlik.Ad,
            RandevuSayisi = d.Randevular.Count,
            SonRandevu = d.Randevular.Max(r => (DateTime?)r.Tarih)
        })
        .FirstOrDefault();

    if (vm == null) return NotFound();
    return View(vm);
}
```

EF Core, Select içindeki VM projeksiyonunu SQL'e çevirir — sadece gereken sütunlar çekilir, Include gerekmez.

AutoMapper veya elle mapping — büyük projeler bu dönüşümü otomatikleştirmek için AutoMapper kütüphanesini kullanır. Küçük ve orta projelerde elle yeterlidir.

4.14.6 ViewModel'de Validation Attribute'ları

Entity'de bir property [MaxLength(100)] iken, view'da o alan farklı validation gerektirebilir (min uzunluk, regex). ViewModel bu ayrımı sağlar:

```
public class KullaniciKayitVM
{
    [Required(ErrorMessage = "Kullanıcı adı zorunlu.")]
    [StringLength(20, MinimumLength = 4, ErrorMessage = "4-20 karakter arası.")]
    public string KullaniciAdi { get; set; }

    [Required(ErrorMessage = "E-posta zorunlu.")]
    [EmailAddress(ErrorMessage = "Geçerli e-posta girin.")]
    public string Email { get; set; }

    [Required(ErrorMessage = "Parola zorunlu.")]
    [DataType(DataType.Password)]
    [StringLength(50, MinimumLength = 8)]
    public string Parola { get; set; }

    [Required(ErrorMessage = "Parola tekrar zorunlu.")]
    [DataType(DataType.Password)]
    [Compare(nameof(Parola), ErrorMessage = "Parolalar eşleşmiyor.")]
    public string ParolaTekrar { get; set; }
}
```

```

public string ParolaTekrar { get; set; }

[Range(typeof(bool), "true", "true", ErrorMessage = "Kullanım koşullarını onaylamalısınız.")]
public bool Onay { get; set; }
}

```

ParolaTekrar ve Onay entity’de yer almazlar; sadece kullanıcı deneyiminin gereğidir. ViewModel’in tek sorumluluğu bu tür alanları taşımaktır.

4.14.7 Örnek 4.14.1 — Karışık Kullanım

Bir hastane randevu formu:

```

// Models/Entities/Randevu.cs (entity – DB'ye eşlenir)
public class Randevu
{
    public int Id { get; set; }
    public int DoktorId { get; set; }
    public Doktor Doktor { get; set; }
    public string HastaAdi { get; set; }
    public string HastaTelefonu { get; set; }
    public DateTime Tarih { get; set; }
    public bool IsDeleted { get; set; } // entity'de var, view'da yok
}

// ViewModels/RandevuCreateVM.cs
public class RandevuCreateVM
{
    public Randevu Randevu { get; set; } = new();
    public IEnumerable<SelectListItem> DoktorListesi { get; set; }
    public string SecilenTarihMetni { get; set; } // JS için ISO format
}

```

Görüldüğü gibi entity DB detayları (IsDeleted, navigation) taşır; ViewModel view’ın gerçekten ihtiyacı olan alanları toplar. IsDeleted form binding’ine hiç ulaşmaz — güvenlik alanı olarak korunur.

4.14.8 Konum Konvansiyonu

Küçük projelerde tek Models/ klasörü altında karışık tutulur. Orta ve büyük projelerde:

```

Models/
├── Entities/           # DB'ye eşlenen sınıflar
│   ├── Doktor.cs
│   └── Randevu.cs
├── ViewModels/        # View'a özel
│   ├── DoktorCreateVM.cs
│   ├── DoktorDetayVM.cs
│   └── RandevuListeVM.cs
└── DTOs/              # (varsa) API için
    └── DoktorDto.cs

```

Bu ayrım, “hangi sınıf DB’ye, hangisi view’a?” sorusunu her seferinde tekrar sormaktan kurtarır.

4.14.9 Yaygın Hatalar

Hatalı Kod	Sonuç	Neden	Düzeltilmiş
Postback’te dropdown listeleri yeniden yüklenmedi	Post-back’te dropdown boş	Model gelen VM’de sadece Doktor property’si dolu	Dropdown’ları controller’da tekrar yükle

Hatalı Kod	Sonuç	Neden	Düzeltilmiş
Entity'yi view'a doğrudan geçirmek + form'a bind	Over-posting saldırısı	Bind kısıtı yok	ViewModel kullan
ViewModel'e her yeni alan eklenince entity'yi kopyalama unutulmak	Yeni alan DB'ye yazılmaz	Elle mapping tutarsız	AutoMapper veya çok sayıda test
ViewModel'de DB navigation property'leri	Recursion, serialization sorunları	ViewModel POCO olmalı	Sadece scalar veya SelectListItem

4.14.10 Bölüm Özeti

- ViewModel = view'a özel sınıf; entity ile view arasında ara katman.
- DTO = veri paketi; ViewModel'e benzer, çoğu zaman API bağlamında.
- Entity'yi form'a bağlamak over-posting saldırısına açar; ViewModel bunu önler.
- Dropdown listeleri, ek UI alanları, farklı validation kuralları ViewModel'de.
- Postback'te (ModelState invalid) dropdown listelerini yeniden yükle — form'dan gelmedi.
- Entity → ViewModel dönüşümü LINQ Select ile SQL'e projekte edilebilir.

İlgili konular: §4.13 (Mass assignment savunması), §4.10 (Model geçme), §4.11 (asp-items), §4.15 (Validation), §5.1 / §5.2 (Proje ViewModel örnekleri).

4.15 Model Validation

4.15.1 Genel Bakış

Model validation, form'dan gelen verinin uygulama iş kurallarına uygunluğunu kontrol etme sürecidir. ASP.NET Core MVC'de üç aşamalıdır:

1. **Data Annotations** ile kural bildirimi (§3.7): [Required], [StringLength], [Range], [EmailAddress].
2. **Client-side validation** (unobtrusive validation): Form gönderilmeden JavaScript ile kontrol; kötü veriyi gitmeden yakalar.
3. **Server-side validation**: Client'a güvenilmez — controller action ModelState.IsValid ile aynı kuralları sunucuda tekrar kontrol eder.

Prensip: **Client-side kolaylık için, server-side güvenlik için.** İkisi de aynı Data Annotation'lardan besleneceği için tek yerde yazılır, iki yerde çalışır.

4.15.2 Data Annotations ile Kural

Bkz. §3.7 detayları. Tekrar en sık kullanılanlar:

```
public class DoktorForm
{
    public int Id { get; set; }

    [Required(ErrorMessage = "Ad zorunlu.")]
    [StringLength(100, MinimumLength = 2)]
    [Display(Name = "Doktor Adı")]
    public string Ad { get; set; }

    [Required(ErrorMessage = "Uzmanlık seçin.")]
    public int UzmanlikId { get; set; }

    [Phone(ErrorMessage = "Geçerli telefon numarası.")]
    public string Telefon { get; set; }

    [EmailAddress]
```

```

public string Email { get; set; }

[Range(0, 120)]
public int Yas { get; set; }
}

```

4.15.3 Server-Side: ModelState

Controller action'da:

```

[HttpPost]
public IActionResult Create(DoktorForm form)
{
    if (!ModelState.IsValid)
    {
        return View(form); // hatalarla birlikte view'a döner
    }

    // ModelState geçerli – kaydı yap
    _context.Doktorlar.Add(new Doktor { ... });
    _context.SaveChanges();
    return RedirectToAction(nameof(Index));
}

```

ModelState, tüm binding + validation hatalarını taşıyan sözlüktür. IsValid true ise tüm alanlar hem başarılı bind edildi hem de kuralları geçti demektir.

Elle hata ekleme (iş kuralı iddiası):

```

if (_context.Doktorlar.Any(d => d.Ad == form.Ad && d.Id != form.Id))
{
    ModelState.AddModelError(nameof(form.Ad), "Bu isimde başka bir doktor var.");
}

if (!ModelState.IsValid) return View(form);

```

AddModelError iki parametre alır: property adı (view'da asp-validation-for bu adı arayacak), hata mesajı. Property adı "" (boş) verilirse "model geneli" hata olur — <div asp-validation-summary="ModelOnly"> bunu gösterir.

4.15.4 Client-Side: Unobtrusive Validation

asp-for Tag Helper (§4.11.5), property üzerindeki attribute'lardan data-val-* HTML attribute'ları üretir. Client-side JavaScript kütüphanesi bunları okur ve form'u submit etmeden önce doğrular.

Gerekli script'ler:

```

@section Scripts {
    <partial name="_ValidationScriptsPartial" />
}

```

_ValidationScriptsPartial.cshtml şablonda hazır gelir:

```

<script src="~/lib/jquery-validation/dist/jquery.validate.min.js"></script>
<script src="~/lib/jquery-validation-unobtrusive/jquery.validate.unobtrusive.min.js"></script>

```

Bu ikili çalıştığında form input alanlarının yanında içine mesaj yazılır; submit butonu tetiklenmez.

Kritik: jQuery da yüklü olmalı. Layout'ta yüklüyse partial'a gerek yok jQuery için, sadece validation kütüphaneleri.

4.15.5 Client-side vs Server-side Karşılaştırma

Yön	Client-side	Server-side
Kullanıcı deneyimi	Anında geri bildirim	Form gönderildikten sonra
Ağ trafiği	Yok (browser'da)	Var (POST)
Güvenlik	☐ (bypass edilebilir)	☐ (kesin)
Zorunlu	☐ (opsiyonel)	☐ (kaçınılmaz)

Client-side kapatılabilir (JS kapalı browser, kötü niyetli client). Server-side her koşulda çalışır.

4.15.6 Örnek 4.15.1 — Tam Form + Validation

ViewModel:

```
public class DoktorFormVM
{
    public int Id { get; set; }

    [Required(ErrorMessage = "Ad zorunlu.")]
    [StringLength(100, MinimumLength = 2, ErrorMessage = "2-100 karakter.")]
    public string Ad { get; set; }

    [Required(ErrorMessage = "Uzmanlık seçin.")]
    public int UzmanlikId { get; set; }

    [EmailAddress]
    public string Email { get; set; }

    public IEnumerable<SelectListItem> UzmanlikListesi { get; set; } = new List<SelectListItem>()
```

Controller:

```
[HttpGet]
public IActionResult Create()
{
    var vm = new DoktorFormVM
    {
        UzmanlikListesi = _context.Uzmanliklar
            .Select(u => new SelectListItem { Value = u.Id.ToString(), Text = u.Ad })
            .ToList()
    };
    return View(vm);
}

[HttpPost]
[ValidateAntiForgeryToken]
public IActionResult Create(DoktorFormVM vm)
{
    // İş kuralı iddiası
    if (_context.Doktorlar.Any(d => d.Ad == vm.Ad))
        ModelState.AddModelError(nameof(vm.Ad), "Bu isimde doktor kayıtlı.");

    if (!ModelState.IsValid)
    {
        vm.UzmanlikListesi = _context.Uzmanliklar
            .Select(u => new SelectListItem { Value = u.Id.ToString(), Text = u.Ad })
            .ToList();
        return View(vm);
    }

    _context.Doktorlar.Add(new Doktor
    {
```

```

        Ad = vm.Ad,
        UzmanlikId = vm.UzmanlikId,
        Email = vm.Email
    });
    _context.SaveChanges();
    TempData["Success"] = "Doktor eklendi.";
    return RedirectToAction(nameof(Index));
}

```

View:

```

@model DoktorFormVM

<h1>Yeni Doktor</h1>

<form asp-action="Create" method="post">
    <div asp-validation-summary="ModelOnly" class="text-danger"></div>

    <div class="mb-3">
        <label asp-for="Ad" class="form-label"></label>
        <input asp-for="Ad" class="form-control" />
        <span asp-validation-for="Ad" class="text-danger"></span>
    </div>

    <div class="mb-3">
        <label asp-for="UzmanlikId" class="form-label">Uzmanlık</label>
        <select asp-for="UzmanlikId" asp-items="Model.UzmanlikListesi" class="form-select">
            <option value="">-- Seçiniz --</option>
        </select>
        <span asp-validation-for="UzmanlikId" class="text-danger"></span>
    </div>

    <div class="mb-3">
        <label asp-for="Email" class="form-label"></label>
        <input asp-for="Email" class="form-control" />
        <span asp-validation-for="Email" class="text-danger"></span>
    </div>

    <button type="submit" class="btn btn-primary">Kaydet</button>
</form>

@section Scripts {
    <partial name="_ValidationScriptsPartial" />
}

```

4.15.7 Özel Validation Attribute Yazma

Standart attribute'lar yeterli değilse ValidationAttribute'tan türeyerek yazılır:

```

public class YetiskinAttribute : ValidationAttribute
{
    protected override ValidationResult IsValid(object value, ValidationContext context)
    {
        if (value is DateTime tarih)
        {
            var yas = DateTime.Today.Year - tarih.Year;
            if (tarih > DateTime.Today.AddYears(-yas)) yas--;

            if (yas < 18)
                return new ValidationResult(ErrorMessage ?? "18 yaşından büyük olmalı.");
        }
        return ValidationResult.Success;
    }
}

```

```

    }
}

// Kullanım
public class KayitVM
{
    [Required, Yetiskin(ErrorMessage = "Kayıt için 18 yaşını doldurmuş olmalısınız.")]
    public DateTime DogumTarihi { get; set; }
}

```

Bu server-side'da çalışır; client-side desteği için IClientModelValidator uygulanmalı — genellikle bu tarz özel kurallar sadece server-side yeterlidir.

4.15.8 IValidatableObject

Sınıf seviyesinde, birden çok property'yi birlikte değerlendiren doğrulama:

```

public class SiparisVM : IValidatableObject
{
    [Required] public DateTime BaslangicTarihi { get; set; }
    [Required] public DateTime BitisTarihi { get; set; }

    public IEnumerable<ValidationResult> Validate(ValidationContext ctx)
    {
        if (BitisTarihi < BaslangicTarihi)
        {
            yield return new ValidationResult(
                "Bitiş tarihi başlangıçtan önce olamaz.",
                new[] { nameof(BitisTarihi) });
        }
    }
}

```

Validate metodu ModelState.IsValid kontrolü sırasında otomatik çağrılır.

4.15.9 Yaygın Hatalar

Hatalı Kod	Sonuç	Neden	Düzeltilmiş
Sadece client-side validation, server'da if (!ModelState.IsValid) yok	JS kapalı client bypass'lar	Server kontrolü eksik	Her POST action'da kontrol
_ValidationScriptsPartial unutulmuş	Client-side çalışmaz	Script yok	@section Scripts içinde partial ekle
Postback'te dropdown eksik yüklendi, ModelState'te "geçersiz UzmanlikId"	View render eder, dropdown boş	Listeleri yeniden yüklemedin	Controller'da tekrar doldur
Custom validation attribute serileşmiyor (JSON API)	System.Text.Json sorun çıkarır	Attribute reflection ile alınmalı	IValidatableObject veya FluentValidation

4.15.10 Anti-Pattern ve İdiyomatik Kullanım

Anti-Pattern	İdiyomatik	Neden Daha İyi
Try-catch içinde int.Parse kullanıcı input	Data Annotation [Range]	Otomatik hata mesajı
Controller'da elle her property kontrolü	Data Annotations + ModelState.IsValid	DRY, tutarlı

Anti-Pattern	İdiyomatik	Neden Daha İyi
Client-side dışına ölçsüz güvenmek	Server-side her zaman yeniden validate	Güvenlik
Genel Exception mesajı	Field-specific AddModelError	Kullanıcı hangi alanı düzeltsin bilsin

4.15.11 Bölüm Özeti

- Validation üç aşama: Data Annotation → client-side → server-side.
- [Required], [StringLength], [Range], [EmailAddress], [Compare] en yaygın.
- ModelState.IsValid sunucuda kesin son sözü söyler.
- AddModelError("prop", "mesaj") ile iş kuralı iddialarını manuel ekle.
- Client-side için _ValidationScriptsPartial (jQuery Validation + Unobtrusive).
- Özel kurallar için ValidationAttribute veya IValidatableObject.

İlgili konular: §3.7 (Data Annotations şeması), §4.11.8 (Validation Tag Helper), §4.14 (View-Model'de validation), §4.13 (Model binding sırasında validation).

4.16 Dependency Injection

4.16.1 Genel Bakış

Dependency Injection (DI), bir sınıfın ihtiyaç duyduğu bağımlılıkları kendisi üretmeyip dışarıdan alması ilkesidir. §1.8.6 (DIP)'te tasarım prensibi olarak ele alındı; bu bölüm ASP.NET Core'un yerleşik DI container'ının pratik kullanımını işler.

ASP.NET Core, ilk sınıf DI desteğiyle gelir; ayrı bir kütüphane kurmaya gerek yoktur. Framework kendisi de DI üzerine kuruludur: Controller, DbContext, ILogger, IHttpContextAccessor gibi çekirdek bileşenler container üzerinden dağıtılır.

DI container'ın üç anahtar yeteneği vardır:

1. **Kayıt (Registration):** "Bu interface istendiğinde bu somut sınıfı ver" bilgisi.
2. **Çözümleme (Resolution):** Bir sınıf ihtiyaç duyduğunda uygun örneği üretmek.
3. **Yaşam yönetimi (Lifetime):** Örneklerin ne kadar yaşayacağını, ne zaman dispose edileceğini yönetmek.

4.16.2 Servis Kayıt Yöntemleri

Program.cs'de builder.Services üzerinden üç ana kayıt metodu vardır:

```
// Interface + concrete
builder.Services.AddScoped<IDoktorService, DoktorService>();

// Sadece concrete (kendi kendine)
builder.Services.AddScoped<DoktorService>();

// Factory (özel üretim mantığı)
builder.Services.AddScoped<IDoktorService>(sp =>
{
    var context = sp.GetRequiredService<AppDbContext>();
    return new DoktorService(context, someConfig);
});

// Instance (singleton olarak sabit örnek)
builder.Services.AddSingleton<IEmailAyar>(new EmailAyar("smtp.host", 587));
```

Metotlar üç yaşam süresi (lifetime) sunar: AddSingleton, AddScoped, AddTransient.

4.16.3 Yaşam Süreleri (Lifetimes)

Lifetime	Ömür	Kullanım
Singleton	Uygulama boyu tek örnek	Konfigürasyon nesnesi, cache, thread-safe stateless servisler
Scoped	Her HTTP request için tek örnek	DbContext, request-specific state
Transient	Her çağrıda yeni örnek	Hafif, stateless helper'lar

Görsel şema:

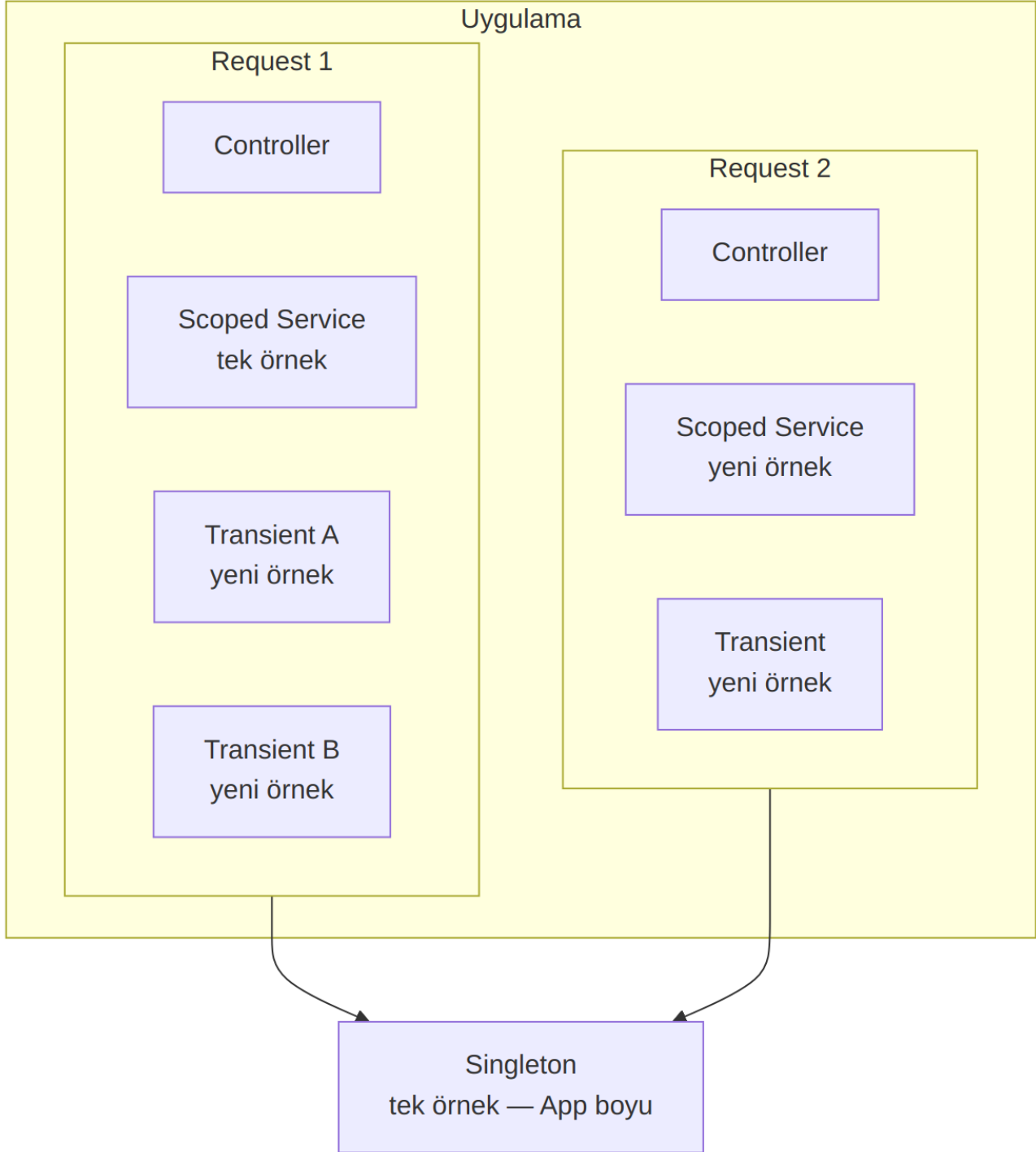


Figure 11: Diyagram 11

Kural:

- Bir servis DB veya request-ölgü state kullanıyorsa → **Scoped**.

- Servis stateless ve hafifse → **Transient**.
- Global konfigürasyon, cache, HTTP client factory gibi → **Singleton**.

DbContext her zaman **Scoped**'dur — AddDbContext<T> bunu otomatik yapar. Singleton yapmak thread-safety kırar; Transient yapmak change tracker'ı işe yaramaz kırar.

4.16.4 Constructor Injection

DI'nin ana kullanım yolu constructor'a bağımlılık geçirmektir. Framework, sınıfı new'lerken constructor parametrelerini container'dan çözer:

```
public class DoktorController : Controller
{
    private readonly IDoktorService _doktorService;
    private readonly ILogger<DoktorController> _logger;

    public DoktorController(IDoktorService doktorService, ILogger<DoktorController> logger)
    {
        _doktorService = doktorService;
        _logger = logger;
    }

    public IActionResult Index()
    {
        var liste = _doktorService.TumunuGetir();
        _logger.LogInformation("Doktor listesi çekildi:{Sayi}", liste.Count());
        return View(liste);
    }
}
```

- IDoktorService, DI container'a önceden kaydedilmiş olmalı.
- ILogger<T> framework tarafından otomatik sağlanır (kayıt gerekmez).

Ne inject edilir, ne edilmez:

- Interface (IXServis) → tercih edilen; test için mock kolay.
- Concrete class → çalışır ama gevşek bağılılığı bozar.
- DbContext, IConfiguration, IHostEnvironment, IHttpContextAccessor → framework standart tipleri.
- ViewModel gibi POCO'lar → hayır; DI'da yerleri yok.

4.16.5 Örnek 4.16.1 — Tam Servis Kaydı ve Kullanımı

Interface:

```
// Services/IDoktorService.cs
public interface IDoktorService
{
    Task<IEnumerable<Doktor>> TumunuGetirAsync();
    Task<Doktor> GetirAsync(int id);
    Task EkleAsync(Doktor doktor);
    Task GuncelleAsync(Doktor doktor);
    Task SilAsync(int id);
}
```

Uygulama:

```
// Services/DoktorService.cs
public class DoktorService : IDoktorService
{
    private readonly AppDbContext _context;

    public DoktorService(AppDbContext context) => _context = context;

    public async Task<IEnumerable<Doktor>> TumunuGetirAsync()
```

```

=> await _context.Doktorlar.AsNoTracking().ToListAsync();

public async Task<Doktor> GetirAsync(int id)
=> await _context.Doktorlar.FindAsync(id);

public async Task EkleAsync(Doktor doktor)
{
    _context.Doktorlar.Add(doktor);
    await _context.SaveChangesAsync();
}

public async Task GuncelleAsync(Doktor doktor)
{
    _context.Doktorlar.Update(doktor);
    await _context.SaveChangesAsync();
}

public async Task SilAsync(int id)
{
    var d = await _context.Doktorlar.FindAsync(id);
    if (d != null) { _context.Doktorlar.Remove(d); await _context.SaveChangesAsync(); }
}
}

```

Kayıt:

```

// Program.cs
builder.Services.AddDbContext<AppDbContext>(o =>
    o.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));

builder.Services.AddScoped<IDoktorService, DoktorService>();

```

Controller:

```

public class DoktorController : Controller
{
    private readonly IDoktorService _service;

    public DoktorController(IDoktorService service) => _service = service;

    public async Task<IActionResult> Index()
    {
        var liste = await _service.TumunuGetirAsync();
        return View(liste);
    }
}

```

Controller DbContext'i doğrudan bilmez — sadece IDoktorService sözleşmesini bilir. Bu, test edilebilirlik ve refactoring kolaylığı sağlar.

4.16.6 Servis Çözümleme Alternatifleri

Constructor injection (önerilen — %95 kullanım).

Method injection — action metoduna [FromServices]:

```

public IActionResult Detay(int id, [FromServices] IDoktorService svc)
{
    var d = svc.Getir(id);
    return View(d);
}

```

Nadiren gerekir; sadece bir metot bir servise ihtiyaç duyuyorsa constructor'ı kirletmemek için.

View injection — view içine servis inject:

```
@inject IStringLocalizer<HomeController> Localizer
```

```
<h1>@Localizer["Hoşgeldiniz"]</h1>
```

Yaygın örnekler: localization, konfigürasyon, layout'ta gösterilecek ad-hoc veri (bir bildirim sayısı gibi).

IServiceProvider ile manuel çözümleme:

```
public class Foo
{
    private readonly IServiceProvider _sp;
    public Foo(IServiceProvider sp) => _sp = sp;

    public void Yap()
    {
        var svc = _sp.GetRequiredService<IDoktorService>();
        // ...
    }
}
```

Anti-pattern — Service Locator olarak bilinir; bağımlılıklar açık değildir. Kaçınılmalıdır; sadece framework yazımında gerekir.

4.16.7 Framework Tarafından Sağlanan Servisler

Kayıt yapmadan doğrudan inject edebileceğin standart servisler:

Tip	Ne için
ILogger<T>	Loglama
IConfiguration	appsettings okuma
IHostEnvironment / IWebHostEnvironment	Ortam bilgisi (Development?)
IHttpContextAccessor	Mevcut HTTP context'e erişim (kayıt gerekir: AddHttpContextAccessor)
IServiceProvider	Container'ın kendisi
IOptions<T>	Options pattern konfigürasyonu (§4.4.3)
IMemoryCache	Bellek cache (AddMemoryCache)
IHttpClientFactory	HTTP istekleri (AddHttpClient)

4.16.8 Yaşam Süresi Çakışmaları

Singleton bir servis, Scoped bir servisi doğrudan constructor'da alamaz — Singleton uygulama boyu yaşarken Scoped her request'te değişir.

```
public class KotuSingleton
{
    public KotuSingleton(AppDbContext ctx) { } // ← RUNTIME HATA
}
```

```
builder.Services.AddSingleton<KotuSingleton>();
```

Uygulama başlatılırken (app.Run() çağrısında) framework bu ihlali tespit eder ve InvalidOperationException fırlatır: "Cannot consume scoped service 'AppDbContext' from singleton 'KotuSingleton'".

Çözüm: IServiceScopeFactory inject edip her ihtiyaç anında bir scope aç:

```
public class DogruSingleton
{
    private readonly IServiceScopeFactory _scopeFactory;

    public DogruSingleton(IServiceScopeFactory scopeFactory) => _scopeFactory = scopeFactory;
}
```

```

public void Yap()
{
    using var scope = _scopeFactory.CreateScope();
    var ctx = scope.ServiceProvider.GetRequiredService<AppDbContext>();
    // ...
}
}

```

4.16.9 Options Pattern

Konfigürasyon değerlerini strongly-typed sınıflara bağlamak için (§4.4.3'te değinildi):

```

// appsettings.json
{
  "EmailAyar": {
    "SmtpHost": "smtp.gmail.com",
    "Port": 587,
    "Kullanici": "..."
  }
}

// Models/EmailAyar.cs
public class EmailAyar
{
    public string SmtpHost { get; set; }
    public int Port { get; set; }
    public string Kullanici { get; set; }
}

// Program.cs
builder.Services.Configure<EmailAyar>(builder.Configuration.GetSection("EmailAyar"));

// Servis
public class EmailServisi
{
    private readonly EmailAyar _ayar;
    public EmailServisi(IOptions<EmailAyar> options) => _ayar = options.Value;
}

```

IOptions<T> singleton'dır; runtime'da değişmez. Değişebilir konfigürasyon için IOptionsMonitor<T> (change bildirim alır).

4.16.10 Yaygın Hatalar

Hatalı Kod	Hata	Neden	Düzeltilmiş
Servis kayıt edilmemiş, controller inject bekliyor	Unable to resolve service for type '...'	Container'da yok	Program.cs'e Add<Lifetime>
Singleton → Scoped bağımlılık	Cannot consume scoped service ... from singleton	Yaşam çakışması	Scope factory kullan veya lifetime yeniden düşün
Constructor'da bağımlılığı new'lemek	Test edilemez, DI atlanmış	Manuel üretim	Inject al
Aynı interface iki defa kayıt (Add<X, A> sonra Add<X, B>)	Son kaydedilen kullanılır	Override	TryAdd veya bilinçli tercih
IDisposable servis kayıtlı ama kayıtsız new üretilmiş	Dispose çağrılmaz	Container yönetmiyor	Container üzerinden çözümle

4.16.11 Anti-Pattern ve İdiyomatik Kullanım

Anti-Pattern	İdiyomatik	Neden Daha İyi
Service Locator (IServiceProvider inject)	Constructor injection	Bağımlılıklar açık
Statik sınıflar iş mantığı için new DoktorService(...) her yerde	Interface + Scoped/Singleton Container'a kaydet, inject al	Test edilebilir Bağımlılık tek yerde tanımlı
AddSingleton her yere (kolay diye)	Doğru lifetime seç	Bellek + thread-safety

4.16.12 Bölüm Özeti

- ASP.NET Core yerleşik DI container ile gelir.
- Üç lifetime: Singleton (app), Scoped (request), Transient (call).
- DbContext Scoped; AddDbContext bunu otomatik yapar.
- Constructor injection ana yol; view injection ve method injection alternatif.
- Singleton → Scoped bağımlılık runtime hatası verir; scope factory çözer.
- Interface kayıt et, mock için ideal; test yazımı kolay.
- Options pattern konfigürasyonu strongly-typed sunar.

İlgili konular: §1.8.6 (DIP), §3.6 (DbContext scoped), §4.3 (Program.cs kayıtları), §4.4 (Options pattern), §5.2 (HastaneMVC — DoktorService/RandevuService kaydı).

4.17 Middleware ve Pipeline

4.17.1 Genel Bakış

Middleware, bir HTTP request'in yolculuğunda sırayla çalışan işlem birimleridir. Her middleware'in iki seçeneği vardır: (a) request'i bir sonraki middleware'e devret (pipeline'ı sürdür), (b) response üretip pipeline'ı kır. Bir bakıma "istek geldiğinde şu şu şu adımlar sırayla çalışsın" bildiriminin kod olarak ifadesidir.

Pipeline olarak adlandırılmasının sebebi, middleware'lerin arka arkaya dizilip request'in bir borudan geçer gibi hepsinden akmasıdır. Her middleware next() çağırarak zinciri sürdürür; çağırmasa response oradan döner.

4.17.2 Program.cs'de Pipeline

Program.cs'in ikinci yarısı pipeline kurar:

```
var app = builder.Build();

if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Home/Error"); // 1. hata yakalayıcı (en dışta)
    app.UseHsts(); // 2. HSTS header
}

app.UseHttpsRedirection(); // 3. HTTP → HTTPS
app.UseStaticFiles(); // 4. wwwroot dosyaları
app.UseRouting(); // 5. routing hazırlığı
app.UseAuthentication(); // 6. authentication (varsa)
app.UseAuthorization(); // 7. authorization
app.MapControllerRoute(...); // 8. endpoint mapping
app.Run(); // 9. sunucuyu başlat
```

Her Use* çağrısı pipeline'a bir middleware ekler. **Sıra kritiktir**: request bu satırların yazıldığı sırayla middleware'leri geçer, response ise tersinden geri döner.



Figure 12: Diyagram 12

4.17.3 Middleware Sırası

Standart sıra kuralları:

1. Exception handler en dışta (her şeyi yakalasın).
2. HTTPS redirect üstte (güvenli protokole çekelim).
3. Static files early (routing'ı atlasın — CSS/JS için gereksiz overhead).
4. Routing sonra (URL'yi endpoint'e eşle).
5. Authentication → Authorization (kimlik → yetki).
6. Endpoints sonda (asıl işi yapan MVC).

4.17.4 Middleware Türleri

Terminal middleware — pipeline'ı sonlandırır, response üretir:

```
app.Run(async context =>
{
    await context.Response.WriteAsync("Merhaba dünya!");
});
```

Run her request için terminal davranış üretir; sonrasına middleware eklenmez.

In-line middleware — Use ile:

```
app.Use(async (context, next) =>
{
    // request öncesi
    var stopwatch = System.Diagnostics.Stopwatch.StartNew();

    await next(); // sonraki middleware'e devret

    // response sonrası
    stopwatch.Stop();
    Console.WriteLine($"{context.Request.Path}: {stopwatch.ElapsedMilliseconds}ms");
});
```

Bu middleware, gelen isteğin süresini ölçüp log'a yazar. `next()` çağrısı öncesindeki kod request tarafı, sonrasındaki response tarafıdır.

Konvansiyonel middleware — ayrı sınıf:

```
public class LoglamaMiddleware
{
    private readonly RequestDelegate _next;
    private readonly ILogger<LoglamaMiddleware> _logger;

    public LoglamaMiddleware(RequestDelegate next, ILogger<LoglamaMiddleware> logger)
    {
        _next = next;
        _logger = logger;
    }

    public async Task InvokeAsync(HttpContext context)
    {
        _logger.LogInformation("İstek: {Path}", context.Request.Path);
        await _next(context);
        _logger.LogInformation("Yanıt: {StatusCode}", context.Response.StatusCode);
    }
}
```

```
// Program.cs
app.UseMiddleware<LoglamaMiddleware>();
```

Kompleks middleware'ler bu şekilde sınıfa çıkarılır. Framework, InvokeAsync metodunu isim konvansiyonu ile tanır.

4.17.5 Örnek 4.17.1 — Global Exception Middleware

Kendi hata yakalayıcın:

```
public class HataYakalaMiddleware
{
    private readonly RequestDelegate _next;
    private readonly ILogger<HataYakalaMiddleware> _logger;

    public HataYakalaMiddleware(RequestDelegate next, ILogger<HataYakalaMiddleware> logger)
    {
        _next = next;
        _logger = logger;
    }

    public async Task InvokeAsync(HttpContext ctx)
    {
        try
        {
            await _next(ctx);
        }
        catch (Exception ex)
        {
            _logger.LogError(ex, "Yakalanmayan istisna: {Path}", ctx.Request.Path);
            ctx.Response.StatusCode = 500;
            ctx.Response.ContentType = "application/json";
            await ctx.Response.WriteAsync("{\"hata\": \"Sunucu hatası\"}");
        }
    }
}

// Program.cs
app.UseMiddleware<HataYakalaMiddleware>(); // en üstte
```

4.17.6 Map ve MapWhen — Şartlı Pipeline

Belirli URL prefix'i için ayrı pipeline dalı:

```
app.Map("/api", api =>
{
    api.UseRouting();
    api.UseAuthentication();
    api.UseEndpoints(...);
});
```

Bu, sadece /api/... istekleri için ayrı bir middleware zinciri kurar.

Koşula göre:

```
app.MapWhen(
    ctx => ctx.Request.Query.ContainsKey("debug"),
    branch => branch.Run(async ctx =>
    {
        await ctx.Response.WriteAsync("DEBUG modu");
    }));
```

Query string'de ?debug=1 gelen tüm request'ler bu dala girer.

4.17.7 HttpContext

Middleware'lerin ana veri yapısıdır. Her request için ayrı bir HttpContext yaratılır ve pipeline boyunca taşınır.

Yaygın özellikler:

Özellik	Ne verir
context.Request.Path	URL yolu
context.Request.Method	GET/POST/...
context.Request.Query["ad"]	Query string değeri
context.Request.Headers["Ad"]	HTTP header
context.Response.StatusCode	Cevap kodu
context.Response.ContentType	MIME
context.User	Kimlik doğrulanmışsa ClaimsPrincipal
context.Items["anahtar"]	Bu request'e özel geçici veri (middleware'ler arası paylaşım)
context.RequestServices	O request için DI scope

context.Items, aynı request boyunca middleware'ler arasında veri taşımak için kullanılır — controller'a kadar ulaşan bir "request scope veri deposu".

4.17.8 Sık Kullanılan Yerleşik Middleware'ler

Middleware	Ne yapar
UseExceptionHandler("/Home/Error")	Yakalanmayan hataları belirli URL'e yönlendirir
UseDeveloperExceptionPage()	Development'ta detaylı stack trace sayfası
UseHsts()	HTTP Strict Transport Security header
UseHttpsRedirection()	HTTP → HTTPS
UseStaticFiles()	wwwroot dosyalarını serve eder
UseRouting()	URL'yi endpoint'e eşle
UseAuthentication()	Kimlik doğrulama
UseAuthorization()	Yetki kontrolü
UseCors(...)	Cross-origin resource sharing
UseSession()	Session desteği
UseResponseCompression()	Response gzip/brotli sıkıştırma
UseResponseCaching()	Response caching

4.17.9 Endpoint Middleware

UseRouting() URL'yi bir endpoint'e karar verir; ardından gelen UseAuthorization() bu endpoint için yetki kontrolü yapar; en sonda MapControllerRoute (endpoint execution) asıl controller action'ı yürütür.

Bu üçlü sırası:

```
app.UseRouting();           // 1. karar ver
app.UseAuthorization();     // 2. izin var mı?
app.MapControllerRoute(name: "default", pattern: "{controller=Home}/{action=Index}/{id?}");
```

UseAuthorization'ın UseRouting sonrasına konulması, "hangi endpoint çağrıldığını" bildiği için [Authorize] attribute'unu doğru değerlendirebilmesini sağlar.

4.17.10 Yaygın Hatalar

Hatalı Kod	Sonuç	Neden	Düzeltilmiş
UseAuthorization UseRouting'ten önce	[Authorize] çalışmaz	Endpoint bilinmiyor	Sıra: Routing → AuthN → AuthZ

Hatalı Kod	Sonuç	Neden	Düzeltilmiş
UseStaticFiles UseRouting'ten sonra	CSS/JS gereksiz middleware zincirinden geçiyor	Verimsiz	StaticFiles önce
Custom middleware'de next() çağırmaı unutmak	Pipeline durur, sonraki middleware çalışmaz	Terminal davranış	await next(context);
Middleware kayıt sıralaması yanlış (auth önce, routing sonra)	[Authorize] çalışmıyor	Sıra kritik	Standart sırayı izle
app.Run middleware'in ortasında	Sonrası çalışmaz	Run terminal	Sadece pipeline sonunda

4.17.11 Bölüm Özeti

- Middleware = pipeline'da sırayla çalışan işlem birimi.
- Her middleware next() çağırarak zinciri sürdürür; çağırılmazsa response oradan döner.
- Sıra kritik: Exception → HTTPS → Static → Routing → Authentication → Authorization → End-points.
- Terminal (Run), in-line (Use), konvansiyonel (sınıf + UseMiddleware<T>).
- HttpContext request-özel veri taşıyıcısı; context.Items middleware'ler arası paylaşım.
- Map / MapWhen şartlı pipeline dalları.
- Global exception handler tipik custom middleware örneğidir.

İlgili konular: §4.3 (Program.cs pipeline), §4.18 (Anti-forgery middleware), §4.16 (DI ile middleware'e servis inject).

4.18 Anti-forgery Koruması

4.18.1 Genel Bakış

Cross-Site Request Forgery (CSRF), kötü niyetli bir sitenin, kullanıcı adına başka bir sitenin oturumunu kullanarak istek göndermesidir. Kullanıcı banka.com'a giriş yapmış, cookie oturumu aktifken; başka bir sitedeki <form action="https://banka.com/para-transferi"> gönderilirse tarayıcı otomatik olarak cookie'yi ekleyerek gönderir. Kullanıcının haberi olmadan işlem başlayabilir.

Anti-forgery token bu saldırıya karşı ana savunmadır. Sunucu her form'a gizli bir token yerleştirir; form gönderildiğinde sunucu token'ı doğrular. Kötü niyetli site bu token'ı bilemez.

ASP.NET Core MVC anti-forgery mekanizmasını yerleşik olarak sağlar; iki katmanı vardır: (a) view'da form'a token ekleme, (b) POST action'da token doğrulama.

4.18.2 View Tarafı — Token Nasıl Eklenir?

Tag Helper <form> her POST form'una otomatik olarak gizli input ekler:

```
<form asp-action="Create" method="post">
  <!-- Tag Helper otomatik ekler:
    <input type="hidden" name="__RequestVerificationToken" value="..." /> -->
  <input asp-for="Ad" />
  <button type="submit">Kaydet</button>
</form>
```

Klasik <form> etiketi (Tag Helper olmadan) için @Html.AntiForgeryToken() çağrısı manuel eklenmelidir:

```
<form action="/Doktor/Create" method="post">
  @Html.AntiForgeryToken()
  <input name="Ad" />
```

```
<button type="submit">Kaydet</button>
</form>
```

Modern MVC kodunda Tag Helper standart olduğu için elle çağrı nadir gereklidir.

4.18.3 Controller Tarafı — İki Yaklaşım

Sunucu tarafında iki yaklaşım vardır:

4.18.3.a Klasik Yaklaşım — Action Başına Attribute Her POST action'a [ValidateAntiForgeryToken] attribute'u eklenir:

```
public class DoktorController : Controller
{
    [HttpPost]
    [ValidateAntiForgeryToken]
    public IActionResult Create(Doktor form) { ... }

    [HttpPost]
    [ValidateAntiForgeryToken]
    public IActionResult Edit(Doktor form) { ... }

    [HttpPost]
    [ValidateAntiForgeryToken]
    public IActionResult Delete(int id) { ... }
}
```

Avantaj: Explicit; hangi action'ın korunduğu görünür.

Dezavantaj: Her POST'ta unutulabilir; bir action'da eksik bırakılırsa açık.

4.18.3.b Modern Yaklaşım — Global Filter Program.cs'te tüm POST action'lara otomatik uygulan global filter:

```
builder.Services.AddControllersWithViews(options =>
{
    options.Filters.Add(new AutoValidateAntiforgeryTokenAttribute());
});
```

Bu yapılandırma, tüm POST/PUT/PATCH/DELETE isteklerine anti-forgery doğrulaması ekler. GET istekleri etkilenmez.

Avantaj: Her action'a tek tek yazmaya gerek yok; unutma riski minimum.

Dezavantaj: Belirli bir POST action'da token beklememek gerekiyorsa (webhook, API endpoint) explicit istisna gerekir:

```
[HttpPost]
[IgnoreAntiforgeryToken]
public IActionResult WebhookAl([FromBody] object payload) { ... }
```

Master defterin standardı: Modern (global) yaklaşım. Yeni projelerde AutoValidateAntiforgeryTokenAttribute global filter olarak eklenir; klasik [ValidateAntiForgeryToken] sadece istisnai action'larda kullanılır. Var olan projelerde iki yaklaşım aynı anda çalışır — token attribute'u zaten varken global filter eklemek zarar vermez.

4.18.4 Doğrulama Nasıl Çalışır?

Token iki parçadan oluşur:

1. **Cookie token:** __RequestVerificationToken isimli cookie; user session ile bağlantılı.
2. **Form token:** __RequestVerificationToken isimli form field'ı; her form için ayrı üretilir.

İkisi kriptografik olarak birbirine bağlıdır. Sunucu POST'ta hem cookie hem form field'ının doğru eşleştiğini kontrol eder. Kötü niyetli site cookie'ye erişemediği için (same-origin policy), form token'ını üretemez.

Doğrulama başarısızsa AntiforgeryValidationException fırlar, framework 400 Bad Request üretir.

4.18.5 AJAX'ta Anti-forgery Token

JavaScript form'suz AJAX POST'ta token'ı elle göndermek gerekir. _Layout.cshtml'de bir helper:

```
@inject Microsoft.AspNetCore.Antiforgery.IAntiforgery Antiforgery

<script>
    window.antiforgeryToken = "@Antiforgery.GetAndStoreTokens(HttpContext).RequestToken";
</script>
```

AJAX kullanımı:

```
$.ajax({
    url: "/Doktor/Sil",
    type: "POST",
    data: { id: 5 },
    headers: { "RequestVerificationToken": window.antiforgeryToken },
    success: function(sonuc) { /* ... */ }
});
```

Sunucu, header'da token'ı kabul eder — konfigürasyon:

```
builder.Services.AddAntiforgery(options =>
{
    options.HeaderName = "RequestVerificationToken"; // varsayılan zaten bu
});
```

Alternatif: token'ı gizli input yerine <meta> etiketinde ver:

```
<meta name="request-verification-token"
      content="@Antiforgery.GetAndStoreTokens(HttpContext).RequestToken" />

const token = document.querySelector('meta[name="request-verification-token"]').content;
$.ajax({ headers: { "RequestVerificationToken": token }, ... });
```

4.18.6 GET Kalıcı Değişiklik Yapmamalı

Anti-forgery koruma sadece POST'ta çalışır. GET isteklerini “yan etkisi olmayan salt-okunur” olarak tasarlamak, hem HTTP RESTful prensiplerine uyar hem CSRF saldırı yüzeyini daraltır. Bir doktoru “GET /Doktor/Sil/5” ile silmek büyük hata: bir etiketi bile silmeyi tetikler.

Kural: Kalıcı değişiklikler (INSERT/UPDATE/DELETE) daima POST — GET veri değiştirmez.

4.18.7 Yaygın Hatalar

Hatalı Kod	Sonuç	Neden	Düzeltilmiş
<form> + Tag Helper yok, token da yok	POST fail: 400	Token eksik	<form asp-action=...> veya @Html.AntiForgeryToken() POST'a taşı + token
GET action ile veri silme	CSRF açık	HTTP method yanlış	POST'a taşı + token
AJAX'ta token eklenmemiş	400 Bad Request	Header yok	Meta tag'den al, header'a koy
Global filter aktif, ama API controller'a [IgnoreAntiforgeryToken] yok	Webhook fail	Filter tüm POST'lara uygulandı	[IgnoreAntiforgeryToken] ekle
Klasik + global iki yaklaşım karışık	Çift kontrol (zararsız ama gereksiz)	Yönetim karmaşası	Bir tane seç, tutarlı kal

4.18.8 Bölüm Özeti

- CSRF saldırısı = kullanıcı adına başka sitenin form göndermesi.
- Anti-forgery token her POST form'una gizli input ile eklenir; sunucu doğrular.
- Tag Helper <form asp-action> token'ı otomatik ekler.
- İki yaklaşım: klasik [ValidateAntiForgeryToken] per-action vs modern AutoValidateAntiForgeryTokenAttribute global.
- Master defterin standardı: global filter.
- AJAX POST'ta token header'da elle geçirilir.
- GET kalıcı değişiklik yapmaz — CSRF ve REST prensipleri gereği.

İlgili konular: §4.11.4 (Tag Helper otomatik ekleme), §4.19 (AJAX token gönderimi), §4.17 (Middleware pipeline).

4.19 AJAX

4.19.1 Genel Bakış

AJAX (Asynchronous JavaScript and XML), sayfanın tamamı yeniden yüklenmeden sunucu ile veri alışverişi yapma tekniğidir. Kısaltmadaki "XML" tarihi kalıntıdır; modern uygulamalarda veri **JSON** ile taşınır. Amaç: kullanıcı deneyimini iyileştirmek — bir dropdown seçildiğinde alt dropdown'ın dolması, bir butonun tıklanmasıyla listeden bir öğenin silinmesi, form doğrulamasının server'da anlık kontrol edilmesi gibi.

ASP.NET Core MVC bağlamında AJAX, iki yönlü çalışır:

- **İstek tarafı:** jQuery \$.ajax, \$.get, \$.post (klasik) veya modern fetch API ile browser'dan sunucuya HTTP isteği.
- **Cevap tarafı:** Controller action JsonResult (return Json(...)) veya PartialViewResult (return PartialView(...)) döner.

4.19.2 jQuery AJAX Metodları

\$.ajax — tam kontrol:

```
$.ajax({
  url: "/Doktor/GetById",
  type: "GET",
  data: { id: 5 },
  dataType: "json",
  success: function (sonuc) {
    console.log(sonuc);
  },
  error: function (xhr, status, err) {
    console.error("Hata:", err);
  },
  complete: function () {
    console.log("Tamamlandı (başarı ya da hata).");
  }
});
```

Ana parametreler:

Parametre	Anlamı
url	Hedef endpoint
type	HTTP method — GET, POST, PUT, DELETE
data	Gönderilecek veri (obje veya string)
dataType	Beklenen cevap formatı — json, html, xml, text
contentType	Gönderilen veri MIME (JSON POST için "application/json")
success	Başarı callback'i
error	Hata callback'i

Parametre	Anlamı
complete	Her durumda çalışan callback
headers	Ek HTTP header'lar

Kısayollar:

```
// GET
$.get("/Doktor/GetById", { id: 5 }, function (data) {
    console.log(data);
});

// POST
$.post("/Doktor/Update", { id: 5, ad: "Yeni Ad" }, function (data) {
    console.log(data);
});

// getJSON - özellikle JSON için
$.getJSON("/Doktor/List", function (data) {
    console.log(data);
});

// load - HTML parçasını doğrudan bir div'e yükle
$("#detay").load("/Doktor/Ozet/5");
```

\$.get, \$.post, \$.getJSON altta hepsi \$.ajax'e döner. Basit çağrılar için kısayol; karmaşık senaryolar için \$.ajax.

4.19.3 Modern fetch API

jQuery gerektirmeyen native alternatif:

```
fetch("/Doktor/GetById?id=5")
    .then(response => response.json())
    .then(data => console.log(data))
    .catch(err => console.error(err));

// POST + JSON
fetch("/Doktor/Update", {
    method: "POST",
    headers: {
        "Content-Type": "application/json",
        "RequestVerificationToken": token
    },
    body: JSON.stringify({ id: 5, ad: "Yeni Ad" })
})
    .then(r => r.json())
    .then(data => { /* ... */ });
```

ASP.NET Core MVC şablonu jQuery'yi zaten getirir; master defterin proje bölümlerinde jQuery + \$.ajax desenini standart alır.

4.19.4 Controller Tarafı — JsonResult

```
public class DoktorController : Controller
{
    public IActionResult GetById(int id)
    {
        var d = _context.Doktorlar.Find(id);
        if (d == null) return Json(new { basarili = false, mesaj = "Bulunamadı." });

        return Json(new
```

```

    {
        basarili = true,
        veri = new
        {
            d.Id,
            d.Ad,
            d.Uzmanlik
        }
    });
}
}

```

Json(...) metodu, verilen nesneyi JSON'a serialize eder ve response'u application/json content-type ile döner.

JavaScript tarafında:

```

$.getJSON("/Doktor/GetById?id=5", function (sonuc) {
    if (sonuc.basarili) {
        $("#ad").text(sonuc.veri.ad);
        $("#uzmanlik").text(sonuc.veri.uzmanlik);
    } else {
        alert(sonuc.mesaj);
    }
});

```

4.19.5 JSON Property Adları — Camel Case

System.Text.Json (default) property adlarını camelCase'e çevirir:

- C#: Ad → JSON: ad
- C#: HastaAdi → JSON: hastaAdi

Bu, JavaScript konvansiyonuna uygundur. Bilinçli PascalCase korumak için:

```

builder.Services.AddControllersWithViews()
    .AddJsonOptions(o => o.JsonSerializerOptions.PropertyNamingPolicy = null);

```

Master defterin standardı camelCase — modern kod tutarlılığı için.

4.19.6 Örnek 4.19.1 — Cascading Dropdown (Katalog C'nin ana senaryosu)

Bölüm seçildikçe o bölümdeki doktorlar dropdown'ı dolar.

HTML:

@model RandevuVM

```

<form asp-action="Kaydet" method="post">
    <div class="mb-3">
        <label asp-for="BolumId" class="form-label">Bölüm</label>
        <select asp-for="BolumId" asp-items="Model.Bolumler" id="bolumSelect" class="form-select">
            <option value="">-- Bölüm seçin --</option>
        </select>
    </div>

    <div class="mb-3">
        <label asp-for="DoktorId" class="form-label">Doktor</label>
        <select asp-for="DoktorId" id="doktorSelect" class="form-select">
            <option value="">-- Önce bölüm seçin --</option>
        </select>
    </div>

    <button type="submit" class="btn btn-primary">Randevu Al</button>
</form>

```

```

@section Scripts {
    <script>
        $(function () {
            $("#bolumSelect").change(function () {
                var bolumId = $(this).val();
                var $doktor = $("#doktorSelect");
                $doktor.empty().append('<option value="">-- Yükleniyor --</option>');

                if (!bolumId) {
                    $doktor.empty().append('<option value="">-- Önce bölüm seçin --</option>');
                    return;
                }

                $.getJSON("/Randevu/DoktorlariGetir", { bolumId: bolumId }, function (liste) {
                    $doktor.empty().append('<option value="">-- Doktor seçin --</option>');
                    $.each(liste, function (i, d) {
                        $doktor.append($('<option>', { value: d.id, text: d.ad }));
                    });
                });
            });
        });
    </script>
}

```

Controller:

```

public class RandevuController : Controller
{
    private readonly AppDbContext _context;
    public RandevuController(AppDbContext context) => _context = context;

    // Sayfayı yükleyen
    public IActionResult YeniRandevu()
    {
        var vm = new RandevuVM
        {
            Bolumler = _context.Bolumler
                .Select(b => new SelectListItem { Value = b.Id.ToString(), Text = b.Ad })
                .ToList()
        };
        return View(vm);
    }

    // AJAX endpoint
    public IActionResult DoktorlariGetir(int bolumId)
    {
        var liste = _context.Doktorlar
            .Where(d => d.BolumId == bolumId)
            .Select(d => new { id = d.Id, ad = d.Ad })
            .ToList();

        return Json(liste);
    }

    [HttpPost]
    public IActionResult Kaydet(RandevuVM vm)
    {
        if (!ModelState.IsValid) { /* dropdown'ları yeniden yükle */ return View(vm); }
        // ...kayıt
        return RedirectToAction(nameof(Index));
    }
}

```

```
}
```

Yolculuğun özeti:

1. Sayfa yüklendiğinde bölüm dropdown dolu, doktor dropdown boş.
2. Kullanıcı bir bölüm seçince change olayı tetiklenir.
3. jQuery, /Randevu/DoktorlariGetir?bolumId=X GET isteği yapar.
4. Controller o bölümdeki doktorları JSON olarak döner.
5. jQuery cevabı okur, #doktorSelect içine <option>'lar ekler.

Bu, Katalog C'deki AspNet7Hastane projesinin ana özelliğidir; §5.3'te bu proje detaylıca ele alınır.

4.19.7 Örnek 4.19.2 — AJAX ile Silme + Anti-forgery

```
$(function () {  
    $(".sil-btn").click(function () {  
        if (!confirm("Silmek istediğinizden emin misiniz?")) return;  
  
        var id = $(this).data("id");  
        var token = $('input[name="__RequestVerificationToken"]').val();  
  
        $.ajax({  
            url: "/Doktor/Sil",  
            type: "POST",  
            data: { id: id },  
            headers: { "RequestVerificationToken": token },  
            success: function (sonuc) {  
                if (sonuc.basarili) {  
                    $("#row-" + id).fadeOut();  
                    toastr.success(sonuc.mesaj);  
                } else {  
                    toastr.error(sonuc.mesaj);  
                }  
            }  
        });  
    });  
});
```

Controller:

```
[HttpPost]  
public IActionResult Sil(int id)  
{  
    var d = _context.Doktorlar.Find(id);  
    if (d == null) return Json(new { basarili = false, mesaj = "Kayıt bulunamadı." });  
  
    _context.Doktorlar.Remove(d);  
    _context.SaveChanges();  
    return Json(new { basarili = true, mesaj = "Doktor silindi." });  
}
```

Silme başarılı olduğunda tabloda ilgili satır JavaScript ile gizlenir; sayfa yenilenmez. Toastr bildirimi kullanıcıya sonucu gösterir (§4.22).

4.19.8 Örnek 4.19.3 — Partial View Dönüşü

Bazen JSON değil, HTML parçası dönmek gerekir — bir kartı, bir tablonun satırlarını, arama sonucu.

Controller:

```
public IActionResult AramaSonucu(string ara)  
{  
    var liste = _context.Doktorlar
```

```

        .Where(d => d.Ad.Contains(ara))
        .Take(10)
        .ToList();

    return PartialView("_DoktorListe", liste);
}

```

View (_DoktorListe.cshtml):

```

@model IEnumerable<Doktor>

@foreach (var d in Model)
{
    <div class="card mb-2">
        <div class="card-body">
            <h5>@d.Ad</h5>
            <p>@d.Uzmanlik</p>
        </div>
    </div>
}

```

JavaScript:

```

$("#ara").keyup(function () {
    var kelime = $(this).val();
    $.get("/Doktor/AramaSonucu", { ara: kelime }, function (html) {
        $("#sonuclar").html(html);
    });
});

```

Sunucu HTML üretir, JavaScript onu doğrudan bir konteynere yerleştirir. Bu, “server-side render + AJAX güncelleme” desenidir; SPA’sız daha zengin etkileşim sağlar.

4.19.9 Yaygın Hatalar

Hatalı Kod	Sonuç	Neden	Düzeltilmiş
POST’ta anti-forgery token eksik	400 Bad Request	Global filter aktif	Header’da veya form’da token gönder
dataType: "json" ama controller HTML dönüyor	Parse hatası	Cevap JSON değil	Ya controller Json() dönsün ya dataType: "html"
Kültür duyarlı sayı (19,99) JSON’da	Bind fail	Sunucu invariant bekliyor	Client’ta 19.99 gönder
Async controller action senkron olarak çağırılıyor	Deadlock potansiyeli (server tarafta)	.Result ile bekleniyor	await kullan (server-side)
CORS hatası (farklı origin’de host)	Tarayıcı isteği bloke eder	AddCors yapılandırılmamış	Program.cs’e UseCors

4.19.10 Bölüm Özeti

- AJAX = sayfa yenilenmeden istemci-sunucu iletişimi; veri JSON ile taşınır.
- jQuery \$.ajax klasik; fetch modern native alternatif.
- Controller return Json(...) ile JSON döner; System.Text.Json camelCase üretir.
- PartialView ile HTML parçası dönmek zengin senaryolar için.
- Cascading dropdown, AJAX silme, canlı arama tipik kullanım örnekleri.
- POST’larda anti-forgery token header üzerinden gönderilir.
- Property adları JSON’da camelCase (hastaAdi), C#’ta PascalCase (HastaAdi).

İlgili konular: §4.7 (JsonResult, PartialViewResult), §4.18 (Anti-forgery), §4.22 (Toastr — AJAX call-back’te), §5.3 (Cascading dropdown proje uygulaması).

4.20 Base Sınıflar: BaseEntity, BaseController, BaseService

4.20.1 Genel Bakış

Bir uygulamanın entity'leri, controller'ları veya servisleri arasında tekrar eden alanlar/davranışlar olduğunda, DRY prensibi (§1.3.2) gereği bunlar bir taban sınıfta toplanır. §1.5 kalıtım ve §1.8 SOLID prensiplerinin doğrudan pratik uygulamasıdır:

- **BaseEntity**: Her entity'de ortak olan Id, oluşturma/güncelleme tarihi, soft-delete işareti gibi alanları toplar.
- **BaseController**: Ortak TempData mesajları, dropdown yükleme metotları, common filter'lar için taban.
- **BaseService / IBaseService**: Generic CRUD implementasyonu; her entity için EkleAsync, GetirAsync, SilAsync kodunu tekrar yazmaktan kurtarır.

Bu desen, master defter uygulamalarından **HastaneMVC** (§5.2) projesinde uygulanır. Bulky projesi bu deseni kullanmaz — küçük proje için gerekmebilir; ama orta ve büyük projelerde neredeyse standart hale gelir.

4.20.2 BaseEntity

Tüm entity'lerin ortak olarak taşıyacağı alanlar:

```
// Models/Entities/BaseEntity.cs
public abstract class BaseEntity
{
    public int Id { get; set; }
    public DateTime OlusturmaTarihi { get; set; } = DateTime.Now;
    public DateTime? GuncellemeTarihi { get; set; }
    public bool IsDeleted { get; set; } = false;
}
```

Her entity bunu kalıtır:

```
// Models/Entities/Doktor.cs
public class Doktor : BaseEntity
{
    public string Ad { get; set; }
    public string Uzmanlik { get; set; }

    // Navigation
    public int BolumId { get; set; }
    public Bolum Bolum { get; set; }
    public ICollection<Randevu> Randevular { get; set; } = new List<Randevu>();
}
```

Kazanımlar:

- Id, OlusturmaTarihi, IsDeleted her yerde ayrıca tekrar tanımlanmaz.
- Global query filter (§3.8.6) BaseEntity'nin IsDeleted'ine göre kurulup soft-delete otomatik uygulanabilir.
- SaveChanges override edilerek GuncellemeTarihi otomatik atanabilir.

4.20.3 OnModelCreating'de BaseEntity Otomasyonu

Tüm BaseEntity türevlerine query filter uygulama:

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    base.OnModelCreating(modelBuilder);

    // Tüm BaseEntity türetenlere IsDeleted filtresi
    foreach (var entityType in modelBuilder.Model.GetEntityTypes())
    {
        if (typeof(BaseEntity).IsAssignableFrom(entityType.ClrType))
        {

```

```

        var param = System.Linq.Expressions.Expression.Parameter(entityType.ClrType, "e");
        var prop = System.Linq.Expressions.Expression.Property(param, nameof(BaseEntity.IsDeleted));
        var kosul = System.Linq.Expressions.Expression.Equal(prop, System.Linq.Expressions.Expression.Constant(false));
        var lambda = System.Linq.Expressions.Expression.Lambda(kosul, param);

        modelBuilder.Entity(entityType.ClrType).HasQueryFilter(lambda);
    }
}

```

Bu blok karmaşık gözükse de tek bir amaca hizmet eder: reflection ile her BaseEntity türevine e => e.IsDeleted == false filtresini otomatik ekler. Yeni bir entity eklendiğinde bu satırı tekrar yazmaya gerek kalmaz.

4.20.4 SaveChanges Override

OlusturmaTarihi ve GuncellemeTarihi'nin otomatik doldurulması:

```

public override int SaveChanges()
{
    OtomatikTarihleriDoldur();
    return base.SaveChanges();
}

public override Task<int> SaveChangesAsync(CancellationToken cancellationTok = default)
{
    OtomatikTarihleriDoldur();
    return base.SaveChangesAsync(cancellationTok);
}

private void OtomatikTarihleriDoldur()
{
    var entries = ChangeTracker.Entries<BaseEntity>();
    foreach (var e in entries)
    {
        if (e.State == EntityState.Added)
        {
            e.Entity.OlusturmaTarihi = DateTime.Now;
        }
        else if (e.State == EntityState.Modified)
        {
            e.Entity.GuncellemeTarihi = DateTime.Now;
        }
    }
}

```

Bu, “her yerde entity.OlusturmaTarihi = DateTime.Now yazma” tekrarını ortadan kaldırır.

4.20.5 Generic BaseService

CRUD işlemlerinin generic implementasyonu:

```

// Services/IBaseService.cs
public interface IBaseService<T> where T : BaseEntity
{
    Task<IEnumerable<T>> TumunuGetirAsync();
    Task<T> GetirAsync(int id);
    Task EkleAsync(T entity);
    Task GuncelleAsync(T entity);
    Task SoftDeleteAsync(int id);
}

```

```
// Services/BaseService.cs
public class BaseService<T> : IBaseService<T> where T : BaseEntity
{
    protected readonly AppDbContext _context;
    protected readonly DbSet<T> _set;

    public BaseService(AppDbContext context)
    {
        _context = context;
        _set = context.Set<T>();
    }

    public virtual async Task<IEnumerable<T>> TumunuGetirAsync()
        => await _set.AsNoTracking().ToListAsync();

    public virtual async Task<T> GetirAsync(int id)
        => await _set.FindAsync(id);

    public virtual async Task EkleAsync(T entity)
    {
        _set.Add(entity);
        await _context.SaveChangesAsync();
    }

    public virtual async Task GuncelleAsync(T entity)
    {
        _set.Update(entity);
        await _context.SaveChangesAsync();
    }

    public virtual async Task SoftDeleteAsync(int id)
    {
        var entity = await _set.FindAsync(id);
        if (entity != null)
        {
            entity.IsDeleted = true;
            await _context.SaveChangesAsync();
        }
    }
}
```

Her entity için ayrı bir servis:

```
// Services/IDoktorService.cs
public interface IDoktorService : IBaseService<Doktor>
{
    // Doktor'a özgü ek metotlar
    Task<IEnumerable<Doktor>> BolumGoreGetirAsync(int bolumId);
}

// Services/DoktorService.cs
public class DoktorService : BaseService<Doktor>, IDoktorService
{
    public DoktorService(AppDbContext context) : base(context) { }

    public async Task<IEnumerable<Doktor>> BolumGoreGetirAsync(int bolumId)
        => await _set.Where(d => d.BolumId == bolumId).AsNoTracking().ToListAsync();
}
```

DoktorService, temel CRUD metotlarını miras alır; sadece doktor-özü metotları yazar. Program.cs kaydı:

```
builder.Services.AddScoped<IDoktorService, DoktorService>();
builder.Services.AddScoped<IRandevuService, RandevuService>();
```

4.20.6 BaseController

Controller'ların ortak metotları:

```
// Controllers/BaseController.cs
public abstract class BaseController : Controller
{
    protected void BasariMesaji(string mesaj)
    {
        TempData["Success"] = mesaj;
    }

    protected void HataMesaji(string mesaj)
    {
        TempData["Error"] = mesaj;
    }

    protected void UyariMesaji(string mesaj)
    {
        TempData["Warning"] = mesaj;
    }

    protected IActionResult IndexeYonlendir() => RedirectToAction("Index");
}
```

Kullanım:

```
public class DoktorController : BaseController
{
    // ...

    [HttpPost]
    public async Task<IActionResult> Create(Doktor form)
    {
        if (!ModelState.IsValid) return View(form);

        await _service.EkleAsync(form);
        BasariMesaji("Doktor eklendi.");
        return IndexeYonlendir();
    }
}
```

BaseController genellikle küçük tutulur — büyük şişerse yeniden düşünülür. Amaç, yaygın deseni kısayola indirmektir.

4.20.7 Örnek 4.20.1 — Tam BaseEntity + BaseService Zinciri

```
// BaseEntity
public abstract class BaseEntity
{
    public int Id { get; set; }
    public DateTime OlusturmaTarihi { get; set; } = DateTime.Now;
    public DateTime? GuncellemeTarihi { get; set; }
    public bool IsDeleted { get; set; }
}

// Entity
public class Randevu : BaseEntity
{
}
```

```

    public string HastaAdi { get; set; }
    public DateTime Tarih { get; set; }
    public int DoktorId { get; set; }
    public Doktor Doktor { get; set; }
}

// IBaseService (generic)
public interface IBaseService<T> where T : BaseEntity
{
    Task<IEnumerable<T>> TumunuGetirAsync();
    Task<T> GetirAsync(int id);
    Task EkleAsync(T entity);
    Task GuncelleAsync(T entity);
    Task SoftDeleteAsync(int id);
}

// IRandevuService (özel)
public interface IRandevuService : IBaseService<Randevu>
{
    Task<IEnumerable<Randevu>> DoktoraGoreGetirAsync(int doktorId);
    Task<IEnumerable<Randevu>> TariheGoreGetirAsync(DateTime tarih);
}

// BaseService (generic implementasyon)
public class BaseService<T> : IBaseService<T> where T : BaseEntity
{
    protected readonly AppDbContext _context;
    protected readonly DbSet<T> _set;

    public BaseService(AppDbContext context) { _context = context; _set = context.Set<T>(); }

    public virtual async Task<IEnumerable<T>> TumunuGetirAsync() => await _set.AsNoTracking().ToListAsync();
    public virtual async Task<T> GetirAsync(int id) => await _set.FindAsync(id);
    public virtual async Task EkleAsync(T entity) { _set.Add(entity); await _context.SaveChangesAsync(); }
    public virtual async Task GuncelleAsync(T entity) { _set.Update(entity); await _context.SaveChangesAsync(); }
    public virtual async Task SoftDeleteAsync(int id)
    {
        var e = await _set.FindAsync(id);
        if (e != null) { e.IsDeleted = true; await _context.SaveChangesAsync(); }
    }
}

// RandevuService (özel implementasyon)
public class RandevuService : BaseService<Randevu>, IRandevuService
{
    public RandevuService(AppDbContext context) : base(context) { }

    public async Task<IEnumerable<Randevu>> DoktoraGoreGetirAsync(int doktorId)
    => await _set.Include(r => r.Doktor)
        .Where(r => r.DoktorId == doktorId)
        .OrderBy(r => r.Tarih)
        .AsNoTracking()
        .ToListAsync();

    public async Task<IEnumerable<Randevu>> TariheGoreGetirAsync(DateTime tarih)
    => await _set.Where(r => r.Tarih.Date == tarih.Date)
        .Include(r => r.Doktor)
        .AsNoTracking()
        .ToListAsync();
}

```

```
// Program.cs
builder.Services.AddScoped(typeof(IBaseService<>), typeof(BaseService<>));
builder.Services.AddScoped<IRandevuService, RandevuService>();
```

Bu iskelet §5.2 HastaneMVC projesinin omurgasıdır.

4.20.8 Yaygın Hatalar

Hatalı Kod	Sonuç	Neden	Düzeltilmiş
BaseEntity abstract değil, new BaseEntity() oluşturulmuş	Anlamsız satır	Semantic yanlış	abstract yap
BaseService generic ama AddScoped<IBaseService<Doktor>, ...> unutulmuş	DI resolve edemez	Kayıt yok	AddScoped(typeof(IBaseService<>), typeof(BaseService<>))
Override edilmemiş Update — _set.Update(entity) kullanılmış ama entity detached değil	Extra UPDATE	Bir kez izleniyor	IsDeleted = true set edip SaveChanges
Query filter var, ama admin kullanıcı silinen kayıtları görmek istiyor	Görmez	Filter aktif	IgnoreQueryFilters()

4.20.9 Bölüm Özeti

- BaseEntity: Id, OlusturmaTarihi, GuncellemeTarihi, IsDeleted — tüm entity’lerde ortak.
- SaveChanges override ile tarihler otomatik doldurulur.
- BaseService: generic CRUD implementasyonu; her entity için tekrar yazılmaz.
- IBaseService: sözleşme; özel servisler genişletir.
- BaseController: TempData mesaj metotları, ortak redirect.
- Global query filter reflection ile tüm BaseEntity’lere IsDeleted filtresi ekler.
- Küçük projelerde gereksiz olabilir; orta+ projelerde standart hale gelir.

İlgili konular: §1.5 (Kalıtım), §3.8 (HasQueryFilter), §4.21 (Soft Delete), §5.2 (HastaneMVC — bu deseni tam uygular).

4.21 Soft Delete

4.21.1 Genel Bakış

Soft delete, bir kaydı fiziksel olarak silmek yerine “silinmiş” olarak işaretleme desenidir. Kayıt veritabanında kalır; sadece bir bayrak (IsDeleted = true) ile listelerden gizlenir. Klasik DELETE (hard delete) yerine UPDATE kullanılır.

Neden soft delete?

- **Denetim izi:** Kim, ne zaman, hangi kaydı sildi — kayıt duruyor, sorgulanabilir.
- **Geri alınabilirlik:** Yanlışlıkla silmek yerine IsDeleted = false ile geri getirmek.
- **Referans bütünlüğü:** FK’larla bağlı kayıtlar silinmez; ilişkiler kırılmaz.
- **Yasal zorunluluk:** Bazı sektörlerde (sağlık, finans) veri retention yasaları vardır.

Trade-off: veritabanı büyümesi, listeleme sorgularının WHERE IsDeleted = 0 filtresine ihtiyaç duyması.

4.21.2 Model Değişikliği

BaseEntity'de IsDeleted alanı zaten var (§4.20.2). Değilse:

```
public class Doktor
{
    public int Id { get; set; }
    public string Ad { get; set; }
    public string Uzmanlik { get; set; }
    public bool IsDeleted { get; set; } // ← eklendi
}
```

Migration ile sütunu ekle:

```
Add-Migration AddIsDeletedToDoktor
Update-Database
```

4.21.3 Global Query Filter

Her sorguya otomatik WHERE IsDeleted = 0 eklenmesi için Fluent API (§3.8.6):

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Doktor>()
        .HasQueryFilter(d => !d.IsDeleted);
}
```

Artık _context.Doktorlar.ToList() çağrısı otomatik olarak WHERE IsDeleted = 0 üretir. Silinmiş kayıtları görmek için IgnoreQueryFilters():

```
var hepsi = _context.Doktorlar.IgnoreQueryFilters().ToList();
var silinenler = _context.Doktorlar.IgnoreQueryFilters().Where(d => d.IsDeleted).ToList();
```

4.21.4 Silme İşlemi

Normal Remove yerine, entity'yi bulup IsDeleted = true işaretlemek:

```
public async Task SoftDeleteAsync(int id)
{
    var d = await _context.Doktorlar.FindAsync(id);
    if (d != null)
    {
        d.IsDeleted = true;
        await _context.SaveChangesAsync();
    }
}
```

Controller:

```
[HttpPost, ActionName("Delete")]
[ValidateAntiForgeryToken]
public async Task<IActionResult> DeleteConfirmed(int id)
{
    await _service.SoftDeleteAsync(id);
    TempData["Success"] = "Doktor silindi.";
    return RedirectToAction(nameof(Index));
}
```

4.21.5 Geri Alma (Restore)

```
public async Task RestoreAsync(int id)
{
    var d = await _context.Doktorlar
        .IgnoreQueryFilters()
```

```

        .FirstOrDefaultAsync(x => x.Id == id);

    if (d != null && d.IsDeleted)
    {
        d.IsDeleted = false;
        await _context.SaveChangesAsync();
    }
}

```

IgnoreQueryFilters() gerekli — çünkü silinmiş bir kaydı normal sorgu göremez.

4.21.6 Cascading Soft Delete

Bir doktor silindiğinde onun tüm randevuları da soft-delete edilmeli mi? Bu iş kuralına bağlıdır:

Manuel yaklaşım — servis içinde:

```

public async Task SoftDeleteWithChildrenAsync(int doktorId)
{
    var doktor = await _context.Doktorlar
        .Include(d => d.Randevular)
        .FirstOrDefaultAsync(d => d.Id == doktorId);

    if (doktor == null) return;

    doktor.IsDeleted = true;
    foreach (var r in doktor.Randevular)
        r.IsDeleted = true;

    await _context.SaveChangesAsync();
}

```

Otomatik cascade EF Core'da soft-delete için yok; navigation üzerinden elle işlenir. Domain ihtiyacına göre karar verilir — bazı projelerde parent silinse de child'lar aktif kalır.

4.21.7 Global Query Filter'ı Devre Dışı Bırakmak

Bazı senaryolarda (admin panel, denetim raporu) tüm kayıtlar gerekir:

```

public async Task<IEnumerable<Doktor>> AdminTumListeAsync()
    => await _context.Doktorlar.IgnoreQueryFilters().ToListAsync();

public async Task<IEnumerable<Doktor>> SilinenListeAsync()
    => await _context.Doktorlar.IgnoreQueryFilters().Where(d => d.IsDeleted).ToListAsync();

```

4.21.8 Unique Constraint ile Soft Delete Çatışması

Bir kullanıcı adının benzersiz olduğu bir tabloda soft-delete edilmiş bir kaydın kullanıcı adını tekrar kayıt eklerken kullanmak istenirse, unique constraint hataya sebep olur.

Çözümler:

1. **Filtered unique index** (§3.14.5): Sadece aktif kayıtlar için benzersizlik.

```

modelBuilder.Entity<Kullanici>()
    .HasIndex(k => k.KullaniciAdi)
    .IsUnique()
    .HasFilter("[IsDeleted] = 0");

```

2. **Kullanıcı adına silme timestamp'i eklemek** — silme anında KullaniciAdi = \$"{Original}_deleted_{Guid.NewGuid()}". Estetik olmayan ama garantili yaklaşım.
3. **Hard delete + audit tablo** — soft-delete yerine kayıt gerçekten silinir, ama bir audit log tablosunda görünür kalır.

4.21.9 Yaygın Hatalar

Hatalı Kod	Sonuç	Neden	Düzeltilmiş
<code>_context.Doktorlar.Remove(kayıt)</code> (soft delete olması gereken)	Kayıt fiziksel silinir	Hard delete yapıldı	<code>d.IsDeleted = true</code> ile update
<code>IsDeleted</code> filter'ı var, silinen kayıt arama sonuçlarında yok — kullanıcı şaşırdı	Kayıt görünmüyor	Query filter aktif	Kayıt zaten silinmiş; ya restore et ya sonucu yok göster
Restore için normal <code>Find</code> kullanılıyor	Kayıt bulunamaz	Query filter siliyor	<code>IgnoreQueryFilters()</code>
Unique constraint + soft delete çatışma	INSERT hatası	Silinen kayıt hâlâ isim tutuyor	Filtered unique index
Cascading soft delete unutulmuş	Parent silindi, child'lar hâlâ aktif	Manuel cascade yok	Servis metoduna ekle

4.21.10 Bölüm Özeti

- Soft delete = fiziksel silme değil, `IsDeleted = true` işaretleme.
- Global query filter (`HasQueryFilter`) tüm sorgulara otomatik `WHERE IsDeleted = 0` ekler.
- Silme fiili = `Remove` yerine `IsDeleted = true` + `SaveChanges`.
- Restore için `IgnoreQueryFilters()` ile önce bul, sonra `IsDeleted = false`.
- Cascade otomatik değil; parent silme → child'lar servis içinde manuel işlenir.
- Unique constraint çatışması filtered index ile çözülür.
- `BaseEntity` (§4.20) ile birleşince tüm entity'lere ortak soft-delete uygulanır.

İlgili konular: §3.8.6 (`HasQueryFilter`), §4.20 (`BaseEntity`), §3.14 (Unique + soft delete), §5.2 (Has-taneMVC uygulaması).

4.22 Toastr Bildirimleri

4.22.1 Genel Bakış

Toastr, tarayıcının köşesinde küçük bildirim kutuları (toast) gösteren jQuery tabanlı bir kütüphanedir. Bir kullanıcı işlemi tamamlandığında (“Kayıt eklendi”, “Hata: ...”) kısa ömürlü, göze çarpmayan geri bildirim sağlar. MVC ile TempData birleşiminde POST-Redirect-GET akışının doğal parçasıdır.

Ekran örneği (mimari):

4.22.2 Kurulum

CDN üzerinden (en pratik) — `_Layout.cshtml`'de:

```
<head>
...
<link rel="stylesheet"
      href="https://cdnjs.cloudflare.com/ajax/libs/toastr.js/2.1.4/toastr.min.css" />
</head>
<body>
...
<script src="~/lib/jquery/dist/jquery.min.js"></script>
<script src="https://cdnjs.cloudflare.com/ajax/libs/toastr.js/2.1.4/toastr.min.js"></script>
...
</body>
```

Local kurulum — `wwwroot/lib/toastr/` altına dosyalar konur; LibMan (Library Manager) veya npm ile.

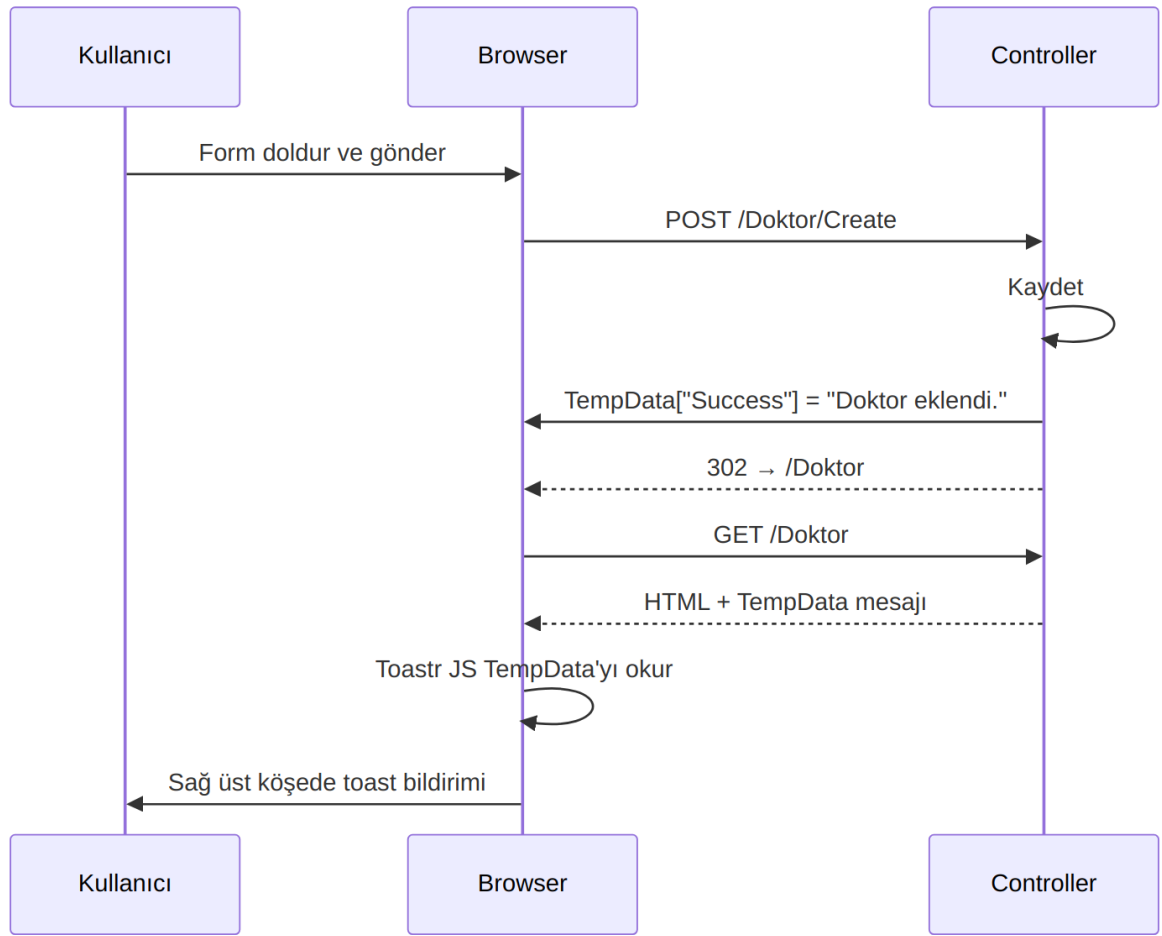


Figure 13: Diyagram 13

4.22.3 Temel Kullanım

Toastr dört tip bildirim sunar:

```
toastr.success("Kayıt eklendi!");  
toastr.info("Bilgi mesajı");  
toastr.warning("Dikkat: bir sorun olabilir");  
toastr.error("Hata oluştu!");
```

Ek başlık parametresi:

```
toastr.success("Doktor eklendi.", "Başarılı");
```

4.22.4 Konfigürasyon

Global ayarlar:

```
toastr.options = {  
    "closeButton": true,  
    "positionClass": "toast-top-right",    // toast-top-right / toast-bottom-right / ...  
    "timeOut": "3000",  
    "extendedTimeOut": "1000",  
    "showEasing": "swing",  
    "hideEasing": "linear",  
    "showMethod": "fadeIn",  
    "hideMethod": "fadeOut",  
    "progressBar": true  
};
```

Bu ayarları Layout'ta bir kez atamak, tüm sayfalarda tutarlı davranış sağlar.

4.22.5 TempData ile Entegrasyon

Controller mesajı TempData'ya yazar, view/layout mesajı Toastr'a çevirir:

Controller:

```
[HttpPost]  
public IActionResult Create(Doktor form)  
{  
    if (!ModelState.IsValid) return View(form);  
  
    _context.Doktorlar.Add(form);  
    _context.SaveChanges();  
  
    TempData["Success"] = "Doktor başarıyla eklendi.";  
    return RedirectToAction(nameof(Index));  
}  
  
[HttpPost]  
public IActionResult Delete(int id)  
{  
    try  
    {  
        _context.Doktorlar.Remove(_context.Doktorlar.Find(id));  
        _context.SaveChanges();  
        TempData["Success"] = "Doktor silindi.";  
    }  
    catch (Exception ex)  
    {  
        TempData["Error"] = $"Silme sırasında hata: {ex.Message}";  
    }  
    return RedirectToAction(nameof(Index));  
}
```

Layout (_Layout.cshtml):

```
<!-- Body sonunda, Toastr JS yüklendikten sonra -->
<script>
    $(function () {
        @if (TempData["Success"] != null)
        {
            <text>toastr.success("@TempData["Success"]");</text>
        }
        @if (TempData["Error"] != null)
        {
            <text>toastr.error("@TempData["Error"]");</text>
        }
        @if (TempData["Warning"] != null)
        {
            <text>toastr.warning("@TempData["Warning"]");</text>
        }
        @if (TempData["Info"] != null)
        {
            <text>toastr.info("@TempData["Info"]");</text>
        }
    });
</script>
```

<text> Razor bloğu, C# içinden JavaScript kodunu çıkışa yazmak için kullanılır. Toastr çağrısı sayfa yüklendiğinde bir kere çalışır ve TempData otomatik olarak temizlenir (§4.10.6).

4.22.6 Örnek 4.22.1 — Partial ile Temiz Layout

Toastr çağrılarını ayrı bir partial'a çıkarmak:

```
@* Views/Shared/_Notifications.cshtml *@
<script>
    $(function () {
        @if (TempData["Success"] != null)
        {
            <text>toastr.success("@TempData["Success"]");</text>
        }
        @if (TempData["Error"] != null)
        {
            <text>toastr.error("@TempData["Error"]");</text>
        }
        @if (TempData["Warning"] != null)
        {
            <text>toastr.warning("@TempData["Warning"]");</text>
        }
        @if (TempData["Info"] != null)
        {
            <text>toastr.info("@TempData["Info"]");</text>
        }
    });
</script>
```

Layout'ta:

```
<script src="~/lib/toastr/toastr.min.js"></script>
<partial name="_Notifications" />
```

Bu ayrım Layout'u temiz tutar; her mesaj tipi eklendiğinde/çıkarıldığında sadece partial değişir.

4.22.7 AJAX Response ile Toastr

AJAX yanıtında sunucu bir mesaj döndürebilir, JavaScript bunu toastr'a bağlayabilir:

```
$.post("/Doktor/AjaxSil", { id: id }, function (sonuc) {
    if (sonuc.basarili) {
        toastr.success(sonuc.mesaj);
        $("#row-" + id).fadeOut();
    } else {
        toastr.error(sonuc.mesaj);
    }
});
```

```
[HttpPost]
public IActionResult AjaxSil(int id)
{
    var d = _context.Doktorlar.Find(id);
    if (d == null) return Json(new { basarili = false, mesaj = "Kayıt bulunamadı." });

    _context.Doktorlar.Remove(d);
    _context.SaveChanges();
    return Json(new { basarili = true, mesaj = "Doktor silindi." });
}
```

AJAX senaryosunda TempData/redirect zinciri yoktur; JSON dönüşü içinde mesaj taşınır.

4.22.8 Alternatifler

Toastr popüler ama tek seçenek değildir:

Kütüphane	Notlar
Toastr	jQuery gerektirir; en yaygın
SweetAlert2	Modal + toast; daha zengin görsel
noty.js	Toastr benzeri, jQuery bağımsız
Bootstrap Toast	Bootstrap 5 yerleşik; ek kütüphane yok
Notyf	Modern, hafif, jQuery bağımsız

Master defterin standardı Toastr'dır — kolay entegre, jQuery ile uyumlu, Bulky ve HastaneMVC projelerinde kullanılır.

4.22.9 Bootstrap 5 Toast Alternatifi (Kütüphane Bağımsız)

```
<div class="toast-container position-fixed bottom-0 end-0 p-3">
  <div id="mesajToast" class="toast" role="alert">
    <div class="toast-header">
      <strong class="me-auto">Bildirim</strong>
      <button type="button" class="btn-close" data-bs-dismiss="toast"></button>
    </div>
    <div class="toast-body" id="mesajBody"></div>
  </div>
</div>

<script>
  function bildirGoster(mesaj) {
    document.getElementById("mesajBody").textContent = mesaj;
    var t = new bootstrap.Toast(document.getElementById("mesajToast"));
    t.show();
  }
</script>
```

Bootstrap 5 varsa ek CSS/JS kütüphane eklemeye gerek yoktur.

4.22.10 Yaygın Hatalar

Hatalı Kod	Sonuç	Neden	Düzeltilmiş
Toastr JS jQuery'den önce yüklenmiş	toastr is not defined veya jQuery hatası	Bağımlılık sırası	jQuery önce, Toastr sonra
TempData set edildi ama layout'ta script yok	Bildirim gözüküyor	Layout entegrasyonu eksik	_Notifications partial ekle
TempData mesajında " karakteri	JS syntax error	Escape edilmedi	Html.Encode veya JavaScriptStringEncode
CDN erişilemez ortamda çalışıyor	Toastr yüklenemez	Offline	Local fallback ekle veya wwwroot'a al
Toast timeout çok kısa	Kullanıcı okuyamıyor	3sn yetersiz	4000-6000ms önerilir

4.22.11 Bölüm Özeti

- Toastr: köşede kısa süreli bildirim kutuları (success/info/warning/error).
- Kurulum: CDN veya local; jQuery bağımlılığı var.
- TempData ile entegrasyon: controller TempData["Success"] = "... " → layout Razor blok → toastr.success(...).
- AJAX response'ta JSON içinde mesaj → JavaScript callback'te toastr çağrısı.
- Partial'a çıkarmak (_Notifications.cshtml) layout'u temiz tutar.
- Alternatifler: SweetAlert2, Bootstrap 5 Toast, Notyf.

İlgili konular: §4.10 (TempData), §4.9 (Layout ve partial), §4.19 (AJAX), §5.1 (BulkyBookWeb — kayıt sonrası Toastr), §5.2 (HastaneMVC — Toastr bildirim yönetimi).

Bölüm V — Proje Dokümantasyonları

Bu bölüm, önceki bölümlerde işlenen tüm kavramların dört farklı projede nasıl bir araya geldiğini gösterir. Her proje ayrı bir odaklanma noktasıyla ele alınmıştır:

- **§5.1 BulkyBookWeb** — Yalın Code First + Data Annotations + Toastr üzerine kurulu, tek modüllü (Kategori CRUD) bir örnek. Bir MVC projesinin baştan sona iskeleti.
- **§5.2 HastaneMVC** — BaseEntity + generic BaseService + soft delete + çoklu modül (Doktor, Randevu, Tahlil) üzerine kurulu, katmanlı ve orta ölçekli bir uygulama. Master defterin en kapsamlı proje bölümü.
- **§5.3 HastaneWeb / AspNet7Hastane** — Database First + Scaffold-DbContext + AJAX cascading dropdown yaklaşımı. Var olan bir veritabanından uygulama üretme senaryosu.
- **§5.4 ÖrnekUygulama** — Ürün yönetimi için minimal CRUD; §5.1'in daha da yalınlaştırılmış eğitim odaklı hali.

Her projenin sonundaki "Beklenen Sonuçlar" tabloları, ilgili modülün akışını test odaklı bir kontrol listesine dönüştürür.

5.1 BulkyBookWeb — Kategori CRUD Modülü

5.1.1 Proje Tanıtımı

BulkyBookWeb, bir online kitap satış sisteminin ana veri modülü olarak tasarlanmış bir MVC projesidir. Kitaplar Kategori'lere ayrılır (Roman, Bilim Kurgu, Ders Kitabı, ...); her kategorinin bir görüntüleme sırası vardır. Bu proje bölümü, sistemin kategori yönetim modülünü — kategori listeleme, ekleme, düzenleme, silme (CRUD) — baştan sona işler.

Amaç: Bir MVC projesinin **tüm iskelet parçalarını** tek bir odaklı modülde göstermek — Program.cs kurulumu, Data Annotations validasyonu, EF Core Code First migration, Bootstrap tema, Toastr bildirimleri. Kod tabanı küçük, kavram yoğunluğu yüksek.

5.1.2 Ortam

Bileşen	Değer
.NET SDK	8.0.x
Proje Şablonu	ASP.NET Core Web App (Model-View-Controller)
Veritabanı	SQL Server (localdb)
EF Core	8.0.x
Bootstrap Teması	Bootswatch Litera
Bildirim	Toastr 2.1.4 (CDN)
Nullable Reference Types	Açık
Implicit Usings	Açık
NuGet Paketleri	Microsoft.EntityFrameworkCore.SqlServer, Microsoft.EntityFrameworkCore.Tools, Microsoft.EntityFrameworkCore.Design

5.1.3 Proje Yapısı

```
BulkyBookWeb/
├── Controllers/
│   ├── HomeController.cs
│   └── KategoriController.cs
├── Data/
│   └── AppDbContext.cs
├── Migrations/
│   └── 20240610120000_InitialCreate.cs
├── Models/
│   ├── ErrorViewModel.cs
│   └── Kategori.cs
├── Views/
│   ├── Home/
│   │   ├── Index.cshtml
│   │   └── Privacy.cshtml
│   ├── Kategori/
│   │   ├── Index.cshtml
│   │   ├── Create.cshtml
│   │   ├── Edit.cshtml
│   │   └── Delete.cshtml
│   ├── Shared/
│   │   ├── _Layout.cshtml
│   │   └── _ValidationScriptsPartial.cshtml
│   ├── _ViewImports.cshtml
│   └── _ViewStart.cshtml
├── wwwroot/
│   ├── css/site.css
│   ├── js/site.js
│   └── lib/
├── Program.cs
├── appsettings.json
└── BulkyBookWeb.csproj
```

Sadelik dikkate değer: tek bir domain sınıfı (Kategori), tek bir controller (KategoriController), dört view. Proje şablonuyla gelen HomeController da korunur ama iş yükü taşımaz.

5.1.4 Veri Modeli

Tek entity: Kategori.

```
// Models/Kategori.cs
using System.ComponentModel.DataAnnotations;
```

```

public class Kategori
{
    [Key]
    public int Id { get; set; }

    [Required(ErrorMessage = "Kategori adı zorunlu.")]
    [MaxLength(50, ErrorMessage = "En fazla 50 karakter.")]
    [Display(Name = "Kategori Adı")]
    public string Ad { get; set; }

    [Range(1, 100, ErrorMessage = "1-100 arasında olmalı.")]
    [Display(Name = "Görüntüleme Sırası")]
    public int GosterimSirasi { get; set; }

    [Display(Name = "Oluşturma Tarihi")]
    public DateTime OluşturmaTarihi { get; set; } = DateTime.Now;
}

```

Id konvansiyonel olarak PK; [Key] açıklık için yazılmış (opsiyonel). Ad ve GosterimSirasi validation ile birlikte. OluşturmaTarihi varsayılan DateTime.Now — kayıt oluşturulunca otomatik atanır.

5.1.5 DbContext

```

// Data/AppDbContext.cs
using Microsoft.EntityFrameworkCore;

public class AppDbContext : DbContext
{
    public AppDbContext(DbContextOptions<AppDbContext> options) : base(options) { }

    public DbSet<Kategori> Kategoriler { get; set; }
}

```

Tek DbSet — proje bir domain nesnesine odaklıdır. Fluent API veya OnModelCreating gerekmez, tüm konfigürasyon Data Annotations üzerinden yapılmıştır.

5.1.6 Program.cs

```

using Microsoft.EntityFrameworkCore;
using BulkyBookWeb.Data;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllersWithViews();
builder.Services.AddDbContext<AppDbContext>(o =>
    o.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));

var app = builder.Build();

if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Home/Error");
    app.UseHsts();
}

app.UseHttpsRedirection();
app.UseStaticFiles();
app.UseRouting();
app.UseAuthorization();

```

```
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");

app.Run();
```

Standart minimal hosting şablonu. Servisler: AddControllersWithViews (MVC), AddDbContext<AppDbContext> (EF Core scoped). Pipeline standart sırayla — bkz. §4.3.

5.1.7 appsettings.json

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Server=(localdb)\\mssqllocaldb;Database=BulkyBookDb;Trusted_Connection="
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*"
}
```

Master defter standardı: DefaultConnection anahtar adı. LocalDB kullanımı geliştirme kolaylığı için ideal.

5.1.8 Migration İş Akışı

Model ilk kez tanımlandığında:

```
Add-Migration InitialCreate
Update-Database
```

Üretilen tablo (özet):

```
CREATE TABLE Kategoriler (
    Id INT IDENTITY PRIMARY KEY,
    Ad NVARCHAR(50) NOT NULL,
    GosterimSirasi INT NOT NULL,
    OlusturmaTarihi DATETIME2 NOT NULL
);
```

Sonraki alan eklemelerinde:

```
Add-Migration AddDescriptionToKategori
Update-Database
```

Migration dosyaları Migrations/ klasöründe versiyonlanır; ekip çalışmasında Git ile paylaşılır.

5.1.9 KategoriController — Tam Kod

```
// Controllers/KategoriController.cs
using BulkyBookWeb.Data;
using BulkyBookWeb.Models;
using Microsoft.AspNetCore.Mvc;

public class KategoriController : Controller
{
    private readonly AppDbContext _context;

    public KategoriController(AppDbContext context)
    {
```

```

        _context = context;
    }

    // GET: /Kategori
    public IActionResult Index()
    {
        var liste = _context.Kategoriler
            .OrderBy(k => k.GosterimSirasi)
            .ToList();
        return View(liste);
    }

    // GET: /Kategori/Create
    public IActionResult Create()
    {
        return View();
    }

    // POST: /Kategori/Create
    [HttpPost]
    [ValidateAntiForgeryToken]
    public IActionResult Create(Kategori kategori)
    {
        // Ek iş kuralı: Ad ve GosterimSirasi aynı olamaz
        if (kategori.Ad?.ToString() == kategori.GosterimSirasi.ToString())
        {
            ModelState.AddModelError(
                nameof(kategori.Ad),
                "Kategori adı ile görüntüleme sırası aynı olamaz.");
        }

        if (!ModelState.IsValid)
        {
            return View(kategori);
        }

        _context.Kategoriler.Add(kategori);
        _context.SaveChanges();

        TempData["Success"] = "Kategori başarıyla eklendi.";
        return RedirectToAction(nameof(Index));
    }

    // GET: /Kategori/Edit/5
    public IActionResult Edit(int? id)
    {
        if (id == null || id == 0) return NotFound();

        var kategori = _context.Kategoriler.Find(id);
        if (kategori == null) return NotFound();

        return View(kategori);
    }

    // POST: /Kategori/Edit/5
    [HttpPost]
    [ValidateAntiForgeryToken]
    public IActionResult Edit(Kategori kategori)
    {
        if (!ModelState.IsValid) return View(kategori);
    }

```

```

        _context.Kategoriler.Update(kategori);
        _context.SaveChanges();

        TempData["Success"] = "Kategori güncellendi.";
        return RedirectToAction(nameof(Index));
    }

    // GET: /Kategori/Delete/5
    public IActionResult Delete(int? id)
    {
        if (id == null || id == 0) return NotFound();

        var kategori = _context.Kategoriler.Find(id);
        if (kategori == null) return NotFound();

        return View(kategori);
    }

    // POST: /Kategori/Delete/5
    [HttpPost, ActionName("Delete")]
    [ValidateAntiForgeryToken]
    public IActionResult DeleteConfirmed(int? id)
    {
        var kategori = _context.Kategoriler.Find(id);
        if (kategori == null) return NotFound();

        _context.Kategoriler.Remove(kategori);
        _context.SaveChanges();

        TempData["Success"] = "Kategori silindi.";
        return RedirectToAction(nameof(Index));
    }
}

```

Notlar:

- Constructor injection ile ApplicationDbContext alınır (§4.16).
- Her POST action'da [ValidateAntiForgeryToken] — proje klasik yaklaşımı kullanır (§4.18.3.a). Master defterin genel önerisi global filter olsa da, bu proje küçük olduğu için action başına attribute görünürdür.
- [HttpPost, ActionName("Delete")] + DeleteConfirmed deseni: GET /Kategori/Delete/5 ile onay formunu göster, POST aynı URL ile silmeyi yap; metod adı çakışmasın diye ActionName ile route'u sabitle.
- Ek iş kuralı olarak Ad ve GosterimSirasi aynı olamaz; Data Annotations dışında controller'da manuel ModelState.AddModelError.
- POST sonrası her zaman RedirectToAction(nameof(Index)) — PRG deseni (§4.7.8).
- TempData["Success"] layout'ta Toastr'la görüntülenecek.

5.1.10 Ana Akış Diyagramı

5.1.11 View'lar — Index

```

@* Views/Kategori/Index.cshtml *@
@model IEnumerable<Kategori>

@{
    ViewData["Title"] = "Kategoriler";
}

<div class="d-flex justify-content-between align-items-center mb-3">
    <h1>Kategoriler</h1>
    <a asp-action="Create" class="btn btn-primary">

```

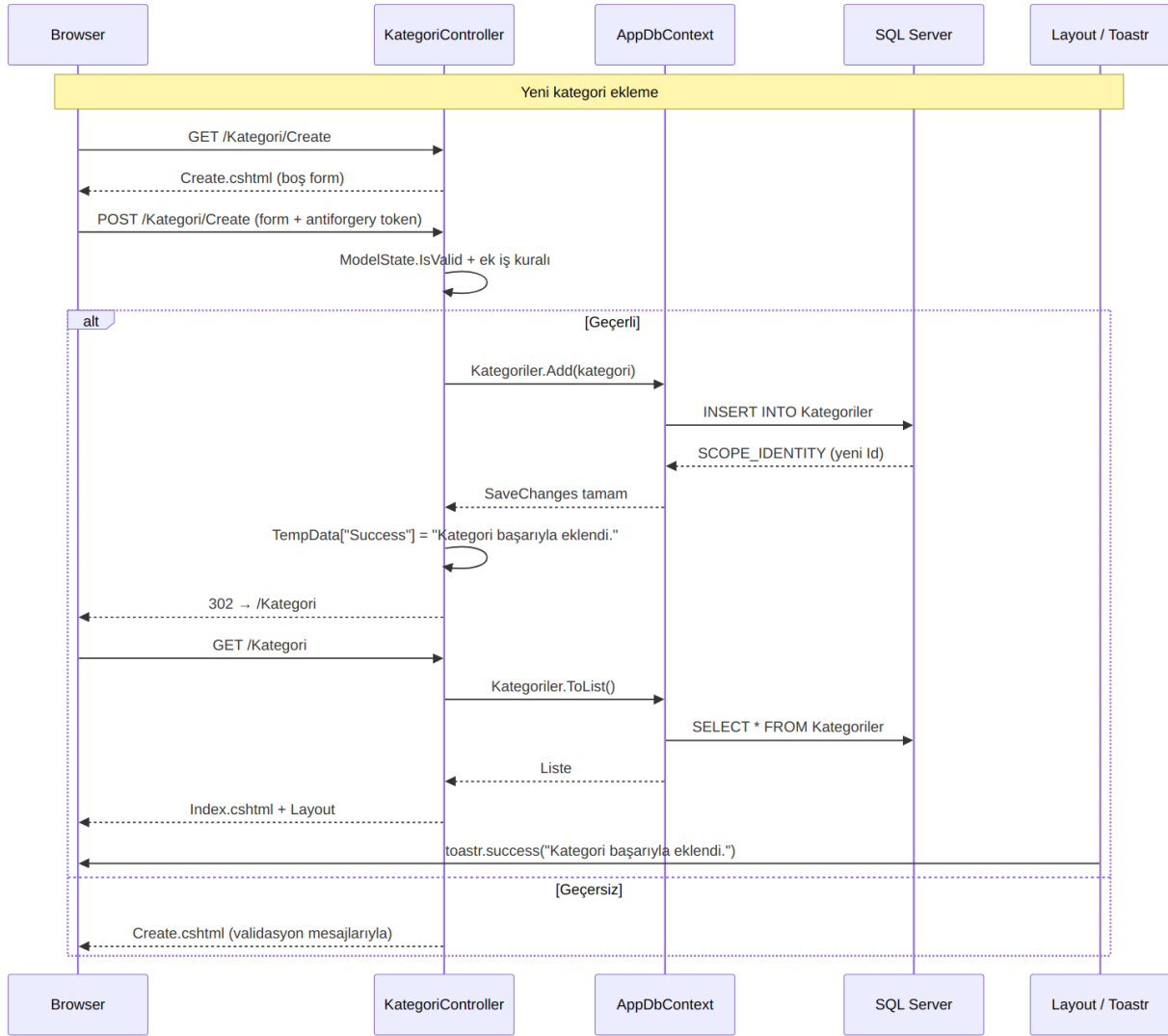


Figure 14: Diyagram 14

```

        <i class="bi bi-plus-circle"></i> Yeni Kategori
    </a>
</div>

<table class="table table-hover">
    <thead class="table-light">
        <tr>
            <th>@Html.DisplayNameFor(m => m.First().Ad)</th>
            <th>@Html.DisplayNameFor(m => m.First().GosterimSirasi)</th>
            <th>@Html.DisplayNameFor(m => m.First().OlusturmaTarihi)</th>
            <th class="text-end">İşlemler</th>
        </tr>
    </thead>
    <tbody>
        @foreach (var k in Model)
        {
            <tr>
                <td>@k.Ad</td>
                <td>@k.GosterimSirasi</td>
                <td>@k.OlusturmaTarihi.ToString("dd.MM.yyyy HH:mm")</td>
                <td class="text-end">
                    <a asp-action="Edit" asp-route-id="@k.Id" class="btn btn-sm btn-warning">Düzenle</a>
                    <a asp-action="Delete" asp-route-id="@k.Id" class="btn btn-sm btn-danger">Sil</a>
                </td>
            </tr>
        }
    </tbody>
</table>

```

- IEnumerable<Kategori> model tipi — liste sayfası.
- Html.DisplayNameFor — [Display(Name = "...")] attribute'undan başlık üretir.
- URL'ler Tag Helper ile (asp-action, asp-route-id).

5.1.12 View'lar — Create

```

@* Views/Kategori/Create.cshtml *@
@model Kategori

@{
    ViewData["Title"] = "Yeni Kategori";
}

<h1>Yeni Kategori</h1>

<form asp-action="Create" method="post">
    <div asp-validation-summary="ModelOnly" class="text-danger"></div>

    <div class="mb-3">
        <label asp-for="Ad" class="form-label"></label>
        <input asp-for="Ad" class="form-control" />
        <span asp-validation-for="Ad" class="text-danger"></span>
    </div>

    <div class="mb-3">
        <label asp-for="GosterimSirasi" class="form-label"></label>
        <input asp-for="GosterimSirasi" class="form-control" />
        <span asp-validation-for="GosterimSirasi" class="text-danger"></span>
    </div>

    <button type="submit" class="btn btn-primary">Kaydet</button>
    <a asp-action="Index" class="btn btn-secondary">İptal</a>
</form>

```

```

</form>

@section Scripts {
    <partial name="_ValidationScriptsPartial" />
}

```

Standart form iskelet — asp-for, asp-validation-for çifti, _ValidationScriptsPartial client-side validation için.

5.1.13 View'lar — Edit

Create ile çok benzer; farklar: Id gizli input olarak eklenir, action Edit, model dolu gelir.

```

@* Views/Kategori/Edit.cshtml *@
@model Kategori

@{
    ViewData["Title"] = "Kategori Düzenle";
}

<h1>Kategori Düzenle</h1>

<form asp-action="Edit" method="post">
    <div asp-validation-summary="ModelOnly" class="text-danger"></div>

    <input type="hidden" asp-for="Id" />
    <input type="hidden" asp-for="OlusturmaTarihi" />

    <div class="mb-3">
        <label asp-for="Ad" class="form-label"></label>
        <input asp-for="Ad" class="form-control" />
        <span asp-validation-for="Ad" class="text-danger"></span>
    </div>

    <div class="mb-3">
        <label asp-for="GosterimSirasi" class="form-label"></label>
        <input asp-for="GosterimSirasi" class="form-control" />
        <span asp-validation-for="GosterimSirasi" class="text-danger"></span>
    </div>

    <button type="submit" class="btn btn-primary">Güncelle</button>
    <a asp-action="Index" class="btn btn-secondary">İptal</a>
</form>

@section Scripts {
    <partial name="_ValidationScriptsPartial" />
}

```

OlusturmaTarihi hidden'da tutulur ki Update çağrısında property atlanmasın (aksi halde DateTime.MinValue olarak DB'ye gider). Alternatif: entity'yi bul, sadece değişen alanları kopyala (Find + property atama).

5.1.14 View'lar — Delete

Onay sayfası. GET ile göster, POST ile sil.

```

@* Views/Kategori/Delete.cshtml *@
@model Kategori

@{
    ViewData["Title"] = "Kategori Sil";
}

```

```

<h1 class="text-danger">Kategori Sil</h1>
<p class="lead">Bu kategoriyi silmek istediğinizden emin misiniz?</p>

<dl class="row">
  <dt class="col-sm-3">@Html.DisplayNameFor(m => m.Ad)</dt>
  <dd class="col-sm-9">@Model.Ad</dd>

  <dt class="col-sm-3">@Html.DisplayNameFor(m => m.GosterimSirasi)</dt>
  <dd class="col-sm-9">@Model.GosterimSirasi</dd>

  <dt class="col-sm-3">@Html.DisplayNameFor(m => m.OlusturmaTarihi)</dt>
  <dd class="col-sm-9">@Model.OlusturmaTarihi.ToString("dd.MM.yyyy HH:mm")</dd>
</dl>

<form asp-action="Delete" method="post">
  <input type="hidden" asp-for="Id" />
  <button type="submit" class="btn btn-danger">Evet, Sil</button>
  <a asp-action="Index" class="btn btn-secondary">Vazgeç</a>
</form>

```

Kullanıcı butona basınca POST tetiklenir; controller'daki DeleteConfirmed action'ı devreye girer.

5.1.15 Layout — Toastr Entegrasyonu

```

@* Views/Shared/_Layout.cshtml (özet) *@
<!DOCTYPE html>
<html lang="tr">
<head>
  <meta charset="utf-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>@ViewData["Title"] - BulkyBook</title>

  <!-- Bootswatch Litera teması -->
  <link rel="stylesheet"
        href="https://bootswatch.com/5/litera/bootstrap.min.css" />
  <link rel="stylesheet"
        href="https://cdn.jsdelivr.net/npm/bootstrap-icons/font/bootstrap-icons.css" />
  <link rel="stylesheet"
        href="https://cdn.jsdelivr.net/npm/toastr.js/dist/toastr.min.css" />
  <link rel="stylesheet" href="~/css/site.css" asp-append-version="true" />
</head>
<body>
  <nav class="navbar navbar-expand-sm navbar-light bg-white border-bottom">
    <div class="container-fluid">
      <a class="navbar-brand" asp-controller="Home" asp-action="Index">BulkyBook</a>
      <ul class="navbar-nav">
        <li class="nav-item">
          <a class="nav-link" asp-controller="Home" asp-action="Index">Ana Sayfa</a>
        </li>
        <li class="nav-item">
          <a class="nav-link" asp-controller="Kategori" asp-action="Index">Kategoriler</a>
        </li>
      </ul>
    </div>
  </nav>

  <div class="container mt-4">
    <main role="main" class="pb-3">
      @RenderBody()
    </main>
  </div>
</body>
</html>

```

```

</div>

<footer class="border-top footer text-muted mt-5">
  <div class="container">
    &copy; @DateTime.Now.Year - BulkyBook
  </div>
</footer>

<script src="~/lib/jquery/dist/jquery.min.js"></script>
<script src="~/lib/bootstrap/dist/js/bootstrap.bundle.min.js"></script>
<script src="https://cdnjs.cloudflare.com/ajax/libs/toastr.js/2.1.4/toastr.min.js"></script>
<script src="~/js/site.js" asp-append-version="true"></script>

<partial name="_Notifications" />

@await RenderSectionAsync("Scripts", required: false)
</body>
</html>

```

Bootswatch Litera Bootstrap 5 üzerine kurulu ücretsiz bir tema; CDN'den doğrudan yüklenir. Bootstrap Icons ikonlar için, Toastr bildirimler için.

5.1.16 _Notifications Partial

```

@* Views/Shared/_Notifications.cshtml *@
<script>
  $(function () {
    @if (TempData["Success"] != null)
    {
      <text>toastr.success("@TempData["Success"]");</text>
    }
    @if (TempData["Error"] != null)
    {
      <text>toastr.error("@TempData["Error"]");</text>
    }
  });
</script>

```

Layout'ta <partial name="_Notifications" /> çağrısı ile aktive edilir. Controller TempData["Success"] = "... yazdığında, bir sonraki sayfa yüklemesinde bildirim otomatik gösterilir.

5.1.17 Beklenen Sonuçlar Tablosu

Senaryo	Girdi	Beklenen Sonuç
Yeni kategori ekleme (geçerli)	Ad="Roman", Sıra=1	Kayıt eklenir, Index'e yönlendirme, Toastr success "Kategori başarıyla eklendi."
Yeni kategori ekleme (boş ad)	Ad="", Sıra=1	Create'e döner, "Kategori adı zorunlu." hatası
Yeni kategori ekleme (sıra 0)	Ad="Deneme", Sıra=0	Create'e döner, "1-100 arasında olmalı." hatası
Yeni kategori ekleme (ad=sıra)	Ad="5", Sıra=5	Create'e döner, "Kategori adı ile görüntüleme sırası aynı olamaz." hatası
Var olan kategoriyi düzenleme	Id=1, Ad="Roman-2", Sıra=1	Kayıt güncellenir, Index'e yönlendirme, Toastr success "Kategori güncellendi."
Olmayan Id ile Edit çağrısı	/Kategori/Edit/999	NotFound() (404)
Kategori silme (onay sayfası)	GET /Kategori/Delete/1	Delete.cshtml onay ekranı

Senaryo	Girdi	Beklenen Sonuç
Kategori silme onayı	POST /Kategori/Delete/1	Silinir, Index'e yönlendirme, Toastr success "Kategori silindi."
Anti-forgery token'sız POST	Elle CURL POST	400 Bad Request

5.1.18 Tasarım Notları

Neden BaseEntity yok?

Proje tek entity içerir; ortak alanları çıkarma ihtiyacı yok. §5.2 HastaneMVC birden fazla entity içerdiği için BaseEntity uygulaması orada başlar.

Neden soft delete yok?

Kategori bir referans veri türüdür — silme sık değildir ve etkilenen kaynaklara IsDeleted bayrağıyla erişme ihtiyacı domain'de yoktur. Bir kategori silindiğinde ilgili kitaplar da hesap verilir; ama bu proje kapsamı dışında.

Neden Service Layer yok?

Tek modülde Controller → DbContext direk erişim yeterlidir; ara servis katmanı ihtiyacı yaratmayı zorlaştırır. §5.2'de servisler devreye girer çünkü orada iş kuralları daha zengin (Randevu çakışması kontrolü, tahlil formu doldurma gibi).

Bootstrap Icons kullanımı

`<i class="bi bi-plus-circle"></i>` gibi ikon eklemeleri, buton içeriğine görsel zenginlik katar. Katalog A'daki "Bulky/Bootstrap Icons kurulumu" konusuna denk düşer (bkz. Ek C).

Modern global anti-forgery filter alternatifi

Bu proje klasik [ValidateAntiForgeryToken] her POST action'da yazar. Bir sonraki iterasyonda Program.cs'e `options.Filters.Add(new AutoValidateAntiforgeryTokenAttribute())` eklenirse attribute'lar kaldırılabilir. Master defterin genel önerisi budur (bkz. §4.18.3.b).

5.1.19 İlişkili Konular

Bu projede tek yerde birden çok bölüm iç içe geçer:

- Model + Data Annotations → §3.7
- Migration → §3.9
- Controller + PRG → §4.7
- Tag Helper + form → §4.11
- Validation → §4.15
- Toastr + TempData → §4.22
- Anti-forgery → §4.18

Bu projede karşılaştığın soruların cevabı çoğu zaman ilgili teori bölümündedir.

5.2 HastaneMVC — Kapsamlı Randevu Yönetimi

5.2.1 Proje Tanıtımı

HastaneMVC, bir hastane işletim sisteminin randevu ve tıbbi tetkik yönetim modüllerini içeren, çok katmanlı bir MVC projesidir. Master defterin en kapsamlı proje bölümüdür; §4.20 base sınıflar deseni, §4.21 soft delete, §3.11 LINQ, §4.19 AJAX gibi tüm ileri konuları tek bir uygulamada birleştirir.

Kapsam:

- **Doktor:** Uzmanlık ve bölüm bilgileriyle CRUD.
- **Bölüm:** Doktorların bağlı olduğu tıbbi bölümler.
- **Randevu:** Doktor-hasta buluşması; bölüm bazlı doktor seçimi (AJAX cascading, §5.3'te de ele alınır).
- **Tahlil:** Bir hastaya yapılan tıbbi test kayıtları.

Yaklaşım: **BaseEntity + generic BaseService** üzerine kurulu, soft delete standart, her modül için kendi servisi ve controller'ı olan çok modüllü bir yapı. Katmanlar arası bağımlılık interface'ler üzerinden kurulur — DI container tüm iskelet omurgasıdır.

5.2.2 Ortam

Bileşen	Değer
.NET SDK	8.0.x
Proje Şablonu	ASP.NET Core Web App (Model-View-Controller)
Veritabanı	SQL Server (localdb veya SQL Server 2019+)
EF Core	8.0.x
Bootstrap	5 (yerleşik)
Bildirim	Toastr 2.1.4
Ek Paketler	Microsoft.EntityFrameworkCore.SqlServer, Microsoft.EntityFrameworkCore.Tools, Microsoft.EntityFrameworkCore.Design, Microsoft.AspNetCore.Mvc.Razor.RuntimeCompilation

5.2.3 Proje Yapısı

```
HastaneMVC/
├── Controllers/
│   ├── HomeController.cs
│   ├── DoktorController.cs
│   ├── BolumController.cs
│   ├── RandevuController.cs
│   └── TahlilController.cs
├── Data/
│   └── ApplicationDbContext.cs
├── Migrations/
├── Models/
│   ├── Entities/
│   │   ├── BaseEntity.cs
│   │   ├── Bolum.cs
│   │   ├── Doktor.cs
│   │   ├── Randevu.cs
│   │   └── Tahlil.cs
│   └── ViewModels/
│       ├── DoktorCreateVM.cs
│       ├── RandevuCreateVM.cs
│       └── TahlilCreateVM.cs
├── Services/
│   ├── IBaseService.cs
│   ├── BaseService.cs
│   ├── IDoktorService.cs
│   ├── DoktorService.cs
│   ├── IBolumService.cs
│   ├── BolumService.cs
│   ├── IRandevuService.cs
│   ├── RandevuService.cs
│   ├── ITahlilService.cs
│   └── TahlilService.cs
├── Views/
│   ├── Home/
│   ├── Doktor/
│   ├── Bolum/
│   ├── Randevu/
│   └── Tahlil/
```

```
|   └─ Shared/  
|   └─ wwwroot/  
|   └─ Program.cs  
|   └─ appsettings.json
```

Katmanlı yapı belirgindir: Models/Entities/ (DB'ye eşlenen), Models/ViewModels/ (view'a özel), Services/ (iş katmanı), Controllers/ (sunum katmanı).

5.2.4 Veri Modeli ve İlişki Diyagramı

- Bolum → Doktor: bir bölümün birçok doktoru olabilir.
- Doktor → Randevu: bir doktorun birçok randevusu olabilir.
- Doktor → Tahlil: bir doktor birçok tahlil isteyebilir.
- Tüm entity'ler BaseEntity'den kalıtır.

5.2.5 BaseEntity

```
// Models/Entities/BaseEntity.cs  
public abstract class BaseEntity  
{  
    public int Id { get; set; }  
    public DateTime OlusturmaTarihi { get; set; } = DateTime.Now;  
    public DateTime? GuncellemeTarihi { get; set; }  
    public bool IsDeleted { get; set; } = false;  
}
```

abstract — doğrudan new BaseEntity() üretilemez. IsDeleted varsayılan false; ekleme sırasında kayıt aktif.

5.2.6 Entity Sınıfları

```
// Models/Entities/Bolum.cs  
using System.ComponentModel.DataAnnotations;  
  
public class Bolum : BaseEntity  
{  
    [Required, MaxLength(100)]  
    [Display(Name = "Bölüm Adı")]  
    public string Ad { get; set; }  
  
    [MaxLength(500)]  
    [Display(Name = "Açıklama")]  
    public string Aciklama { get; set; }  
  
    public ICollection<Doktor> Doktorlar { get; set; } = new List<Doktor>();  
}  
  
// Models/Entities/Doktor.cs  
public class Doktor : BaseEntity  
{  
    [Required, MaxLength(100)]  
    [Display(Name = "Ad Soyad")]  
    public string Ad { get; set; }  
  
    [Required, MaxLength(50)]  
    [Display(Name = "Uzmanlık")]  
    public string Uzmanlik { get; set; }  
  
    [Display(Name = "Bölüm")]  
    public int BolumId { get; set; }
```

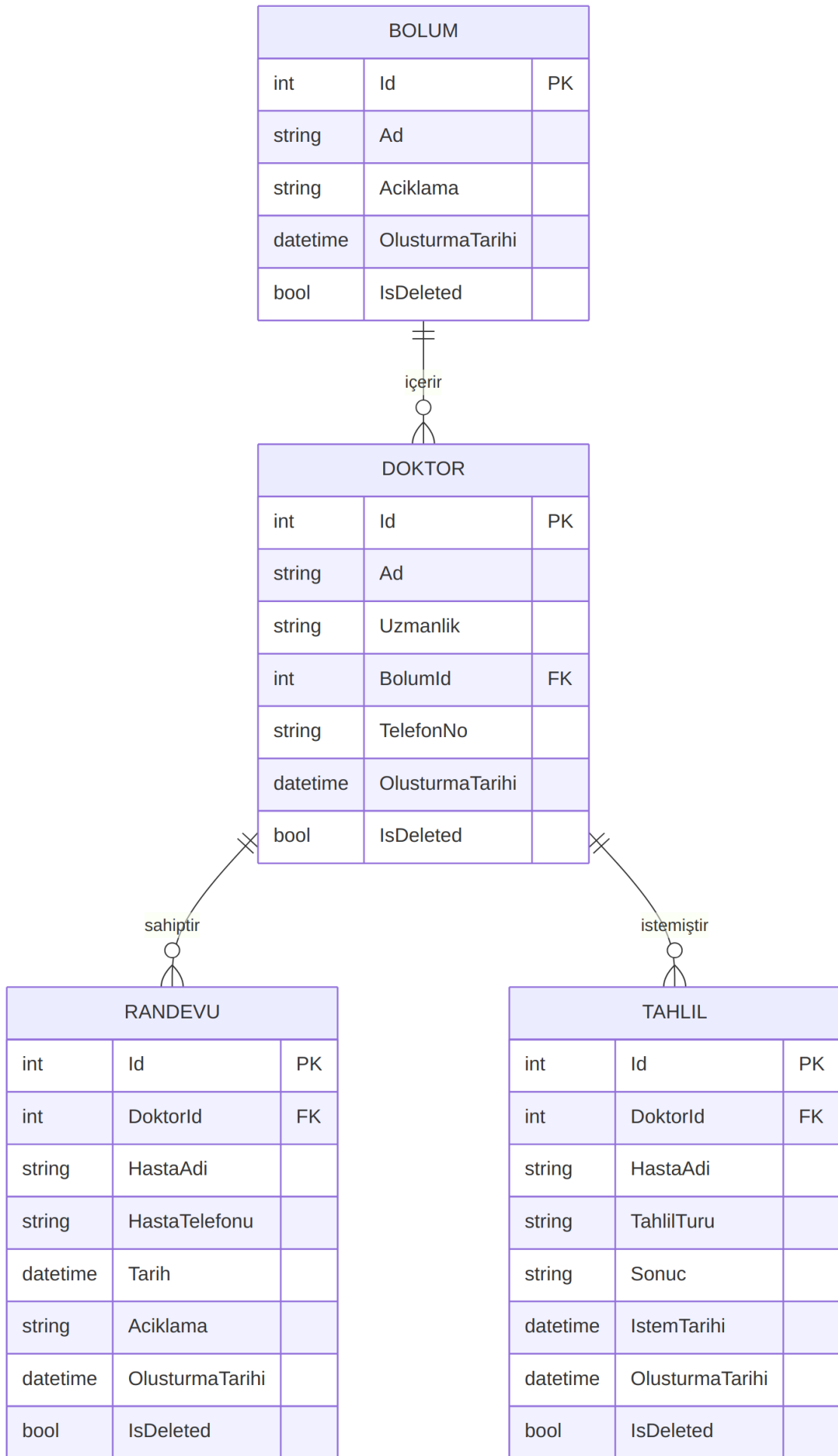


Figure 15: Diyagram 15

```

public Bolum Bolum { get; set; }

[Phone, MaxLength(20)]
[Display(Name = "Telefon")]
public string TelefonNo { get; set; }

public ICollection<Randevu> Randevular { get; set; } = new List<Randevu>();
public ICollection<Tahlil> Tahliller { get; set; } = new List<Tahlil>();
}

// Models/Entities/Randevu.cs
public class Randevu : BaseEntity
{
    [Display(Name = "Doktor")]
    public int DoktorId { get; set; }
    public Doktor Doktor { get; set; }

    [Required, MaxLength(100)]
    [Display(Name = "Hasta Adı")]
    public string HastaAdi { get; set; }

    [Phone, MaxLength(20)]
    [Display(Name = "Hasta Telefonu")]
    public string HastaTelefonu { get; set; }

    [Required]
    [Display(Name = "Randevu Tarihi")]
    public DateTime Tarih { get; set; }

    [MaxLength(500)]
    [Display(Name = "Açıklama")]
    public string Aciklama { get; set; }
}

// Models/Entities/Tahlil.cs
public class Tahlil : BaseEntity
{
    [Display(Name = "Doktor")]
    public int DoktorId { get; set; }
    public Doktor Doktor { get; set; }

    [Required, MaxLength(100)]
    [Display(Name = "Hasta Adı")]
    public string HastaAdi { get; set; }

    [Required, MaxLength(100)]
    [Display(Name = "Tahlil Türü")]
    public string TahlilTuru { get; set; }

    [MaxLength(2000)]
    [Display(Name = "Sonuç")]
    public string Sonuc { get; set; }

    [Required]
    [Display(Name = "İstem Tarihi")]
    public DateTime IstemTarihi { get; set; } = DateTime.Now;
}

```

Her entity BaseEntity'den türer; Id, OluşturmaTarihi, GuncellemeTarihi, IsDeleted otomatik gelir. [Display] attribute'ları form etiketleri ve tablo başlıkları için etiket sağlar.

5.2.7 DbContext

```
// Data/AppDbContext.cs
using Microsoft.EntityFrameworkCore;
using System.Linq.Expressions;

public class AppDbContext : DbContext
{
    public AppDbContext(DbContextOptions<AppDbContext> options) : base(options) { }

    public DbSet<Bolum> Bolumler { get; set; }
    public DbSet<Doktor> Doktorlar { get; set; }
    public DbSet<Randevu> Randevular { get; set; }
    public DbSet<Tahlil> Tahliller { get; set; }

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);

        // Cascade davranışı: doktor silinemez randevusu varsa
        modelBuilder.Entity<Randevu>()
            .HasOne(r => r.Doktor)
            .WithMany(d => d.Randevular)
            .HasForeignKey(r => r.DoktorId)
            .OnDelete(DeleteBehavior.Restrict);

        modelBuilder.Entity<Tahlil>()
            .HasOne(t => t.Doktor)
            .WithMany(d => d.Tahliller)
            .HasForeignKey(t => t.DoktorId)
            .OnDelete(DeleteBehavior.Restrict);

        modelBuilder.Entity<Doktor>()
            .HasOne(d => d.Bolum)
            .WithMany(b => b.Doktorlar)
            .HasForeignKey(d => d.BolumId)
            .OnDelete(DeleteBehavior.Restrict);

        // Tüm BaseEntity türevlerine IsDeleted global query filter
        foreach (var entityType in modelBuilder.Model.GetEntityTypes())
        {
            if (typeof(BaseEntity).IsAssignableFrom(entityType.ClrType))
            {
                var param = Expression.Parameter(entityType.ClrType, "e");
                var prop = Expression.Property(param, nameof(BaseEntity.IsDeleted));
                var kosul = Expression.Equal(prop, Expression.Constant(false));
                var lambda = Expression.Lambda(kosul, param);

                modelBuilder.Entity(entityType.ClrType).HasQueryFilter(lambda);
            }
        }

        // OlusturmaTarihi / GuncellemeTarihi otomatik doldurma
        public override int SaveChanges()
        {
            OtomatikTarihleriDoldur();
            return base.SaveChanges();
        }

        public override Task<int> SaveChangesAsync(CancellationToken ct = default)
```

```

    {
        OtomatikTarihleriDoldur();
        return base.SaveChangesAsync(ct);
    }

    private void OtomatikTarihleriDoldur()
    {
        var entries = ChangeTracker.Entries<BaseEntity>();
        foreach (var e in entries)
        {
            if (e.State == EntityState.Added)
                e.Entity.OlusturmaTarihi = DateTime.Now;
            else if (e.State == EntityState.Modified)
                e.Entity.GuncellemeTarihi = DateTime.Now;
        }
    }
}

```

Bu DbContext üç ana özelliği bir arada uygular:

1. **Cascade davranışı özelleştirmesi** — silme davranışı Restrict, çünkü doktor silinirse randevular kaybolmamalı.
2. **Global soft-delete query filter** — reflection ile tüm BaseEntity'lere `e => !e.IsDeleted` otomatik eklenir (§4.20.3).
3. **SaveChanges override** — ekleme/güncelleme zaman damgaları otomatik doldurulur (§4.20.4).

5.2.8 Servis Katmanı

```

// Services/IBaseService.cs
public interface IBaseService<T> where T : BaseEntity
{
    Task<IEnumerable<T>> TumunuGetirAsync();
    Task<T> GetirAsync(int id);
    Task EkleAsync(T entity);
    Task GuncelleAsync(T entity);
    Task SoftDeleteAsync(int id);
}

// Services/BaseService.cs
public class BaseService<T> : IBaseService<T> where T : BaseEntity
{
    protected readonly AppDbContext _context;
    protected readonly DbSet<T> _set;

    public BaseService(AppDbContext context)
    {
        _context = context;
        _set = context.Set<T>();
    }

    public virtual async Task<IEnumerable<T>> TumunuGetirAsync()
        => await _set.AsNoTracking().ToListAsync();

    public virtual async Task<T> GetirAsync(int id)
        => await _set.FindAsync(id);

    public virtual async Task EkleAsync(T entity)
    {
        _set.Add(entity);
        await _context.SaveChangesAsync();
    }
}

```

```

    }

    public virtual async Task GuncelleAsync(T entity)
    {
        _set.Update(entity);
        await _context.SaveChangesAsync();
    }

    public virtual async Task SoftDeleteAsync(int id)
    {
        var entity = await _set.FindAsync(id);
        if (entity != null)
        {
            entity.IsDeleted = true;
            await _context.SaveChangesAsync();
        }
    }
}

```

Her modül için ayrı servis:

```

// Services/IDoktorService.cs
public interface IDoktorService : IBaseService<Doktor>
{
    Task<IEnumerable<Doktor>> BolumGoreGetirAsync(int bolumId);
    Task<IEnumerable<Doktor>> TumunuIleBolumGetirAsync();
}

// Services/DoktorService.cs
public class DoktorService : BaseService<Doktor>, IDoktorService
{
    public DoktorService(AppDbContext context) : base(context) { }

    public async Task<IEnumerable<Doktor>> BolumGoreGetirAsync(int bolumId)
    => await _set.Where(d => d.BolumId == bolumId)
        .AsNoTracking()
        .ToListAsync();

    public async Task<IEnumerable<Doktor>> TumunuIleBolumGetirAsync()
    => await _set.Include(d => d.Bolum)
        .AsNoTracking()
        .ToListAsync();
}

// Services/IRandevuService.cs
public interface IRandevuService : IBaseService<Randevu>
{
    Task<IEnumerable<Randevu>> DoktoraGoreGetirAsync(int doktorId);
    Task<IEnumerable<Randevu>> TariheGoreGetirAsync(DateTime tarih);
    Task<bool> CakismaKontrolAsync(int doktorId, DateTime tarih);
}

// Services/RandevuService.cs
public class RandevuService : BaseService<Randevu>, IRandevuService
{
    public RandevuService(AppDbContext context) : base(context) { }

    public async Task<IEnumerable<Randevu>> DoktoraGoreGetirAsync(int doktorId)
    => await _set.Include(r => r.Doktor)
        .Where(r => r.DoktorId == doktorId)
        .OrderBy(r => r.Tarih)
        .AsNoTracking()

```

```

        .ToListAsync();

    public async Task<IEnumerable<Randevu>> TariheGoreGetirAsync(DateTime tarih)
    => await _set.Include(r => r.Doktor)
        .Where(r => r.Tarih.Date == tarih.Date)
        .OrderBy(r => r.Tarih)
        .AsNoTracking()
        .ToListAsync();

    public async Task<bool> CakismaKontrolAsync(int doktorId, DateTime tarih)
    => await _set.AnyAsync(r => r.DoktorId == doktorId && r.Tarih == tarih);
}

```

Genel servisler benzer şekilde BolumService, TahlilService olarak yazılır.

5.2.9 Program.cs — Servis Kayıtları

```

using Microsoft.EntityFrameworkCore;
using HastaneMVC.Data;
using HastaneMVC.Services;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllersWithViews(options =>
{
    // Modern anti-forgery: tüm POST'lara otomatik doğrulama
    options.Filters.Add(new Microsoft.AspNetCore.Mvc.AutoValidateAntiforgeryTokenAttribute());
});

builder.Services.AddDbContext<AppDbContext>(o =>
    o.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));

// Generic + özel servisler
builder.Services.AddScoped(typeof(IBaseService<>), typeof(BaseService<>));
builder.Services.AddScoped<IBolumService, BolumService>();
builder.Services.AddScoped<IDoktorService, DoktorService>();
builder.Services.AddScoped<IRandevuService, RandevuService>();
builder.Services.AddScoped<ITahlilService, TahlilService>();

var app = builder.Build();

if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Home/Error");
    app.UseHsts();
}

app.UseHttpsRedirection();
app.UseStaticFiles();
app.UseRouting();
app.UseAuthorization();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");

app.Run();

```

Master defterin çelişki çözümü: Anti-forgery için modern global filter (AutoValidateAntiforgeryTokenAttribute) kullanılır — her POST action'a otomatik uygulanır. Controller'larda [ValidateAntiForgeryToken] yazmaya gerek yoktur (bkz. §4.18.3.b).

5.2.10 RandevuController — Cascading Dropdown

Randevu formu **bölüm seçilince o bölümdeki doktorların dropdown'ı dolan** AJAX'lı yapıya sahiptir.

```
// Controllers/RandevuController.cs
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.Rendering;
using HastaneMVC.Models.Entities;
using HastaneMVC.Models.ViewModels;
using HastaneMVC.Services;

public class RandevuController : BaseController
{
    private readonly IRandevuService _randevu;
    private readonly IDoktorService _doktor;
    private readonly IBolumService _bolum;

    public RandevuController(
        IRandevuService randevu,
        IDoktorService doktor,
        IBolumService bolum)
    {
        _randevu = randevu;
        _doktor = doktor;
        _bolum = bolum;
    }

    // GET: /Randevu
    public async Task<IActionResult> Index()
    {
        var liste = (await _randevu.TumunuGetirAsync())
            .OrderByDescending(r => r.Tarih);
        return View(liste);
    }

    // GET: /Randevu/Create
    public async Task<IActionResult> Create()
    {
        var vm = new RandevuCreateVM
        {
            Randevu = new Randevu { Tarih = DateTime.Now.AddDays(1) },
            Bolumler = await BolumSelectListAsync()
        };
        return View(vm);
    }

    // POST: /Randevu/Create
    [HttpPost]
    public async Task<IActionResult> Create(RandevuCreateVM vm)
    {
        // Çakışma kontrolü
        if (await _randevu.CakismaKontrolAsync(vm.Randevu.DoktorId, vm.Randevu.Tarih))
        {
            ModelState.AddModelError(
                nameof(vm.Randevu.Tarih),
                "Bu doktorun aynı saatte başka randevusu var.");
        }

        if (!ModelState.IsValid)
        {
            vm.Bolumler = await BolumSelectListAsync();
        }
    }
}
```

```

        return View(vm);
    }

    await _randevu.EkleAsync(vm.Randevu);
    BasariMesaji("Randevu eklendi.");
    return IndexeYonlendir();
}

// AJAX endpoint – bölüme göre doktorlar
[HttpGet]
public async Task<IActionResult> DoktorlariGetir(int bolumId)
{
    var doktorlar = await _doktor.BolumeGoreGetirAsync(bolumId);
    var sonuc = doktorlar.Select(d => new { id = d.Id, ad = d.Ad });
    return Json(sonuc);
}

// GET: /Randevu/Detay/5
public async Task<IActionResult> Detay(int id)
{
    var r = await _randevu.GetirAsync(id);
    if (r == null) return NotFound();
    return View(r);
}

// POST: /Randevu/Sil/5
[HttpPost, ActionName("Sil")]
public async Task<IActionResult> SilOnayli(int id)
{
    await _randevu.SoftDeleteAsync(id);
    BasariMesaji("Randevu silindi.");
    return IndexeYonlendir();
}

// Yardımcı
private async Task<IEnumerable<SelectListItem>> BolumSelectListAsync()
{
    var bolumler = await _bolum.TumunuGetirAsync();
    return bolumler.Select(b => new SelectListItem
    {
        Value = b.Id.ToString(),
        Text = b.Ad
    });
}
}

```

Constructor injection üç servisi birden alır — üç interface tek noktada birleşir. **DoktorlariGetir** action'ı cascading dropdown için AJAX endpoint'idir; JSON döner.

5.2.11 RandevuCreateVM

```

// Models/ViewModels/RandevuCreateVM.cs
using Microsoft.AspNetCore.Mvc.Rendering;

public class RandevuCreateVM
{
    public Randevu Randevu { get; set; } = new();
    public IEnumerable<SelectListItem> Bolumler { get; set; } = new List<SelectListItem>();
}

```

ViewModel iki alan taşır: form'un ana verisi (Randevu) + dropdown listeleri (Bolumler). Doktor

listesi client-side'da AJAX ile geleceği için burada yer almaz.

5.2.12 Randevu Create View

```
@* Views/Randevu/Create.cshtml *@
@model RandevuCreateVM

@{
    ViewData["Title"] = "Yeni Randevu";
}

<h1>Yeni Randevu</h1>

<form asp-action="Create" method="post">
    <div asp-validation-summary="ModelOnly" class="text-danger mb-3"></div>

    <div class="mb-3">
        <label class="form-label">Bölüm</label>
        <select id="bolumSelect" asp-items="Model.Bolumler" class="form-select">
            <option value="">-- Bölüm seçin --</option>
        </select>
    </div>

    <div class="mb-3">
        <label asp-for="Randevu.DoktorId" class="form-label"></label>
        <select id="doktorSelect" asp-for="Randevu.DoktorId" class="form-select">
            <option value="">-- Önce bölüm seçin --</option>
        </select>
        <span asp-validation-for="Randevu.DoktorId" class="text-danger"></span>
    </div>

    <div class="mb-3">
        <label asp-for="Randevu.HastaAdi" class="form-label"></label>
        <input asp-for="Randevu.HastaAdi" class="form-control" />
        <span asp-validation-for="Randevu.HastaAdi" class="text-danger"></span>
    </div>

    <div class="mb-3">
        <label asp-for="Randevu.HastaTelefonu" class="form-label"></label>
        <input asp-for="Randevu.HastaTelefonu" class="form-control" />
        <span asp-validation-for="Randevu.HastaTelefonu" class="text-danger"></span>
    </div>

    <div class="mb-3">
        <label asp-for="Randevu.Tarih" class="form-label"></label>
        <input asp-for="Randevu.Tarih" type="datetime-local" class="form-control" />
        <span asp-validation-for="Randevu.Tarih" class="text-danger"></span>
    </div>

    <div class="mb-3">
        <label asp-for="Randevu.Aciklama" class="form-label"></label>
        <textarea asp-for="Randevu.Aciklama" class="form-control" rows="3"></textarea>
    </div>

    <button type="submit" class="btn btn-primary">Randevu Al</button>
    <a asp-action="Index" class="btn btn-secondary">İptal</a>
</form>

@section Scripts {
    <partial name="_ValidationScriptsPartial" />
    <script>
```

```

$(function () {
    $("#bolumSelect").change(function () {
        var bolumId = $(this).val();
        var $doktor = $("#doktorSelect");
        $doktor.empty();

        if (!bolumId) {
            $doktor.append('<option value="">-- Önce bölüm seçin --</option>');
            return;
        }

        $doktor.append('<option value="">-- Yükleniyor... --</option>');

        $.getJSON("/Randevu/DoktorlariGetir", { bolumId: bolumId }, function (liste) {
            $doktor.empty();
            $doktor.append('<option value="">-- Doktor seçin --</option>');
            $.each(liste, function (i, d) {
                $doktor.append('<option>', { value: d.id, text: d.ad });
            });
        });
    });
});
</script>
}

```

Kritik parça: \$("#bolumSelect").change(...) handler'ı, bölüm seçimi değişince AJAX ile doktor listesini yükler. Sunucu tarafında DoktorlariGetir action'ı JSON döner (§4.19.6 pattern'i).

5.2.13 Randevu Ekleme Akış Diyagramı

5.2.14 BaseController

```

// Controllers/BaseController.cs
using Microsoft.AspNetCore.Mvc;

public abstract class BaseController : Controller
{
    protected void BasariMesaji(string mesaj) => TempData["Success"] = mesaj;
    protected void HataMesaji(string mesaj) => TempData["Error"] = mesaj;
    protected void UyariMesaji(string mesaj) => TempData["Warning"] = mesaj;

    protected IActionResult IndexeYonlendir() => RedirectToAction("Index");
    protected IActionResult IndexeYonlendir(string ctrl) => RedirectToAction("Index", ctrl);
}

```

Tüm controller'lar bunu kalıtır; BasariMesaji("...") gibi kısayollar kod tekrarını azaltır.

5.2.15 Doktor Modülü — Standart CRUD

```

// Controllers/DoktorController.cs
public class DoktorController : BaseController
{
    private readonly IDoktorService _doktor;
    private readonly IBolumService _bolum;

    public DoktorController(IDoktorService doktor, IBolumService bolum)
    {
        _doktor = doktor;
        _bolum = bolum;
    }
}

```

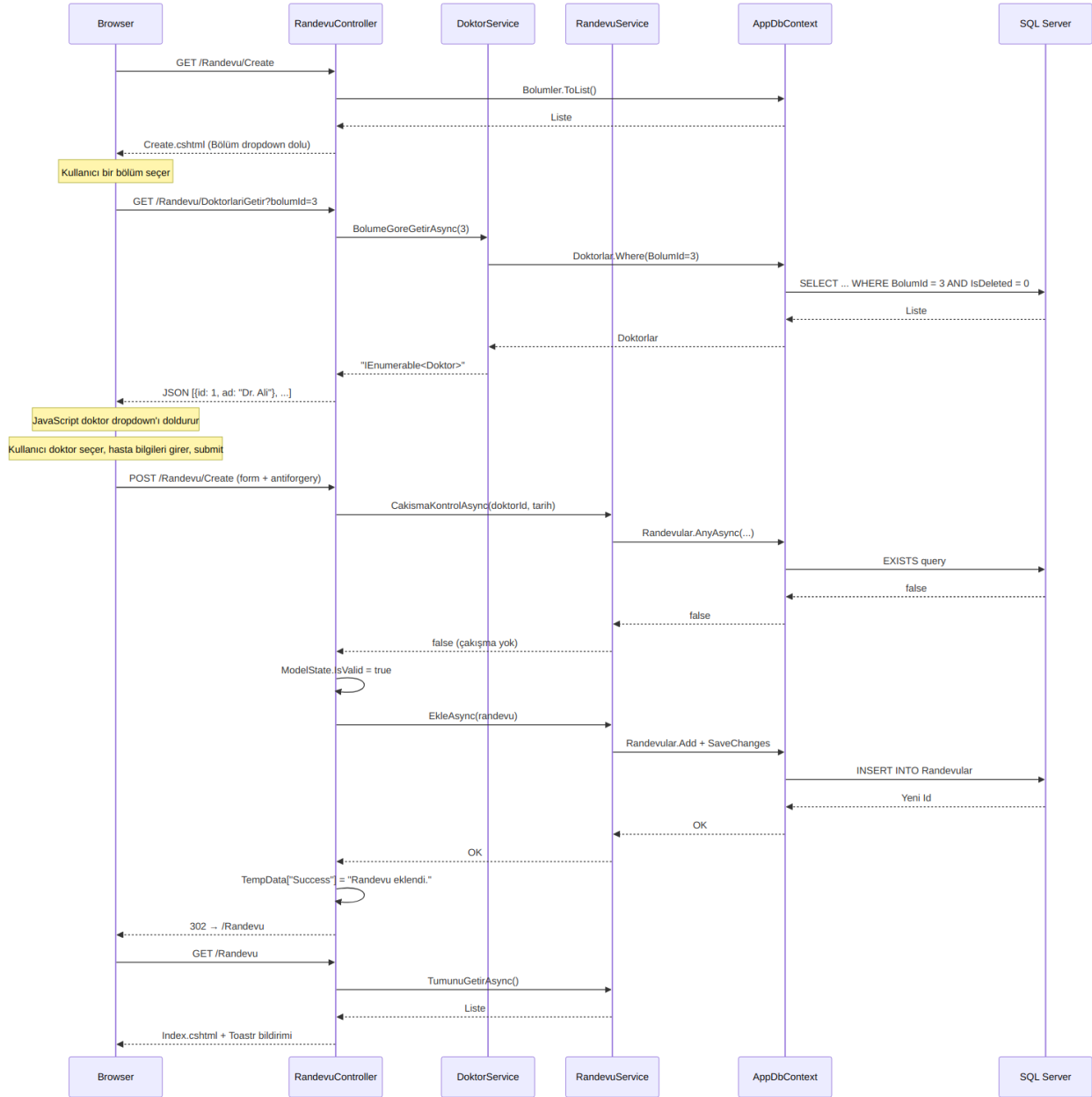


Figure 16: Diyagram 16

```

public async Task<IActionResult> Index()
{
    var liste = await _doktor.TumunuIleBolumGetirAsync();
    return View(liste);
}

public async Task<IActionResult> Create()
{
    var vm = new DoktorCreateVM
    {
        Doktor = new Doktor(),
        Bolumler = await BolumSelectListAsync()
    };
    return View(vm);
}

[HttpPost]
public async Task<IActionResult> Create(DoktorCreateVM vm)
{
    if (!ModelState.IsValid)
    {
        vm.Bolumler = await BolumSelectListAsync();
        return View(vm);
    }

    await _doktor.EkleAsync(vm.Doktor);
    BasariMesaji("Doktor eklendi.");
    return IndexeYonlendir();
}

public async Task<IActionResult> Edit(int id)
{
    var doktor = await _doktor.GetirAsync(id);
    if (doktor == null) return NotFound();

    var vm = new DoktorCreateVM
    {
        Doktor = doktor,
        Bolumler = await BolumSelectListAsync()
    };
    return View(vm);
}

[HttpPost]
public async Task<IActionResult> Edit(DoktorCreateVM vm)
{
    if (!ModelState.IsValid)
    {
        vm.Bolumler = await BolumSelectListAsync();
        return View(vm);
    }

    await _doktor.GuncelleAsync(vm.Doktor);
    BasariMesaji("Doktor güncellendi.");
    return IndexeYonlendir();
}

[HttpPost]
public async Task<IActionResult> Sil(int id)
{
    await _doktor.SoftDeleteAsync(id);
}

```

```

        BasariMesaji("Doktor silindi.");
        return IndexeYonlendir();
    }

    private async Task<IEnumerable<SelectListItem>> BolumSelectListAsync()
    {
        var bolumler = await _bolum.TumunuGetirAsync();
        return bolumler.Select(b => new SelectListItem
        {
            Value = b.Id.ToString(),
            Text = b.Ad
        });
    }
}

```

Standart CRUD, tek fark: TumunuIleBolumGetirAsync() liste sayfasında bölüm adını da çeker (Include(d => d.Bolum) — §3.12).

5.2.16 Tahlil Modülü — Doktor Bazlı Liste

Bir doktorun sayfasından “tahlillerini gör” akışı:

```

// Controllers/TahlilController.cs
public class TahlilController : BaseController
{
    private readonly ITahlilService _tahlil;
    private readonly IDoktorService _doktor;

    public TahlilController(ITahlilService tahlil, IDoktorService doktor)
    {
        _tahlil = tahlil;
        _doktor = doktor;
    }

    // GET: /Tahlil?doktorId=5
    public async Task<IActionResult> Index(int? doktorId = null)
    {
        IEnumerable<Tahlil> liste;
        if (doktorId.HasValue)
            liste = await _tahlil.DoktoraGoreGetirAsync(doktorId.Value);
        else
            liste = await _tahlil.TumunuGetirAsync();

        ViewBag.DoktorId = doktorId;
        return View(liste);
    }

    // GET: /Tahlil/Create?doktorId=5
    public async Task<IActionResult> Create(int? doktorId = null)
    {
        var vm = new TahlilCreateVM
        {
            Tahlil = new Tahlil { DoktorId = doktorId ?? 0, IstemTarihi = DateTime.Now },
            Doktorlar = await DoktorSelectListAsync()
        };
        return View(vm);
    }

    [HttpPost]
    public async Task<IActionResult> Create(TahlilCreateVM vm)
    {
        if (!ModelState.IsValid)

```

```

    {
        vm.Doktorlar = await DoktorSelectListAsync();
        return View(vm);
    }

    await _tahlil.EkleAsync(vm.Tahlil);
    BasariMesaji("Tahlil kaydedildi.");
    return RedirectToAction(nameof(Index), new { doktorId = vm.Tahlil.DoktorId });
}

private async Task<IEnumerable<SelectListItem>> DoktorSelectListAsync()
{
    var doktorlar = await _doktor.TumunuIleBolumGetirAsync();
    return doktorlar.Select(d => new SelectListItem
    {
        Value = d.Id.ToString(),
        Text = $"{d.Ad} - {d.Bolum?.Ad}"
    });
}
}

```

ITahlilService içindeki DoktoraGoreGetirAsync(doktorId) bir doktora ait tüm tahlilleri döner. Controller filtreleme yeteneğini opsiyonel URL parametresiyle sunar: /Tahlil tüm tahlilleri, /Tahlil?doktorId=5 sadece belirli doktorun tahlillerini gösterir.

5.2.17 Beklenen Sonuçlar Tablosu — Randevu Modülü

Senaryo	Girdi	Beklenen Sonuç
Yeni randevu ekleme (geçerli)	Bölüm=Dahiliye, Doktor=Dr. Ali, Hasta=Mehmet, Tarih=yarın 10:00	Randevu eklenir, Toastr success, Index'te en üstte
Randevu ekleme (bölüm seçilmedi)	Bölüm=boş	Doktor dropdown "Önce bölüm seçin", form submit engellenmez ama DoktorId=0 → validasyon hatası
Bölüm seçildi, doktor dropdown otomatik doldu	Bölüm=Cildiye	AJAX çağırısı, #doktorSelect doldu, iki <option> görünür (Dr. X, Dr. Y)
Aynı saat çakışması	Var olan bir randevunun tarihiyle aynı doktor+tarih	ModelState hata: "Bu doktorun aynı saatte başka randevusu var."
Randevu silme	POST /Randevu/Sil/5	IsDeleted=true, Index'te görünmez, Toastr success Görünmez (query filter aktif)
Silinmiş randevuyu Index'te gör	GET /Randevu	
Anti-forgery token'sız POST Doktor silinemez randevusu varsa	Elle curl POST POST /Doktor/Sil/1 (Dr. Ali'nin randevuları var)	400 Bad Request (global filter) Soft delete başarılı; IsDeleted=true — RESTRICT etkilenmez çünkü aslında DELETE değil UPDATE

5.2.18 Tasarım Notları

Neden generic BaseService?

5 entity + 5 controller = 5 servis. Her serviste TumunuGetir, EkleAsync, GuncelleAsync gibi metotları tekrar yazmak DRY ihlali. Generic base bu tekrarı %90 azaltır; entity-özel iş kuralları (çakışma kontrolü, bölüme göre filtre) alt servislerde ek metotlar olarak yazılır.

Neden Restrict + soft delete birlikte?

Hard delete davranışı Restrict (§3.2.5) — DB seviyesinde parent silinemez child varsa. Ama soft

delete Remove çağırılmaz, IsDeleted = true UPDATE eder. Yani Restrict tetiklenmez, silme her koşulda mümkündür. Bu, “veri hiç kaybolmasın, ama listelerde de görünmesin” ihtiyacına uygun.

Neden global anti-forgery filter?

Master defterin modern önerisi (§4.18.3.b). Her POST action’a [ValidateAntiForgeryToken] yazmayı unutmak riskini eleman kaldırır. Webhook gibi istisnalar için [IgnoreAntiforgeryToken] explicit yazılır.

Neden ViewModel entity’yi taşıyor?

“Doktor + Bolum SelectList” gibi kompozit modellerde iki seçenek var: (a) ViewModel içinde entity nesnesi + ek property’ler, (b) ViewModel’de entity’nin alanları tek tek yazılı + mapping. (a) daha kısa ama entity property’lerinin form’a bind olmasını gerektirir. Bu proje (a) yaklaşımını kullanır — küçük ekiplerde pragmatik seçim.

Cascading dropdown neden AJAX?

Alternatif: bölüm seçilince form yeniden yüklenir (onchange="submit()"). Bu, kullanıcı deneyimini bozar — girilen diğer alanlar kaybolur. AJAX sadece bir dropdown’ı günceller; form durumunu korur.

5.2.19 İlişkili Konular

- BaseEntity + BaseService → §4.20
- Soft delete + global query filter → §4.21, §3.8.6
- Cascading dropdown AJAX → §4.19
- Global anti-forgery filter → §4.18
- Constructor injection çoklu servis → §4.16.4
- FK Restrict davranışı → §3.2.5
- LINQ Include + Where → §3.11, §3.12

Bu proje master defterin en kapsamlı kısmıdır; §4’teki konuların çoğunun uygulamalı karşılığıdır.

5.3 HastaneWeb / AspNet7Hastane — Database First + AJAX

5.3.1 Proje Tanıtımı

HastaneWeb (proje isim varyantı: **AspNet7Hastane**), bir hastane yönetim sistemini var olan bir SQL Server veritabanından türeten Database First yaklaşımlı bir MVC projesidir. §5.2 HastaneMVC (Code First) ile aynı domain’i kapsar, ancak yaklaşım tersinedir: DB önce, uygulama sonra. Bu projenin en belirgin teknik özelliği **AJAX ile cascading dropdown**’ın uygulanışdır — bölüm seçilince o bölümdeki doktorlar otomatik yüklenir.

Kapsam:

- **6 tablo şeması**: Bolum, Doktor, Hasta, Randevu, Personel, Tetkik.
- **Scaffold-DbContext** ile entity ve DbContext üretimi.
- **AJAX cascading dropdown** ile bölüm-doktor bağlantısı.
- **Standart CRUD** ile her tablo için yönetim ekranları.

5.3.2 Ortam

Bileşen	Değer
.NET SDK	8.0.x
Proje Şablonu	ASP.NET Core Web App (Model-View-Controller)
Veritabanı	SQL Server (localdb veya SQL Server 2019+)
EF Core	8.0.x
Yaklaşım	Database First (Scaffold-DbContext)
Bootstrap	5 (yerleşik)
jQuery	3.x (şablonla gelir)
Ek Paketler	Microsoft.EntityFrameworkCore.SqlServer, Microsoft.EntityFrameworkCore.Tools, Microsoft.EntityFrameworkCore.Design

5.3.3 Veritabanı Şeması (SQL)

Proje başlamadan önce SQL Server Management Studio ile veritabanı ve tablolar oluşturulur:

```
-- Veritabanı
CREATE DATABASE hastanedb;
GO
USE hastanedb;
GO

-- Bolum
CREATE TABLE Bolum (
    Id          INT IDENTITY PRIMARY KEY,
    Ad          NVARCHAR(100) NOT NULL,
    Aciklama    NVARCHAR(500) NULL
);

-- Doktor
CREATE TABLE Doktor (
    Id          INT IDENTITY PRIMARY KEY,
    Ad          NVARCHAR(100) NOT NULL,
    Soyad       NVARCHAR(100) NOT NULL,
    Uzmanlik    NVARCHAR(100) NULL,
    TelefonNo   NVARCHAR(20)  NULL,
    BolumId     INT NOT NULL,
    CONSTRAINT FK_Doktor_Bolum FOREIGN KEY (BolumId) REFERENCES Bolum(Id)
);

-- Hasta
CREATE TABLE Hasta (
    Id          INT IDENTITY PRIMARY KEY,
    TC          NVARCHAR(11)  NOT NULL,
    Ad          NVARCHAR(100) NOT NULL,
    Soyad       NVARCHAR(100) NOT NULL,
    DogumTarihi DATE NULL,
    Cinsiyet    NCHAR(1) NULL,
    TelefonNo   NVARCHAR(20)  NULL,
    Email       NVARCHAR(150) NULL,
    Adres       NVARCHAR(500) NULL
);

-- Randevu
CREATE TABLE Randevu (
    Id          INT IDENTITY PRIMARY KEY,
    DoktorId    INT NOT NULL,
    HastaId     INT NOT NULL,
    Tarih       DATETIME NOT NULL,
    Aciklama    NVARCHAR(500) NULL,
    CONSTRAINT FK_Randevu_Doktor FOREIGN KEY (DoktorId) REFERENCES Doktor(Id),
    CONSTRAINT FK_Randevu_Hasta  FOREIGN KEY (HastaId)  REFERENCES Hasta(Id)
);

-- Personel
CREATE TABLE Personel (
    Id          INT IDENTITY PRIMARY KEY,
    Ad          NVARCHAR(100) NOT NULL,
    Soyad       NVARCHAR(100) NOT NULL,
    Gorev       NVARCHAR(100) NOT NULL,
    TelefonNo   NVARCHAR(20)  NULL,
    Maas        DECIMAL(18,2) NULL,
    BolumId     INT NULL,
    CONSTRAINT FK_Personel_Bolum FOREIGN KEY (BolumId) REFERENCES Bolum(Id)
```

```
);
```

```
-- Tetkik
```

```
CREATE TABLE Tetkik (  
    Id INT IDENTITY PRIMARY KEY,  
    HastaId INT NOT NULL,  
    DoktorId INT NULL,  
    Ad NVARCHAR(150) NOT NULL,  
    Sonuc NVARCHAR(2000) NULL,  
    IstemTarihi DATETIME NOT NULL,  
    Ucret DECIMAL(18,2) NULL,  
    CONSTRAINT FK_Tetkik_Hasta FOREIGN KEY (HastaId) REFERENCES Hasta(Id),  
    CONSTRAINT FK_Tetkik_Doktor FOREIGN KEY (DoktorId) REFERENCES Doktor(Id)  
);
```

Ana ilişkiler:

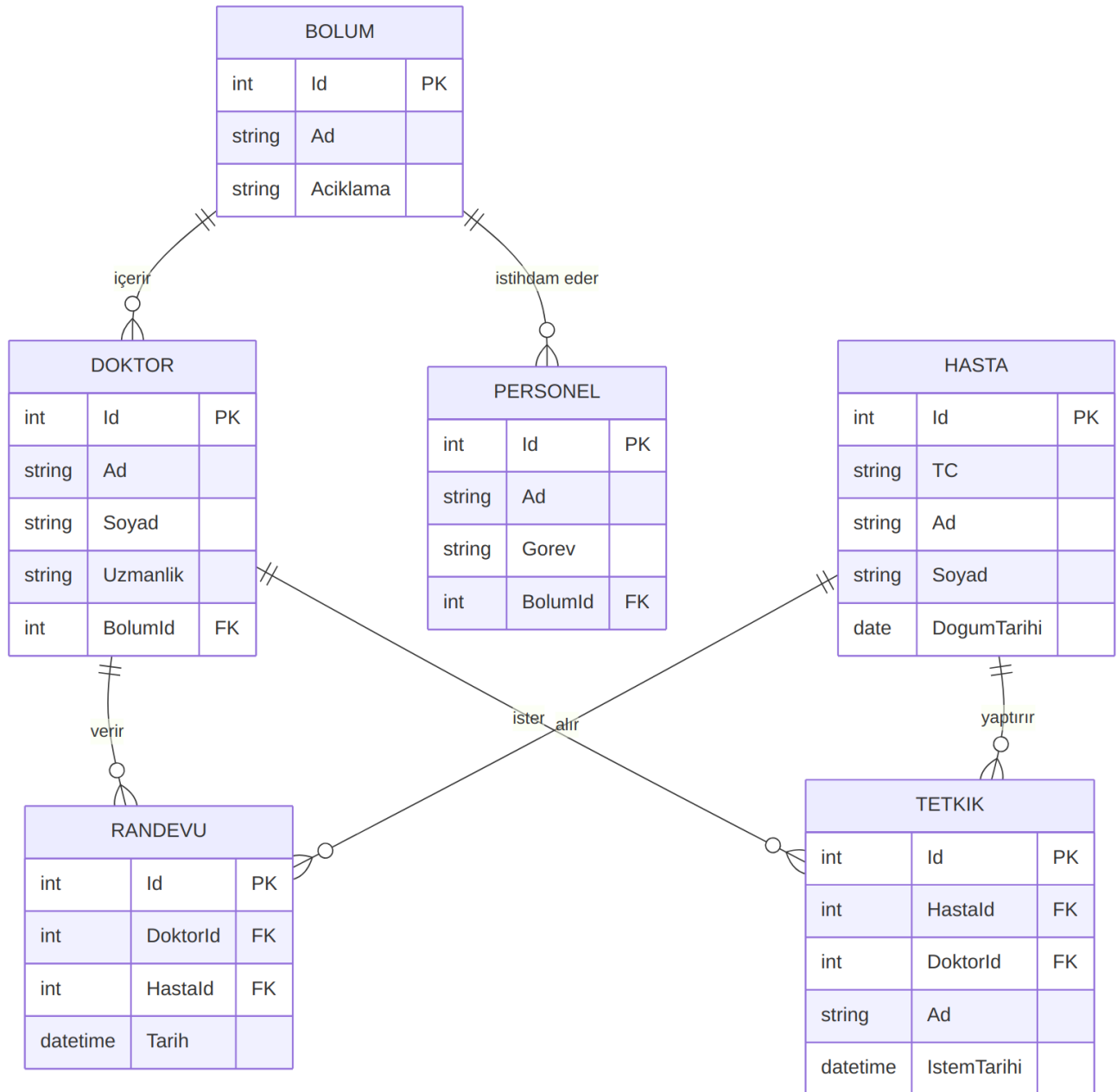


Figure 17: Diyagram 17

5.3.4 Scaffold Adımı

Yeni ASP.NET Core MVC projesi oluşturulur, NuGet paketleri eklenir (§3.5.2), PMC’de:

```
Scaffold-DbContext "Server=(localdb)\mssqllocaldb;Database=hastanedb;Trusted_Connection=True" `
    Microsoft.EntityFrameworkCore.SqlServer `
    -OutputDir Models `
    -Context HastaneContext `
    -Force
```

CLI karşılığı:

```
dotnet ef dbcontext scaffold "Server=(localdb)\mssqllocaldb;Database=hastanedb;Trusted_Connection=True" `
    Microsoft.EntityFrameworkCore.SqlServer `
    --output-dir Models `
    --context HastaneContext `
    --force
```

Sonuç:

- Models/HastaneContext.cs — DbContext, tüm DbSet’ler, Fluent API konfigürasyonu.
- Models/Bolum.cs, Models/Doktor.cs, Models/Hasta.cs, Models/Randevu.cs, Models/Personel.cs, Models/Tetkik.cs — entity sınıfları (partial).

5.3.5 Üretilen Entity Örneği

```
// Models/Doktor.cs (scaffold ile üretilmiş – özet)
public partial class Doktor
{
    public Doktor()
    {
        Randevular = new HashSet<Randevu>();
        Tetkikler = new HashSet<Tetkik>();
    }

    public int Id { get; set; }
    public string Ad { get; set; } = null!;
    public string Soyad { get; set; } = null!;
    public string? Uzmanlik { get; set; }
    public string? TelefonNo { get; set; }
    public int BolumId { get; set; }

    public virtual Bolum Bolum { get; set; } = null!;
    public virtual ICollection<Randevu> Randevular { get; set; }
    public virtual ICollection<Tetkik> Tetkikler { get; set; }
}
```

- partial işaretlemesi — geliştirici aynı sınıfı Doktor.Partial.cs gibi bir dosyada genişletebilir; scaffold yeniden çalıştığında elle yazılan uzantı korunur.
- null! null-forgiving operator (§2.4.10) — nullable reference types açıkken required alanlar için scaffold’ın ürettiği kalıp.
- virtual navigation’lar — lazy loading proxy yüklendiğinde çalışabilmesi için hazır. Master defterin önerisi lazy loading’i kapatmak (§3.12.5); bu proje varsayılan davranışı kabul eder.

5.3.6 Üretilen DbContext Örneği

```
// Models/HastaneContext.cs (scaffold ile üretilmiş – özet)
public partial class HastaneContext : DbContext
{
    public HastaneContext() { }
    public HastaneContext(DbContextOptions<HastaneContext> options) : base(options) { }

    public virtual DbSet<Bolum> Boluns { get; set; } = null!;
```

```

public virtual DbSet<Doktor> Doktors { get; set; } = null!;
public virtual DbSet<Hasta> Hastas { get; set; } = null!;
public virtual DbSet<Randevu> Randevus { get; set; } = null!;
public virtual DbSet<Personel> Personels { get; set; } = null!;
public virtual DbSet<Tetkik> Tetkiks { get; set; } = null!;

protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
{
    if (!optionsBuilder.IsConfigured)
    {
        optionsBuilder.UseSqlServer("Server=(localdb)\\mssqllocaldb;Database=hastanedb;Trust
    }
}

protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Doktor>(entity =>
    {
        entity.ToTable("Doktor");
        entity.Property(e => e.Ad).HasMaxLength(100);
        entity.Property(e => e.Soyad).HasMaxLength(100);
        entity.HasOne(d => d.Bolum)
            .WithMany(p => p.Doktors)
            .HasForeignKey(d => d.BolumId)
            .OnDelete(DeleteBehavior.ClientSetNull);
    });

    // ... diğer entity'ler için de benzer Fluent API blokları
}
}

```

Not: Scaffold varsayılan olarak DbSet adlarını İngilizce çoğulla verir (Doktors, Bolums). Bu, İngilizce olmayan tablo adlarıyla estetik uyumsuzluk üretir. İki çözüm:

1. Elle düzenle — Doktors → Doktorlar. Ancak bir sonraki -Force scaffold bu düzeltmeyi geri alır.
2. -DataAnnotations parametresi ile scaffold — Fluent API yerine attribute'lar üretilir; DbSet adı hâlâ İngilizce çoğuldur.

Master defter kapsamında bu detay partial extension ile korunabilir; ama yaygın uygulama scaffold sonuçlarını kabul etmek ve devam etmektir.

5.3.7 Elle Düzeltme: Connection String Konvansiyonu

Scaffold, connection string'i OnConfiguring içine hardcode olarak koyar. Master defter standardı (§3.5.4): connection string appsettings.json'da olmalı, DI ile inject edilmeli.

Düzeltme:

appsettings.json:

```

{
  "ConnectionStrings": {
    "DefaultConnection": "Server=(localdb)\\mssqllocaldb;Database=hastanedb;Trusted_Connection=
  }
}

```

Program.cs:

```

using Microsoft.EntityFrameworkCore;
using AspNet7Hastane.Models;

var builder = WebApplication.CreateBuilder(args);

```

```

builder.Services.AddControllersWithViews();
builder.Services.AddDbContext<HastaneContext>(o =>
    o.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));

var app = builder.Build();

if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Home/Error");
    app.UseHsts();
}

app.UseHttpsRedirection();
app.UseStaticFiles();
app.UseRouting();
app.UseAuthorization();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");

app.Run();

```

Scaffold tarafından üretilen OnConfiguring metodu, IsConfigured kontrolü sayesinde artık atlanır — dış konfigürasyon önceliklidir. Yine de bakım için scaffold-üretilmiş OnConfiguring bloğunu silmek veya -NoOnConfiguring parametresi ile hiç üretmemek tercih edilir.

5.3.8 Proje Yapısı (Scaffold Sonrası)

```

AspNet7Hastane/
├── Controllers/
│   ├── HomeController.cs
│   ├── DoktorController.cs
│   ├── RandevuController.cs
│   ├── HastaController.cs
│   ├── PersonelController.cs
│   └── TetkikController.cs
├── Models/
│   ├── HastaneContext.cs
│   ├── Bolum.cs
│   ├── Doktor.cs
│   ├── Hasta.cs
│   ├── Randevu.cs
│   ├── Personel.cs
│   └── Tetkik.cs
├── Views/
│   ├── Home/
│   ├── Doktor/
│   ├── Randevu/
│   └── ...
├── wwwroot/
├── Program.cs
├── appsettings.json
└── AspNet7Hastane.csproj

```

← Scaffold çıkışları

Entity ve DbContext hazır olduğu için geliştirici doğrudan controller ve view yazımına başlar.

5.3.9 Ana Özellik: Randevu Ekleme Ekranı ve AJAX Cascading

Proje kod tabanının en belirleyici tercih noktası: **randevu formunda bölüm seçilince doktor listesinin AJAX ile otomatik yüklenmesi**. §4.19.6’da açıklanan cascading dropdown deseninin

bu projedeki karşılığıdır.

Controller:

```
// Controllers/RandevuController.cs
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.Rendering;
using Microsoft.EntityFrameworkCore;
using AspNet7Hastane.Models;

public class RandevuController : Controller
{
    private readonly HastaneContext _context;

    public RandevuController(HastaneContext context)
    {
        _context = context;
    }

    // GET: /Randevu
    public async Task<IActionResult> Index()
    {
        var liste = await _context.Randevus
            .Include(r => r.Doktor).ThenInclude(d => d.Bolum)
            .Include(r => r.Hasta)
            .OrderByDescending(r => r.Tarih)
            .AsNoTracking()
            .ToListAsync();
        return View(liste);
    }

    // GET: /Randevu/Create
    public async Task<IActionResult> Create()
    {
        ViewBag.Bolumler = new SelectList(
            await _context.Bolums.AsNoTracking().ToListAsync(),
            nameof(Bolum.Id),
            nameof(Bolum.Ad));

        ViewBag.Hastalar = new SelectList(
            await _context.Hastas.AsNoTracking().ToListAsync(),
            nameof(Hasta.Id),
            nameof(Hasta.Ad));

        return View();
    }

    // POST: /Randevu/Create
    [HttpPost]
    [ValidateAntiForgeryToken]
    public async Task<IActionResult> Create(Randevu randevu)
    {
        if (!ModelState.IsValid)
        {
            ViewBag.Bolumler = new SelectList(
                await _context.Bolums.AsNoTracking().ToListAsync(),
                nameof(Bolum.Id),
                nameof(Bolum.Ad));
            ViewBag.Hastalar = new SelectList(
                await _context.Hastas.AsNoTracking().ToListAsync(),
                nameof(Hasta.Id),
                nameof(Hasta.Ad));
        }
    }
}
```

```

        return View(randevu);
    }

    _context.Randevus.Add(randevu);
    await _context.SaveChangesAsync();

    TempData["Success"] = "Randevu oluşturuldu.";
    return RedirectToAction(nameof(Index));
}

// AJAX: bölüme göre doktor listesi
[HttpGet]
public async Task<IActionResult> DoktorGetir(int bolumId)
{
    var doktorlar = await _context.Doktors
        .Where(d => d.BolumId == bolumId)
        .Select(d => new
        {
            id = d.Id,
            adSoyad = d.Ad + " " + d.Soyad
        })
        .ToListAsync();

    return Json(doktorlar);
}
}

```

AJAX endpoint (DoktorGetir) — JSON döner. Property adları client-side’da rahat kullanım için camelCase (id, adSoyad).

5.3.10 Randevu Create View

```

@* Views/Randevu/Create.cshtml *@
@model Randevu

@{
    ViewData["Title"] = "Yeni Randevu";
}

<h1>Yeni Randevu</h1>

<form asp-action="Create" method="post">
    <div asp-validation-summary="ModelOnly" class="text-danger"></div>

    <div class="mb-3">
        <label class="form-label">Bölüm</label>
        <select id="bolumSelect" class="form-select" asp-items="ViewBag.Bolumler">
            <option value="">-- Bölüm seçin --</option>
        </select>
    </div>

    <div class="mb-3">
        <label asp-for="DoktorId" class="form-label">Doktor</label>
        <select id="doktorSelect" asp-for="DoktorId" class="form-select">
            <option value="">-- Önce bölüm seçin --</option>
        </select>
        <span asp-validation-for="DoktorId" class="text-danger"></span>
    </div>

    <div class="mb-3">
        <label asp-for="HastaId" class="form-label">Hasta</label>
    </div>

```

```

        <select asp-for="HastaId" class="form-select" asp-items="ViewBag.Hastalar">
            <option value="">-- Hasta seçin --</option>
        </select>
        <span asp-validation-for="HastaId" class="text-danger"></span>
    </div>

    <div class="mb-3">
        <label asp-for="Tarih" class="form-label"></label>
        <input asp-for="Tarih" type="datetime-local" class="form-control" />
        <span asp-validation-for="Tarih" class="text-danger"></span>
    </div>

    <div class="mb-3">
        <label asp-for="Aciklama" class="form-label"></label>
        <textarea asp-for="Aciklama" class="form-control" rows="3"></textarea>
    </div>

    <button type="submit" class="btn btn-primary">Kaydet</button>
    <a asp-action="Index" class="btn btn-secondary">İptal</a>
</form>

@section Scripts {
    <partial name="_ValidationScriptsPartial" />
    <script>
        $(function () {
            $("#bolumSelect").change(function () {
                var bolumId = $(this).val();
                var $doktor = $("#doktorSelect");
                $doktor.empty();

                if (!bolumId) {
                    $doktor.append('<option value="">-- Önce bölüm seçin --</option>');
                    return;
                }

                $doktor.append('<option value="">-- Yükleniyor... --</option>');

                $.getJSON("@Url.Action("DoktorGetir", "Randevu")", { bolumId: bolumId },
                    function (liste) {
                        $doktor.empty();
                        $doktor.append('<option value="">-- Doktor seçin --</option>');
                        $.each(liste, function (i, d) {
                            $doktor.append(
                                $('<option>', { value: d.id, text: d.adSoyad })
                            );
                        });
                    })
                .fail(function () {
                    $doktor.empty();
                    $doktor.append('<option value="">-- Hata oluştu --</option>');
                });
            });
        });
    </script>
}

```

@Url.Action("DoktorGetir", "Randevu") — reverse routing (§4.6.6) ile URL derleme zamanında üretilir; route şeması değişirse otomatik güncellenir.

.fail(...) — jQuery AJAX'ta hata handler'ı. Sunucu 500 veya ağ hatası durumunda kullanıcıya feedback vermek için eklenir.

5.3.11 Ana Akış Diyagramı

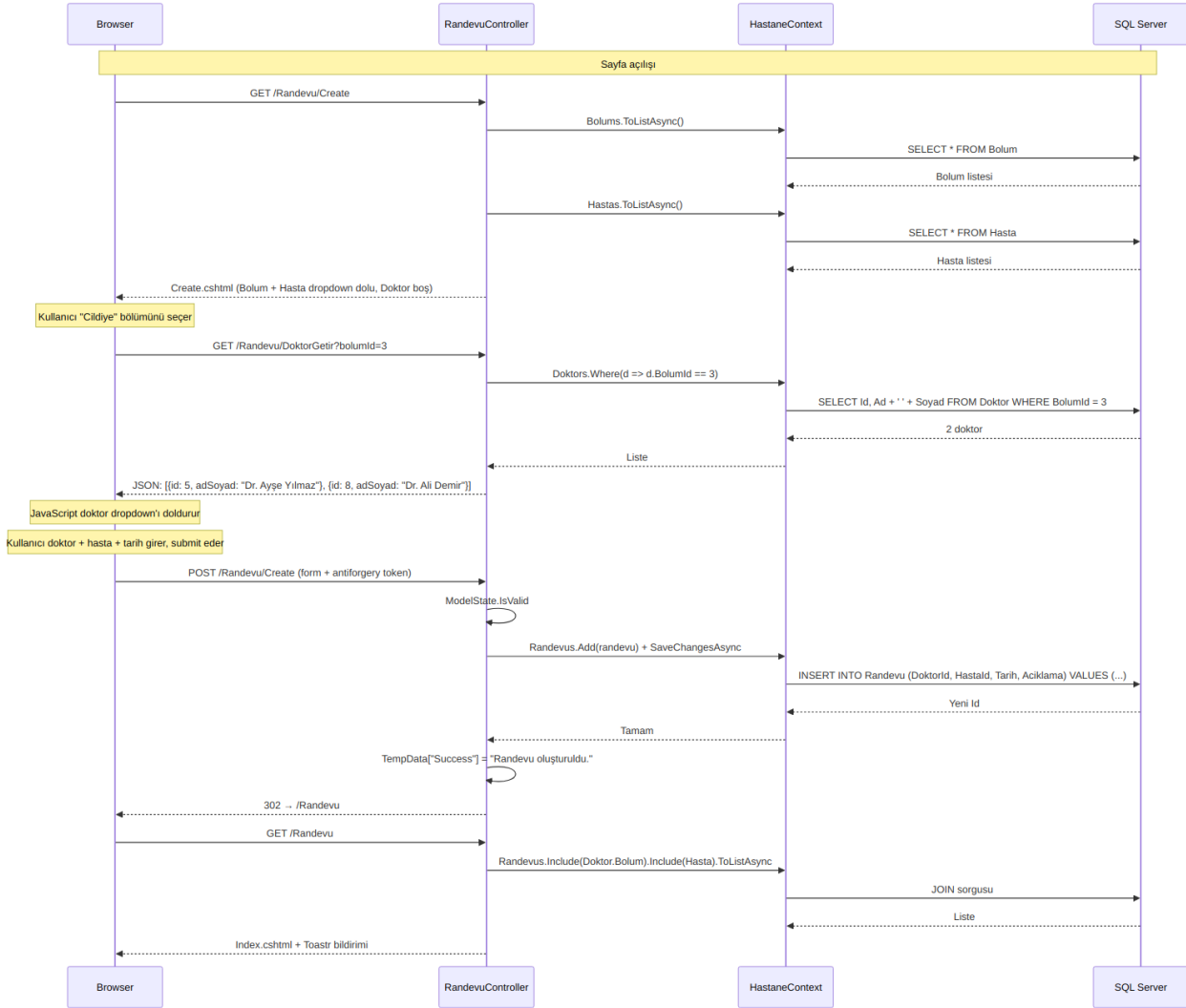


Figure 18: Diyagram 18

5.3.12 Layout — Toastr Entegrasyonu

Master defter §4.22'deki standart entegrasyon:

```

@* Views/Shared/_Layout.cshtml (özet) *@
<link rel="stylesheet"
      href="https://cdnjs.cloudflare.com/ajax/libs/toastr.js/2.1.4/toastr.min.css" />

<!-- body sonunda -->
<script src="~/lib/jquery/dist/jquery.min.js"></script>
<script src="~/lib/bootstrap/dist/js/bootstrap.bundle.min.js"></script>
<script src="https://cdnjs.cloudflare.com/ajax/libs/toastr.js/2.1.4/toastr.min.js"></script>

<script>
    $(function () {
        @if (TempData["Success"] != null)
        {
            <text>toastr.success("@TempData["Success"]");</text>
        }
        @if (TempData["Error"] != null)
        {
            <text>toastr.error("@TempData["Error"]");</text>
        }
    })
  
```

```
});  
</script>
```

5.3.13 Beklenen Sonuçlar Tablosu

Senaryo	Girdi	Beklenen Sonuç
Sayfa açılışı	GET /Randevu/Create	Bolum dropdown dolu (SQL SELECT çalıştı), Doktor dropdown “Önce bölüm seçin”, Hasta dropdown dolu
Bölüm seçimi	Bölüm=Cildiye	AJAX çağrısı /Randevu/DoktorGetir?bolumId=3, JSON döner, doktor dropdown 2 seçenekle dolar
Bölüm seçimi (o bölümde doktor yok)	Bölüm=X (doktoru olmayan)	JSON boş [], doktor dropdown “- Doktor seçin -” tek seçenekle dolu
Randevu ekleme (geçerli)	Bölüm=3, Doktor=5, Hasta=1, Tarih=yarın 10:00	Kayıt eklenir, Toastr success, Index’te en üstte görünür
Randevu ekleme (DoktorId=0)	Doktor dropdown “Önce bölüm seçin” bırakıldı	ModelState hata, form geri döner
AJAX endpoint hatası (500)	Sunucu hata verirse	.fail() handler çalışır, “- Hata oluştu -” görünür
Anti-forgery token’sız POST	Elle curl POST	400 Bad Request

5.3.14 Tasarım Notları

Database First → Code First geçişi mümkün mü?

Evet — scaffold’la üretilen sınıflar POCO’dur; onları elle düzenleyip migration desteği eklemek mümkün. Ama pratikte proje ya Database First (DB DBA’in yönetiminde) ya Code First (uygulama takımı yönetimi) olarak sabitlenir; karma yaklaşım karışıklık üretir.

Neden BaseEntity yok?

Database First projede tablolar zaten belirli; entity’lerin ortak alanları DB şemasına bağlı. Scaffold-üretilmiş sınıflara BaseEntity kalıtımı eklemek -Force scaffold’da kaybolur. Alternatif: partial uzantı dosyalarda interface uygulaması eklemek (partial class Doktor : IAuditable { ... }) ama bu Bulky ve HastaneMVC’ye kıyasla dolaylı yoldur.

Neden generic BaseService yok?

Scaffold-üretilmiş sınıflar farklı isimlendirme (Bolums vs beklenen Bolumler) taşıdığı için generic servis kurulumu ek uğraş gerektirir. Bu projenin öğretici odağı DB First + AJAX olduğu için servis katmanı atlanır — controller doğrudan DbContext’e erişir.

Neden -Force her seferinde?

DB şeması değiştiğinde en garantili senkron yol tekrar scaffold’dur. Ancak scaffold -Force elle yazılan tüm değişiklikleri (partial extension’lar hariç) yer. Bu, proje disiplini belirler: elle yazılan tüm ekler partial sınıflarda yer alır, aslı hiç değiştirilmez.

Data Annotations mı Fluent API mi?

Scaffold varsayılan olarak Fluent API üretir (OnModelCreating içinde). -DataAnnotations parametresi ile attribute’lu üretim istenebilir. Master defterin standardı ne olursa olsun scaffold’un ne ürettiğini kabul etmek — sonradan değiştirmek -Force’ta kaybolur.

5.3.15 İlişkili Konular

- Database First + Scaffold-DbContext → §3.10
- AJAX cascading dropdown → §4.19
- Anti-forgery koruma → §4.18
- Connection string konvansiyonu → §3.5.4
- DI ile DbContext → §4.16
- LINQ Include + projection → §3.11, §3.12

- Toastr + TempData → §4.22

Bu proje, Database First workflow'una odaklanmış olsa da MVC'nin tüm temel bloklarını tekrar tekrar kullanır. HastaneMVC (§5.2) ile karşılaştırılınca “aynı domain, iki farklı yaklaşım” perspektifi ortaya çıkar.

5.4 ÖrnekUygulama — Basit Product CRUD

5.4.1 Proje Tanıtımı

ÖrnekUygulama, bir ürün kataloğunu yöneten en yalın MVC uygulamasıdır. Amacı: Bir MVC projesinin en temel akışını — model tanımla, DbContext yapılandır, controller ile CRUD yaz — mümkün olan en az kavramla sunmak. §5.1 BulkyBookWeb ve §5.2 HastaneMVC'nin “bir önceki adımı” — bu projede henüz Toastr, ViewModel, servis katmanı yok; yalın controller + entity + view.

Bu bölüm eğitim odaklı bir dip nottur; üretim kalitesi bir uygulama değildir. Kod tabanının küçük-lüğü bilinçlidir.

5.4.2 Ortam

Bileşen	Değer
.NET SDK	8.0.x
Proje Şablonu	ASP.NET Core Web App (MVC)
Veritabanı	SQL Server (localdb)
EF Core	8.0.x
Yaklaşım	Code First + Migration
NuGet Paketleri	Microsoft.EntityFrameworkCore.SqlServer, Microsoft.EntityFrameworkCore.Tools, Microsoft.EntityFrameworkCore.Design

5.4.3 Proje Yapısı

```
OrnekUygulama/
├── Controllers/
│   ├── HomeController.cs
│   └── UrunController.cs
├── Data/
│   └── AppDbContext.cs
├── Migrations/
├── Models/
│   ├── ErrorViewModel.cs
│   └── Urun.cs
├── Views/
│   ├── Home/
│   └── Urun/
│       ├── Index.cshtml
│       ├── Create.cshtml
│       ├── Edit.cshtml
│       └── Delete.cshtml
│   └── Shared/
├── wwwroot/
├── Program.cs
└── appsettings.json
```

Sadelik: tek entity (Urun), tek controller (UrunController), dört view.

5.4.4 Veri Modeli

```
// Models/Urun.cs
using System.ComponentModel.DataAnnotations;

public class Urun
{
    public int Id { get; set; }

    [Required(ErrorMessage = "Ürün adı zorunlu.")]
    [MaxLength(100)]
    [Display(Name = "Ürün Adı")]
    public string Ad { get; set; }

    [Required]
    [Range(0.01, 1000000, ErrorMessage = "Fiyat 0.01 ile 1.000.000 arasında olmalı.")]
    [Column(TypeName = "decimal(18,2)")]
    [Display(Name = "Fiyat")]
    public decimal Fiyat { get; set; }

    [Range(0, int.MaxValue)]
    [Display(Name = "Stok")]
    public int Stok { get; set; }

    [Display(Name = "Eklenme Tarihi")]
    public DateTime EklenmeTarihi { get; set; } = DateTime.Now;
}
```

Data Annotations (§3.7) ile hem şema hem validasyon.

5.4.5 DbContext

```
// Data/AppDbContext.cs
using Microsoft.EntityFrameworkCore;

public class AppDbContext : DbContext
{
    public AppDbContext(DbContextOptions<AppDbContext> options) : base(options) { }
    public DbSet<Urun> Urunler { get; set; }
}
```

5.4.6 Program.cs

```
using Microsoft.EntityFrameworkCore;
using OrnekUygulama.Data;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllersWithViews();
builder.Services.AddDbContext<AppDbContext>(o =>
    o.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));

var app = builder.Build();

if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Home/Error");
    app.UseHsts();
}

app.UseHttpsRedirection();
```

```

app.UseStaticFiles();
app.UseRouting();
app.UseAuthorization();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");

app.Run();

```

Standart iskelet — §4.3.

5.4.7 appsettings.json

```

{
  "ConnectionStrings": {
    "DefaultConnection": "Server=(localdb)\\mssqllocaldb;Database=OrnekUygulamaDb;Trusted_Connection=true;
  }
}

```

5.4.8 UrunController

```

// Controllers/UrunController.cs
using Microsoft.AspNetCore.Mvc;
using OrnekUygulama.Data;
using OrnekUygulama.Models;

public class UrunController : Controller
{
    private readonly AppDbContext _context;
    public UrunController(AppDbContext context) => _context = context;

    // GET: /Urun
    public IActionResult Index()
    {
        var liste = _context.Urunler.OrderBy(u => u.Ad).ToList();
        return View(liste);
    }

    // GET: /Urun/Create
    public IActionResult Create() => View();

    // POST: /Urun/Create
    [HttpPost]
    [ValidateAntiForgeryToken]
    public IActionResult Create(Urun urun)
    {
        if (!ModelState.IsValid) return View(urun);
        _context.Urunler.Add(urun);
        _context.SaveChanges();
        return RedirectToAction(nameof(Index));
    }

    // GET: /Urun/Edit/5
    public IActionResult Edit(int id)
    {
        var urun = _context.Urunler.Find(id);
        if (urun == null) return NotFound();
        return View(urun);
    }
}

```

```

// POST: /Urun/Edit/5
[HttpPost]
[ValidateAntiForgeryToken]
public IActionResult Edit(Urun urun)
{
    if (!ModelState.IsValid) return View(urun);
    _context.Urunler.Update(urun);
    _context.SaveChanges();
    return RedirectToAction(nameof(Index));
}

// GET: /Urun/Delete/5
public IActionResult Delete(int id)
{
    var urun = _context.Urunler.Find(id);
    if (urun == null) return NotFound();
    return View(urun);
}

// POST: /Urun/Delete/5
[HttpPost, ActionName("Delete")]
[ValidateAntiForgeryToken]
public IActionResult DeleteConfirmed(int id)
{
    var urun = _context.Urunler.Find(id);
    if (urun != null)
    {
        _context.Urunler.Remove(urun);
        _context.SaveChanges();
    }
    return RedirectToAction(nameof(Index));
}
}

```

Beş action, standart CRUD. TempData/Toastr yok — sade PRG.

5.4.9 Index View

```

@* Views/Urun/Index.cshtml *@
@model IEnumerable<Urun>

@{
    ViewData["Title"] = "Ürünler";
}

<h1>Ürünler</h1>
<a asp-action="Create" class="btn btn-primary mb-3">Yeni Ürün</a>

<table class="table table-striped">
    <thead>
        <tr>
            <th>@Html.DisplayNameFor(m => m.First().Ad)</th>
            <th>@Html.DisplayNameFor(m => m.First().Fiyat)</th>
            <th>@Html.DisplayNameFor(m => m.First().Stok)</th>
            <th>@Html.DisplayNameFor(m => m.First().EklenmeTarihi)</th>
            <th></th>
        </tr>
    </thead>
    <tbody>
        @foreach (var u in Model)

```

```

        {
            <tr>
                <td>@u.Ad</td>
                <td>@u.Fiyat.ToString("C")</td>
                <td>@u.Stok</td>
                <td>@u.EklenmeTarihi.ToString("dd.MM.yyyy")</td>
                <td>
                    <a asp-action="Edit"    asp-route-id="@u.Id" class="btn btn-sm btn-warning">D
                    <a asp-action="Delete" asp-route-id="@u.Id" class="btn btn-sm btn-danger">Si
                </td>
            </tr>
        }
    </tbody>
</table>

```

5.4.10 Create View

```

@* Views/Urun/Create.cshtml *@
@model Urun

@{
    ViewData["Title"] = "Yeni Ürün";
}

<h1>Yeni Ürün</h1>

<form asp-action="Create" method="post">
    <div asp-validation-summary="ModelOnly" class="text-danger"></div>

    <div class="mb-3">
        <label asp-for="Ad" class="form-label"></label>
        <input asp-for="Ad" class="form-control" />
        <span asp-validation-for="Ad" class="text-danger"></span>
    </div>

    <div class="mb-3">
        <label asp-for="Fiyat" class="form-label"></label>
        <input asp-for="Fiyat" class="form-control" step="0.01" />
        <span asp-validation-for="Fiyat" class="text-danger"></span>
    </div>

    <div class="mb-3">
        <label asp-for="Stok" class="form-label"></label>
        <input asp-for="Stok" class="form-control" />
        <span asp-validation-for="Stok" class="text-danger"></span>
    </div>

    <button type="submit" class="btn btn-primary">Kaydet</button>
    <a asp-action="Index" class="btn btn-secondary">İptal</a>
</form>

@section Scripts {
    <partial name="_ValidationScriptsPartial" />
}

```

step="0.01" ondalık girişe izin verir; HTML5 number input'un ondalık hassasiyet ayarı.

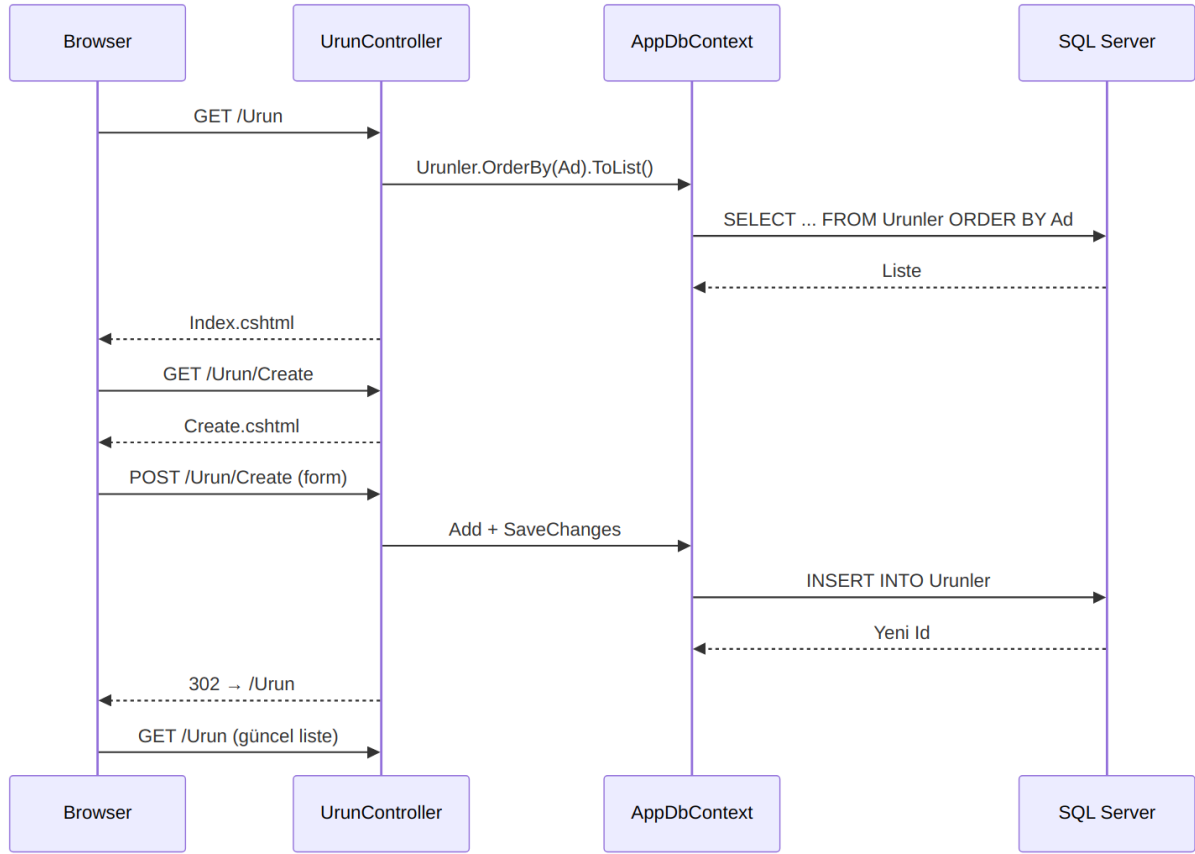


Figure 19: Diyagram 19

5.4.11 Ana Akış

5.4.12 Migration İş Akışı

Standart:

Add-Migration Init
Update-Database

Üretilen tablo (özet):

```

CREATE TABLE Urunler (
    Id INT IDENTITY PRIMARY KEY,
    Ad NVARCHAR(100) NOT NULL,
    Fiyat DECIMAL(18,2) NOT NULL,
    Stok INT NOT NULL,
    EklenmeTarihi DATETIME2 NOT NULL
);
  
```

5.4.13 Beklenen Sonuçlar Tablosu

Senaryo	Girdi	Beklenen Sonuç
İlk çalıştırma, Index Yeni ürün ekleme	GET /Urun Ad="Kalem", Fiyat=5.50, Stok=100	Boş tablo, "Yeni Ürün" butonu Kayıt eklenir, Index'e yönlendirme
Yeni ürün — geçersiz fiyat	Fiyat=0	Range hatası: "0.01 ile 1.000.000 arasında olmalı"
Yeni ürün — boş ad	Ad=""	Required hatası: "Ürün adı zorunlu."
Ürün düzenleme	Id=1, Ad="Kalem-2"	Kayıt güncellenir

Senaryo	Girdi	Beklenen Sonuç
Ürün silme onayı	GET /Urun/Delete/1	Delete.cshtml onay ekranı
Ürün silme	POST /Urun/Delete/1	Silinir, Index'e yönlendirme

5.4.14 Tasarım Notları

Ne yok?

- **ViewModel yok** — entity direkt view'a geçiyor. Bu üretim kalitesi bir yaklaşım değildir; mass assignment riskine açık (§4.14.2).
- **Service Layer yok** — controller doğrudan DbContext ile konuşur.
- **Soft delete yok** — hard Remove.
- **Toastr / TempData yok** — sadece PRG redirect.
- **Global anti-forgery filter yok** — her POST'ta klasik attribute.

Ne var?

- Data Annotations validasyonu.
- Constructor injection.
- Migration ile Code First.
- Tag Helper form.
- Client-side validation partial.
- PRG deseni.

Bu proje nereye evrilir?

Şu adımlar sırasıyla eklenerek §5.1 BulkyBookWeb seviyesine, sonra §5.2 HastaneMVC seviyesine geçilir:

1. **ViewModel katmanı:** Urun yerine UrunFormVM — sadece form-relevant alanlar.
2. **TempData + Toastr:** post sonrası bildirimler.
3. **Global anti-forgery:** filter Program.cs'e.
4. **Service Layer:** IUrunService + UrunService.
5. **BaseEntity:** ortak alanları çıkar.
6. **Soft delete:** IsDeleted + query filter.

Bu evrim, master defterin bölümlerinin uygulamalı öğrenme sırasındır — küçük bir uygulama ile başla, katman katman genişlet.

5.4.15 İlişkili Konular

- Data Annotations → §3.7
- Migration → §3.9
- Tag Helper form → §4.11
- Validation → §4.15
- CRUD controller → §4.13
- PRG deseni → §4.7.8

Bu proje sonrasında geliştirilen tüm ek özellikler (Toastr, ViewModel, Service, BaseEntity, Soft Delete) master defterin ilgili bölümlerine bakılarak eklenebilir.

Ekler

Bu bölüm, ana bölümlere referans veren pratik başvuru tabloları ve kısa notlar içerir. Her ek bir kez okunup sık ihtiyaç anında geri dönülecek biçimde tasarlanmıştır.

Ek A. Tag Helper Hızlı Referansı

Master defter §4.11'deki Tag Helper'ların özet tablosu. Detay için ilgili bölüme bakılır.

A.1 Anchor Tag Helper (<a>)

Attribute	Anlam	Örnek
asp-controller	Hedef controller adı	asp-controller="Doktor"
asp-action	Hedef action adı	asp-action="Detay"
asp-area	Alan (area) adı	asp-area="Admin"
asp-route-<param>	Route parametresi	asp-route-id="@Model.Id"
asp-route	Named route adı	asp-route="detail"
asp-fragment	URL sonuna #fragment	asp-fragment="randevular"
asp-host	Host override	asp-host="admin.site.com"
asp-protocol	http / https	asp-protocol="https"

A.2 Form Tag Helper (<form>)

Attribute	Anlam
asp-controller	Hedef controller
asp-action	Hedef action
asp-route-<param>	Route parametresi
method="post"	HTTP method — POST için token otomatik eklenir
enctype="multipart/form-data"	Dosya upload'u için

A.3 Input Tag Helper

Attribute	Anlam
asp-for="Property"	Property'ye bağla; name, id, value, type, data-val-* üretir
asp-format="{0:dd/MM/yyyy}"	Değerin format string'i

A.4 Label Tag Helper

Attribute	Anlam
asp-for="Property"	[Display(Name = "...")] etiketini kullanır, yoksa property adı

A.5 Select Tag Helper

Attribute	Anlam
asp-for="Property"	Property'ye bağla
asp-items="IEnumerable<SelectListItem>"	Seçenek listesi

A.6 Textarea Tag Helper

Attribute	Anlam
asp-for="Property"	Property'ye bağla; rows, cols klasik HTML attribute

A.7 Validation Tag Helper'leri

Tag	Attribute	Anlam
	asp-validation-for="Property"	Belirli property için hata mesajı yeri
<div>	asp-validation-summary="ModelOnly"	Genel hata özeti; None/ModelOnly/All

A.8 Environment Tag Helper

```
<environment include="Development">
  <link href="~/css/site.css" rel="stylesheet" />
</environment>
<environment exclude="Development">
  <link href="~/css/site.min.css" rel="stylesheet" asp-append-version="true" />
</environment>
```

Attribute	Anlam
include="Development,Staging"	Sadece belirtilen ortamlarda göster
exclude="Production"	Belirtilen ortamlar dışında göster

A.9 Cache Tag Helper

```
<cache expires-after="@TimeSpan.FromMinutes(5)">
  <div>@DateTime.Now</div>
</cache>
```

Attribute	Anlam
expires-after	Süre
expires-on	Belirli tarih
expires-sliding	Yumuşak süre; erişim varsa uzar
vary-by-header	Header'a göre farklı cache
vary-by-query	Query string'e göre
vary-by-cookie	Cookie'ye göre
vary-by-user	Kullanıcıya göre

A.10 Partial Tag Helper

```
<partial name="_Karnesi" model="@Model.Rapor" view-data="@ViewData" />
```

Attribute	Anlam
name	Partial view adı
model	Modeli geç
view-data	ViewData sözlüğü
for	ModelExpression

A.11 Script ve Link Tag Helper'ları

```
<script src="~/js/site.js" asp-append-version="true"></script>
<link href="~/css/site.css" rel="stylesheet" asp-append-version="true" />
```

Attribute	Anlam
asp-append-version="true"	Dosya hash'ini query string'e ekler (cache-busting)
asp-src-include, asp-src-exclude	Glob pattern ile birden çok script

Attribute	Anlam
asp-fallback-src	CDN başarısızsa yedek URL
asp-fallback-test	JS ifadesi; falsy ise fallback yüklenir
crossorigin, integrity	Standart HTML attribute'ları

A.12 Image Tag Helper

```

```

asp-append-version="true" — cache-busting hash. Diğer attribute'lar standart HTML.

A.13 Kayıt (_ViewImports.cshtml)

Tüm Tag Helper'lar için gerekli:

```
@addTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers
```

Yerleşik proje şablonu bu satırı içerir. Kendi yazdığın Tag Helper'lar için:

```
@addTagHelper *, MyProject
```

Ek B. Migration Komutları

Master defter §3.9'daki komutların yan yana karşılaştırılması.

B.1 PMC (Package Manager Console) vs .NET CLI

Görev	PMC	.NET CLI
Yeni migration	Add-Migration <ad>	dotnet ef migrations add <ad>
Veritabanı güncelle	Update-Database	dotnet ef database update
Belirli sürüme geri git	Update-Database <ad>	dotnet ef database update <ad>
Tüm migration'ları sıfırla	Update-Database -Migration 0	dotnet ef database update 0
Son migration'ı sil	Remove-Migration	dotnet ef migrations remove
Migration listesi	Get-Migration	dotnet ef migrations list
SQL script üret	Script-Migration	dotnet ef migrations script
İki migration arası SQL	Script-Migration -From X -To Y	dotnet ef migrations script X Y
Idempotent SQL script	Script-Migration -Idempotent	dotnet ef migrations script --idempotent
DbContext scaffold	Scaffold-DbContext "..."	dotnet ef dbcontext scaffold "..."
DbContext bilgisi	Get-DbContext	dotnet ef dbcontext info
Optimize edilmiş model	Optimize-DbContext	dotnet ef dbcontext optimize
EF Core sürümü	Get-Help entityframework	dotnet ef --version

B.2 Çoklu DbContext

Bir projede birden fazla DbContext varsa hangisi hedefleniyor belirtilir:

PMC:

```
Add-Migration Init -Context SalesDbContext
Update-Database -Context SalesDbContext
```

CLI:

```
dotnet ef migrations add Init --context SalesDbContext
dotnet ef database update --context SalesDbContext
```

B.3 Çoklu Proje

Solution içinde EF Core migration'ları farklı bir projeden alınıyorsa:

PMC — -Project (migration'ın gideceği) ve -StartupProject (Program.cs olan):

```
Add-Migration Init -Project MyApp.Data -StartupProject MyApp.Web
Update-Database -Project MyApp.Data -StartupProject MyApp.Web
```

CLI — proje klasöründe çalıştırarak veya --project ile:

```
dotnet ef migrations add Init --project ../MyApp.Data --startup-project .
dotnet ef database update --project ../MyApp.Data --startup-project .
```

B.4 Migration Geri Alma Adım Adım

Yanlış migration eklendi, uygulanmadan geri almak isteniyor:

```
Remove-Migration
```

Uygulandıysa önce DB'yi bir önceki sürüme al, sonra sil:

```
Update-Database -Migration <ÖncekiMigrationAdı>
Remove-Migration
```

Tüm migration'ları sıfırla (DB boşalır ama tablolar kalır):

```
Update-Database -Migration 0
```

B.5 dotnet-ef Kurulumu

CLI komutları için global tool:

```
dotnet tool install --global dotnet-ef
dotnet tool update --global dotnet-ef
dotnet ef --version
```

Yol çakışması varsa (Windows):

```
$env:Path += ";$env:USERPROFILE\.dotnet\tools"
```

Ek C. Bootswatch ve Bootstrap Icons Kurulumu

C.1 Bootswatch — Hazır Bootstrap Temaları

Bootswatch, Bootstrap 5 üzerine kurulu ücretsiz temalar koleksiyonudur (Litera, Cosmo, Flatly, Sketchy, ...).

CDN ile kullanım — _Layout.cshtml'de mevcut Bootstrap link'ini değiştir:

```
<!-- Varsayılan Bootstrap yerine -->
<link rel="stylesheet"
      href="https://bootswatch.com/5/litera/bootstrap.min.css" />
```

Tema listesi ve önizleme: <https://bootswatch.com/>.

Local kurulum — LibMan ile:

```
# LibMan CLI kur (bir kez)
dotnet tool install --global Microsoft.Web.LibraryManager.Cli

# Proje kökünde bootswatch kur
libman install bootswatch@5.3.0 -d wwwroot/lib/bootswatch
```

Sonra layout'ta:

```
<link rel="stylesheet" href="~/lib/bootswatch/dist/litera/bootstrap.min.css" />
```

C.2 Bootstrap Icons

Bootstrap Icons, Bootstrap ekibinin ürettiği ikon setidir (2000+ ikon).

CDN ile:

```
<link rel="stylesheet"
      href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.11.3/font/bootstrap-icons.min.css" />
```

Kullanım:

```
<button class="btn btn-primary">
  <i class="bi bi-plus-circle"></i> Yeni Kayıt
</button>

<a asp-action="Delete" class="btn btn-sm btn-danger">
  <i class="bi bi-trash"></i>
</a>
```

İkon araması: <https://icons.getbootstrap.com/>.

Local kurulum — LibMan:

```
libman install bootstrap-icons@1.11.3 -d wwwroot/lib/bootstrap-icons
```

```
<link rel="stylesheet" href="~/lib/bootstrap-icons/font/bootstrap-icons.css" />
```

C.3 Sık Kullanılan İkonlar (MVC Bağlamında)

İkon Sınıfı	Kullanım
bi-plus-circle	Yeni kayıt ekleme
bi-pencil / bi-pencil-square	Düzenle
bi-trash / bi-trash-fill	Sil
bi-eye / bi-eye-slash	Detay / gizle
bi-search	Arama
bi-house	Ana sayfa
bi-person / bi-people	Kullanıcı(lar)
bi-gear	Ayarlar
bi-list / bi-list-ul	Menü / liste
bi-x-circle / bi-check-circle	İptal / onay
bi-arrow-left / bi-arrow-right	Yön
bi-download / bi-upload	İndir / yükle
bi-envelope	E-posta
bi-telephone	Telefon
bi-calendar-event	Tarih / randevu

C.4 LibMan Yerine npm

Modern frontend araçlarıyla uyumlu çalışıyorsa:

```
npm install bootstrap@5.3.3 bootstrap-icons@1.11.3 bootswatch@5.3.3
```

Dosyalar node_modules/ altına iner; build sistemiyle (Webpack, esbuild) wwwroot'a kopyalanır. Şablon MVC projeleri LibMan tercih eder — daha basit.

Ek D. Razor Runtime Compilation

D.1 Ne İşe Yarar?

Varsayılan olarak, Razor view'ları uygulama başlatıldığında derlenir. Bir view'da değişiklik yaparsan projeyi yeniden başlatman gerekir. Bu, geliştirme döngüsünü yavaşlatır — özellikle küçük CSS/HTML değişikliklerinde.

Razor Runtime Compilation açıkken, view'lar her istekte gerekirse yeniden derlenir. Değişikliği kaydet, F5 yenile, sonucu gör — restart yok.

Sadece geliştirme ortamı için önerilir; üretimde performansı düşürür.

D.2 Kurulum

NuGet paketi:

```
Install-Package Microsoft.AspNetCore.Mvc.Razor.RuntimeCompilation
```

CLI:

```
dotnet add package Microsoft.AspNetCore.Mvc.Razor.RuntimeCompilation
```

D.3 Program.cs'de Aktivasyon

```
builder.Services.AddControllersWithViews()  
    .AddRazorRuntimeCompilation();
```

Bir kez eklendiğinde .cshtml dosyalarındaki değişiklik proje çalışırken görülür.

D.4 Yalnız Development İçin Açmak

Üretimde performans için kapatmak istersen:

```
var builder = WebApplication.CreateBuilder(args);  
  
var mvc = builder.Services.AddControllersWithViews();  
if (builder.Environment.IsDevelopment())  
{  
    mvc.AddRazorRuntimeCompilation();  
}
```

D.5 Hot Reload İle Karşılaştırma

.NET 6+ ile birlikte gelen **Hot Reload** özelliği, sadece view değil, C# kodunda da bazı değişiklikleri restart'sız uygulayabilir. `dotnet watch run` komutuyla çalıştırılır:

```
dotnet watch run
```

Runtime compilation ile Hot Reload birlikte kullanılabilir; Hot Reload C# kodunu, runtime compilation view'ları hedefler.

Ek E. Git ve GitHub Entegrasyonu

E.1 Genel Bakış

Git, dağıtık bir sürüm kontrol sistemidir; kaynak kodun tarihçesini, dallanmaları ve ekip çalışmasını yönetir. GitHub, Git repository'lerini barındıran bir platformdur (alternatifleri: GitLab, Bitbucket, Azure DevOps).

ASP.NET Core MVC projeleri için Git standart bir bileşendir; bu bölüm proje başlatma ve GitHub'a yükleme akışını özetler.

E.2 Sık Kullanılan Git Komutları

Komut	Ne yapar
git init	Klasörde Git repository başlatır
git status	Değişiklik durumunu gösterir
git add <dosya>	Bir dosyayı staging area'ya alır
git add .	Tüm değişiklikleri staging'e alır
git commit -m "mesaj"	Staging'deki değişiklikleri kaydeder
git log --oneline	Commit tarihçesini kısa listeler
git branch	Mevcut branch'i gösterir
git branch <ad>	Yeni branch oluşturur
git checkout <branch>	Branch değiştirir
git switch <branch>	Modern branch değiştirme
git checkout -b <ad>	Branch oluştur + geç
git merge <branch>	Belirtilen branch'i mevcut branch'e birleştirir
git remote add origin <url>	Uzak repository ekler
git push -u origin main	Local main branch'i uzağa gönderir + upstream kur
git push	Sonraki push'lar için
git pull	Uzaktaki değişiklikleri çeker
git clone <url>	Uzak repository'yi kopyalar
git diff	Değişiklikleri gösterir
git stash	Değişiklikleri geçici olarak saklar
git reset --hard HEAD	Tüm değişiklikleri geri alır (dikkatli)
git revert <commit>	Belirli commit'i tersine çeviren yeni commit üretir
git config --global user.name "Ad"	Kullanıcı adı
git config --global user.email "e"	E-posta

E.3 Yeni Bir MVC Projesini GitHub'a Yükleme

Adım adım:

1. Local repository başlat:

```
cd MyMvcProject
git init
```

2. .gitignore oluştur — .NET projeleri için standart ignore dosyası:

```
dotnet new gitignore
```

Bu komut, .gitignore dosyasını üretir; bin/, obj/, kullanıcı ayarları, .vs/, secrets dosyaları gibi Git'e girmemesi gerekenler otomatik listelenir.

3. İlk commit:

```
git add .
git commit -m "İlk MVC projesi"
```

4. GitHub'da yeni repository oluştur:

- GitHub arayüzünde "New repository" tıkla.
- Repo adı gir (MyMvcProject), README/gitignore ekleme (zaten local'de var).
- Repository oluştur; verilen URL'yi kopyala (<https://github.com/kullanici/MyMvcProject.git>).

5. Remote ekle ve push:

```
git remote add origin https://github.com/kullanici/MyMvcProject.git
git branch -M main
git push -u origin main
```

6. Sonraki commit'ler:

```
git add .
git commit -m "Doktor CRUD eklendi"
git push
```

E.4 .NET Projeleri İçin .gitignore (Özet)

dotnet new gitignore komutu ile oluşan dosyanın kritik girdileri:

```
# Build çıktıları
bin/
obj/

# Visual Studio
.vs/
*.user
*.suo

# Rider
.idea/

# NuGet
*.nupkg
packages/

# Kullanıcı secret'ları
appsettings.Production.json
appsettings.Local.json
secrets.json

# Log
*.log

# Temp
*.tmp
```

E.5 Branch Workflow — Basit Feature Branch

Yeni bir özellik için branch aç:

```
git checkout -b feature/doktor-modulu
# ... çalış ...
git add .
git commit -m "Doktor listesi eklendi"
git push -u origin feature/doktor-modulu
```

GitHub'da Pull Request aç, ekip inceler, main'e merge edilir. Sonra local:

```
git checkout main
git pull
git branch -d feature/doktor-modulu # local branch sil
```

E.6 Migration Dosyaları Git'e Girer

Kritik: Migrations/ klasörü **repository'ye commit edilir**. Ekip üyeleri aynı migration'ları paylaşır; DB şeması senkron kalır.

Sadece bin/ ve obj/ klasörleri commit edilmez; Migrations/ altındaki .cs dosyaları kaynak koddur.

E.7 Commit Mesajı Konvansiyonu

Genel kural: kısa (50 karakter altı) özet + boş satır + gerekirse detay. Sık kullanılan prefix'ler (Conventional Commits):

```
feat: Doktor CRUD eklendi
fix: Randevu tarih validasyonu düzeltildi
docs: README güncellendi
refactor: DoktorService AsNoTracking eklendi
test: RandevuService unit testleri
chore: NuGet paketleri güncellendi
```

Detay: <https://www.conventionalcommits.org/>.

E.8 Visual Studio Git Entegrasyonu

Visual Studio, Git için grafik arayüz sunar:

- **Team Explorer** (VS 2019) veya **Git Changes** (VS 2022): staging, commit, push.
- **Menü: Git → Manage Branches**: branch yönetimi.
- **Git menüsü → Clone Repository**: uzak repo indirme.

Ekip çalışması için: **Git menüsü → Fetch/Pull/Push** üçlüsü günlük döngüdür.

Ek F. Terim Sözlüğü

Belge boyunca kullanılan teknik terimlerin alfabetik sözlüğü. Türkçe karşılıklar + kısa açıklamalar.

Abstract class (soyut sınıf) — Doğrudan örneklenemeyen, kısmen tamamlanmış taban sınıf. Bazı üyeleri abstract (imza), bazıları concrete (gövdeli).

Action (aksiyon) — Controller içindeki public metot; bir HTTP endpoint'ini karşılar.

ActionResult — Bir action'ın döndürdüğü sonuç. ViewResult, JsonResult, RedirectResult gibi alt tipleri vardır.

AJAX (Asynchronous JavaScript and XML) — Sayfa yenilenmeden client-server veri alışverişi. Modern kodda veri JSON ile taşınır.

Anti-forgery token — CSRF saldırısına karşı gizli input token'ı; form ile birlikte gönderilir, sunucu doğrular.

appsettings.json — ASP.NET Core konfigürasyon dosyası; connection string, log seviyeleri, uygulama ayarları.

Assembly — Derlenmiş .NET çıktısı (.dll veya .exe). "Aynı assembly" ≈ aynı proje.

BaseEntity — Tüm entity'lerde ortak alanları toplayan taban sınıf (Id, OluşturmaTarihi, IsDeleted gibi).

BaseService — Generic CRUD implementasyonu; her entity servisi için tekrarı azaltır.

Bind (bağlama) — HTTP request verisinin action parametrelerine otomatik doldurulması (model binding).

Boxing — Değer tipinin object içine sarılıp heap'e taşınması. Unboxing tersidir.

Cascading dropdown — Bir dropdown'ın seçimine göre başka bir dropdown'ın AJAX ile dolması.

CASCADE (foreign key davranışı) — Parent silinirse child'ların da otomatik silinmesi.

Change tracker — DbContext'in bellekteki entity değişikliklerini izleyen bileşen.

Class (sınıf) — Nesnelerin nasıl üretileceğini belirleyen şablon.

Client-side validation — Form gönderilmeden önce browser'da JavaScript ile yapılan doğrulama.

Code First — Sınıfları önce yaz, DB'yi migration ile üret yaklaşımı.

Constructor — Nesne üretilirken çalışan özel metot; başlangıç durumunu ayarlar.

Constructor injection — Bağımlılıkların constructor parametreleriyle DI container'dan alınması.

Controller — MVC'nin kontrol katmanı; HTTP request'i alır, model'i işler, view'ı döndürür.

CSRF (Cross-Site Request Forgery) — Kullanıcı adına başka bir sitenin form göndermesine dayalı saldırı.

Data Annotation — Property'ye eklenen attribute; hem şema hem validasyon kuralı bildirir.

Database First — Var olan DB'den Scaffold-DbContext ile sınıf üretme yaklaşımı.

DbContext — EF Core'un giriş noktası; entity setlerini, change tracker'ı, bağlantıyı tutar.

DbSet — Bir entity tipinin veritabanı tablo karşılığı; sorgu ve komut kaynağı.

Deep copy — Bir nesnenin ve iç referanslarının hepsinin ayrı kopyalarının üretilmesi.

Dependency Injection (DI) — Bağımlılıkların container üzerinden dışarıdan alınması ilkesi.

Deferred execution — LINQ sorgusunun materialize edilene (ToList, First) kadar çalışmaması.

DIP (Dependency Inversion Principle) — SOLID'in D'si: bağımlılıklar somut değil, soyut olmalı.

DTO (Data Transfer Object) — Katmanlar arası veri paketi; ViewModel'e benzer.

Eager loading — İlişkili entity'lerin ilk sorguda Include ile yüklenmesi.

Entity — Veritabanına eşlenen sınıf; her instance bir DB satırı.

EntityState — Change tracker'daki entity durumu: Detached, Added, Unchanged, Modified, Deleted.

Expression tree — Kodu veri olarak temsil eden ağaç; LINQ IQueryable → SQL çevirisinin ara temsili.

Fluent API — OnModelCreating içinde method chaining ile model konfigürasyonu.

Foreign key (yabancı anahtar) — Başka bir tablonun primary key'ine referans veren sütun.

Global query filter — EF Core'da bir entity'nin tüm sorgularına otomatik eklenen filtre (soft delete için ideal).

Hot Reload — .NET'in bazı kod değişikliklerini restart'sız uygulama özelliği.

HTTP method — GET, POST, PUT, PATCH, DELETE gibi HTTP fiil'leri.

HttpContext — Bir HTTP request için oluşturulan bağlam; middleware'ler arasında taşınır.

IEnumerable — Sadece ileri iterasyon; foreach ile.

ICollection — Count, Add, Remove, Contains ekler; IEnumerable<T> üzerine kurulu.

IList — İndeksleyici (this[i]), Insert, RemoveAt ekler; ICollection<T> üzerine kurulu.

Immutable — Oluşturulduktan sonra durumu değiştirilemeyen tip (örn. string).

Include — EF Core'da eager loading için LINQ metodu.

Injection — DI container'ın bir bağımlılığı bir sınıfa vermesi.

Interface — Sadece imzalardan oluşan sözleşme; I prefix'i konvansiyonel.

IQueryable — Sorgunun SQL'e çevrileceği koleksiyon; expression tree üretir.

ISP (Interface Segregation Principle) — SOLID'in I'sı: küçük, odaklı interface'ler.

Kalıtım (Inheritance) — Bir sınıfın başka bir sınıfın üyelerini otomatik olarak edinmesi.

Kapsam (Scope) — Değişkenin görünür olduğu bölge; blok ({ }) ile sınırlanır.

Kapsülleme (Encapsulation) — İç veriyi private tutup kontrollü erişim vermek.

Layout — Sayfaların ortak iskeleti; _Layout.cshtml.

Lazy loading — Navigation property'ye erişildiği anda otomatik ek sorgu çalıştırma.

Lifetime — DI container'ın servis ömrü politikası: Singleton, Scoped, Transient.

LINQ (Language Integrated Query) — C#'ın entegre sorgulama dili.

LSP (Liskov Substitution Principle) — SOLID'in L'si: alt sınıf üst sınıfın yerine kullanılabilir.

Mass assignment — Kullanıcının form'dan istemediği property'leri bind ettirme saldırısı.

Materialization — LINQ sorgusunun çalıştırılıp bellekteki nesnelere dönüştürülmesi.

Middleware — HTTP request pipeline’ında sırayla çalışan bileşen.

Migration — EF Core’un versiyonlanmış şema değişikliği.

Model — MVC’nin veri ve iş kuralları katmanı; entity, ViewModel, servis.

Model Binding — HTTP request verisinin action parametrelerine otomatik dönüştürülmesi.

Model Validation — Form verisinin iş kurallarına uygunluk kontrolü.

MVC (Model-View-Controller) — Sunum katmanını üç sorumluluğa ayıran mimari desen.

Navigation property — Bir entity’nin ilişkili entity’lere erişim property’si.

Nullable — null alabilme yeteneği; değer tipleri için ? ile açılır.

OCP (Open/Closed Principle) — SOLID’in O’su: genişletmeye açık, değişikliğe kapalı.

OOP (Object-Oriented Programming) — Nesne yönelimli programlama paradigması.

ORM (Object-Relational Mapping) — Nesne dünyası ile ilişkisel DB arasında köprü.

Overload — Aynı isimli birden fazla metod; farklı parametre imzasıyla.

Override — Alt sınıfta taban virtual metodun yerini almak.

Partial view — Tekrar kullanılabilir küçük view parçası; ad alt çizgi ile başlar.

Pattern matching — C#’ta is, switch üzerinde tip/property/tuple deseni.

Pipeline — Middleware’lerin ardarda dizildiği zincir.

Polimorfizm (Polymorphism) — Aynı çağrının farklı tiplerde farklı davranması.

PRG (Post-Redirect-Get) — POST sonrası redirect ile refresh sorununu önleme deseni.

Primary key (birincil anahtar) — Bir tabloda her satırı benzersiz tanımlayan sütun.

Projection — LINQ Select ile veri şeklini değiştirme.

Property — Dış dünyaya alan gibi görünen ama gerçekte get/set olan üye.

Query filter — EF Core’un HasQueryFilter ile tanımlanan otomatik sorgu filtresi.

Razor — ASP.NET Core’un view template motoru; .cshtml dosyalarında @ sözdizimi.

Reference type (referans tipi) — Değeri heap’te, referansı stack’te tutan tip (class, string).

Refactoring — Davranışı değiştirmeden kod yapısını iyileştirmek.

Repository pattern — DbContext’in üstüne bir katman koyup veri erişimini soyutlama deseni.

Response — HTTP cevabı; status kodu, header’lar, body.

Route — URL şablonu ve controller/action eşlemesi.

Route data — URL’nin route slot’larından çıkarılan değerler.

Runtime Compilation (Razor) — View’ların çalışma zamanında derlenmesi; geliştirme kolaylığı.

Scaffold — Şablondan otomatik kod üretimi; DB’den entity üretmek için Scaffold-DbContext.

Scoped — Her HTTP request için tek DI örneği ömrü.

Section — Layout’un ayrılmış bölgesi; view’dan içerik enjekte edilir.

SelectList / SelectListItem — Dropdown için <option> üretimi.

Server-side validation — Sunucuda ModelState.IsValid ile yapılan doğrulama.

Shallow copy — Nesnenin alan değerleri kopyalanır, iç referanslar paylaşılır.

Singleton — Uygulama boyu tek DI örneği ömrü.

SOLID — OOP’nin beş tasarım prensibi (SRP, OCP, LSP, ISP, DIP).

Soft delete — Kaydı fiziksel silmek yerine IsDeleted = true işaretleme.

Soyutlama (Abstraction) — Ne yaptığını göster, nasıl yaptığını gizle.

SRP (Single Responsibility Principle) — SOLID’in S’si: tek sorumluluk.

Static class — Sadece static üyeler taşıyan, örneklenmeyen sınıf.

Stack — Yerel değişkenlerin LIFO düzende tutulduğu bellek bölgesi.

Startup — Klasik iki dosyalı (Program.cs + Startup.cs) MVC şablonu. .NET 6+ ile tek Program.cs'ye taşındı.

Strong typing (statik tiplendirme) — Değişken tipinin derleme zamanında bilinmesi.

Switch expression — C# 8+ ile gelen => sözdizimli modern switch; değer üretir.

Tag Helper — Razor'da asp-* prefix'li HTML attribute'ları ile server-side işleme.

TempData — Sadece bir sonraki request'e taşınan veri; PRG mesajları için.

Toastr — jQuery tabanlı bildirim (toast) kütüphanesi.

Transient — Her DI çağrısında yeni örnek ömrü.

TryParse — String'i tipe çeviren, başarısızlıkta exception atmayan güvenli metot.

Tuple — Birden fazla değeri geçici olarak birlikte taşıyan yapı.

Unboxing — Boxed object'ten orijinal değer tipini çıkarmak.

Unique constraint — Bir sütunun (veya sütun grubunun) benzersiz olmasını garanti eden DB kısıtı.

Value type (değer tipi) — Kendi değerini stack'te tutan tip (int, bool, struct).

Var — C#'ta örtük tiplendirme; derleyici tipi sağdaki ifadeden çıkarır.

ViewBag — dynamic wrapper; view'a küçük ek veriler geçirmek için.

ViewComponent — Kendi verisini kendi çeken, mini controller + view çifti.

ViewData — Sözlük tabanlı view veri taşıyıcısı; ViewBag ile aynı depoyu paylaşır.

ViewModel — View'a özel sınıf; entity ile view arasında ara katman.

Virtual — Alt sınıflarda override edilebilir olarak işaretleme.
